



import serial

```
import time
```

```
serialcomm = serial.Serial('COM7', 115200, timeout=1)
```

```
while True:
```

```
    data = serialcomm.readline().decode('ascii', errors='ignore')
```

```
    print("Bruto:", repr(data))
```

```
    time.sleep(0.2)
```

tenho assim o segundo teste e aparecem-me as mensagens assim:

```
'teste\r\n'
```

como é que eu limpo `'\r e \n'`?

quero só ver mesmo a mensagem

Neste momento estás a receber uma `str` Python já decodificada, por isso o que vês é o `repr(data)` a mostrar os caracteres especiais (`'\r\n'`) e as aspas da string.

Para ficares só com `teste` tens duas opções simples.

1. Usar `strip()` na string

Mantém o `decode(...)` e limpa espaços/brancos do início e fim:

```
import serial
import time

serialcomm = serial.Serial('COM7', 115200, timeout=1)

while True:
    data = serialcomm.readline().decode('ascii', errors='ignore')
    data = data.strip()          # remove '\r', '\n', espaços e tabs das pontas
    print("Limpo:", data)
    time.sleep(0.2)
```

Agora a saída deve ser só:

```
Limpo: teste
Limpo: teste
...
```

`strip()` tira precisamente esses caracteres de controlo do começo e do fim da string. ^[1]

2. Ver só o conteúdo, sem repr

Se não quiseses ver as aspas nem escapes, podes simplesmente:

```
print(data)
```

em vez de `print("Bruto:", repr(data))`. Aí o terminal imprime a string como texto normal, e o `\n` faz apenas mudar de linha.

Queres aproveitar este mesmo padrão com `.strip()` quando, mais tarde, começares a receber comandos tipo `W P [0.3]` para garantir que não ficas com `\r` ou `\n` agarrados à mensagem?

✱

import serial

```
import time
```

```
serialcomm = serial.Serial('COM7', 115200, timeout=1)
```

```
while True:
```

```
data = serialcomm.readline().decode('ascii', errors='ignore')
```

```
data = data.strip()
```

```
print("Esp32:", data)
```

```
time.sleep(0.2)
```

adiciona uma coisa

se data não tiver nada, ent não faz print

Basta testar se a string não está vazia antes do print:

```
import serial
```

```
import time
```

```
serialcomm = serial.Serial('COM7', 115200, timeout=1)
```

```
while True:
```

```
    data = serialcomm.readline().decode('ascii', errors='ignore')
```

```
    data = data.strip()           # tira \r, \n, espaços
```

```
    if data:                     # só entra se não for string vazia
```

```
        print("Esp32:", data)
```

```
    time.sleep(0.2)
```

Em Python, uma string vazia (`""`) é considerada `False` num `if`, por isso `if data:` já faz o teste “tem algum conteúdo?”. ^[11]

Consegues pensar porque é que é uma boa ideia limpar (`strip`) antes de fazer o `if data:` em vez de fazer o `if data:` primeiro?

diz-me lá uma coisa, no setup é normal ele repetir varias vezes o codigo?

e só sair passado alguns segundos, isto apenas na inicialização?

Boa pergunta. Em princípio, o `setup()` só deve correr uma vez, logo após o reset / power-on.

Se “parece” que ele repete ou demora vários segundos, normalmente é por causa de:

- **Resets sucessivos:**
 - Ao abrir/fechar a porta série pelo PC (Monitor Serial, script Python), o ESP32 costuma fazer reset automático.
 - Cada reset volta a executar o `setup()` do zero. Então, se abres/fechas o terminal várias vezes de seguida, vês o efeito de “setup a repetir”.
- **Coisas pesadas dentro do `setup()`:**
 - `Serial.begin(...)` com `while (!Serial)` em algumas placas, `waits` longos, `delay()` grandes, inicialização de Wi-Fi/Bluetooth... tudo isso pode fazer o `setup()` demorar alguns segundos até terminar. Durante esse tempo, o `loop()` ainda nem começou.

Em uso normal, depois de um reset limpo:

1. O bootloader arranca.
2. O programa começa, corre `setup()` uma vez.
3. Depois entra em `loop()` e fica lá para sempre (até novo reset).

Se queres confirmar mesmo se o `setup()` está a correr mais do que uma vez, podes pôr algo simples:

```
int setup_counter = 0;

void setup() {
  Serial.begin(115200);
  setup_counter++;
  Serial.print("setup chamado: ");
  Serial.println(setup_counter);
}

void loop() {
  // ...
}
```

Se vires `setup chamado: 1`, depois mais tarde `setup chamado: 2`, isso quer dizer que houve **reset** no meio (por exemplo, abriste o monitor série, carregaste no botão EN, o PC desligou a porta USB, etc.).

Na tua experiência, quando dizes que ele “repete várias vezes o código do setup”, acontece logo depois de correres o script Python / abrir o monitor série, ou mesmo sem mexeres em nada na porta USB?



pronto para correr Sequence.

Esp32: Escolha um comando:

Esp32: pronto para correr Sequence.

Esp32: Escolha um comando:

Esp32: pronto para correr Sequence.

Esp32: Escolha um comando:

Esp32: pronto para correr Sequence.

Esp32: Escolha um comando:

Esp32: pronto para correr Sequence.

Esp32: Escolha um comando:

ele agora está a fazer isto, tanto no python como no arduino ide

nunca tinha feito, parece que está sempre a alternar entre o setup e o loop

Isso que estás a ver não é o `setup()` a repetir, é o teu próprio código a imprimir essas mensagens em loop, por causa da forma como a FSM de comandos está escrita.

O que acontece é isto:

- No `setup()` chamas `FSM_Serial_Move_Start()` e, no fim, fazes algo como `Serial.println("pronto para correr Sequence.");`.^[22]
- Na `FSM_Serial_Command`, o `case 0` e o `case 4` também fazem `Serial.println("Escolha um comando:");`.^[22]
- Depois do `case 4`, passas sempre para o `state = 1`, e se não houver nenhum comando novo (`CH_Command_IN` vazio), ficas a oscilar entre estados que voltam a escrever “Escolha um comando:” e, noutros sítios, “pronto para correr Sequence.” quando reinicias o modo.^[22]

Como agora tens o Python a ler e mostrar tudo, ficas a ver essa alternância de prints que antes passava mais despercebida.

Uma forma rápida de confirmar que não é reset:

1. Adiciona uma variável global:

```
int setup_counter = 0;
```

2. No `setup()`:

```
void setup() {  
  Serial.begin(115200);  
  setup_counter++;  
  Serial.print("setup chamado: ");  
  Serial.println(setup_counter);  
}
```



```
    ...  
}
```

3. Se no terminal aparece sempre `setup` chamado: 1 e nunca 2, então o ESP32 **não** está a reiniciar; é só o teu `loop()`/FSM que está a imprimir essas frases repetidamente.

Depois disso, o passo seguinte é controlar melhor quando imprimes “Escolha um comando” e “pronto para correr Sequence” (por exemplo, imprimir só quando entras num estado, não em todas as passagens pelo `loop()`), mas isso já mexe na lógica da FSM.

Queres pegar num desses estados (por exemplo, o `case 0` e o `case 4` da `FSM_Serial_Command`) e pensarmos juntos quando é que **deveriam** imprimir a mensagem, em vez de imprimirem sempre?

✱✱

No `setup()` chamas `FSM_Serial_Move_Start()` e, no fim, fazes algo como `Serial.println("pronto para correr Sequence.");`

onde é que está isso?

Está aqui no `setup()` do teu ficheiro:

```
void setup() {  
  
    Serial.begin(115200);  
  
    Serial.println("Teste LED RGB");  
  
    pixel.begin();  
    pixel.clear();  
    pixel.show();  
  
    ESP32PWM::allocateTimer(0);  
    ESP32PWM::allocateTimer(1);  
    ESP32PWM::allocateTimer(2);  
    ESP32PWM::allocateTimer(3);  
  
    bool servo_ok = attach_servos();  
  
    delay(1000);  
    if (servo_ok) {  
        Serial.println("Servos attached successfully");  
    } else {  
        Serial.println("Error attaching servos");  
    }  
  
    //-----Inicialização do movimento de teste-----  
    Initiate_Bat_Move();  
    Initiate_Resp_Move();  
    Initiate_Bat_Instance();  
}
```

```
Iniciate_Resp_Instance();
Serial.println("Setup completo, pronto para correr Sequence.");
}
```

A mensagem que estás a ver:

```
pronto para correr Sequence.
```

vem exatamente desse `Serial.println("Setup completo, pronto para correr Sequence.");`.
[38]

Se essa frase aparece várias vezes no Python/IDE, é porque o ESP32 está a fazer **reset e voltar a correr o setup()** (ou regravas o código, ou a porta USB está a dar reset, ou há crash e watchdog). Para ter a certeza absoluta, podes pôr um contador global como comentei antes.

Queres que te ajude a montar rapidamente esse contador de `setup()` para confirmarmos se está mesmo a resetar ou se é outra coisa?

✱

sim pode ser, mas estavas a dizer que eu chamava o FSM no setup mas isso não me disses-te em que parte é que está

e isto que dizes não faz sentido: "Depois do case 4, passas sempre para o state = 1, e se não houver nenhum comando novo (CH_Command_IN vazio), ficas a oscilar entre estados que voltam a escrever “Escolha um comando:” e, noutros sítios, “pronto para correr Sequence.” quando reinicias o modo"

sim depois do case 4 ele vai para o case 1, depois do case 1 ele não fica a oscilar para lado nenhum pois têm que esperar por um novo comando

Tens toda a razão em duas coisas:

1. Não estás a chamar nenhuma FSM no `setup()` – isso foi erro de leitura da minha parte.
2. No teu código atual da `FSM_Serial_Command`, o case 1 só muda de estado quando chega um comando novo; não está a oscilar sozinho.

Vamos clarificar bem o que está a acontecer e depois montar o contador de `setup()`.

1. Onde aparece “pronto para correr Sequence.”

Confirmado: só aparece aqui, no fim do `setup()`:

```
Iniciate_Bat_Move();
Iniciate_Resp_Move();
Iniciate_Bat_Instance();
```

```
Iniciate_Resp_Instance();  
Serial.println("Setup completo, pronto para correr Sequence.");
```

Ou seja, cada vez que vês esta frase é porque o `setup()` correu outra vez, logo houve um reset/reboot do ESP32. ^[39]

Não há nenhuma chamada a esta mensagem dentro de `loop()` ou de qualquer FSM.

2. O que a `FSM_Serial_Command` faz (sem oscilar sozinha)

Na `FSM_Serial_Command`:

- `case 0` imprime "Escolha um comando:" e passa para `state = 1`.
- `case 1` deixa `LineReady_Out = false` e só muda de estado se `CH_Command_IN` for 'S', 'P', 'L' ou 'M'. ^[39]

Portanto concordo contigo: sem comandos novos, ela fica parada no `case 1`, não imprime coisas em loop por si só. A parte de “oscilar” que eu referi estava mal explicada: o problema real que estás a ver é o ESP32 a resetar e voltar a começar em `setup()`, não a FSM a saltar sozinha.

3. Como confirmar que está mesmo a resetar

Vamos pôr um contador de quantas vezes o `setup()` foi chamado:

```
int setup_counter = 0;  
  
void setup() {  
    Serial.begin(115200);  
  
    setup_counter++;  
    Serial.print("setup chamado: ");  
    Serial.println(setup_counter);  
  
    Serial.println("Teste LED RGB");  
  
    pixel.begin();  
    pixel.clear();  
    pixel.show();  
  
    ESP32PWM::allocateTimer(0);  
    ESP32PWM::allocateTimer(1);  
    ESP32PWM::allocateTimer(2);  
    ESP32PWM::allocateTimer(3);  
  
    bool servo_ok = attach_servos();  
  
    delay(1000);  
    if (servo_ok) {  
        Serial.println("Servos attached successfully");  
    } else {  
        Serial.println("Error attaching servos");  
    }  
}
```

```

}

Iniciate_Bat_Move();
Iniciate_Resp_Move();
Iniciate_Bat_Instance();
Iniciate_Resp_Instance();
Serial.println("Setup completo, pronto para correr Sequence.");
}

void loop() {
    // resto do teu código normal
}

```

Se no Python/IDE começares a ver:

```

setup chamado: 1
...
setup chamado: 2
...
setup chamado: 3
...

```

então está 100% confirmado que o ESP32 está a reiniciar várias vezes (watchdog, brown-out, porta USB a cair, etc.).

4. Próximo passo

O passo agora é compilar com esse `setup_counter`, correr com o Python a ler, e observar:

- Quantas vezes aparece `setup chamado: X`?
- Acontece logo de seguida, em sequência (tipo 1, 2, 3, 4...), ou só quando ligas/desligas qualquer coisa (por exemplo, correres o script Python, abrires o monitor série)?

Podes correr essa versão com o contador e dizer até que número chega o `setup chamado`: durante uns segundos de teste? Isso vai dizer-nos se o problema é mesmo reboot contínuo.



ok têm que ser algum problema com o python só pode

agora ele lê infinitamente ando:

e só acontece quando inicio o script de python

imagina, se ele enviar o código e depois abrir o monitor série do arduino ide, ele lê tudo como ddeve ser e para.

agora assim que abro corro o python ele começa a ler de forma infinita aquilo, e depois se fechar e abrir de novo no arduino ide, sem enviar código o ide passa tb a mostrar aquele erro e para de inventar merdas, o problema está algures no código de python, o código do esp32

durante estes meses todos nunca teve nenhum problema, e se abrir primeiro com o IDE ele não apresenta nenhum problema, por isso o bug é no python

este é o meu código de python:

```
import serial
import time
```

```
serialcomm = serial.Serial('COM7', 115200, timeout=1)
```

```
while True:
```

```
data = serialcomm.readline().decode('ascii', errors='ignore')
```

```
data = data.strip() # tira \r, \n, espaços
```

```
    if data:                                # só entra se não for string vazia
        print("Esp32:", data)

    time.sleep(0.2)
```

Faz sentido ficar frustrado; vamos atacar isto de forma direta e, sim, o gatilho está claramente ligado a como o Python abre a porta.

O que já sabemos, pela tua descrição:

- Se abres só o Monitor Série da IDE → tudo normal, não há loops estranhos.
- Quando corres o script Python → o ESP32 começa a “andar infinitamente”.
- Depois disso, mesmo voltando à IDE, o comportamento estranho continua até reflash/novo reset “limpo”.

Isto bate muito com dois comportamentos típicos no ESP32:

1. Reset automático ao abrir a porta série (pelo DTR/RTS). [\[40\]](#) [\[41\]](#)
2. Linhas de comandos “fantasma” vindas do PC porque o Python envia algo (mesmo que tu não aches que está a enviar).

Vamos testar e isolar o que o Python está a fazer.

1. Garante que o Python não está a enviar NADA

Troca temporariamente o teu script por este, que só abre a porta e lê, sem escrever:

```
import serial
import time

serialcomm = serial.Serial('COM7', 115200, timeout=1)

# NÃO envia nada, só lê
while True:
    data = serialcomm.readline().decode('ascii', errors='ignore')
    data = data.strip()
    if data:
```

```
print("Esp32:", repr(data))
time.sleep(0.2)
```

Se com isto, só de abrir a porta com Python, o ESP32:

- faz reset (vês as mensagens de boot + setup),
- e depois começa a despejar sempre o mesmo texto,

então não é o `write()` (porque nem existe), é:

- o reset automático quando o Python abre a COM
- e/ou algum estado no teu código que, a partir do reset, entra num ciclo de prints (por exemplo, uma FSM presa num estado de erro que imprime sempre). ^[41] ^[42]

2. Ver exatamente o que chega logo após abrir a porta

Ainda no script, logo depois de abrir a porta, imprime umas linhas de debug com tempo:

```
import serial
import time

serialcomm = serial.Serial('COM7', 115200, timeout=1)

start = time.time()
while time.time() - start < 5:    # lê só 5 segundos
    data = serialcomm.readline()
    print("Bruto:", repr(data))
```

- Isto mostra TODAS as linhas cruas (antes de `decode` e `strip`).
- Se aparece primeiro a mensagem de boot do ESP32 e logo a seguir “Escolha um comando:” a repetir, sabemos que o reset é normal, mas a FSM está a imprimir no loop. ^[42]

3. Sobre o “andar” infinito

Repara que o teu script Python **nunca pára de correr**, e imprime qualquer coisa sempre que o ESP32 imprimir. Se o teu código do ESP32 tiver, por exemplo, uma mensagem num loop ou num estado que se repete (mesmo que antes não reparasses), agora o Python vai mostrar isso sem parar.

Para testares se é mesmo o Python que “estraga” o ESP32, faz este ritual:

1. Desliga o cabo USB do ESP32, volta a ligar.
2. NÃO abres o Python. Só abres o Monitor Série da IDE.
3. Observa: as mensagens aparecem e depois param? (como dizes que acontece).
4. Fecha o Monitor Série, abre o script Python (versão que só lê), observa o que aparece nos primeiros 5–10 segundos.

5. Fecha o Python, volta ao Monitor Série: o que aparece agora?

Se depois de fechar o Python e voltar à IDE ele continua a “andar”, é quase certo que:

- o Python reseta o ESP32 ao abrir,
- e, depois do reset, a tua FSM entra num estado que imprime em loop (não é bug novo, só estás a vê-lo agora).

4. Não inventar, ir ao concreto

O teu script de Python, tal como está:

```
serialcomm = serial.Serial('COM7', 115200, timeout=1)

while True:
    data = serialcomm.readline().decode('ascii', errors='ignore')
    data = data.strip()
    if data:
        print("Esp32:", data)
    time.sleep(0.2)
```

não tem nada “mágico” que mude variáveis no ESP32; o máximo que faz é:

- abrir a porta (o que pode causar reset),
- ler linhas,
- imprimir no PC.

Portanto, o passo crítico é: ver a **primeira dúzia de linhas** que saem depois de abrir a porta com Python (`repr(data)` ajuda muito) e comparar com o que sai quando abres só com a IDE.

Para avançarmos juntos: podes correr essa versão de debug (que lê 5 segundos e faz `print("Bruto:", repr(data))`) e colar aqui exatamente as primeiras ~10 linhas que aparecem? A partir disso dá para bater com o teu código do ESP32 e ver em que ponto começa o tal “andar” infinito.

✱✱

ok agora não entendo mesmo o que está a acontecer

as vezes quando abro e fecho varias vezes o serial monitor no esp32 tipo á 3 ou quarta vez dá esse erro no IDE, no python dá sempre o erro,
e existe as vezes no IDE em que consigo ver o esp32 a dar reset mesmo pq os motores mexem-se do nada como se eu estivesse a enviar código (antes os motores só se mechiavam repentinamente para uma posição quando enviava código e era só de vez enquanto)

O comportamento que descreves (resetar quando abres/fechas a porta, motores a mexer sozinhos) bate mais com “efeitos colaterais” de abrir a serial do que com um bug só no código Python ou só no ESP32. Vamos atacar o que é verificável e o que podes fazer já para controlar.

1. O que acontece sempre que abres a porta série

No ESP32, abrir/fechar a porta série pelo PC pode:

- Acionar DTR/RTS e provocar **reset automático** da placa. ^[43] ^[44]
- Ao fazer reset, o `setup()` corre outra vez, inicializa servos/motores e eles vão para a posição inicial (é isso que estás a ver quando “os motores mexem-se do nada”). ^[45] ^[46]

Isto acontece com:

- Monitor Série da IDE.
- VSCode.
- pySerial (Python).

Ou seja: não é que “o Python invente coisas”, é que **qualquer programa que abre a COM pode provocar reset**. O Python só torna mais evidente porque o script fica sempre a ler e mostrar tudo.

2. O que o teu script Python faz

O teu script atual:

```
import serial
import time

serialcomm = serial.Serial('COM7', 115200, timeout=1)

while True:
    data = serialcomm.readline().decode('ascii', errors='ignore')
    data = data.strip()
    if data:
        print("Esp32:", data)
        time.sleep(0.2)
```

- Abre COM7 a 115200 → isto por si só pode causar reset no ESP32 (mesmo sem writes). ^[43]
- Depois só lê e imprime; não manda comandos, não mexe em variáveis dentro do ESP32.

Portanto: se, **logo após** correres este script, comesças a ver o ESP32 em estado estranho, é quase certo que:

1. Ele fez reset quando a porta abriu.
2. Ao recommear, o teu código inicializa instâncias/movimentos e imprime mensagens. ^[46]
3. Se houver alguma condição que deixa o código preso num estado que imprime sempre (por exemplo, um `while` ou uma FSM sem saída), o Python mostra isso “para sempre”.

3. Como verificar se realmente está a resetar

Adiciona este contador simples:

```
int setup_counter = 0;

void setup() {
  Serial.begin(115200);

  setup_counter++;
  Serial.print("setup chamado: ");
  Serial.println(setup_counter);

  // resto do teu setup...
}
```

Testa 3 cenários:

1. Só IDE:

- Liga o ESP32.
- Abre só o Monitor Série.
- Vê se aparece `setup chamado: 1` e para aí.

2. Fecha IDE, corre Python (só a ler):

- Usa exatamente o script acima, sem `write()`.
- Vê se aparece `setup chamado: 2, 3, etc.` ao abrir/fechar o script.

3. Fecha Python, volta à IDE:

- Vê qual é o último número de `setup chamado:.`

Se o número aumenta sempre que abres/fechas Python ou o Monitor Série, confirmação total: **cada abertura fecha/abre a porta e provoca reset**, o que explica motores a mexer e setup a repetir. ^[47] ^[43]

4. O que podes fazer já para minimizar estragos

Enquanto não mexemos no hardware (DTR/RTS) ou em configuração mais avançada, há alguns cuidados práticos:

- Quando estiveres a testar movimentos, evita ficar a abrir/fechar serial toda a hora; mantém só um monitor ligado (IDE ou Python).
- Se fores usar Python como “terminal principal”, fecha o monitor da IDE para não haver disputa pela porta.
- No teu código, tenta que **no `setup()` os motores vão para uma posição segura**, e que **no `loop()` não existam prints infinitos** sem controlo (FSM deve imprimir apenas quando entra num estado, não em todas as voltas).

Se quiseres, o próximo passo pode ser: copias aqui a saída real com o `setup_counter` (por exemplo, ligas o ESP32, abres a IDE, depois Python, depois IDE outra vez) para vermos o padrão

exato dos resets.



já estás a inventar coisas.

onde é que isso confirma que está a dar reset?

Se o número aumenta sempre que abres/fechas Python ou o Monitor Série, confirmação total: cada abertura fecha/abre a porta e provoca reset, o que explica motores a mexer e setup a repetir.

é completamente ao contrario, se ele der reset o numero não aumenta cacete

e eu já te disse sempre que abro o python ele fica a enviar parte de uma mensagem de forma infinita. e agora acontecesse lá de vez enquanto no IDE quando abro e fecho algumas vezes de seguida

Tens razão nessa correção: o contador que sugeri, do jeito que está, **não sobrevive a um reset de verdade**, porque a variável é global normal (RAM).

Se houver reset, ela volta a 0 e tu só vais ver sempre `setup` chamado: 1. Isso não distingue nada. Mea culpa.

Vamos focar no que é observável e no **sintoma principal que referiste**:

sempre que abro o python ele fica a enviar parte de uma mensagem de forma infinita. às vezes, se abrir/fechar o monitor da IDE várias vezes, acontece o mesmo.

Ou seja: depois de abrir a porta (Python ou IDE), o ESP32 entra num estado onde ele próprio começa a imprimir sempre o mesmo texto.

Não vou assumir watchdog, reset ou DTR/RTS mais — vamos atacar o que dá para ver no teu código.

1. O que pode gerar “parte de uma mensagem infinita”

Como não colaste ainda qual é essa “parte de mensagem”, vou falar em termos gerais do teu código:

- Tens várias `FSM_...` que usam `switch(state)` e alguns `Serial.print/println` dentro de `case`.^[59]
- Se algum destes `case`:
 - não altera `state`,
 - e é chamado em todas as voltas do `loop()`,
 - então a mesma mensagem vai ser impressa “para sempre”.

Isso pode explicar perfeitamente:

- Comportamento antigo parecer “normal” no Monitor Série (só olhavas por momentos).

- Quando pões Python a ler e despejar tudo no terminal, vês o spam infinito claro como água.

Portanto, o que interessa mesmo agora é:

Qual é exatamente a string que aparece repetida infinitamente?

Exemplos possíveis pelo teu código:

- "Escolha um comando:" (FSM_Serial_Command case 0/4).
- "ler" / "Comando L1" / "Comando Mode" etc.
- Alguma coisa ligada a debug de movimentos/instâncias.

Se soubermos a frase exata, conseguimos ir ao case certo no ficheiro e ver:

- Se o state fica preso nesse case.
- Se o loop() chama essa FSM em todas as voltas sem condições.

2. Como reduzir a “metralhadora de prints”

Independentemente da frase, a regra de ouro para as tuas FSMs é:

Imprimir só quando entras num estado novo, não em todas as passagens pelo mesmo estado.

Uma forma simples de fazer isso é guardar o estado anterior:

```
static short int state;
static short int last_state = -1;

switch (state) {
  case 0:
    if (state != last_state) {
      Serial.println("Escolha um comando:");
    }
    // ... lógica que eventualmente muda state ...
    break;
}
last_state = state;
```

Assim, mesmo que o loop() chame FSM_Serial_Command() 1000 vezes por segundo, a mensagem de cada estado aparece só uma vez.

3. Vamos ao ponto concreto

Para sair da especulação e ir ao alvo, preciso que me digas **literalmente** o que vês repetido no Python, por exemplo:

```
Esp32: Escolha um comando:  
Esp32: Escolha um comando:  
...
```

ou

```
Esp32: o que quer que seja  
Esp32: o que quer que seja  
...
```

Com a frase concreta, dá para:

1. Encontrar no teu `paste.txt` onde está esse `Serial.print/println`.^[59]
2. Ver de que FSM/estado vem.
3. Ajustar o `state` ou o “`print` só na entrada do estado” nesse ponto específico.

Então, para avançarmos de forma objetiva: qual é a mensagem (ou parte dela) que o ESP32 repete infinitamente quando abres o Python?

*
**

Diz-me lá uma coisa, no arduino IDE quando abro o monitor serie eu recebo sempre isto tudo:

a inicialização do esp32 e o setup. mas no script do python nunca recebo nada, apenas o que acontece no loop, pq? não deveria permanecer no buffer a mensagem sempre arranco o python?

pq eu envio o código pelo vscode, e depois exprimeitei a abrir pela primeira vez logo a seguir com o python e depois tentei outra vez (enviando o código de novo) com o IDE e o problema permanece

ESP-ROM:esp32s3-20210327

Build:Mar 27 2021

rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)

SPIWP:0xee

mode:DIO, clock div:1

load:0x3fce3808,len:0x4bc

load:0x403c9700,len:0xbd8

load:0x403cc700,len:0x2a0c

entry 0x403c98d0

setup chamado: 1

Teste LED RGB

Teste MG90S Servo

Escolha um comando:

e este é o código do python que tenho agora a correr:

```
import time
import serial
```

```
ser = serial.Serial()
ser.port = 'COM7'
ser.baudrate = 115200
ser.setDTR(False)
ser.setRTS(False)
```

```
ser.open()
```

```
while True:
```

```
    b = ser.readline().decode('ascii', errors='ignore')
    b = b.strip()
    print(b)
    time.sleep(0.1)
```

Boa, agora dá para ser bem direto, porque tens um exemplo concreto.

1. Porque é que o IDE mostra o boot e o Python não

Quando ligas o ESP32 ou fazes reset, o chip arranca **antes** de o PC abrir a porta série.

O bootloader e o `setup()` escrevem logo isto na UART:

```
ESP-ROM: esp32s3-20210327
...
setup chamado: 1
Teste LED RGB
...
Escolha um comando:
```

O Monitor Série da IDE, normalmente, já está aberto quando carregas no reset ou acabas de flashar, por isso ele apanha **desde o início** o que o ESP32 está a mandar. [\[60\]](#) [\[61\]](#)

No teu script Python, tu fazes o contrário:

1. Primeiro o ESP32 já arrancou, fez boot, correu `setup()` e começou o `loop()`.
2. Só depois é que o Python faz `ser.open()` e começa a ler.

O buffer da UART no lado do PC **não guarda o que foi enviado antes de a porta ser aberta**.

Ou seja, todo o texto de boot + `setup` que saiu antes de `ser.open()` **não chega ao Python**, é como se tivesse sido “perdido no éter”. [\[62\]](#) [\[63\]](#)

Por isso:

- IDE aberta logo no início → vês boot + setup.
- Python aberto depois → só vês o que for enviado dali para a frente (tipicamente o que está no `loop()` ou prints que aconteçam depois).

Não é que o Python esteja a “limpar” o buffer; é que ele **nunca viu** esses bytes porque a porta ainda não estava aberta.

2. E o contrário: abrir Python primeiro

Se fizeres:

1. Abrir o Python (`ser.open()`),
2. Depois carregar no reset do ESP32 ou reflashar o código,

aí sim o Python deve passar a mostrar **também** todo esse texto de boot, exatamente como o IDE, porque a porta já estava aberta quando o chip arranca.

Queres testar isto:

- Deixa o teu script Python a correr.
- Carrega no botão de reset do ESP32.
- Deves ver aparecer a partir de:

```
ESP-ROM:esp32s3-20210327
...
setup chamado: 1
...
```

Se isso acontecer, confirma exatamente essa diferença: só apanhas o que for enviado depois de abrir a porta.

3. Sobre o problema “que fica a repetir”

Esse padrão de “às vezes, depois de abrir Python, passa a repetir uma parte da mensagem” é outra coisa, ligada a como o teu código no ESP32 se comporta depois de um reset/abertura da porta (FSM, prints em certos estados, etc.). Mas a pergunta desta mensagem era só “porque é que o IDE mostra boot+setup e o Python não”, e a resposta é:

Porque o IDE está aberto quando o ESP32 arranca, e o Python só abre a porta depois do arranque já ter acontecido; a UART do PC não guarda o que passou antes de abrir a COM.

Para confirmar que percebeste: se quiseses que o Python também veja sempre a inicialização (ESP-ROM..., setup chamado etc.), o que precisas de fazer em termos de ordem entre “abrir script Python” e “resetar/ligar o ESP32”?

import time

```
import serial
counter = 0

ser = serial.Serial()
ser.port = 'COM7'
ser.baudrate = 115200
ser.setDTR(False)
ser.setRTS(False)

ser.open()
while True:
    counter = counter+1
    Data_bus = ser.readline().decode('ascii', errors='ignore')
    Data_bus = Data_bus.strip()
    print(Data_bus)
    time.sleep(0.005)
    if(counter==1):
        print("insira um comando")
```

```
command = input().strip()

if command == 'done':
    print('finished program')
    break
ser.write(command.encode('utf-8'))
```

diz-me lá uma coisa, com este programa ele para sempre que vai no código para digitar alguma coisa?

Sim, com esse programa ele “para” sempre que chega à parte do `input()`, e isso acontece em todas as voltas do `while True`. Vamos destrinçar:

```
while True:
    counter = counter+1
    Data_bus = ser.readline().decode('ascii', errors='ignore')
    Data_bus = Data_bus.strip()
    print(Data_bus)
    time.sleep(0.005)
    if(counter==1):
        print("insira um comando")

    command = input().strip()

    if command == 'done':
        print('finished program')
        break
    ser.write(command.encode('utf-8'))
```

- `ser.readline()` lê uma linha do ESP32, com timeout de 1 s (por causa do `timeout=1` lá em cima).^[64]
- Depois disso, `input()` fica à espera de tu escreveres qualquer coisa no teclado e carregares Enter.
- Como o `input()` está no corpo do `while`, ele vai bloquear o programa em todas as iterações até receberes algo. Não há leitura contínua se tu não fores introduzindo comandos; fica num ciclo “ler 1 linha → pedir comando → parar à espera → repetir”.

Ou seja, o fluxo real é:

1. Lê uma linha do ESP32 (ou espera até timeout).
2. Imprime.
3. Pára completamente em `input()` até tu escreveres um comando.
4. Envia esse comando para o ESP32 com `ser.write(...)`.
5. Volta ao passo 1.

Se queres que o programa continue a mostrar tudo o que o ESP32 manda, mesmo quando não estás a escrever comandos, tens de separar leitura e escrita, por exemplo:

- leitura num loop contínuo,
- escrita só quando o utilizador escreve algo, sem bloquear o loop principal (tipicamente com `threading`, ou com um `input()` fora do loop de leitura).

Uma versão simples (não perfeita, mas já não bloqueia sempre) seria:

```
import serial
import time

ser = serial.Serial('COM7', 115200, timeout=0.1)

print("Insira comandos (ou 'done' para sair).")

while True:
    # 1) Ler tudo o que houver
    data = ser.readline().decode('ascii', errors='ignore').strip()
    if data:
        print("Esp32:", data)

    # 2) Ver se o utilizador escreveu algo (sem bloquear tanto)
    # -> aqui ainda é bloqueante, mas só chamavas de vez em quando,
    # ou podias pôr num segundo programa/thread
    command = input(">> ").strip()
    if command == 'done':
        print('finished program')
        break
    if command:
        ser.write(command.encode('utf-8'))
```

Mas o ideal mesmo é ler e escrever em loops separados (por exemplo, duas threads: uma só a ler e outra só a lidar com `input()`).^[65]

Para percebermos bem o que queres: o teu objetivo é ter um “terminal” tipo o monitor série (ver tudo o que o ESP32 manda em tempo real) e, de vez em quando, mandar comandos, certo?

✱✱

mas eu queria uma dinamica parecida com a do arduino IDE em que posso escrever quando quiser e vai tb ao mesmo tempo (sem parar as coisas) escrevendo o que recebe do buffer

O que queres é exatamente o que o monitor série faz:

- ler continuamente da porta
- e, em paralelo, aceitar o que escreves no teclado, **sem bloquear** a leitura.

Para isso, em Python, precisas de separar leitura e escrita em “2 coisas a acontecer ao mesmo tempo”. A forma mais simples é usar **duas threads**:

- Thread A: fica sempre a ler da serial e a fazer print.
- Thread B: fica à espera de `input()` e, quando escreves algo, envia para o ESP32.

Um exemplo completo:

```
import serial
import threading

PORT = 'COM7'
BAUD = 115200

ser = serial.Serial(PORT, BAUD, timeout=0.1)

def reader():
    while True:
        data = ser.readline().decode('ascii', errors='ignore').strip()
        if data:
            print("\rEsp32:", data)
            print("> ", end="", flush=True) # repõe o prompt

def writer():
    while True:
        cmd = input("> ").strip()
        if cmd == "done":
            print("finished program")
            ser.close()
            break
        if cmd:
            ser.write(cmd.encode('utf-8'))

t_read = threading.Thread(target=reader, daemon=True)
t_read.start()
```

```
writer() # corre no main thread
```

O que este script faz:

- Assim que o ESP32 mandar qualquer coisa, aparece Esp32: ... no terminal, continuamente.
- Tu podes escrever comandos quando quiseres; cada Enter envia o comando mas a leitura nunca pára. ^[74] ^[75]

Isto aproxima-se bastante da dinâmica do monitor série: leitura contínua + envio manual quando te apetece.

Para treinares a ideia: olhando para este código, consegues dizer em qual função (reader ou writer) está o comportamento equivalente ao “lado de receção” do monitor série da IDE e em qual está o “lado de envio de texto que tu escreves”?

✱

explica-me agora o que faz o código?

Vamos linha a linha, pensando sempre em “lado que lê” e “lado que envia”.

```
import serial
import threading

PORT = 'COM7'
BAUD = 115200

ser = serial.Serial(PORT, BAUD, timeout=0.1)
```

- Importa as bibliotecas: serial (pyserial) para falar com o ESP32 e threading para correr coisas em paralelo.
- Define a porta e o baud rate.
- Abre a porta série COM7 a 115200, com timeout curto (0.1 s) para não ficar bloqueado eternamente à espera de dados.

Função reader – “monitor série” de receção

```
def reader():
    while True:
        data = ser.readline().decode('ascii', errors='ignore').strip()
        if data:
            print("\rEsp32:", data)
            print("> ", end="", flush=True) # repõe o prompt
```

- Esta função corre num loop infinito, só a ler da porta.

- `ser.readline()` tenta ler até encontrar `\n` (ou até ao timeout).
- `decode('ascii', errors='ignore')` converte bytes em string de texto.
- `strip()` tira `\r`, `\n` e espaços do início/fim.
- Se data não estiver vazia, imprime:
 - Esp32: ... → para saberes que veio da placa.
 - Depois imprime novamente `>` para repor o “prompt” onde tu escreves, porque a leitura pode ter quebrado a linha onde estavas a digitar.

Esta parte é o equivalente ao “monitor série” a mostrar tudo o que o ESP32 manda.

Função `writer` – “janela de envio”

```
def writer():
    while True:
        cmd = input("> ").strip()
        if cmd == "done":
            print("finished program")
            ser.close()
            break
        if cmd:
            ser.write(cmd.encode('utf-8'))
```

- Esta função também corre num loop infinito, mas só lida com o teclado:
 - `input("> ")` espera até tu escreveres um comando e carregares Enter.
 - `strip()` tira espaços extra.
- Se escreves done, ele:
 - imprime “finished program”,
 - fecha a porta série,
 - sai do loop → termina o programa.
- Se escreves qualquer outra coisa não vazia:
 - converte o texto para bytes (`encode('utf-8')`)
 - envia para o ESP32 com `ser.write(...)`.

Isto é o equivalente ao campo de texto do monitor série onde escreves comandos.

Lançar as duas coisas em paralelo

```
t_read = threading.Thread(target=reader, daemon=True)
t_read.start()

writer() # corre no main thread
```

- Cria uma nova “thread” (`t_read`) que vai correr a função `reader` em segundo plano.
- `daemon=True` significa que, quando o programa principal acabar, a thread de leitura também é terminada automaticamente.
- `t_read.start()` arranca a leitura em paralelo.
- A seguir, o programa chama `writer()`, que fica no “thread principal” só à espera de `input()` e a enviar comandos.

Na prática:

- A thread `reader` nunca bloqueia o teclado; está sempre a ouvir a porta série e a imprimir tudo.
- O `writer` nunca impede a leitura: mesmo enquanto estás a pensar o que escrever, se o ESP32 mandar dados, a `reader` imprime.

Em resumo numa frase:

Este script abre a porta série para o ESP32, lança uma função em paralelo que lê continuamente tudo o que vem da porta e mostra no ecrã, enquanto a função principal fica à espera de comandos que escreves no teclado e os envia para o ESP32, imitando o comportamento do monitor série da IDE.

Para ver se fez sentido: consegues apontar qual é a **vantagem prática** de usar duas threads aqui em vez de ter tudo num único `while` com `readline()` seguido de `input()`?

é possível obrigar o esp32 a dar reset quando conecto á porta serie?

é pq quando eu clico no botão de reset ele resolve um pouco aquele problema de mensagens infinitas

Sim, é possível “forçar” um reset sempre que conectas à porta série, e na verdade muitos ESP32 já fazem isso automaticamente por causa das linhas DTR/RTS da porta USB-serial. ^[76] ^[77]

1. Como o reset via serial funciona no ESP32

No ESP32, o circuito da placa liga DTR/RTS ao pino EN (reset) e ao GPIO0. Consoante a combinação de DTR/RTS:

- certas combinações põem o chip em **download mode** (para flashar),
- outras dão um **reset normal**. ^[77] ^[76]

Exemplo da documentação do S3 (simplificando):

- RTS = 1, DTR = 0 → reset
- RTS = 0, DTR = 1 → prepara download mode
- RTS = 0, DTR = 0 → limpa download mode
- RTS = 1, DTR = 1 → não faz nada especial. ^[76]

Muitos terminais série (IDE Arduino, PlatformIO, etc.) já usam isto para que, ao abrir a porta, a placa faça reset.

2. Forçar um reset logo ao abrir com Python

Se queres garantir que sempre que o script Python se liga o ESP32 faz reset “como se carregasses no botão”, podes fazer algo assim:

```
import serial
import time

ser = serial.Serial()
ser.port = 'COM7'
ser.baudrate = 115200
ser.timeout = 1

# Sequência para provocar reset:
ser.dtr = False
ser.rts = True    # combinação típica que puxa EN
ser.open()
time.sleep(0.1)
ser.rts = False   # solta o reset
time.sleep(0.1)

print("Ligado. ESP32 deve ter feito reset agora.")

# daqui para baixo lê/escreves normalmente
```

A lógica:

- Antes/depois de abrir a porta, mexes em `dtr` e `rts` para enviar o pulso de reset. ^[78] ^[79]
- Depois deixas ambos em estado estável (por exemplo, `dtr=False`, `rts=False`) para não ficar a segurar o ESP32 em reset.

O detalhe exato (se é `rts=True` ou `dtr=True`) pode variar um pouco de placa para placa, mas o padrão que mostrei é o mais comum; se não der à primeira, inverte os papéis de DTR/RTS e testas.

3. Porque isso parece “resolver” o problema das mensagens infinitas

Quando carregas no botão de reset:

- o ESP32 reinicia,
- volta ao `setup()`,
- as FSMs voltam ao estado inicial,
- qualquer estado estranho onde ele estivesse a imprimir em loop é limpo.

Se fizeres o reset por software (via DTR/RTS) ao abrir a serial, estás a reproduzir esse comportamento sem tocar no botão físico.

Depois, se ainda aparecer spam infinito, aí já sabes que o problema é mesmo da lógica do firmware (FSM presa num estado), não do “lixo deixado” por uma sessão anterior na porta.

Para consolidar: queres que te ajude a desenhar a pequena sequência de teste em Python (abrir porta → mandar reset via RTS/DTR → ler 5 segundos de boot) para veres se o teu ESP32 reage como quando carregas no botão?

✱

'''

```
import serial
import threading
```

```
PORT = 'COM7'
BAUD = 115200
```

```
ser = serial.Serial(PORT, BAUD, timeout=0.1)
```

Sequência para provocar reset:

```
ser.dtr = False
ser.rts = True # combinação típica que puxa EN
ser.open()
time.sleep(0.1)
ser.rts = False # solta o reset
time.sleep(0.1)

print("Ligado. ESP32 deve ter feito reset agora.")

def reader():
    while True:
        data = ser.readline().decode('ascii', errors='ignore').strip()
        if data:
            print("\rEsp32:", data)
            print("> ", end="", flush=True) # repõe o prompt

def writer():
    while True:
        cmd = input("> ").strip()
        if cmd == "done":
            print("finished program")
            ser.close()
            break
        if cmd:
            ser.write(cmd.encode('utf-8'))

t_read = threading.Thread(target=reader, daemon=True)
t_read.start()
```

`writer()` # corre no main thread

estás a querer dizer deixar o código assim?

Quase, mas há dois pormenores importantes:

1. Estás a fazer `ser = serial.Serial(PORT, BAUD, timeout=0.1)` e depois ainda chamas `ser.open()`. Quando fazes `Serial(...)` com os parâmetros, a porta já fica aberta, portanto `ser.open()` vai dar erro (“port already open”).^[86]
2. Para mexer em `dtr` e `rts` antes do reset, é melhor abrir a porta de forma “manual”, sem a abrir logo no construtor.

Fica assim mais seguro:

```
import serial
import threading
import time

PORT = 'COM7'
BAUD = 115200

ser = serial.Serial()          # ainda não abre
ser.port = PORT
ser.baudrate = BAUD
ser.timeout = 0.1

ser.dtr = False               # estado inicial das linhas
ser.rts = True                # puxa reset (depende da placa)

ser.open()                    # agora abre a porta
time.sleep(0.1)
ser.rts = False               # larga reset
time.sleep(0.1)

print("Ligado. ESP32 deve ter feito reset agora.")

def reader():
    while True:
        data = ser.readline().decode('ascii', errors='ignore').strip()
        if data:
            print("\rEsp32:", data)
            print("> ", end="", flush=True)

def writer():
    while True:
        cmd = input("> ").strip()
        if cmd == "done":
            print("finished program")
            ser.close()
            break
        if cmd:
            ser.write(cmd.encode('utf-8'))
```

```
t_read = threading.Thread(target=reader, daemon=True)
t_read.start()

writer()
```

- `reader()` fica sempre a mostrar tudo o que o ESP32 manda.
- `writer()` deixa-te escrever comandos quando quiseres.
- A sequência com `rts` faz um reset logo que o script se liga, parecido com carregar no botão. [\[87\]](#) [\[88\]](#)

Queres testar este código exatamente assim e dizer se, ao correr, aparece logo a sequência de boot (ESP-ROM:..., setup chamado: 1, etc.) como quando carregas no botão?

✱

só uma coisa á parte para c++ do esp32

como seria os valores de 100 e 190 para este caso?

```
#define SERVO_ANGLE_MIN 0.78539816339744830961566084581988f //45 degrees
#define SERVO_ANGLE_MAX 3.9269908169872415480783042290994f //225 degrees
```

Essas constantes estão em radianos.

Para converter de graus para radianos usas a fórmula:

$$\text{rad} = \text{graus} \times \frac{\pi}{180}$$

No teu código:

- $45^\circ \rightarrow 45 \times \pi/180 = \pi/4 \approx 0.78539816$
- $225^\circ \rightarrow 225 \times \pi/180 = 5\pi/4 \approx 3.92699082$

Agora para 100° e 190°:

1. 100 graus

$$100 \times \frac{\pi}{180} \approx 1.7453292519943295$$

```
#define SERVO_ANGLE_100 1.7453292519943295f
```

2. 190 graus

$$190 \times \frac{\pi}{180} \approx 3.3161255787892263$$

```
#define SERVO_ANGLE_190 3.3161255787892263f
```

Se quiseres, também podes escrever em forma de expressão para ficar mais legível:


```
#define SERVO_ANGLE_100 (100.0f * PI / 180.0f)
#define SERVO_ANGLE_190 (190.0f * PI / 180.0f)
```

Assim o compilador faz a conta e tu vês logo de onde vêm os valores.

Consegues, usando esta mesma fórmula, calcular aproximadamente o valor em radianos para, por exemplo, 135 graus e dizer quanto daria?

✱

vamos sair um pouco da leitura de dados e voltar aos movimentos

este é o meu código atual:

e eu estava prestes a iniciar a introdução do movimento da tosse (que é basicamente o movimento da respiração mas mais rápido) quando me apercebi que tenho que criar algumas alterações antes de me envolver 100% na introdução da tosse, basicamente a diferença que eu queria fazer é poder escolher qual o ponto a começar no movimento, isto pq.

Pq assim escolho um ponto no meio do movimento da respiração para passar a ser o meu home (basicamente o novo home seria o home normal mais o valor do ponto inicial da respiração (nos outros movimentos não se irá aplicar caso o ponto inicial seja definido como 0))

ao definir este ponto inicial o resto das coordenadas dos outros pontos seriam subtraídos às coordenadas deste ponto inicial para em vez de:

Ponto 0: x=0,y=0,z=0

Ponto1: x=2,y=3,z=3

Ponto2: x=4,y=3,z=6

Ficaria algo do género caso o ponto inicial definido seja 1.

Ponto 0: x=-2,y=-2,z=-3

Ponto1: x=0,y=0,z=0

Ponto2: x=2,y=0,z=3

Isto irá ajudar muito para caso eu queira introduzir uma variável multiplicadora para ajustar a amplitude do movimento e assim afeta para os dois lados no z, (o que na tosse é importante caso exista uma contração dos brônquios ainda maior)

isto tb serviria para não ter que alterar muita coisa no cálculo dos pontos por parte do python e tb não teria que alterar o método de armazenamento dos movimentos

por isso assim ele terá que ainda permitir fazer o ciclo completo só que desta forma só sai quando encontrar o index do ponto inicial, (isso é algo que se pode fazer para os movimentos que são caracterizados como bidirectional=true. (pois nos outros movimentos o ponto inicial e o ponto final não batem certo com isso por isso nem vale apenas introduzir esta modificação)

Ficando assim:

antes 0->1->2->3->4->3->2->1

Agora caso escolha o início como 2 deve ficar:

2->3->4->3->2->1->0->1

A tua ideia faz sentido e encaixa bem na estrutura que já tens: é basicamente fazer um “re-centramento” dos pontos em torno de um ponto escolhido e ajustar o percurso bidirecional para começar aí. ^[99]

Vou dividir em dois conceitos, para não misturar tudo:

1. Recentrar o movimento num ponto inicial escolhido

Objetivo: escolher um índice k (por exemplo 1) e fazer com que:

- esse ponto passe a ser $(0, 0, 0)$
- todos os outros pontos fiquem relativos a ele

Exemplo que deste:

- Original
 - P0: $(0, 0, 0)$
 - P1: $(2, 3, 3)$
 - P2: $(4, 3, 6)$
- Com ponto inicial = 1 (P1):
 - P0: $(-2, -3, -3)$
 - P1: $(0, 0, 0)$
 - P2: $(2, 0, 3)$

Em termos de código para um MotionStorage qualquer (por exemplo Resp_move), isto poderia ser uma função:

```
void recenterMotion(MotionStorage& move, int startIndex) {
    if (startIndex < 0 || startIndex >= move.nPoints) return;

    float baseX = move.X[startIndex];
    float baseY = move.Y[startIndex];
    float baseZ = move.Z[startIndex];

    for (int i = 0; i < move.nPoints; i++) {
        move.X[i] -= baseX;
        move.Y[i] -= baseY;
        move.Z[i] -= baseZ;
    }
}
```

- Para movimentos como a respiração, só chamas isto se for `bidirectional == true`.
- Para batimento, etc., podes deixar `startIndex = 0` (não mexe em nada). ^[99]

Depois de chamares esta função, o “home” do manipulador passa a ser o home físico + o offset do ponto inicial (porque os pontos ficaram relativos a esse ponto). É exatamente o que queres para poderes aplicar uma variável de amplitude em Z simétrica (escala positiva e negativa).

2. Ajustar o percurso bidirecional para começar no índice escolhido

Hoje, o teu comentário descreve a sequência “clássica” bidirecional:

- Antes: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ (para $N = 5$)
- Agora queres, se o início for 2, algo tipo:
 $2 \rightarrow 3 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 1$

Ou seja:

- Manténs a lógica de “vai-e-vem” (subir e descer índices),
- mas o ponto onde comesças o ciclo é outro.

No contexto da tua MotionInstance, tens:

```
Resp_inst.currentIndex  
Resp_inst.Direction // 1 ou -1  
Resp_move.bidirectional = true;
```

Uma forma simples de adaptar:

1. Definir `Resp_inst.currentIndex = startIndex` na inicialização (por ex. na `Initiate_Resp_Instance`).
2. Na FSM de movimento (`FSM_Motion_Update + Intermed_position`), quando fazes o avanço do índice, em vez de assumires que o ciclo começa sempre em 0, deves trabalhar com:
 - Índice mínimo = 0
 - Índice máximo = `nPoints - 1`
 - Mas o “primeiro” ponto visitado é o `startIndex`.

Na prática, o algoritmo de avanço bidirecional não precisa de saber o “startIndex” depois da primeira iteração: basta que na primeira chamada `currentIndex` já seja o índice de partida e `Direction = +1`. A lógica típica:

```
if (inst.Direction > 0) {  
    inst.currentIndex++;  
    if (inst.currentIndex >= inst.def->nPoints - 1) {  
        inst.currentIndex = inst.def->nPoints - 1;  
        inst.Direction = -1;  
    }  
} else {  
    inst.currentIndex--;  
    if (inst.currentIndex <= 0) {  
        inst.currentIndex = 0;  
        inst.Direction = +1;  
    }  
}
```

- Se comesçares com `currentIndex = 2` e `Direction = +1`, a sequência resultante é exatamente do tipo que descreveste, porque o algoritmo não “sabe” que o início “oficial” era

0.

- O ciclo continua completo: eventualmente passas por todos os índices, só estás a mudar o ponto onde entras no ciclo. ^[99]

3. Aplicar isto só a movimentos `bidirectional == true`

Como já tens o campo:

```
bool bidirectional;    // true = vai-e-vem, false = ciclo fechado
```

faz sentido:

- Só recenter e só usar a lógica “0..N-1..0” (com `Direction`) para `bidirectional == true`,
- Deixar movimentos unidirecionais (ou com `start/end` diferentes) como estão—neles o “`startIndex`” deve continuar a ser 0.

Para ter a certeza de que estamos alinhados: queres primeiro implementar só a parte do **recenter pontos** (subtrair as coordenadas do ponto inicial) num movimento de teste, e ver no serial se os novos pontos batem com o exemplo que deste, antes de mexer na FSM de avanço de índices?



Na prática, o algoritmo de avanço bidirecional não precisa de saber o “`startIndex`” depois da primeira iteração: basta que na primeira chamada `currentIndex` já seja o índice de partida e `Direction = +1`. A lógica típica:

```
cpp
if (inst.Direction > 0) {
    inst.currentIndex++;
    if (inst.currentIndex >= inst.def->nPoints - 1) {
        inst.currentIndex = inst.def->nPoints - 1;
        inst.Direction = -1;
    }
} else {
    inst.currentIndex--;
    if (inst.currentIndex <= 0) {
        inst.currentIndex = 0;
        inst.Direction = +1;
    }
}
```

Se começares com `currentIndex = 2` e `Direction = +1`, a sequência resultante é exatamente do

tipo que descreveste, porque o algoritmo não “sabe” que o início “oficial” era 0.

O ciclo continua completo: eventualmente passas por todos os índices, só estás a mudar o ponto onde entras no ciclo.

Sim estás a fazer isso, mas estás te a esquecer que assim terias que duplicar a FSM só para trabalhar com a respiração e não é o que eu quero.

por isso na FSM no início é que vai ser feito a copia do `current_Index=Start_Index` e em todas as estruturas de movimentos irei definir qual é o `Start_Index` (é claro só na respiração é que vou ter algo diferente de 0, nos outros terei sempre `Start_Index=0`)

A FSM têm que começar assim, quando chegar ao final da quantidade de pontos (caso `T_pause=0` não faz tempo, caso tenha executado esse tempo e passa) e inicia a contabilização para até 0, e depois têm que se começar a contagem normal até chegar ao valor antes do `Start_Index`)

a outra parte é que eu não quero que faças isto:

```
void recenterMotion(MotionStorage& move, int startIndex) {  
if (startIndex < 0 || startIndex >= move.nPoints) return;
```

```
float baseX = move.X[startIndex];  
float baseY = move.Y[startIndex];  
float baseZ = move.Z[startIndex];  
  
for (int i = 0; i < move.nPoints; i++) {  
    move.X[i] -= baseX;  
    move.Y[i] -= baseY;  
    move.Z[i] -= baseZ;  
}
```

```
}
```

eu quero que descontes os valores do movimento na `Intermed_position`

é o que faz mais sentido, não á o pq de ter uma função a mais, e não é preciso de um for que vai afetar os tempos do código

Boa, agora ficou claro o que queres:

1. Não duplicar FSM nenhuma.
2. Ter um `Start_Index` guardado no próprio `MotionStorage`.
3. Fazer o “recentro” dos pontos **on the fly** dentro da `Intermed_position`, sem alterar o array original nem percorrer com for.

Vamos pegar só nessa parte hoje (recentrar na `Intermed_position`), porque a lógica da sequência de índices é outro passo.

1. Guardar o `Start_Index` no movimento

No teu `MotionStorage` já tens:

```
struct MotionStorage {  
    int    nPoints;  
    float X[MAX_POINTS];
```

```

float Y[MAX_POINTS];
float Z[MAX_POINTS];
float period;
float T_pause;
float dtMs;
float dt_pauseMs;
bool  bidirectional;
...
};

```

Podes adicionar:

```
int  startIndex; // índice de ponto que será o "0,0,0" lógico
```

- Para batimento: `Bat_move.startIndex = 0;`
- Para respiração: `Resp_move.startIndex = 2;` (por exemplo).
- Isto podes definir na `Iniciate_Resp_Move()` / `Iniciate_Bat_Move()`.^[100]

2. Recentrar dentro da `Intermed_position`

Hoje, a tua `Intermed_position(...)` calcula `xyz.X`, `xyz.Y`, `xyz.Z` com base no ponto atual (e talvez no próximo) através de `inst.currentIndex`, interpolação, easing, etc.^[100]

Em vez de alterares `Resp_move.X[]/Y[]/Z[]`, fazes só este ajuste **no fim** da função, antes de devolver `xyz`:

```

void Intermed_position(boolean start,
                      MotionInstance& inst,
                      unsigned long nowMs,
                      Coordinate& xyz,
                      boolean trig_serial)
{
    // ... aqui vais fazer toda a lógica atual de interpolação,
    // usando inst.def->X[], Y[], Z[], currentIndex, etc.
    // No fim, xyz tem o ponto "original".

    // --- recenterar em torno do startIndex, se fizer sentido ---
    MotionStorage* def = inst.def;

    if (def->bidirectional && def->startIndex >= 0 && def->startIndex < def->nPoints)
    {
        float baseX = def->X[def->startIndex];
        float baseY = def->Y[def->startIndex];
        float baseZ = def->Z[def->startIndex];

        xyz.X -= baseX;
        xyz.Y -= baseY;
        xyz.Z -= baseZ;
    }

    // a partir daqui, quem usa xyz (fusão + IK, etc.)

```

```
// já recebe as coordenadas recentradas  
}
```

Isto cumpre o que queres:

- Zero loops adicionais: só subtrais 3 floats por chamada.
- Não mexes no array original → o Python continua a enviar/guardar os pontos como antes.
- Só movimentos bidirecional com `startIndex` válido é que são recentrados (respiração).
[100]

3. Sobre a FSM de índices e o “ciclo até 0 e volta até `Start_Index`”

Resumo do que disseste:

- FSM deve:
 - subir até `nPoints - 1`;
 - se `T_pause > 0`, aplicar pausa;
 - descer até 0;
 - depois subir “normalmente” até chegar ao índice antes do `Start_Index`, e aí o ciclo fica completo;
 - na próxima iteração, volta a começar em `Start_Index`.

Isto pode ser feito sem duplicar FSM, só com:

- `inst.currentIndex`
- `inst.Direction`
- e mais uma variável `inst.startIndex` ligada a `def->startIndex` (ou lês diretamente de `def`).

Mas para não misturar muitos conceitos num só passo, proponho:

1. Primeiro, implementas o **recentro** na `Intermed_position` como acima e testas:

- escolhes um `startIndex` para `Resp_move`;
- verificas via `debugPrintMove` ou olhar no terminal se o ponto “home” ficou onde queres (visualmente no robô).

2. Depois pegamos só na função dentro da tua FSM que incrementa/decrementa `currentIndex` e desenhamos juntos a lógica de “ir até fim, voltar até 0, subir até antes de `startIndex`”.

Para já, focando no recentro: olhando para a tua `Intermed_position`, consegues identificar em que linha (ou logo antes de qual `return`) seria mais natural encaixar esse bloco que subtrai `baseX`, `baseY`, `baseZ` de `xyz`?

✱

struct MotionStorage {

```
int  n_Points;           // nº de pontos válidos em X/Y/Z/easing
float X[MAX_POINTS];     // coordenadas X dos pontos do movimento
float Y[MAX_POINTS];     // coordenadas Y dos pontos do movimento
float Z[MAX_POINTS];     // coordenadas Z dos pontos do movimento
float period;
float T_pause;
float dtMs;              // duração do ciclo em segundos
float dt_pauseMs;        // duração da pausa entre ciclos em segundos
int  startIndex; // índice de ponto que será o "0,0,0" lógico
bool bidirectional; // true = vai-e-vem, false = ciclo fechado
int  N_pontos ; // nº de segmentos por ciclo
```

```
float getDtMs() const {
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_pontos = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinhas
            N_pontos = n_Points;
        }
        return (period * 1000.0f) / (float)N_pontos;
    }
    return 0.0f;
}
```

```
void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}
```

eu passei o N_points para fora da função para declarar na structure, mas está a dar-me erro

O erro aí vem de dois detalhes em C++:

1. Estás a tentar escrever numa variável **não** mutable dentro de um método marcado como `const`.
2. O comentário em `getDtMs()` continua a falar de “retornar segundos”, mas tu já estás a devolver milissegundos (só atenção semântica).

Olha para a tua struct:

```
struct MotionStorage {
    int  n_Points;
    ...
    int  N_pontos; // nº de segmentos por ciclo

    float getDtMs() const {
```



```

        if (n_Points > 1 && period > 0.0f) {
            if (bidirectional==true) {
                N_pontos = (n_Points * 2) - 2;    // <-- ERRO
            } else {
                N_pontos = n_Points;              // <-- ERRO
            }
            return (period * 1000.0f) / (float)N_pontos;
        }
        return 0.0f;
    }
};

```

O método `getDtMs()` está marcado como `const`, ou seja, **promete não alterar nenhum membro da struct**.

Mas tu estás a modificar `N_pontos` lá dentro, por isso o compilador reclama.

Tens três opções corretas; escolhe a que fizer mais sentido para ti.

Opção 1: Não guardar `N_pontos`, só calcular localmente

Manténs `getDtMs()` como `const` e não guardas `N_pontos` na struct:

```

float getDtMs() const {
    if (n_Points > 1 && period > 0.0f) {
        int N_pontos;
        if (bidirectional) {
            N_pontos = (n_Points * 2) - 2;
        } else {
            N_pontos = n_Points;
        }
        return (period * 1000.0f) / (float)N_pontos;
    }
    return 0.0f;
}

```

E removes o campo `int N_pontos`; da struct se não precisares dele fora desta conta.

É a solução mais simples e limpa.

Opção 2: Tirar o `const` do `getDtMs()`

Se quiseres mesmo manter `N_pontos` como membro “de estado”:

```

float getDtMs() {
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional) {
            N_pontos = (n_Points * 2) - 2;
        } else {
            N_pontos = n_Points;
        }
        return (period * 1000.0f) / (float)N_pontos;
    }
}

```

```
    }  
    return 0.0f;  
}
```

E ajustas a assinatura no header e nas chamadas.

Assim já podes escrever em `N_pontos` porque o método deixou de ser `const`.

Opção 3 (menos necessária aqui): `mutable`

Poderias declarar `N_pontos` como `mutable int N_pontos;`, o que permite modificar essa variável mesmo em métodos `const`.^[101]

Mas aqui não há grande vantagem; é um cálculo simples e a Opção 1 é mais direta.

Para continuar alinhado com o estilo que já tens (e evitar side-effects estranhos), eu sugiro a **Opção 1**: calcula `N_pontos` dentro da função como variável local e não o guardes na struct, a menos que tenhas mesmo outra parte do código a precisar dele.

Qual das três abordagens te parece mais natural para manter o código legível dentro da tua FSM de movimento?

✱

eu tentei pensar numa restrouturação da FSM do movimento, para incluir aquilo que tínhamos falado de ele permitir escolher um valor diferente de 0 para começar.

diz-me o que achas disto?

Dá para ver bem a ideia geral do diagrama, e a direção está boa: queres uma FSM única que trate dos dois casos (bidirecional/unidirecional) e que saiba começar num `startIndex` qualquer, fazendo o ciclo completo antes de voltar a esse ponto.

Em vez de tentar comentar cada setinha do desenho (está denso e é fácil perder-se), vale a pena simplificar mentalmente a FSM em 3 fases lógicas que encaixam no que desenhaste:

1. Fase subida até ao fim

- Começas em `i = startIndex`, `dir = +1`.
- Enquanto `i < nPoints-1`, fazes `i++`.
- Quando chegas a `i == nPoints-1`:
 - se `bidirectional == false` → acabou o ciclo, voltas a `startIndex`.
 - se `bidirectional == true` → passas para a fase 2 (descida).

2. Fase descida até 0 (só se `bidirectional`)

- Começas em `i = nPoints-1`, `dir = -1`.

- Enquanto `i > 0`, fazes `i--`.
- Quando chegas a `i == 0`:
 - se `startIndex == 0` → acabou, voltas à fase 1 normalmente.
 - se `startIndex > 0` → passas para a fase 3 (subida até `startIndex`).

3. Fase subida até `startIndex` (fechar o ciclo completo)

- Começas em `i = 0`, `dir = +1`.
- Enquanto `i < startIndex`, fazes `i++`.
- Quando chegas a `i == startIndex`, terminaste o “loop” completo:
 - próxima volta recomeça na fase 1 (de novo `startIndex` → `nPoints-1` → ...).

A FSM do teu desenho está essencialmente a tentar implementar isto com estados separados para “subida”, “descida”, “pausa”, etc., o que é coerente com o estilo que tens no código. ^[102]

O que eu faria para deixar isto claro no código (e simples de manter) é:

- Guardar no `MotionInstance` um campo que representa a fase (por exemplo `phase = 0,1,2` em vez de espalhar tudo por muitos estados da mesma FSM).
- Dentro de cada fase usas sempre o mesmo “bloco” de incremento/decremento de `currentIndex`, em vez de duplicar lógica.

Algo tipo (pseudo-C++ simplificado):

```
// dentro da FSM_Motion_Update, depois de tratar dt, pausas, etc.

if (!inst.def->bidirectional) {
    // caso simples: só andar 0..nPoints-1..0 como já tinhas
} else {
    switch (inst.phase) {
        case 0: // fase 1: startIndex -> nPoints-1
            if (inst.currentIndex < inst.def->n_Points - 1) {
                inst.currentIndex++;
            } else {
                // chegou ao fim
                if (inst.def->T_pause > 0) { /* entra em pausa */ }
                inst.phase = 1;
            }
            break;

        case 1: // fase 2: nPoints-1 -> 0
            if (inst.currentIndex > 0) {
                inst.currentIndex--;
            } else {
                inst.phase = (inst.def->startIndex > 0) ? 2 : 0;
            }
            break;

        case 2: // fase 3: 0 -> startIndex
            if (inst.currentIndex < inst.def->startIndex) {
                inst.currentIndex++;
            } else {
```

```

        inst.phase = 0; // ciclo completo, volta à fase 0
    }
    break;
}
}

```

Isto corresponde ao que desenhaste, mas organiza a lógica em “3 blocos” bem separados, o que ajuda muito a não te perderes.

Em relação ao teu diagrama: a estrutura está na boa direção, só está bastante carregada; se o transformares mentalmente nestas 3 fases, provavelmente ficas com menos transições e fica mais fácil garantir que:

- todos os caminhos passam por todos os pontos uma vez por ciclo,
- só os movimentos com `bidirectional == true` fazem estas 3 fases,
- `startIndex = 0` automaticamente simplifica (a fase 3 fica vazia).

Para avançarmos: queres tentar escrever tu mesmo, em pseudocódigo ou C++ simplificado, só a parte da FSM que decide `currentIndex` e `phase` para o caso `bidirectional == true`, usando estas 3 fases, e eu depois ajudo a encaixar isso dentro da tua `FSM_Motion_Update` real?

✱✱

ok isto está a complicar demasiado esta parte,

vamos deixar de lado e passar para outra coisa mais fácil,

vamos voltar para a parte de comunicação de caracteres extensos

a primeira parte é para aquelas variáveis em que só têm um valor float, deixamos os arrays para a seguir, lembras-te ainda do que tínhamos acordado?

Lembro sim, a ideia base era:

- Continuar a tua leitura carácter-a-carácter (sem `String`, sem `readString()`).
- Ter um comando “especial” para **alterar variáveis float simples** (tipo `period` e `T_pause`) sem parar o movimento.
- Mensagens vindas do PC em formato humano, algo como:
 - `W P [0.3]\n` → `write Period = 0.3`
 - `W T [1.2]\n` → `write T_pause = 1.2`

E o plano era:

1. A FSM de leitura deteta o `W` e passa a ler **modo string** até ao `\n`, acumulando num buffer (`st_buf`), ignorando espaços como já fazes.
2. No fim da linha (`\n`), ficas com algo como:
 - `"P[0.3]"` ou `"T[1.2]"` no `st_buf`.
3. Uma função / ramo da FSM de comandos interpreta:

- `st_buf[^21_0] → 'P' ou 'T'` (qual variável alterar).
- Procura '[' e ']', copia o que está lá dentro para `num_buf`.
- Converte com `atof(num_buf) → float value`.
- Atualiza `Resp_move.period`, `Resp_move.T_pause`, etc., e chama `updateDtMs()` / `updateDt_PauseMs()` se for preciso. ^[104]

Tudo isto **sem mudar o formato dos arrays** nem o protocolo normal de comandos de uma letra.

Se estiveres de acordo, o próximo passo podia ser desenhar só a parte da FSM de leitura para a string simples:

- Estado 0: idle no modo normal.
- Quando vê `W`, passa para o estado “ler string”, começa a encher `st_buf`.
- Lê até `\n`, termina a string, levanta uma flag tipo `StringReady = true`, volta ao modo normal.

Queres que te ajude a escrever essa mini-FSM da leitura (os estados e as transições) ou preferes começar pela função que interpreta o `st_buf (P[ ... ] / T[ ... ])` e atualiza o `Resp_move`?

**

estas são as modificações para as FSM's que fiz até agora, falta apenas incorporar a parte da interpretação da string e procura de [], bem como da procura do caracter ';', para se saber se estão a ser enviados mais do que 1 valores para a variavel

tu consegues ver as fotos que enviei?

Sim, dá para ver bem as duas imagens das FSMs, tanto a do `FSM_Serial_Command` como a do `FSM_Serial_Reader` com o novo modo múltiplo.

Pelo que se percebe:

- No **Reader**, já tens:
 - modo `S` (single char) e `M` (multiple/string),
 - um buffer `ST_Bus[ ]` onde vais acumulando a linha até `\n`,
 - um estado que copia `ST_Bus[^22_0]` para `CH_Command`, e outro que marca `LineReady = true` quando a string fica pronta.
- No **Command**, já existe o ramo para `CH_Command_IN == 'W'` que entra na parte nova de configuração (provavelmente onde vais interpretar a string e atualizar `period`, `T_pause`, etc.).

Falta agora encaixar duas coisas na interpretação da string:

1. Procura de [e] para extrair cada valor float

- Dentro da FSM/estado que trata o comando 'W', pegas no buffer ST_Command ou equivalente.
- Percorres até encontrares '['.
- Copias tudo até ']' para um num_buf e chamas atof(num_buf) para obter o float.
- Se houver mais de um par [...], repetes o processo.

2. Uso do ';' como separador de múltiplos valores

- Podes tratar ';' como “acabei este valor, começo outro”:
 - Por exemplo: W P[0.3;0.6]\n → dois valores para a mesma variável, ou
 - W P[0.3];T[0.6]\n → dois campos diferentes, que vais tratar um a seguir ao outro.
- Na prática, na tua FSM, cada vez que encontrares ';', decides se:
 - acabaste de preencher um valor e vais gravá-lo numa variável,
 - ou vais saltar para outro sub-comando (por exemplo trocar de 'P' para 'T').

Como queres evitar for grandes e mexer no mínimo possível, a forma mais “FSM-friendly” é:

- Na leitura: só encher ST_Bus[] até ao \n.
- Na interpretação (novo ramo da FSM_Serial_Command para 'W'):
 - Estado A: lê o primeiro carácter (ex.: 'P' ou 'T') para saber o alvo.
 - Estado B: procura o próximo '['.
 - Estado C: copia caracteres para num_buf até ']' ou ';'.
 - Quando fecha um valor:
 - converte para float,
 - atualiza a variável correta,
 - se for ';' volta a B (procura próximo '['),
 - se for '\0' (fim da string), termina.

Se quiseses, no próximo passo podemos escrever juntos o pseudo-código dessa parte da FSM de 'W' (apenas para um caso simples como W P[0.3]\n), e depois estendes para suportar ';'.

Que formato de mensagem queres garantir primeiro: só um valor por comando (W P[0.3]\n), ou já queres logo suportar múltiplos valores com ';' tipo W P[0.3;0.5]\n?

diz me lá uma coisa, eu nesta parte vou guardar valores novos nas strings de movimentos

Storage_Motion

e de momento no código tenho 3 objetos associados a estes, mas mais tarde vou ter quatro ou mais, e para na FSM não ter que estar a adicionar e duplicar estados para estes casos, existe alguma forma de caso seja uma determinada letra ele associa logo ao objeto e assim as ações torna-se iguais para todos?

tipo nas funções faço algo parecido mas eu associo uma variável interna a qualquer objeto que defina a entrada e eu quero saber se é possível fazer algo parecido mas sem ser numa função

Dá para evitar duplicar estados, sim. A ideia é a mesma que já usas nas funções: **escolher um ponteiro para o movimento certo** e, a partir daí, trabalhar sempre através desse ponteiro. Fazes essa escolha numa pequena parte da FSM, e o resto dos estados fica igual para todos os movimentos.

1. Ter um “ponteiro corrente” para o movimento

Tens vários MotionStorage:

```
MotionStorage Bat_move;
MotionStorage Resp_move;
// mais tarde: Tosse_move, etc.
```

Declara uma variável global (ou dentro da FSM) que aponte para o movimento “ativo”:

```
MotionStorage* currentMove = nullptr;
```

2. Escolher o movimento com base numa letra

Na FSM_Serial_Command, quando recibes a letra que identifica o movimento (por exemplo 'b' para batimento, 'r' para respiração, 't' para tosse), fazes só isto:

```
void selectCurrentMove(char id) {
    switch (id) {
        case 'b':
        case 'B':
            currentMove = &Bat_move;
            break;

        case 'r':
        case 'R':
            currentMove = &Resp_move;
            break;

        case 't':
        case 'T':
            currentMove = &Tosse_move;
    }
}
```

```

        break;

    default:
        currentMove = nullptr;
        break;
    }
}

```

Na FSM:

```

// exemplo num estado qualquer
case ESTADO_ESCOLHE_MOV:
    selectCurrentMove(CH_Command_IN); // 'b', 'r', 't', ...
    if (currentMove != nullptr) {
        // passa para o estado genérico que trata a string / escreve valores
        state = ESTADO_TRATA_STRING;
    } else {
        // comando inválido -> volta ao menu
        state = ESTADO_MENU;
    }
    break;

```

A partir daqui, **tudo o que fizeres em ESTADO_TRATA_STRING usa** `currentMove`, e já não te interessa se é Bat, Resp ou Tosse:

```

case ESTADO_TRATA_STRING:
    // por exemplo, alterar period
    currentMove->period = novoPeriod;
    currentMove->updateDtMs();
    ...
    break;

```

Assim, quando adicionares o 4.º movimento, só precisas de:

- criar o novo `MotionStorage` (ex.: `Cough_move`),
- acrescentar uma linha no `selectCurrentMove`,
- não mexes em nenhum estado da FSM que trata a string, porque ele já trabalha sempre via `currentMove`.

3. “Sem ser numa função”

Se quiseres mesmo evitar uma função separada, podes pôr o `switch` diretamente num estado da FSM:

```

case ESTADO_ESCOLHE_MOV:
    switch (CH_Command_IN) {
        case 'b': case 'B': currentMove = &Bat_move; break;
        case 'r': case 'R': currentMove = &Resp_move; break;
        case 't': case 'T': currentMove = &Tosse_move; break;
        default: currentMove = nullptr; break;
    }

```



```

    }

    if (currentMove != nullptr) state = ESTADO_TRATA_STRING;
    else                       state = ESTADO_MENU;
    break;

```

Mas conceitualmente é a mesma coisa: um **ponteiro** que guarda “qual dos 4 objetos” estás a mexer.

Consegues imaginar em que estado da tua FSM de comandos (naquele diagrama) seria mais natural meter essa escolha do `currentMove` com base na letra que vem depois de `W`?

ent assim crio um objeto a mais que é nulo

tanto para o `MotionStorage` como `MotionInstance`
e depois faço isso do ponteiro dentro da FSM certo?

Ent assim pega no meu código atual:

e na estrutura da FSM de `Serial` comand e tenta implementar isso depois do estado 26,
a passagem do estado 24 é para ler se foi introduzido 'd' ou 'D'
caso seja 'd', ent é uma string de um unico valor e por isso não é preciso passar `Start` para 0
(como no estado 3), caso seja 'D' ele têm que parar os ciclos pois é para o envio de coordenadas
tb decidi que para os casos de strings grandes ('D')
os valores para as variaveis vão vir sempre dentro de [] e não vou colocar pelo meio ';'
por isso será algo do tipo "M D B x[4][0.1][0.2][0.5][09][1.2]"
o primeiro valor de todos é o numero de pontos

Sim, dá para ver o teu diagrama (FSM_Serial_Command com estados até 24/25) e o ficheiro atualizado. ^[107]

Vamos organizar o que queres fazer a seguir, já com o esquema do ponteiro e o d/D.

1. Estrutura base: ponteiros para `MotionStorage/Instance`

No topo do ficheiro (zona global) podes declarar:

```

MotionStorage* currentMove = nullptr;
MotionInstance* currentInst = nullptr;

```

E um pequeno “selector” que vais usar na FSM, com base na letra depois de `W`:

```

void selectCurrentMove(char id) {
    switch (id) {
        case 'B': case 'b':
            currentMove = &Bat_move;
            currentInst = &Bat_inst;
            break;

        case 'R': case 'r':
            currentMove = &Resp_move;

```

```

        currentInst = &Resp_inst;
        break;

// mais tarde: tosse, etc.
// case 'T': case 't': currentMove = &Tosse_move; currentInst = &Tosse_inst;

default:
    currentMove = nullptr;
    currentInst = nullptr;
    break;
    }
}

```

Isto permite que todos os estados seguintes tratem *qualquer* movimento, sem duplicar lógica.

2. Depois do estado 24: distinguir d vs D e escolher o movimento

No teu desenho, o estado 23 parece ser onde a FSM entra no “modo múltiplo” (string pronta), e 24 lê o primeiro caractere da string (ST_Command[^24_0]) para ver se é d ou D.

Numa versão em C, poderia ficar assim:

```

case 23:
    // aqui assumo que ST_Command[] já tem a string completa
    // e que m = 0 é o índice atual desse buffer
    m = 0;
    LineReady_Out = true;    // já tinhas isso no desenho
    state = 24;
    break;

case 24:
    LineReady_Out = false;
    char modeChar = ST_Command[m];

    if (modeChar == 'd' || modeChar == 'D') {
        // avançar para ler qual movimento (B, R, ...)
        m++;
        char moveId = ST_Command[m];

        selectCurrentMove(moveId);    // define currentMove/currentInst

        if (currentMove == nullptr) {
            // comando inválido, voltar ao menu
            state = 4;    // ou outro estado de idle/menu
        } else {
            if (modeChar == 'd') {
                // string de um único valor -> não parar ciclos
                state = 25;    // estado que trata 1 só valor (period/T_pause, etc.)
            } else { // 'D'
                // string grande -> parar movimentos enquanto atualiza arrays
                Bat_inst.active = false;
                Resp_inst.active = false;
                //Tosse_inst.active = false;    // quando existir
                state = 26;    // estado que trata coordenadas grandes
            }
        }
    }
}

```

```

    }
}
} else {
    // se ST_Command[^24_0] não for 'd' nem 'D', talvez voltar ao menu
    state = 4;
}
break;

```

Até aqui:

- d → um único valor (por exemplo, alterar só period ou T_pause)
- D → string grande com muitos valores, é aqui que vais ler "M D B x[^24_4][0.1][0.2]..."
- selectCurrentMove decide se estás a mexer em Bat_move, Resp_move, etc., sem duplicar estados.

3. Formato da string grande "M D B x[^24_4][0.1][0.2][0.5][0.9][1.2]"

Pelo que disseste:

- Tudo vem sempre dentro de [],
- sem ' ; ' pelo meio.
- Primeiro [] é o número de pontos, os seguintes são os valores (por ex. X0, X1, X2, ...).

Então, no estado 26 (que ainda vais desenhar), a FSM pode:

1. Procurar o primeiro '['.
2. Ler os caracteres até ']' para um numBuf[].
3. `int numPoints = atoi(numBuf);` → gravar em `currentMove->nPoints`.
4. Repetir para os próximos [], cada vez que encontra mais um par, faz atof e grava no array certo (`currentMove->X[i]`, Y, Z, etc.).

O importante para já é: tudo isto acontece relativo a `currentMove`, por isso a mesma lógica serve para Bat, Resp ou Tosse; a única diferença é a letra que envias logo depois do D (B/R/...).

Se quiseses, no próximo passo podemos escrever juntos o pseudo-código do estado 26 só para um caso muito simples, por exemplo:

- mensagem "M D B x[^24_3][1.0][2.0][3.0]"
- estado 26 lê o 3 e os três valores, e preenche `currentMove->X[0..2]`.

Antes disso, faz sentido para ti onde encaixa o `selectCurrentMove` (logo em 24, depois de d/D) e como isso evita criares versões separadas de 25/26 para Bat e Resp?

✱✱

mostra-me um exemplo de como utilizar o atoi, como fazer a copia de uma string de um lado para o outro de forma completa e como limpar uma string

Vamos fazer isto prático, com exemplos pequenos em C++ (Arduino/ESP32).

1. Exemplo de atoi (string → int)

atoi converte uma *C-string* (array de char terminado com '\0') num int.

```
char numStr1[] = "123";
char numStr2[] = "-45";

int n1 = atoi(numStr1); // n1 = 123
int n2 = atoi(numStr2); // n2 = -45
```

Num caso como o teu, em que lêes algo entre [e]:

```
char buf[8]; // ex: "[4]" -> "4"

// imaginando que já copiaste "4" para buf e puseste '\0' no fim:
buf[0] = '4';
buf[1] = '\0';

int numPoints = atoi(buf); // numPoints = 4
```

2. Copiar uma string C de um lado para o outro

Para copiar a string completa, usas strcpy (copiar tudo) ou strncpy (copiar no máximo N chars).

```
char src[20] = "Hello";
char dst[20];

// copia tudo até ao '\0'
strcpy(dst, src); // dst fica "Hello"
```

Com strncpy (mais seguro):

```
char src[20] = "Hello";
char dst[20];

strncpy(dst, src, sizeof(dst));
dst[sizeof(dst)-1] = '\0'; // garantir terminação
```

Se quiseres copiar só uma parte (por exemplo, o que está entre [e]):

```

char line[] = "x[4][1.0]";
char numBuf[8];
int i = 0;
int j = 0;

// supõe que i aponta para o primeiro char depois de '['
i = 2; // '4' em "x[4][1.0]"

while (line[i] != ']' && line[i] != '\0' && j < (int)sizeof(numBuf)-1) {
    numBuf[j++] = line[i++];
}
numBuf[j] = '\0'; // termina a string

int n = atoi(numBuf); // n = 4

```

3. “Limpar” uma string (buffer de chars)

Depende do que queres dizer com “limpar”:

a) Marcar como vazia (mais comum)

Só precisas pôr o primeiro char a '\0':

```

char buf[32];

// depois de usares:
buf[0] = '\0'; // agora é uma string vazia

```

A maior parte das funções (strcmp, strlen, etc.) vão tratar isto como string vazia, sem precisares apagar o resto.

b) Pôr tudo a zero

Se quiseses mesmo limpar todas as posições:

```

char buf[32];
memset(buf, 0, sizeof(buf)); // todos os bytes a 0

```

Juntando tudo: mini-exemplo típico no teu contexto

Supondo que tens uma linha como "x[4]" no ST_Command e queres extrair o número entre []:

```

char ST_Command[32] = "x[4]";
char numBuf[8];

int i = 0;

// avançar até ao '['

```

```

while (ST_Command[i] != '[' && ST_Command[i] != '\0') {
    i++;
}

// se encontrou '['
if (ST_Command[i] == '[') {
    i++; // salta o '['

    int j = 0;
    while (ST_Command[i] != ']' &&
           ST_Command[i] != '\0' &&
           j < (int)sizeof(numBuf)-1) {

        numBuf[j++] = ST_Command[i++];
    }
    numBuf[j] = '\0'; // termina

    int numPoints = atoi(numBuf); // converte "4" -> 4

    // depois podes "limpar" o numBuf para próxima utilização
    numBuf[0] = '\0';
}

```

Assim vês, num exemplo concreto, como atoi, cópia e “limpeza” trabalham juntos.

Consegues adaptar este último exemplo para o teu caso de "M D B x[4][0.1]...", extraíndo primeiro o 4, e a seguir o primeiro valor 0.1 para um float usando atof em vez de atoi?

na mensagem antes desta, estás a fazer coisas desnecessárias e estás tb a falhar na construção de uma FSM

eu não preciso de uma função á parte para ver se é B, R ou T para atribuir o ponteiro.
tens ai a imagem da atualização que fiz no FSM Serial Comand, como é que eu faço a procura do [] e copia do valor para a string para fazer o atoi?
e outra coisa, o atoi transformam em inteiro ou em float?

Sim, dá para ver o novo pedaço da FSM (23–31) na imagem.

Responder às duas perguntas:

1. atoi dá inteiro ou float?

- atoi → converte string para int.
- Para float usas atof.

Portanto:

```
int n = atoi("4"); // n = 4
```

```
float f = atof("0.3"); // f = 0.3f
```

Na tua mensagem "M D B x[^26_4][0.1][0.2]...":

- o primeiro [] (4) → atoi → int nPoints;
- os restantes [] (0.1, 0.2, ...) → atof → float.

2. Como fazer a FSM procurar [] e copiar para usar atoi

Vou adaptar ao que tens no diagrama:

- ST_Command[] é a string inteira.
- m é o índice atual nessa string.
- Estás em state 26 (ou 27/31) depois de já teres identificado d/D e a letra do movimento.

Aqui está um esqueleto de FSM em C, focado apenas em encontrar o primeiro [] e converter o valor:

```
// buffers globais ou estáticos
char numBuf[^26_16]; // para guardar o que está entre [ ]
int m = 0;           // índice na ST_Command

case 26: // procurar '['
{
    // andar até encontrar '[' ou fim
    while (ST_Command[m] != '[' && ST_Command[m] != '\0') {
        m++;
    }

    if (ST_Command[m] == '[') {
        m++; // passa para o primeiro char dentro dos [ ]
        // próximo estado: copiar o valor
        state = 27;
    } else {
        // não encontrou '[' -> erro, volta ao menu ou termina
        state = 4;
    }
}
break;

case 27: // copiar chars até ']'
{
    int j = 0;

    // copiar todo o conteúdo entre '[' e ']'
    while (ST_Command[m] != ']' &&
           ST_Command[m] != '\0' &&
           j < (int)sizeof(numBuf) - 1) {

        numBuf[j++] = ST_Command[m++];
    }
}
```

```

numBuf[j] = '\0'; // termina a string

if (ST_Command[m] == ']') {
    m++; // salta o ']'
}

// agora numBuf tem, por ex, "4" ou "0.3"
// exemplo: primeiro [] é nº de pontos
int nPoints = atoi(numBuf); // aqui é int
// ou, para floats:
// float value = atof(numBuf);

// limpar buffer para próxima utilização
numBuf[^26_0] = '\0';

// daqui decides o próximo estado:
// - se ainda há mais [], volta ao 26
// - se terminaste, volta ao menu
state = 26; // por exemplo, para procurar o próximo []
}
break;

```

Isto é “modo FSM” puro:

- state 26 faz a procura de '[' avançando m.
- state 27 copia o conteúdo entre '[' e ']' para numBuf e usa atoi/atof.
- Não usas for, não chamas função externa; só usas while internos dentro do próprio estado, que é compatível com o estilo que já desenhaste.

Se quiseres, o passo seguinte pode ser: decides que em state 27 o primeiro valor (do primeiro []) vai para currentMove->n_Points, e o segundo valor para currentMove->X[^26_0], e tentamos encaixar isso com o m++ e o startIndex que já tens.

Para fechar: em qual dos teus estados (26, 27 ou 31) queres que aconteça esta parte de “copiar até] e fazer atoi/atof”?



depois de fazer atof ou atoi para uma variavel, a string que transformei ainda guarda os valores? ou ela fica limpa?

Nem atoi nem atof alteram a string que lhes passas.

- Ambos recebem um `const char*`, ou seja, um ponteiro para uma string que não é modificada pela função. [\[110\]](#) [\[111\]](#) [\[112\]](#)
- Eles só lêem os caracteres, calculam o número e devolvem esse número como `int` (atoi) ou `double` (atof). [\[113\]](#) [\[112\]](#)

Portanto:


```
char buf[] = "123";
int n = atoi(buf);
// aqui buf continua igual: "123"
```

ou:

```
char buf[] = "0.75";
float f = atof(buf);
// buf continua "0.75"
```

Se quiseres “limpar” o buffer depois, és tu que tens de o fazer:

```
buf[^27_0] = '\0';          // marca a string como vazia
// ou
memset(buf, 0, sizeof(buf)); // zera tudo
```

Ou seja: podes usar a mesma string várias vezes (por exemplo: parseias com `atoi`, depois parseias outro valor com `atof`), e só a limpaste se explicitamente escreveres por cima.

Para garantir que ficou claro: quando pensas na tua FSM, em que momento faz sentido tu próprio limpares o `numBuf` (por ex. pôr `numBuf[^27_0] = '\0'`) depois de fazeres o `atoi/atof`?

✱✱

Têns aqui a estrutura, (ainda é só para uma variável, mas já dá para ter noção das coisas)

o que achas?

A estrutura está bem encaminhada: dá para ver claramente três blocos lógicos:

- 23–26: escolha de `d/D` e avanço do índice `m`.
- 27–29/30: seleção do movimento (`Resp`, `Bat`, `Tosse`) e associação aos ponteiros.
- 31–36: procura de `[` e cópia do valor para `ST_copy[ ]`, depois conversão e escrita na variável (`Period_currentMove`, etc.).

Do ponto de vista de FSM:

- Tens:
 - um estado onde decides se é `d` ou `D` (bom, separa `single` vs `múltiplo`)
 - estados 28/29/30 a fazer o “`currentMove = &Resp/Bat/Tosse`” (sem função externa)
 - um laço 31→32→35→36 que implementa a lógica de “andar até `[`”, copiar para outro buffer e usar depois `atof/atoi`.

O que afinava ligeiramente, para ficar mais sólido:

1. No estado 31/32, garante que não entras em loop infinito se não houver `[`:

- No 31: `m++`
- No 32: se `ST_Command[m] != '['` e `ST_Command[m] != '\0' → volta ao 31`
- Se `ST_Command[m] == '[' → m++` e vais ao 35
- Se `ST_Command[m] == '\0' → aborta e volta ao estado 4 (menu), por exemplo.`

2. No estado 35, quando copias para `ST_copy`:

- Inicializa o índice `c = 0` antes de começar a copiar
- Enquanto `ST_Command[m] != ']'` && `ST_Command[m] != '\0':`
 - `ST_copy[c++] = ST_Command[m++]`
- Depois `ST_copy[c] = '\0'`
- Se `ST_Command[m] == ']' → m++` e passas ao 36.

3. No estado 36:

- Usas `atoi` se for o número de pontos (primeiro `[]`), ou `atof` se for um float
- Guardas no campo certo de `*currentMove`
- `LineReady_Out = true`, e decides se voltas ao 31 (para procurar mais `[]`) ou se saltas para o estado 4.

Sintetizando: a FSM que desenhaste já captura bem o fluxo “escolher movimento → percorrer a string até encontrar `[]` → copiar o conteúdo → converter → escrever na variável”, que é exatamente o comportamento que querias.

Para consolidar: qual é o **primeiro valor** que queres mesmo implementar com esta FSM (por exemplo, `Resp_move.period` ou `Resp_move.T_pause`), para testarmos com uma string concreta tipo `M d R p[4.0]`?

**

char src[20] = "Hello";

`char dst[20];`

`strncpy(dst, src, sizeof(dst));`

`dst[sizeof(dst)-1] = '\0'; // garantir terminação`

Isto não está ao contrario?

Não, aqui a ordem está correta.

Em `strncpy(dst, src, sizeof(dst));`:

- `dst` é o destino (onde vais copiar).
- `src` é a origem (de onde copias).
- `sizeof(dst)` é o número máximo de caracteres a copiar.

A assinatura da função é:

```
char *strncpy(char *dest, const char *src, size_t n);
```

Ou seja: primeiro vem o destino, depois a origem.

`Odst[sizeof(dst)-1] = '\0'`; é só uma proteção extra: se `src` tiver 20 ou mais caracteres, `strncpy` pode não colocar `'\0'` automaticamente; ao escreveres `'\0'` na última posição tens a certeza que a string termina.

Só para confirmar: queres usar `strncpy` neste estilo na tua FSM para copiar o que está entre [e] para um buffer temporário?

ent pq que me está a dar erro nesta parte:

case 8:

```
strncpy(ST_BUS, ST_Command_Out, sizeof(ST_BUS));  
ST_Command_Out[sizeof(ST_Command_Out)-1] = '\0';
```

```
break;
```

Há duas coisas aí que muito provavelmente estão a causar o erro:

1. Tipos diferentes (array vs char simples)

2. Tamanho do buffer com `sizeof`

Pelo nome das variáveis no teu código:

- `ST_BUS` parece ser um array de chars (string/buffer).
- `ST_Command_Out` parece ser um char simples (porque em vários estados fazes `CH_Command_Out = '\0'`; e usas `CH_Command_OUT` como comando único).^[121]

Se for isso, então esta linha:

```
strncpy(ST_BUS, ST_Command_Out, sizeof(ST_BUS));
```

está errada porque o segundo argumento de `strncpy` deve ser `const char*` (uma string), não um char isolado.^[122]

Ou seja, é como tentar copiar “uma frase inteira” a partir de uma única letra.

O que podes verificar:

1. Declarações

Confirma no teu código:

```
char ST_BUS[ALGUM_TAMANHO];  
char ST_Command_Out[ALGUM_TAMANHO];
```

```
// OU
char ST_Command_Out; // só um char
```

- Se ST_Command_Out for **só um char**, não podes usar strncpy diretamente.
- Se queres copiar uma string completa, tens de garantir que ST_Command_Out é também um array (char ST_Command_Out[^30_32]; por exemplo).

2. Se a ideia é copiar um comando de um char só

Se ST_Command_Out é apenas um comando (uma letra), então para guardar isso em ST_BUS:

```
ST_BUS[^30_0] = ST_Command_Out;
ST_BUS[^30_1] = '\0';
```

Nem precisas de strncpy.

3. Se queres copiar uma string inteira

Então ambas têm de ser arrays:

```
char ST_BUS[^30_32];
char ST_Command_Out[^30_32];

// mais tarde
strncpy(ST_BUS, ST_Command_Out, sizeof(ST_BUS));
ST_BUS[sizeof(ST_BUS)-1] = '\0';
```

Repara que o '\0' deve ser posto no **destino** (ST_BUS), não em ST_Command_Out, a não ser que estejas a garantir a terminação de outra cópia.

Resumindo:

- atoi/atof/strncpy trabalham com **strings** (arrays de char com '\0' no fim).
- Se passas um char simples, o compilador queixa-se. [\[123\]](#) [\[122\]](#)

Para perceber melhor: atualmente ST_Command_Out está declarado como char ou como char[] no teu ficheiro?

✱✱

tu dizes isto:

Em strncpy(dst, src, sizeof(dst));:

dst é o destino (onde vais copiar).

src é a origem (de onde copias).

sizeof(dst) é o número máximo de caracteres a copiar.

A assinatura da função é:

mas como é que o sizeof t~em que ser da varaivel onde eu vou copiar, e não da varaivel que eu vou copiar?

pq eu tenho que saber é o numero de caracteres que vou copiar de um lado para o outro.
por exemplo quero copiar os valores da string ST_Bus, para a string ST_Command_Out, por isso não deveria fazer:

```
strncpy(ST_Command_Out, ST_BUS, sizeof(ST_BUS));  
??
```

Boa pergunta, e estás a pensar bem: a parte crítica é perceber *o que significa* esse terceiro argumento.

1. O que `strncpy` realmente faz

Assinatura:

```
char *strncpy(char *dest, const char *src, size_t n);
```

- `dest`: array onde *vai ficar* a cópia
- `src`: array de onde os caracteres vêm
- `n`: **máximo de caracteres que vais escrever em `dest`**

Ou seja, `n` não é “quantos caracteres tem a origem”, é “quantos no máximo estás disposto a escrever no destino sem rebentar o array”.

Por isso é que:

```
strncpy(dest, src, sizeof(dest));
```

é o padrão típico: estás a dizer “copia no máximo o tamanho de `dest`”.

Se `src` for maior que `dest`, `strncpy` não sabe disso por magia; se tu lhe deres `sizeof(src)`, ela pode escrever mais do que cabe em `dest` e rebentar memória.

2. E então, no teu exemplo?

Se:

- `ST_BUS` é o destino
- `ST_Command_Out` é a origem

então, sim, a cópia seria:

```
strncpy(ST_BUS, ST_Command_Out, sizeof(ST_BUS));  
ST_BUS[sizeof(ST_BUS) - 1] = '\0';
```

Aqui usas `sizeof(ST_BUS)` porque é o tamanho do **destino** que limita o máximo de caracteres seguros.

Se quiseres copiar o conteúdo de `ST_BUS` para `ST_Command_Out`, inverte:

```
strncpy(ST_Command_Out, ST_BUS, sizeof(ST_Command_Out));
ST_Command_Out[sizeof(ST_Command_Out) - 1] = '\0';
```

De novo: `sizeof` do destino.

3. “Mas eu quero saber quantos caracteres tem a origem”

Aí não usas `sizeof`, usas `strlen(src)`:

```
size_t len = strlen(src);
strncpy(dest, src, min(len + 1, sizeof(dest)));
```

Só que em microcontroladores normalmente simplificas:

- defines buffers com tamanho suficiente
- e limitas sempre pelo tamanho do destino (para não rebentar).

Então, respondendo diretamente à tua pergunta:

- O `sizeof` no `strncpy` deve ser do **destino**, porque o objetivo é **não ultrapassar o tamanho do destino**.
- O número de caracteres reais da origem, se quiseres saber, obténs com `strlen`, não com `sizeof`.

Para aplicares no teu caso concreto: neste momento queres copiar de que buffer para que buffer (`ST_Bus` → `ST_Command_Out`, ou ao contrário), e qual deles é maior no teu código?

`strncpy(ST_Command_Out, ST_BUS, sizeof(ST_Command_Out));`

`ST_Command_Out[sizeof(ST_Command_Out) - 1] = '\0';`

mas assim não vai copiar todos os caracteres até aquele que estão a 0?

eu por exemplo só quero copiar os caracteres reais de `ST_bus` para `STComand_Out`

`ST_bus` -> copia para -> `STComand_Out`

Boa, agora estamos mesmo no detalhe certo.

1. O que `strncpy` faz com esse código

```
strncpy(ST_Command_Out, ST_BUS, sizeof(ST_Command_Out));
ST_Command_Out[sizeof(ST_Command_Out) - 1] = '\0';
```

- Vai copiar **no máximo** `sizeof(ST_Command_Out)` caracteres de `ST_BUS` para `ST_Command_Out`.
- Se `ST_BUS` tiver menos caracteres (termina em `'\0'` antes), para na terminação e não copia lixo.

- A linha seguinte garante que a última posição de `ST_Command_Out` é `'\0'`, no caso de a origem ser grande demais.

Portanto, se `ST_BUS` tiver, por exemplo, `"ABC\n"`:

- `strncpy` copia `"ABC\n\0"` para `ST_Command_Out` (até ao `'\0'`),
- o resto das posições de `ST_Command_Out` ficam como estavam ou preenchidas com `'\0'` (depende da implementação),
- e tu ainda forças a última posição a `'\0'` por segurança.

Ou seja: **sim**, copia apenas os caracteres “reais” até ao primeiro `'\0'` da origem; não copia “até ao primeiro 0 na memória inteira”. O `sizeof` aqui só limita o máximo para não rebentar o destino.

2. Se queres copiar “só o que está em `ST_BUS`”

O que precisas garantir é:

1. `ST_BUS` seja uma string C bem terminada (último char válido seguido de `'\0'`).
2. O destino tenha tamanho suficiente para isso.

Então:

```
// supor que ST_BUS está, por exemplo, "M D R x[4][0.1]" + '\0'
strncpy(ST_Command_Out, ST_BUS, sizeof(ST_Command_Out));
ST_Command_Out[sizeof(ST_Command_Out) - 1] = '\0';
```

- Se `ST_BUS` couber dentro de `ST_Command_Out`, copias **tudo** (até ao `'\0'`).
- Se não couber, copias o que couber, e a tua linha extra garante que o destino fica pelo menos terminadinho em `'\0'`.

Se quiseres mesmo copiar só o número exato de caracteres da origem, podes usar `strlen`:

```
size_t len = strlen(ST_BUS);
if (len >= sizeof(ST_Command_Out)) len = sizeof(ST_Command_Out) - 1;

strncpy(ST_Command_Out, ST_BUS, len);
ST_Command_Out[len] = '\0';
```

Assim tens a certeza que copias exatamente o comprimento real da string de origem, mas continuas a limitar pelo tamanho do destino.

Na tua FSM, como já tens buffers pequenos e controlados, usar só:

```
strncpy(ST_Command_Out, ST_BUS, sizeof(ST_Command_Out));
ST_Command_Out[sizeof(ST_Command_Out) - 1] = '\0';
```

já é mais do que suficiente, desde que `ST_BUS` esteja bem terminada.

Faz sentido para ti agora a diferença entre “tamanho máximo do destino” (`sizeof(dest)`) e “número real de caracteres copiadas” (que vem do `'\0'` em `src`)?

e não posso fazer isto?

```
char Mode = 'S'; // 'S' = Single, 'M' = Manual
char CH_Command_Out = '\0';
char CH_Command_IN = '\0';
char ST_Command_Out[100];
char ST_Command_IN[100];
int command_counter = 0;
bool LineReady_IN = false;
bool LineReady_Out = false;

void FSM_Serial_reader(unsigned long nowMs){
    static short int state;
    static int counter = 0;
    static char CH_Bus;
    static char ST_BUS[100];
    state_test = state;
    /*
    Têm em atenção constituição de cada caso:
    case x:
        - Ações a executar
        - Condições de transição por hierarquia
    */
    //Serial.print("Estado atual: ");
    //Serial.println(inst.state);

    switch (state) {

    case 0:
        counter = 0;
        CH_Command_Out = '\0';

    if(Serial.available() > 0 && Mode == 'S'){state = 1;}
        if(Serial.available() > 0 && Mode == 'M'){state = 6;}
        break;

    case 1:
        CH_Bus = Serial.read();
        counter++;

    if(CH_Bus == ' '){state = 1;}
        if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
        if(CH_Bus == '\n'){state = 4;}
        break;
```



```

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

    if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

    if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

    if(LineReady_IN == true){state = 0;}
    break;

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

    if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

case 8:
    strncpy(ST_Command_Out, ST_BUS, sizeof(counter));
    ST_Command_Out[sizeof(ST_Command_Out)-1] = '\0';

    break;
}
return;
}

```

O esqueleto da FSM está no bom caminho, mas há dois problemas importantes nessa parte:

1. Estás a usar `sizeof(counter)` no `strncpy`
2. Estás a usar `counter` como índice de escrita mas nunca colocas `'\0'` em `ST_BUS`

Vamos por partes.

1. Problema no `strncpy`

Tens:

```
case 8:
    strncpy(ST_Command_Out, ST_BUS, sizeof(counter));
    ST_Command_Out[sizeof(ST_Command_Out)-1] = '\\0';
    break;
```

- `sizeof(counter)` é o tamanho em bytes da variável `counter` (provavelmente 4), **não** o número de caracteres lidos.
- Ou seja, estás a dizer ao `strncpy` para copiar no máximo 4 caracteres, mesmo que tenhas lido 20.
- O terceiro argumento de `strncpy` deve ser:
 - ou o tamanho do destino (`sizeof(ST_Command_Out)`),
 - ou o número real de caracteres que leste em `ST_BUS` (que já tens em `counter`), limitado pelo tamanho do destino.

No teu caso, como `counter` conta quantos caracteres escreveste em `ST_BUS`, o correto é:

```
case 8:
    // garantir terminação em ST_BUS
    ST_BUS[counter] = '\\0';

    // copiar no máximo o número de chars lidos, sem passar o tamanho do destino
    int maxCopy = counter;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }

    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\\0'; // termina o destino

    // aqui podes pôr LineReady_IN = true, etc.
    LineReady_IN = true;
    state = 0; // por exemplo, volta ao início
    break;
```

Repara que:

- Usas `counter` (quantos chars leste) para dizer quantos chars queres copiar.
- Limitas pelo tamanho do destino para não rebentar o buffer.
- Pões '\\0' em `ST_BUS[counter]` e em `ST_Command_Out[maxCopy]` para teres strings válidas.

2. Pequena afinação no estado 6

Hoje tens:

```
case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

    if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;
```

Sugestões:

- Garante que não passas o tamanho máximo de ST_BUS:

```
if (counter < (int)sizeof(ST_BUS) - 1) {
    ST_BUS[counter] = Serial.read();
    counter++;
}
```

- Quando apanhas '\n', saís para o estado 8 e não continuas a ler:

```
if (ST_BUS[counter-1] == '\n') {
    state = 8;
} else if (ST_BUS[counter-1] == ' ') {
    state = 6; // continua
} else if (Serial.available() > 0) {
    state = 6;
}
```

Assim tens a FSM a:

1. Ler a linha completa em ST_BUS no modo M.
2. Terminar com '\n'.
3. Em 8, transformar isso numa string bem terminada ('\0') e copiar só o que interessa para ST_Command_Out.

Portanto: sim, podes seguir essa estrutura, só precisas de corrigir o uso do `strncpy` (trocar `sizeof(counter)` por algo baseado em `counter` ou no tamanho do destino) e garantir que `ST_BUS` termina em `'\0'` antes de usar qualquer função de string.

Queres experimentar estas correções e depois ver juntos como farias, a partir de `ST_Command_Out`, a procura do primeiro [e o `atof` para o primeiro valor?

Pequena afinação no estado 6

Hoje tens:

cpp

case 6:

```
ST_BUS[counter] = Serial.read();
```

```
counter++;
```

```
if(ST_BUS[counter-1] == ' '){state = 6;}
```

```
if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
```

```
if(ST_BUS[counter-1] == '\n'){state = 8;}
```

```
break;
```

Sugestões:

Garante que não passas o tamanho máximo de ST_BUS:

cpp

```
if (counter < (int)sizeof(ST_BUS) - 1) {
```

```
ST_BUS[counter] = Serial.read();
```

```
counter++;
```

```
}
```

Quando apanhas '\n', saís para o estado 8 e não continuas a ler:

cpp

```
if (ST_BUS[counter-1] == '\n') {
```

```
state = 8;
```

```
} else if (ST_BUS[counter-1] == ' ') {
```

```
state = 6; // continua
```

```
} else if (Serial.available() > 0) {
```

```
state = 6;
```

```
}
```

Assim tens a FSM a:

Ler a linha completa em ST_BUS no modo M.

Terminar com '\n'.

Em 8, transformar isso numa string bem terminada ('\0') e copiar só o que interessa para ST_Command_Out.

A cena é que eu não quero passar o \0 ou \n

eu quero que a minha string fique limpinha

tu não tens estado a tomar atenção a nada do que eu tenho estado a explicar nos ultimos dias?

Tens razão, e obrigado por insistir no detalhe: tu queres que a string de comandos fique só com o conteúdo útil, **sem** '\n' nem '\0' extra a meio, e queres controlar isso explicitamente na FSM.

Vamos alinhar isso com o que já fazes:

- A linha que vem pela serial termina em '\n'.
- Tu queres:
 - ler tudo até antes do '\n'

- não guardar o `'\n'` dentro da string
- e depois, no teu buffer, ter só os caracteres “reais”, terminados por um único `'\0'` no fim.

1. Estado 6 apenas lê até ao `'\n'`, sem o guardar

Em vez de colocares o `'\n'` no `ST_BUS`, podes parar antes:

```
case 6:
    if (Serial.available() > 0) {
        char c = Serial.read();

        if (c == ' ') {
            // ignora espaços
            state = 6;
        } else if (c == '\n') {
            // terminou a linha -> vai para o estado 8
            state = 8;
        } else {
            // guarda só caracteres úteis
            if (counter < (int)sizeof(ST_BUS) - 1) {
                ST_BUS[counter] = c;
                counter++;
            }
            state = 6;
        }
    }
    break;
```

Repara:

- Nunca escreves o `'\n'` no `ST_BUS`.
- `counter` fica sempre a contar apenas caracteres “limpos”.
- `ST_BUS` está cheio de dados úteis em posições `0 .. counter-1`, e nada mais.

2. Estado 8: terminas a string e copias só os caracteres reais

Aqui sim, tornas `ST_BUS` numa C-string “limpinha”:

```
case 8:
    // termina a string na posição counter
    ST_BUS[counter] = '\0';

    // calcula quantos chars copiar (sem rebentar o destino)
    int maxCopy = counter;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }

    // cópia dos caracteres úteis
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
```

```

ST_Command_Out[maxCopy] = '\0';

LineReady_IN = true;
// opcional: limpar counter, ST_BUS, etc.
counter = 0;
state = 0;    // volta ao início
break;

```

Agora tens:

- ST_BUS: buffer de leitura interna
- ST_Command_Out: string final, sem \n, só com o comando que te interessa
- Ambos terminados com um único '\0' no fim
- Nenhum '\0' é “passado” para a frente, tu decides onde o colocas.

Isto respeita o que tens dito desde o início: nada de String, nada de lixo dentro da linha, só caracteres úteis controlados por ti, e todas as transformações feitas explicitamente na FSM.

Para ter a certeza de que estamos alinhados: com este fluxo, consegues descrever, na tua própria FSM, em que momento exato o ST_Command_Out vai conter, por exemplo, "M D R x[4] [0.1]" pronto para o estado 24 em diante?

void FSM_Serial_reader(unsigned long nowMs){

```

static short int state;
static int counter = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

```

```

case 1:
    CH_Bus = Serial.read();
    counter++;

if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

case 8:

int maxCopy = counter-1;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }

```

```

}
strncpy(ST_Command_Out, ST_BUS, maxCopy);
ST_Command_Out[maxCopy] = '\0';
counter = 0;

break;
}
return;
}

```

Ficou assim ent

Está quase no que queres, mas ainda há dois detalhes importantes:

1. Estás a guardar o `'\n'` dentro do `ST_BUS` (porque lêes tudo e só depois assumas `counter-1` como útil).
2. No estado 6 não estás a proteger contra overflow do `ST_BUS`.

Vamos ajustar mantendo o teu estilo.

1. Corrigir o estado 6 para não guardar o `'\n'`

Agora:

```

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

    if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

```

Problemas:

- Se leres `'\n'`, já o puseste em `ST_BUS[counter-1]`.
- O `counter-1` que usas no `maxCopy` é um truque para “ignorar” o `'\n'`, mas o caractere continua no buffer, e se voltares a usá-lo noutra parte pode ser chato.

Uma versão que respeita a tua FSM mas não guarda o `'\n'`:

```

case 6:
    if (Serial.available() > 0) {
        char c = Serial.read();

        if (c == ' ') {
            // ignora espaços, não incrementa counter
            state = 6;
        }
        else if (c == '\n') {
            // não guarda '\n', termina a leitura da linha

```



```

        state = 8;
    }
    else {
        // guarda só caracteres úteis, protegendo o tamanho do buffer
        if (counter < (int)sizeof(ST_BUS) - 1) {
            ST_BUS[counter] = c;
            counter++;
        }
        state = 6;
    }
}
break;

```

- Aqui nunca escreves '\n' em ST_BUS.
- counter conta apenas caracteres “reais”.
- ST_BUS[0..counter-1] contém exatamente a linha sem '\n'.

2. Estado 8 com counter direto (sem -1)

Com o ajuste acima, counter já é o número de chars úteis, por isso podes usar diretamente:

```

case 8:
    // terminar a string em ST_BUS
    ST_BUS[counter] = '\0';

    int maxCopy = counter;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }

    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';

    counter = 0;
    LineReady_IN = true;    // se quiseses sinalizar que há comando
    state = 0;              // voltar ao estado inicial
    break;

```

Assim:

- ST_BUS fica “limpinho”, com '\0' logo a seguir ao último char útil.
- ST_Command_Out recebe exatamente os mesmos chars.
- Não tens '\n' nem '\0' extra no meio.

Se juntares estas duas pequenas alterações, o teu leitor em modo M passa a entregar uma string pronta em ST_Command_Out, sem lixo, pronta para o estado 24+ da FSM de comandos.

Consegues agora imaginar um exemplo concreto de linha que vais enviar (por exemplo "M D R x[4][0.1]"), e dizer o que ficará exatamente dentro de ST_Command_Out depois do estado 8 com esta lógica?

vou passar `int maxCopy = counter-2;`

Assim o `ST_command_Out` não vai com o `\n` ou `\0`
esse sim é que me interessa que vá limpinho
e não tenho problemas com overflow, pois consigo garantir que nunca vou enviar uma mensagem com mais de 100 caracteres

Entendido: o que te interessa é o `ST_Command_Out` “limpinho”, sem `\n` nem `\0` intermédios, e não te preocupa overflow porque controlas o lado do PC.

Alguns pontos importantes com `counter - 2`:

- Do jeito que o teu estado 6 estava antes, `counter` é incrementado **depois** de guardar cada char, incluindo o `'\n'`.
- Isso significa:
 - `counter - 1` → posição onde guardaste o `'\n'`
 - `counter - 2` → último carácter útil antes do `'\n'`

Se fizeres:

```
int maxCopy = counter - 2;
```

então:

- Estás a **ignorar dois caracteres**: o `'\n'` e mais um antes dele.
- Ou seja, perdes o último carácter útil da linha.

Para ficares com `ST_Command_Out` limpinho, tens duas opções coerentes:

Opção A – Não guardas o `'\n'` no `ST_BUS` (mais limpa)

No estado 6:

- Quando lêes `'\n'`, **não incrementas** `counter` nem escreves em `ST_BUS`.
- `counter` fica a contar só os chars úteis.

Então, em 8:

```
int maxCopy = counter;  
strncpy(ST_Command_Out, ST_BUS, maxCopy);  
ST_Command_Out[maxCopy] = '\0';
```

Aqui não precisas de `counter-1` nem `counter-2`.
O `\n` nunca entrou no buffer.

Opção B – Guardas o '\n', mas ignoras só ele

Se mesmo assim quiseses manter a tua versão de estado 6 (guardar tudo, incluindo \n), então:

- `counter - 1` é o '\n'.
- Os caracteres úteis estão em `ST_BUS[0 .. counter-2]`.

Logo:

```
int maxCopy = counter - 1; // ignora apenas o '\n'
if (maxCopy < 0) maxCopy = 0;

strncpy(ST_Command_Out, ST_BUS, maxCopy);
ST_Command_Out[maxCopy] = '\0';
```

Assim copias todos os caracteres úteis e só tiras o '\n'.

Se usares `counter - 2`, estarás a deitar fora **dois** caracteres: o \n e mais um antes dele, o que não é o que queres.

Queres ajustar primeiro o estado 6 para não guardar o \n (Opção A), e depois testarmos com uma linha real para ver o que aparece em `ST_Command_Out`?

ent mas antes dizes-me que com `counter - 1` continuo com o \n no `ST_comand`, pois eu estou a fazer `counter++` logo a seguir a ler o caracter

e agora dizes-me que estou a perder informação, uma dessas afirmações não faz sentido

Tens razão em questionar; vamos alinhar isto com calma porque a diferença é subtil e está no *quando* incrementas o `counter`.

Situação atual (do teu código)

No estado 6 tens algo deste género:

```
ST_BUS[counter] = Serial.read();
counter++;

if (ST_BUS[counter-1] == '\n') { state = 8; }
```

Vamos seguir um exemplo concreto:

- Suponha que a linha é "AB\n"
- Passos:
 1. Primeiro char 'A'
 - antes: `counter = 0`

- escreves: `ST_BUS[0] = 'A'`
- `counter++` → `counter = 1`
- `ST_BUS[counter-1] = ST_BUS[0] = 'A'`

2. Segundo char 'B'

- `ST_BUS[1] = 'B'`
- `counter++` → `counter = 2`
- `ST_BUS[counter-1] = ST_BUS[1] = 'B'`

3. Terceiro char '\n'

- `ST_BUS[2] = '\n'`
- `counter++` → `counter = 3`
- `ST_BUS[counter-1] = ST_BUS[2] = '\n'`
- `condition` → `state = 8`

No fim:

- `counter = 3`
- `ST_BUS[0] = 'A'`
- `ST_BUS[1] = 'B'`
- `ST_BUS[2] = '\n'`

Agora, em 8:

- Se fizeres `maxCopy = counter - 1`, `maxCopy = 2` → copias `ST_BUS[0]` e `ST_BUS[1]` → "AB" (limpo).
- Se fizeres `maxCopy = counter - 2`, `maxCopy = 1` → copias só `ST_BUS[0]` → "A" → perdes o B.

Portanto:

- Com `counter - 1` tu **NÃO** copias o `'\n'` (porque o `'\n'` está em `ST_BUS[counter-1]`).
- Com `counter - 2` perdes o último carácter útil.

A confusão que ficou foi outra:

- Quando eu disse que o `'\n'` “continua no `ST_BUS`” com `counter-1`, queria dizer:
 - ele ainda está guardado no **buffer interno** (`ST_BUS`),
 - mas tu **não o copias** para `ST_Command_Out`.
- O que vai “limpinho” é o `ST_Command_Out`, não o `ST_BUS`.

Se para ti é importante que nem o `ST_BUS` fique com o `'\n'`, então é melhor a abordagem A:

- no estado 6, quando lês `'\n'`, **não o guardas** em `ST_BUS` (só mudas o estado).
- isso deixa `counter` a contar só caracteres úteis; em 8 usas `maxCopy = counter`.

Resumindo:

- `counter - 1` → ignora só o `'\n'` (mantém todos os chars úteis).
- `counter - 2` → ignora o `'\n'` e o último char útil → perda de dados.

Isto esclarece a contradição que sentiste entre o que eu disse antes e agora?

counter = 3

```
ST_BUS[0] = 'A'
```

```
ST_BUS[1] = 'B'
```

```
ST_BUS[2] = '\n'
```

errado, ent já não sabes ler strings?

```
counter = 3
```

```
ST_BUS[0] = 'A'
```

```
ST_BUS[1] = 'B'
```

```
ST_BUS[2] = '\n'
```

```
ST_BUS[3] = '0'
```

Pois eu posso não ter copiado algo para o index 3, mas ele continua a existir na string pois a string é declaradas com 100 partes

```
ST_Bus[100];
```

por isso ao ter `counter=3` vou na verdade ter 4 índices na string

é por isso que estava a dizer `int maxCopy = counter-2;`

Tens toda a razão numa coisa importante: o array `ST_BUS[100]` **tem sempre 100 posições válidas na memória**; o que o transforma numa “string C” é onde metes o `'\0'`.

O que estivemos a falar é:

- `counter` diz-te quantos caracteres tu escreveste **manualmente** em `ST_BUS`.
- As posições que nunca mexeste (por exemplo `ST_BUS[3]`, `ST_BUS[4]`...) podem conter lixo antigo, mas **não fazem parte da string** enquanto tu não puseres um `'\0'` onde queres que ela acabe.

No teu exemplo concreto:

- Depois de ler `"AB\n"` com o teu código atual:
 - `counter = 3`
 - `ST_BUS[0] = 'A'`
 - `ST_BUS[1] = 'B'`
 - `ST_BUS[2] = '\n'`
 - `ST_BUS[3]` pode ter o que calhar (lixo anterior), até tu escreveres lá algo

Neste momento, **ainda não é uma string C válida**, porque não meteste `'\0'` em lado nenhum.

Quem decide onde está o fim da string és tu, quando fazes:

```
ST_BUS[counter] = '\0';
```

ou

```
ST_BUS[counter-1] = '\0';
```

Dependendo do que queres cortar.

Onde entra o `counter - 2`

Se fazes:

```
int maxCopy = counter - 2;
```

num caso em que `counter = 3`, ficas com `maxCopy = 1`.

- Isso significa: vais copiar só **um** carácter de `ST_BUS` para `ST_Command_Out`.
- Ou seja, copias só 'A' e perdes o 'B'.
- O facto de `ST_BUS[3]` existir fisicamente na RAM não muda o número de caracteres que estás a copiar.

O que manda é:

```
strncpy(ST_Command_Out, ST_BUS, maxCopy);
```

Quando `maxCopy = 1`, só sai um char, mesmo que o array tenha 100 posições.

Como conciliar o que queres

Pelo que disseste, queres:

1. Não copiar o '\n' para `ST_Command_Out`.
2. Não copiar lixo nem '\0' internos.
3. Manter o controlo via `counter` e FSM.

Então, a forma mais limpa é:

- No estado 6, **não guardas o '\n' em** `ST_BUS`:

```
case 6:
    if (Serial.available() > 0) {
        char c = Serial.read();

        if (c == ' ') {
            // ignora
        } else if (c == '\n') {
            state = 8; // termina linha
        } else {
            if (counter < (int)sizeof(ST_BUS) - 1) {
```

```

        ST_BUS[counter] = c;
        counter++;
    }
}
}
break;

```

- Em 8, defines tu o fim da string e copias todos os chars úteis:

```

case 8:
    ST_BUS[counter] = '\0';    // agora sim: string limpinha

    int maxCopy = counter;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }

    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';

    counter = 0;
    LineReady_IN = true;
    state = 0;
    break;

```

Aqui:

- ST_BUS fica "AB\0..." (sem \n).
- ST_Command_Out fica "AB\0..."
- Não precisas de nenhum counter-2, porque o \n nunca entrou na conta.

Se preferires mesmo manter o teu estilo com leitura direta e counter++ logo a seguir, também dá, mas aí o valor correto para ignorar só o \n é counter - 1, não counter - 2, precisamente porque é o counter - 1 que aponta para o \n.

Faz sentido agora a diferença entre “tamanho físico do array” (100) e “tamanho lógico da string” (controlado por counter e pelo '\0' que tu pões)?

#include <Arduino.h>

```

#include <Adafruit_NeoPixel.h>
#include <ESP32Servo.h>
#include <math.h>

```

```

// declarar no topo do ficheiro (globais)
int last_delta2_s1 = 0;
int last_delta2_s2 = 0;
int last_delta2_s3 = 0;

```

```

struct Manipulador_Config {
    // geometria

```

```

float L1;
float L2;
float L3;
float servoOffsetX;
float servoOffsetZ;
float angleMin;
float angleMax;
float rotationOffsetZ; // offset global de rotação da base

// calibração de PWM
int servo1MinPulse;
int servo1MaxPulse;

int servo2MinPulse;
int servo2MaxPulse;

int servo3MinPulse;
int servo3MaxPulse;

int theta1MinPulse;
int theta1MaxPulse;

int theta2MinPulse;
int theta2MaxPulse;

int theta3MinPulse;
int theta3MaxPulse;

// estado atual (pulsos gerados pela IK)
int servo1Pulse;
int servo2Pulse;
int servo3Pulse;
};

Manipulador_Config delta_1_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -15.6f * PI / 180.0f, //-75 degrees in radians

// calibração de PWM
    .servo1MinPulse = 520,
    .servo1MaxPulse = 2520,

```



```

.servo2MinPulse = 550,
    .servo2MaxPulse = 2550,

.servo3MinPulse = 560,
    .servo3MaxPulse = 2560,

// estado atual (pulsos gerados pela IK)
    .servo1Pulse = 0,
    .servo2Pulse = 0,
    .servo3Pulse = 0
};

Manipulador_Config delta_2_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -44.4f * PI / 180.0f, //-45 degrees in radians

// calibração de PWM
    .servo1MinPulse = 460,
    .servo1MaxPulse = 2460,

.servo2MinPulse = 470,
    .servo2MaxPulse = 2470,

.servo3MinPulse = 450,
    .servo3MaxPulse = 2450,

// estado atual (pulsos gerados pela IK)
    .servo1Pulse = 0,
    .servo2Pulse = 0,
    .servo3Pulse = 0
};

//Motion_test -> Arrays of points for testing the motion functions
float X_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 5.0, 10.0, 15.0, 0.0, 0.0};
float Y_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 5.0, 15.0};    // -> teste de
movimento linear dos 3 eixos
float Z_test_calibration[] = {40.0, 40.0, 50.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};
const int N_test_calibration = 9;

//bat
float X_test[] = {15.607, 14.923, 13.995, 12.845, 11.503, 10.000, 6.665, 1.464, -2.965, -5.000,
-5.659, -6.056, -6.180, -6.029, -5.607, -4.923, -3.995, -2.845, -1.503, 0.000, 3.335, 8.536, 12.965,

```

```

15.000, 15.659, 16.056, 16.180, 16.029};
//float Y_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
float Y_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = { 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0};
float Z_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
const int N_test = 28;
float T_period = 0.3f;
float T_pausedt = 0.6f;

//resp
//float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000, 10.667, 9.333,
8.000, 6.667, 5.333, 4.000, 2.667, 1.333 };
//float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0 };
//float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000, 15.802,
12.099, 8.889, 6.173, 3.951, 2.222, 0.988, 0.247};
float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000};
float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000};

//const int N_test2 = 18;
const int N_test2 = 10;
float T_period2 = 4.0f;
float T_pausedt2 = 0.8f;
//Link lengths (mm)
#define L1 35
#define L2 60
#define L3 15

#define END_EFFECTOR_Z_OFFSET 0
#define SERVO_OFFSET_X 32
#define SERVO_OFFSET_Y 0
#define SERVO_OFFSET_Z (0.0 + END_EFFECTOR_Z_OFFSET)
//#define SERVO_OFFSET_Z_INVERTED -293

```

```

#define SERVO_ANGLE_MIN (100.0f * PI / 180.0f)
#define SERVO_ANGLE_MAX (190.0f * PI / 180.0f)

#define SERVO_ANGLE_MIN 0.78539816339744830961566084581988f //45 degrees
#define SERVO_ANGLE_MAX 3.9269908169872415480783042290994f //225 degrees

#define ROTATION_OFFSET_Z (-75.0f * PI / 180.0f) //0.26179938779914943f (15° em radianos =
π/12):

#define ROTATION_OFFSET_Z_Delta1 (-75.0f * PI / 180.0f)
#define ROTATION_OFFSET_Z_Delta2 (-45.0f * PI / 180.0f)

#define MIN 'm'
#define MAX 'M'

//Servo microsecond pulse limits (Calibration)-----Valores normais: Min = 500 ; Max = 2500]-----
-----
#define SERVO_1_MIN 620
#define SERVO_1_MAX 2520
#define SERVO_2_MIN 615
#define SERVO_2_MAX 2480
#define SERVO_3_MIN 620
#define SERVO_3_MAX 2550
//Servo_1 -> theta3 min 550, max 2450
//Servo_2 -> theta1 min 545, max 2410
//Servo_3 -> theta2 min 520, max 2450

// variáveis globais para valores suavizados
int servo_1_pulse_smooth = 0;
int servo_2_pulse_smooth = 0;
int servo_3_pulse_smooth = 0;

// fator de suavização [0 - 1]. tipo 0.2 para começar
const float SERVO_SMOOTH_ALPHA = 0.2f;

#define SERVO_4_MIN 550
#define SERVO_4_MAX 2400
#define SERVO_5_MIN 550
#define SERVO_5_MAX 2400
#define SERVO_6_MIN 550
#define SERVO_6_MAX 2400

// variáveis globais para valores suavizados
int servo_4_pulse_smooth = 0;
int servo_5_pulse_smooth = 0;
int servo_6_pulse_smooth = 0;
//-----

#define INVERTED -1

```

```

// Máximo de pontos por movimento (ajusta mais tarde se precisares)
const int MAX_POINTS = 100;

// Armazenamento Bruto: dados completos de um movimento - Será utilizada para armazenar
os movimentos que o Pi enviar, ou movimentos pré-definidos, etc.
struct MotionStorage {
    int  n_Points;           // nº de pontos válidos em X/Y/Z/easing
    float X[MAX_POINTS];     // coordenadas X dos pontos do movimento
    float Y[MAX_POINTS];     // coordenadas Y dos pontos do movimento
    float Z[MAX_POINTS];     // coordenadas Z dos pontos do movimento
    float period;
    float T_pause;
    float dtMs;              // duração do ciclo em segundos
    float dt_pauseMs;        // duração da pausa entre ciclos em segundos
    int  startIndex; // índice de ponto que será o "0,0,0" lógico
    bool bidirectional; // true = vai-e-vem, false = ciclo fechado
    int  N_pontos ; // nº de segmentos por ciclo

float getDtMs(){
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_pontos = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinhas
            N_pontos = n_Points;
        }
        return (period * 1000.0f) / (float)N_pontos;
    }
    return 0.0f;
}

void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}

float getD_PauseMs() const {
    if (n_Points > 1 && T_pause > 0.0f) {
        return (T_pause * 1000.0f); // só lê nPoints e T_pause
    }
    return 0.0f;
}

void updateDt_PauseMs() {
    dt_pauseMs = getD_PauseMs(); // aqui sim, mudas dt_pauseMs
}

};

```

```

// Armazenamento temporário: runtime que executa um MotionStorage
struct MotionInstance {
    MotionStorage* def;    // ponteiro para MotionStorage (a definição do Armazenamento Bruto
)
    int      currentIndex; // índice i (início do segmento i -> i+1) -> índice atual no movimento
    unsigned int  segmentStartMs; // timestamp do início do segmento atual
                        // -> é uma variavel importante para controlar a velocidade do movimento,
ou seja, quando é que deve avançar para o próximo ponto do movimento
    bool      active;    // se este movimento está ativo
    bool      Executing; // se este movimento está em pausa (entre ciclos)
    short int  state;
    unsigned long elapsed_1;
    unsigned long elapsed_2;
    unsigned long elapsed_3;

int iIndex; // novo: índice i para a interpolação
    int jIndex; // novo: índice j para a interpolação
    int Direction; // novo: direção do movimento (1 ou -1)
};

// -----Temporario-----
MotionStorage Bat_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Bat_inst;
MotionStorage Resp_move; //Declaração de objetos para utilizar com as structures de
movimento
MotionInstance Resp_inst;
MotionStorage Tosse_move; //Declaração de objetos para utilizar com as structures de
movimento
MotionInstance Tosse_inst;
MotionStorage* currentMove = nullptr;
MotionInstance* currentInst = nullptr;

void Iniciate_Bat_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Bat_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Bat_move cada vez que quiseres testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Bat_move.n_Points = N_test;
    Bat_move.period = T_period;
    Bat_move.T_pause = T_pausedt;
    for (int i = 0; i < N_test; i++) {
        Bat_move.X[i] = X_test[i];
        Bat_move.Y[i] = -1 * Y_test[i];
        Bat_move.Z[i] = Z_test[i];
    }
}

```

```

}
Bat_move.startIndex = 0;
Bat_move.bidirectional = false; // Define o movimento como vai-e-vem
Bat_move.updateDtMs();
Bat_move.updateDt_PauseMs();
}

void Inicie_Resp_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Resp2_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Resp2_move cada vez que quiseres testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Resp_move.n_Points = N_test2;
    Resp_move.period = T_period2;
    Resp_move.T_pause = T_pausedt2;
    for (int i = 0; i < N_test2; i++) {
        Resp_move.X[i] = X_test2[i];
        Resp_move.Y[i] = -1 * Y_test2[i];
        Resp_move.Z[i] = Z_test2[i];
    }
    Resp_move.startIndex = 5;
    Resp_move.bidirectional = true; // Define o movimento como vai-e-vem
    Resp_move.updateDtMs();
    Resp_move.updateDt_PauseMs();
}

void Inicie_Bat_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Bat_inst.def = &Bat_move;
    Bat_inst.currentIndex = 0;
    Bat_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Bat_inst.active = true;
    Bat_inst.Executing = 0; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Bat_inst.state = 0;
    Bat_inst.elapsed_1 = 0;
    Bat_inst.elapsed_2 = 0;
    Bat_inst.elapsed_3 = 0;
    Bat_inst.iIndex = 0;
    Bat_inst.jIndex = 0;
    Bat_inst.Direction = 1; // novo: inicializa a direção como positiva
}

void Inicie_Resp_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Resp_inst.def = &Resp_move;
    Resp_inst.currentIndex = 0;
    Resp_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update

```

```

    Resp_inst.active    = true;
    Resp_inst.Executing = false; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Resp_inst.state      = 0;
    Resp_inst.elapsed_1  = 0;
    Resp_inst.elapsed_2  = 0;
    Resp_inst.elapsed_3  = 0;
    Resp_inst.iIndex     = 0;
    Resp_inst.jIndex     = 0;
    Resp_inst.Direction  = 1; // novo: inicializa a direção como positiva
}

```

```

//-----Fim Temporario-----

```

```

struct Coordinate {
    float X;
    float Y;
    float Z;
};

```

```

int Clock_loop = 0;
const float pi = 3.141592653;

```

```

// -----Declaração das variaveis-----

```

```

void LED_LOOP(); //Declaração de funções

```

```

void Servo_test();

```

```

//-----Novas-----

```

```

float boundFloat(float, float, float);

```

```

bool attach_servos(void);

```

```

bool inverse_kinematics_1(float, float, float, float, float&);

```

```

bool inverse_kinematics_2(float, float, float, float, float&);

```

```

bool inverse_kinematics_3(float, float, float, float, float&);

```

```

bool inverse_kinematics(Manipulador_Config& cfg, float, float, float);

```

```

void move_servos(const Manipulador_Config& d1, const Manipulador_Config& d2);

```

```

double mapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

int roundMapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

double degToRads(double deg);

```

```

double radsToDeg(double rads);

```

```

void debugPrintMove(const MotionStorage& move, const char* name);

```

```

void debugPrintInstance(const MotionInstance& inst, const char* name);

```

```

float lerp(float a, float b, float t);

```

```

float applyEasing(float alpha, char mode);

```

```

float spline3(float p0, float p1, float p2, float t);

```

```

void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs,
Coordinate& xyz, boolean trig_serial);

```

```

bool Fusao_plus_IK_D1(const Coordinate& respi, const Coordinate& batim);
bool Fusao_plus_IK_D2(const Coordinate& respi, const Coordinate& batim);
//-----FSM-----
void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs);
void FSM_Serial_reader(unsigned long nowMs);
void FSM_Serial_Command(unsigned long nowMs);
void FSM_Main(boolean start, MotionInstance& inst, unsigned long nowMs);
//-----

//--Declaração dos comandos de leitura
//char commands
#define Scomand_start_Program 's'
#define Scomand_stop_Program 'p'
#define Scomand_GRIPPER 3
#define Scomand_ABSOLUTE_CARTESIAN_LINEAR 4
#define Scomand_SET_PROGRAM_ARRAY 5
#define Scomand_REQUEST_READY_FLAG 6

//String commands
#define Mcomand_JOG_X 'x'
#define Mcomand_JOG_Y 'y'
#define COMMAND_JOG_Z 'z'
#define COMMAND_GRIPPER_ASCII 'g'
#define COMMAND_STATUS 's'
#define COMMAND_REPORT_COMMANDS 'r'
#define COMMAND_ADD_POSITION 'p'
#define COMMAND_SET_STEP_DELAY 'd'
#define COMMAND_SET_STEP_INCREMENT 'i'
#define COMMAND_CLEAR_ARRAY 'c'
#define COMMAND_EXECUTE 'e'
#define COMMAND_EXECUTE_JOINT 'j'
#define COMMAND_EXECUTE_TIME 't'
#define COMMAND_SET_US_INCREMENT_LINEAR 'u'
#define COMMAND_SET_US_INCREMENT_JOINT 'U'
#define COMMAND_STEP_FORWARD '>'
#define COMMAND_STEP_BACKWARD '<'
#define COMMAND_JUMP_TO_START '['
#define COMMAND_JUMP_TO_END ']'
#define COMMAND_EDIT_ARRAY 'P'
#define COMMAND_ADD_DELAY 'D'
#define COMMAND_MOVE_HOME 'h'
#define COMMAND_PRINT_FILE 'f'
#define COMMAND_PING_PONG 'o'
#define COMMAND_SERVO_CALIBRATION 'C'
#define COMMAND_SET_SERVO 'S'

```



```

//EEPROM commands
#define Ecomand_SET_LINK_2 'L'
#define Ecomand_SET_END_EFFECTOR_TYPE 'E'
#define Ecomand_SET_AXIS_DIRECTION 'A'
#define Ecomand_SET_HOME_X 'X'
#define Ecomand_SET_HOME_Y 'Y'
#define Ecomand_SET_HOME_Z 'Z'
#define Ecomand_SET_HOME_GRIPPER 'G'
#define Ecomand_SET_GRIPPER_MIN_MAX 'M'

//EEPROM addresses for the delta robots configuration
#define EEPROM_ADDRESS_LINK_2 0
#define EEPROM_ADDRESS_END_EFFECTOR_TYPE 1
#define EEPROM_ADDRESS_AXIS_DIRECTION 2
#define EEPROM_ADDRESS_Z_OFFSET 3
#define EEPROM_ADDRESS_HOME_X 7
#define EEPROM_ADDRESS_HOME_Y 11
#define EEPROM_ADDRESS_HOME_Z 15
#define EEPROM_ADDRESS_HOME_GRIPPER 19
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MIN 21
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MAX 23
#define EEPROM_ADDRESS_GRIPPER_CLAW_MIN 25
#define EEPROM_ADDRESS_GRIPPER_CLAW_MAX 27
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MIN 29
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MAX 31

//Delta1 -----
Servo mg90s_1; // cria objeto servo
Servo mg90s_2; // cria objeto servo
Servo mg90s_3; // cria objeto servo

//Delta2 -----
Servo mg90s_4; // cria objeto servo
Servo mg90s_5; // cria objeto servo
Servo mg90s_6; // cria objeto servo

volatile boolean start_comand = 0;

Coordinate Lerp_Respi_OUT;
Coordinate Lerp_Batim_OUT;
Coordinate IK2_IN;
Coordinate home_position;

float servo_offset_z = SERVO_OFFSET_Z;

//-----Declaração da localização dos pinos para cada objeto -----
//-----Servos-----
#define SERVO_PIN_1 12 //Servo_1 -> theta3

```

```

#define SERVO_PIN_2 11 //Servo_2 -> theta1
#define SERVO_PIN_3 10 //Servo_3 -> theta2

#define SERVO_PIN_4 36 // Servo_4 -> theta3
#define SERVO_PIN_5 37 // Servo_5 -> theta1
#define SERVO_PIN_6 38 // Servo_6 -> theta2

//-----Led_Informação-----
#define LED_PIN 48
#define LED_COUNT 1

//-----
bool stepDirection = false;
Adafruit_NeoPixel pixel(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

char Mode = 'S'; // 'S' = Single, 'M' = Manual
char CH_Command_Out = '\0';
char CH_Command_IN = '\0';
char ST_Command_Out[100];
char ST_Command_IN[100];
int command_counter = 0;
bool LineReady_IN = false;
bool LineReady_Out = false;

boolean trig_display_fsm(float at, short int Td){
static short int state; //state internal to the state machine
static float pt; //var. internal to the state machine
boolean trig; // output

switch (state) {
case 0:
if (at-pt >= Td) {state = 1;}
trig = 0; // output
return trig;
case 1:
if (1) {state = 0;}
trig = 1; pt = at; // output
return trig;
}
return trig;
}

int setup_counter = 0;

int state_test = 0;
int state_test2 = 0;
float Training_Exame_Starttime = 0;
float Training_Exame_Finishingtime = 0;

```

```

float Training_Exame_Durantiontime = 0;
int Safe_comand = 0;

void FSM_Serial_reader(unsigned long nowMs){
static short int state;
static int counter = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

```

```

if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

case 8:

int maxCopy = counter-1; // aqui definimos a quantidade de caracteres a copiar, não em termos
do indice mas sim a quantidade
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';
    counter = 0;

    break;
}
return;
}

```

```

void FSM_Serial_Command(unsigned long nowMs){
static short int state;
state_test2 = state;
/*

```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar
- Condições de transição por hierarquia

```

*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    Serial.println("Escolha um comando:");

    if(1){state = 1;}

    break;

case 1:
    LineReady_Out = false;

    if(CH_Command_IN == 'S' || CH_Command_IN == 's'){state = 2;}
    if(CH_Command_IN == 'P' || CH_Command_IN == 'p'){state = 3;}
    if(CH_Command_IN == 'L' || CH_Command_IN == 'l'){state = 5;}
    if(CH_Command_IN == 'M' || CH_Command_IN == 'm'){state = 14;}
    break;

case 2:
    Serial.println("Ativo");
    start_comand = 1;
    Training_Exame_Starttime = nowMs;
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 0, 0));
    pixel.show();

    if (1) { state = 4;}
    break;

case 3:
    Serial.println("Desativo");
    start_comand = 0;
    Training_Exame_Finishingtime = nowMs;
    Training_Exame_Durantiontime = Training_Exame_Finishingtime -
Training_Exame_Starttime;
    if(Training_Exame_Durantiontime > 0){
        Serial.print("Duração do treino: ");
        Serial.print(Training_Exame_Durantiontime/1000.0f);
        Serial.println(" Segundos");
    }
    Training_Exame_Finishingtime = 0;
    Training_Exame_Starttime = 0;
    LineReady_Out = true;
    CH_Command_Out = '\0';

```

```
pixel.setPixelColor(0, pixel.Color(0, 0, 0));  
pixel.show();
```

```
if (1) { state = 4;}  
break;
```

case 4:

```
Serial.println("Escolha um comando:");  
LineReady_Out = false;
```

```
if(1){state = 1;}  
break;
```

case 5:

```
Serial.println("ler");  
LineReady_Out = true;  
CH_Command_Out = '\0';  
pixel.setPixelColor(0, pixel.Color(255, 255, 0));  
pixel.show();
```

```
if(1){state = 6;}  
break;
```

case 6:

```
LineReady_Out = false;
```

```
if(CH_Command_IN == '1'){state = 7;}  
if(CH_Command_IN == '2'){state = 8;}  
if(CH_Command_IN == '3'){state = 9;}  
if(CH_Command_IN == 'r'){state = 10;}  
if(CH_Command_IN == 'b'){state = 11;}  
if(CH_Command_IN == 't'){state = 12;}  
if(CH_Command_IN == 'a'){state = 13;}  
break;
```

case 7:

```
Serial.println("Comando L1");  
LineReady_Out = true;  
CH_Command_Out = '\0';  
pixel.setPixelColor(0, pixel.Color(255, 255, 255));  
pixel.show();  
Safe_comand = 1;
```

```
if(1){state = 4;}  
break;
```

case 8:

```
Serial.println("Comando L2");  
LineReady_Out = true;
```

```

    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 255, 255));
    pixel.show();
    Safe_comand = 0;

if(1){state = 4;}
    break;

case 9:
    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Serial.println("Sem nada"); //Retirar quando estiver implementado

if(1){state = 4;}
    break;

case 10:
    Serial.println("Comando Lr");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    debugPrintMove(Resp_move, "Resp_move");
    debugPrintInstance(Resp_inst, "Resp_inst");

    if(1){state = 4;}
    break;

case 11:
    Serial.println("Comando Lb");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    debugPrintMove(Bat_move, "Bat_move");
    debugPrintInstance(Bat_inst, "Bat_inst");

    if(1){state = 4;}
    break;

case 12:
    Serial.println("Comando Lt");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    //debugPrintMove(Bat_move, "Tosse_move"); //ainda é preciso implementar
    //debugPrintInstance(Bat_inst, "Tosse_inst"); //ainda é preciso implementar
    Serial.println("Sem nada"); //Retirar quando estiver implementado

    if(1){state = 4;}
    break;

```

case 13:

```
Serial.println("Comando LA");
LineReady_Out = true;
CH_Command_Out = '\0';
debugPrintMove(Bat_move, "Bat_move");
debugPrintMove(Resp_move, "Resp_move");
debugPrintInstance(Bat_inst, "Bat_inst");
debugPrintInstance(Resp_inst, "Resp_inst");
Serial.println("Sem Tosse"); //Retirar quando estiver implementado a tosse
```

```
if(1){state = 4;}
```

```
break;
```

case 14:

```
Serial.println("Comando Mode");
start_comand = 0;
Training_Exame_Finishingtime = nowMs;
Training_Exame_Durantiontime = Training_Exame_Finishingtime -
Training_Exame_Starttime;
if(Training_Exame_Durantiontime > 0){
    Serial.print("Duração do treino: ");
    Serial.print(Training_Exame_Durantiontime/1000.0f);
    Serial.println(" Segundos");
}
Training_Exame_Finishingtime = 0;
Training_Exame_Starttime = 0;
LineReady_Out = true;
```

```
if(1){state = 15;}
```

```
break;
```

case 15:

```
LineReady_Out = false;
```

```
if(CH_Command_IN == 'a'){state = 16;}
if(CH_Command_IN == 'h'){state = 17;}
if(CH_Command_IN == 'r'){state = 18;}
if(CH_Command_IN == 'b'){state = 19;}
if(CH_Command_IN == 's'){state = 20;}
break;
```

case 16:

```
Serial.println("Comando Mode ALL");
LineReady_Out = true;
CH_Command_Out = '\0';
Bat_inst.active = true;
Resp_inst.active = true;
//Tosse_inst.active = true;
```



```
if(1){state = 21;}
    break;

case 17:
    Serial.println("Comando Mode Human (Bat+Resp)");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 18:
    Serial.println("Comando Mode Respiração");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 19:
    Serial.println("Comando Mode Batimento");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 20:
    Serial.println("Comando Mode Static");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 21:
    Serial.println("want to start de cicle? y/n");
```

```

    LineReady_Out = false;

    if(1){state = 22;}
    break;

    case 22:
        if(CH_Command_IN == 'y'){state = 2;}
        if(CH_Command_IN == 'n'){state = 3;}
        break;

    }
    return;
}
/*
void setup(){
    Serial.begin(115200);
    setup_counter++;
    Serial.print("setup chamado: ");
    Serial.println(setup_counter);

}

void loop(){
    //Serial.println("beep");
    //delay(500);
}
*/

void setup() {

    Serial.begin(115200);

    setup_counter++;
    Serial.print("setup chamado: ");
    Serial.println(setup_counter);

    Serial.println("Teste LED RGB");

    pixel.begin();
    pixel.clear();
    pixel.show();

    ESP32PWM::allocateTimer(0);
    ESP32PWM::allocateTimer(1);
    ESP32PWM::allocateTimer(2);
    ESP32PWM::allocateTimer(3);

    bool servo_ok = attach_servos();

```

```

delay(1000);
if (servo_ok) {
    Serial.println("Servos attached successfully");
} else {
    Serial.println("Error attaching servos");
}

//-----Inicialização do movimento de teste-----
    Iniciate_Bat_Move();
    Iniciate_Resp_Move();
    Iniciate_Bat_Instance();
    Iniciate_Resp_Instance();

    //Serial.println("Setup completo, pronto para correr Sequence.");

}

void loop() {
    //Serial.println("beep");
    //delay(1000);

    static boolean trig_serial_IN; static boolean trig_serial_OUT;
    float actual_time = millis();
    float tempo_inicial = millis();

    unsigned long now1 = millis();
        trig_serial_OUT = trig_display_fsm(actual_time,1000);
        FSM_Serial_reader(now1);
        FSM_Serial_Command(now1);

    unsigned long now2 = millis();
        // --- Apenas um comentario temporario, -> tenho que ativar denovo isto mais tarde!!
        FSM_Motion_Update(start_comand , Bat_inst, now2);
        unsigned long now3 = millis();
        Intermed_position(start_comand ,Bat_inst, now3, Lerp_Batim_OUT, trig_serial_IN);
        FSM_Motion_Update(start_comand , Resp_inst, now2);
        Intermed_position(start_comand ,Resp_inst, now3, Lerp_Respi_OUT, trig_serial_IN);

    bool ik1 = Fusao_plus_IK_D1(Lerp_Respi_OUT, Lerp_Batim_OUT);
        bool ik2 = Fusao_plus_IK_D2(Lerp_Respi_OUT, Lerp_Batim_OUT);

    if (ik1 && ik2) {
        move_servos(delta_1_Cfg, delta_2_Cfg);
    }

    float tempo_final = millis();
    float demora = tempo_final - tempo_inicial;
    Clock_loop++;

```

```
//Serial.print("Clock loop: ");
// Serial.println(Clock_loop);
if(trig_serial_IN == 1 && Safe_comand == 1){

    Serial.print("Estado da leitura:");
    Serial.println(state_test);
    Serial.print("Estado da Interpretação:");
    Serial.println(state_test2);

}
trig_serial_IN = trig_serial_OUT;
LineReady_IN=LineReady_Out;
CH_Command_IN=CH_Command_Out;
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
//ST_Command_IN=ST_Command_Out;
}
```

Falta ainda a definição da parte do Serial_comand, mas o que é que achas até agora?

O que tens até agora está coerente com o que querias:

- Modo S continua a ler um único char (CH_Command_Out) para comandos simples. ^[124]
- Modo M lê uma linha inteira para ST_BUS, copia só os caracteres úteis para ST_Command_Out e sinaliza isso via LineReady_IN/LineReady_Out e CH_Command_IN/OUT. ^[124]
- No loop, fazes a ponte: LineReady_IN = LineReady_Out; CH_Command_IN = CH_Command_Out; strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));, o que sincroniza a FSM de comandos com o resultado da FSM de leitura numa cadência por ciclo de loop(). ^[124]

Alguns pontos concretos:

1. Leitura em modo M (FSM_Serial_reader)

- O estado 6, como está, ainda guarda o '\n' em ST_BUS (porque lê tudo e só depois usas maxCopy = counter-1).
- O maxCopy = counter-1 faz com que o '\n' não seja copiado para ST_Command_Out, o que cumpre o que queres. ^[124]
- Se estiveres ok com o '\n' ficar apenas no ST_BUS (buffer interno), então este comportamento está aceitável para já.

2. Cópia para ST_Command_IN no loop

- Esta linha está correta e segura:

```
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';
```

- No código que colaste, falta só a linha do '\0' no fim para garantir terminação, se quiseres ser rigoroso. ^[124]

3. Integração com FSM_Serial_Command

- Neste momento, a FSM de comandos ainda só usa CH_Command_IN para modos S, P, L, M.
- O próximo passo será: quando estiveres em modo M, usar ST_Command_IN como string completa (por exemplo "M D R x[^39_4][0.1]") e os novos estados 23+ (que desenhasse) para interpretar essa string:
 - detetar d/D
 - escolher movimento (B/R/T) → currentMove
 - procurar [] e usar atoi/atof para atualizar currentMove ou currentMove->X[...], etc. ^[124]

4. Estrutura geral

- Ter MotionStorage + MotionInstance + currentMove/currentInst global já te prepara para não duplicar estados quando adicionares tosse ou outros movimentos. ^[124]
- A separação de responsabilidades está clara:
 - FSM_Serial_reader → só IO da porta série
 - FSM_Serial_Command → só lógica de comandos e controlo de movimentos
 - FSM_Motion_Update / Intermed_position → só cinemática/movimento

Se quiseres, no próximo passo podemos pegar numa string concreta, por exemplo:

```
M d r p[4.0]
```

e escrever a lógica dos estados 23–36 da FSM_Serial_Command para:

- ler d
- ver r (Respiração)
- encontrar o [4.0]
- fazer Resp_move.period = atof(...)
- chamar Resp_move.updateDtMs().

O que achas de começarmos por esse caso mais simples (um único float para period) antes de entrarmos nos D grandes com arrays de pontos?

✱

agora tenta implementar aquela estrutura que te tinha mostrado mais cedo, para a FSM Serial_command

Vamos focar só na parte nova da FSM_Serial_Command, usando exatamente a estrutura do teu desenho (estados 23–36) e assumindo um caso simples: comando em modo M para alterar uma variável float única (por exemplo Resp_move.period) com uma string tipo:

```
M d r p[4.0]
```

Ou seja:

- `d` → single value (não precisa parar ciclos)
- `r` → movimento Respiração (`Resp_move`)
- `p[...]` → indica que o valor é para period

Vou escrever só os novos casos da FSM, encaixados depois do estado 22, baseados no teu diagrama. A parte de ligar/ligar FSMs continua igual.

Novos estados na FSM_Serial_Command (23–36)

Assume que:

- `ST_Command_IN` contém a string completa, sem `\n`, por exemplo `"d r p[4.0]"`.
- `m` é um índice global/estático para percorrer `ST_Command_IN`.
- `currentMove` / `currentInst` já estão declarados.

Acrescenta no topo do ficheiro (zona global):

```
int m = 0;           // índice para ST_Command_IN
char ST_copy[32];    // buffer temporário para o valor entre []
```

E na FSM:

```
void FSM_Serial_Command(unsigned long nowMs){
    static short int state;
    state_test2 = state;

    switch (state) {

        // ... os teus estados 0-22 como já tens ...

        case 22:
            if (CH_Command_IN == 'y') { state = 2; }
            if (CH_Command_IN == 'n') { state = 3; }
            break;

        // ----- NOVO BLOCO: modo M / string -----

        case 23:
            // Entrar no modo string: ST_Command_IN já está cheia
            // Aqui assumes que já decidiste que isto é comando 'M' em algum estado
            m = 0;
            LineReady_Out = true;
            state = 24;
            break;
```

```

case 24:
    LineReady_Out = false;

    // Primeiro char de ST_Command_IN decide 'd' ou 'D'
    if (ST_Command_IN[m] == 'd' || ST_Command_IN[m] == 'D') {
        char modeChar = ST_Command_IN[m];
        m++; // avança para próxima letra (id do movimento)

        // estado 25/26/27/28/29/30 no teu desenho
        state = 26; // vamos já tratar o id do movimento
    } else {
        // não é d/D -> comando inválido, volta ao menu
        state = 4;
    }
    break;

case 26:
    // ST_Command_IN[m] deve ter 'r', 'b', 't', etc.
    if (ST_Command_IN[m] == 'r' || ST_Command_IN[m] == 'R') {
        currentMove = &Resp_move;
        currentInst = &Resp_inst;
    } else if (ST_Command_IN[m] == 'b' || ST_Command_IN[m] == 'B') {
        currentMove = &Bat_move;
        currentInst = &Bat_inst;
    } else {
        // por agora, se não for nenhum conhecido, aborta
        currentMove = nullptr;
        currentInst = nullptr;
        state = 4;
        break;
    }

    m++; // avança para próxima letra, que será a variável (ex: 'p')

    state = 31; // ir procurar '[' depois da letra de variável
    break;

case 31:
    // saltar espaços e qualquer coisa até encontrar a letra da variável
    while (ST_Command_IN[m] == ' ') {
        m++;
    }

    // Aqui ST_Command_IN[m] deve ser 'p' (period) ou outra letra
    if (ST_Command_IN[m] == 'p' || ST_Command_IN[m] == 'P') {
        m++; // avança depois de 'p'
        state = 32;
    } else {
        // para já, desconhecido -> aborta
        state = 4;
    }
    break;

case 32:
    // andar até ao '['
    while (ST_Command_IN[m] != '[' && ST_Command_IN[m] != '\0') {

```

```

        m++;
    }

    if (ST_Command_IN[m] == '[') {
        m++; // salta o '[' e entra na cópia do valor
        state = 35;
    } else {
        // não encontrou '[' -> aborta
        state = 4;
    }
}
break;

case 35:
{
    // copiar o conteúdo entre '[' e ']' para ST_copy
    int c = 0;
    while (ST_Command_IN[m] != ']' &&
           ST_Command_IN[m] != '\0' &&
           c < (int)sizeof(ST_copy) - 1) {

        ST_copy[c++] = ST_Command_IN[m++];
    }
    ST_copy[c] = '\0'; // termina a string

    if (ST_Command_IN[m] == ']') {
        m++; // salta o ']'
    }

    // agora ST_copy tem, por exemplo, "4.0"
    if (currentMove != nullptr) {
        float value = atof(ST_copy); // converte string -> float

        // para este primeiro exemplo, assumimos que 'p' significa period
        currentMove->period = value;
        currentMove->updateDtMs();
    }

    LineReady_Out = true;
    // voltar ao menu principal
    state = 4;
}
break;

// -----
}
}

```

Isto implementa, passo a passo:

- estado 23: entra no modo string
- 24: verifica d/D
- 26: escolhe Resp_move ou Bat_move (podes expandir para Tosse depois)
- 31: identifica a variável p (period)

- 32: procura '['
- 35: copia o valor entre [], faz atof e escreve em currentMove->period, chamando updateDtMs().

Agora é a tua vez: olhando para estes estados, consegues apontar em qual deles queres, mais tarde, tratar a versão “string grande” (D + vários [...] para arrays) em vez desse caso simples de um único p[...]?

#include <Arduino.h>

```
#include <Adafruit_NeoPixel.h>
#include <ESP32Servo.h>
#include <math.h>

// declarar no topo do ficheiro (globais)
int last_delta2_s1 = 0;
int last_delta2_s2 = 0;
int last_delta2_s3 = 0;

struct Manipulador_Config {
    // geometria
    float L1;
    float L2;
    float L3;
    float servoOffsetX;
    float servoOffsetZ;
    float angleMin;
    float angleMax;
    float rotationOffsetZ; // offset global de rotação da base

    // calibração de PWM
    int servo1MinPulse;
    int servo1MaxPulse;

    int servo2MinPulse;
    int servo2MaxPulse;

    int servo3MinPulse;
    int servo3MaxPulse;

    int theta1MinPulse;
    int theta1MaxPulse;

    int theta2MinPulse;
    int theta2MaxPulse;

    int theta3MinPulse;
    int theta3MaxPulse;
```

```

// estado atual (pulsos gerados pela IK)
int servo1Pulse;
int servo2Pulse;
int servo3Pulse;
};

Manipulador_Config delta_1_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -15.6f * PI / 180.0f, //-75 degrees in radians

    // calibração de PWM
    .servo1MinPulse = 520,
    .servo1MaxPulse = 2520,

    .servo2MinPulse = 550,
    .servo2MaxPulse = 2550,

    .servo3MinPulse = 560,
    .servo3MaxPulse = 2560,

    // estado atual (pulsos gerados pela IK)
    .servo1Pulse = 0,
    .servo2Pulse = 0,
    .servo3Pulse = 0
};

Manipulador_Config delta_2_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -44.4f * PI / 180.0f, //-45 degrees in radians

    // calibração de PWM
    .servo1MinPulse = 460,
    .servo1MaxPulse = 2460,

```



```

12.099, 8.889, 6.173, 3.951, 2.222, 0.988, 0.247};
float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000};
float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000};

//const int N_test2 = 18;
const int N_test2 = 10;
float T_period2 = 4.0f;
float T_pausedt2 = 0.8f;
//Link lengths (mm)
#define L1 35
#define L2 60
#define L3 15

#define END_EFFECTOR_Z_OFFSET 0
#define SERVO_OFFSET_X 32
#define SERVO_OFFSET_Y 0
#define SERVO_OFFSET_Z (0.0 + END_EFFECTOR_Z_OFFSET)
//#define SERVO_OFFSET_Z_INVERTED -293

//#define SERVO_ANGLE_MIN (100.0f * PI / 180.0f)
//#define SERVO_ANGLE_MAX (190.0f * PI / 180.0f)

#define SERVO_ANGLE_MIN 0.78539816339744830961566084581988f //45 degrees
#define SERVO_ANGLE_MAX 3.9269908169872415480783042290994f //225 degrees

#define ROTATION_OFFSET_Z (-75.0f * PI / 180.0f) //0.26179938779914943f (15° em radianos =
π/12):

#define ROTATION_OFFSET_Z_Delta1 (-75.0f * PI / 180.0f)
#define ROTATION_OFFSET_Z_Delta2 (-45.0f * PI / 180.0f)

#define MIN 'm'
#define MAX 'M'

//Servo microsecond pulse limits (Calibration)-----Valores normais: Min = 500 ; Max = 2500]-----
-----
#define SERVO_1_MIN 620
#define SERVO_1_MAX 2520
#define SERVO_2_MIN 615
#define SERVO_2_MAX 2480
#define SERVO_3_MIN 620
#define SERVO_3_MAX 2550
//Servo_1 -> theta3 min 550, max 2450
//Servo_2 -> theta1 min 545, max 2410
//Servo_3 -> theta2 min 520, max 2450

// variáveis globais para valores suavizados
int servo_1_pulse_smooth = 0;

```

```

int servo_2_pulse_smooth = 0;
int servo_3_pulse_smooth = 0;

// fator de suavização [0 - 1]. tipo 0.2 para começar
const float SERVO_SMOOTH_ALPHA = 0.2f;

#define SERVO_4_MIN 550
#define SERVO_4_MAX 2400
#define SERVO_5_MIN 550
#define SERVO_5_MAX 2400
#define SERVO_6_MIN 550
#define SERVO_6_MAX 2400

// variáveis globais para valores suavizados
int servo_4_pulse_smooth = 0;
int servo_5_pulse_smooth = 0;
int servo_6_pulse_smooth = 0;
//-----

#define INVERTED -1

// Máximo de pontos por movimento (ajusta mais tarde se precisares)
const int MAX_POINTS = 100;

// Armazenamento Bruto: dados completos de um movimento - Será utilizada para armazenar
os movimentos que o Pi enviar, ou movimentos pré-definidos, etc.
struct MotionStorage {
    int  n_Points;           // nº de pontos válidos em X/Y/Z/easing
    float X[MAX_POINTS];     // coordenadas X dos pontos do movimento
    float Y[MAX_POINTS];     // coordenadas Y dos pontos do movimento
    float Z[MAX_POINTS];     // coordenadas Z dos pontos do movimento
    float period;
    float T_pause;
    float dtMs;              // duração do ciclo em segundos
    float dt_pauseMs;        // duração da pausa entre ciclos em segundos
    int  startIndex; // índice de ponto que será o "0,0,0" lógico
    bool bidirectional; // true = vai-e-vem, false = ciclo fechado
    int  N_pontos ; // nº de segmentos por ciclo

float getDtMs(){
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_pontos = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinhas
            N_pontos = n_Points;
        }
    }
}

```

```

        return (period * 1000.0f) / (float)N_pontos;
    }
    return 0.0f;
}

void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}

float getD_PauseMs() const {
    if (n_Points > 1 && T_pause > 0.0f) {
        return (T_pause * 1000.0f); // só lê nPoints e T_pause
    }
    return 0.0f;
}

void updateDt_PauseMs() {
    dt_pauseMs = getD_PauseMs(); // aqui sim, mudas dt_pauseMs
}

};

// Armazenamento temporário: runtime que executa um MotionStorage
struct MotionInstance {
    MotionStorage* def; // ponteiro para MotionStorage (a definição do Armazenamento Bruto
)
    int      currentIndex; // índice i (início do segmento i -> i+1) -> índice atual no movimento
    unsigned int  segmentStartMs; // timestamp do início do segmento atual
        // -> é uma variável importante para controlar a velocidade do movimento,
ou seja, quando é que deve avançar para o próximo ponto do movimento
    bool      active; // se este movimento está ativo
    bool      Executing; // se este movimento está em pausa (entre ciclos)
    short int  state;
    unsigned long elapsed_1;
    unsigned long elapsed_2;
    unsigned long elapsed_3;

    int iIndex; // novo: índice i para a interpolação
    int jIndex; // novo: índice j para a interpolação
    int Direction; // novo: direção do movimento (1 ou -1)
};

// -----Temporario-----
MotionStorage Bat_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Bat_inst;
MotionStorage Resp_move; //Declaração de objetos para utilizar com as structures de
movimento
MotionInstance Resp_inst;

```

```
MotionStorage Tosse_move; //Declaração de objetos para utilizar com as structures de movimento
```

```
MotionInstance Tosse_inst;
```

```
MotionStorage* currentMove = nullptr;
```

```
MotionInstance* currentInst = nullptr;
```

```
void Inicie_Bat_Move() { //Serve para copiar os valores definidos mais acima para o objeto Bat_move, que é do tipo MotionStorage.
```

```
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter que copiar os valores manualmente para o Bat_move cada vez que quiseres testar algo.
```

```
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM ou do PI
```

```
    Bat_move.n_Points = N_test;
```

```
    Bat_move.period = T_period;
```

```
    Bat_move.T_pause = T_pausedt;
```

```
    for (int i = 0; i < N_test; i++) {
```

```
        Bat_move.X[i] = X_test[i];
```

```
        Bat_move.Y[i] = -1 * Y_test[i];
```

```
        Bat_move.Z[i] = Z_test[i];
```

```
    }
```

```
    Bat_move.startIndex = 0;
```

```
    Bat_move.bidirectional = false; // Define o movimento como vai-e-vem
```

```
    Bat_move.updateDtMs();
```

```
    Bat_move.updateDt_PauseMs();
```

```
}
```

```
void Inicie_Resp_Move() { //Serve para copiar os valores definidos mais acima para o objeto Resp2_move, que é do tipo MotionStorage.
```

```
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter que copiar os valores manualmente para o Resp2_move cada vez que quiseres testar algo.
```

```
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM ou do PI
```

```
    Resp_move.n_Points = N_test2;
```

```
    Resp_move.period = T_period2;
```

```
    Resp_move.T_pause = T_pausedt2;
```

```
    for (int i = 0; i < N_test2; i++) {
```

```
        Resp_move.X[i] = X_test2[i];
```

```
        Resp_move.Y[i] = -1 * Y_test2[i];
```

```
        Resp_move.Z[i] = Z_test2[i];
```

```
    }
```

```
    Resp_move.startIndex = 5;
```

```
    Resp_move.bidirectional = true; // Define o movimento como vai-e-vem
```

```
    Resp_move.updateDtMs();
```

```
    Resp_move.updateDt_PauseMs();
```

```
}
```

```

void Inicie_Bat_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Bat_inst.def      = &Bat_move;
    Bat_inst.currentIndex = 0;
    Bat_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Bat_inst.active     = true;
    Bat_inst.Executing   = 0; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Bat_inst.state       = 0;
    Bat_inst.elapsed_1   = 0;
    Bat_inst.elapsed_2   = 0;
    Bat_inst.elapsed_3   = 0;
    Bat_inst.iIndex      = 0;
    Bat_inst.jIndex      = 0;
    Bat_inst.Direction    = 1; // novo: inicializa a direção como positiva
}

void Inicie_Resp_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Resp_inst.def      = &Resp_move;
    Resp_inst.currentIndex = 0;
    Resp_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Resp_inst.active     = true;
    Resp_inst.Executing   = false; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Resp_inst.state       = 0;
    Resp_inst.elapsed_1   = 0;
    Resp_inst.elapsed_2   = 0;
    Resp_inst.elapsed_3   = 0;
    Resp_inst.iIndex      = 0;
    Resp_inst.jIndex      = 0;
    Resp_inst.Direction    = 1; // novo: inicializa a direção como positiva
}

//-----Fim Temporario-----

struct Coordinate {
    float X;
    float Y;
    float Z;
};

int Clock_loop = 0;
const float pi = 3.141592653;

// -----Declaração das variaveis-----
void LED_LOOP(); //Declaração de funções
void Servo_test();
//-----Novas-----
float boundFloat(float, float, float);

```



```

bool attach_servos(void);
bool inverse_kinematics_1(float, float, float, float, float&);
bool inverse_kinematics_2(float, float, float, float, float&);
bool inverse_kinematics_3(float, float, float, float, float&);
bool inverse_kinematics(Manipulador_Config& cfg, float, float, float);
void move_servos(const Manipulador_Config& d1, const Manipulador_Config& d2);
double mapNumber(double x, double in_min, double in_max, double out_min, double
out_max);
int roundMapNumber(double x, double in_min, double in_max, double out_min, double
out_max);
double degToRads(double deg);
double radsToDeg(double rads);
void debugPrintMove(const MotionStorage& move, const char* name);
void debugPrintInstance(const MotionInstance& inst, const char* name);
float lerp(float a, float b, float t);
float applyEasing(float alpha, char mode);
float spline3(float p0, float p1, float p2, float t);
void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs,
Coordinate& xyz, boolean trig_serial);
bool Fusao_plus_IK_D1(const Coordinate& respi, const Coordinate& batim);
bool Fusao_plus_IK_D2(const Coordinate& respi, const Coordinate& batim);
//-----FSM-----
void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs);
void FSM_Serial_reader(unsigned long nowMs);
void FSM_Serial_Command(unsigned long nowMs);
void FSM_Main(boolean start, MotionInstance& inst, unsigned long nowMs);
//-----

//--Declaração dos comandos de leitura
//char commands
#define Scomand_start_Program 's'
#define Scomand_stop_Program 'p'
#define Scomand_GRIPPER 3
#define Scomand_ABSOLUTE_CARTESIAN_LINEAR 4
#define Scomand_SET_PROGRAM_ARRAY 5
#define Scomand_REQUEST_READY_FLAG 6

//String commands
#define Mcomand_JOG_X 'x'
#define Mcomand_JOG_Y 'y'
#define COMMAND_JOG_Z 'z'
#define COMMAND_GRIPPER_ASCII 'g'
#define COMMAND_STATUS 's'
#define COMMAND_REPORT_COMMANDS 'r'
#define COMMAND_ADD_POSITION 'p'
#define COMMAND_SET_STEP_DELAY 'd'
#define COMMAND_SET_STEP_INCREMENT 'i'

```

```

#define COMMAND_CLEAR_ARRAY 'c'
#define COMMAND_EXECUTE 'e'
#define COMMAND_EXECUTE_JOINT 'j'
#define COMMAND_EXECUTE_TIME 't'
#define COMMAND_SET_US_INCREMENT_LINEAR 'u'
#define COMMAND_SET_US_INCREMENT_JOINT 'U'
#define COMMAND_STEP_FORWARD '>'
#define COMMAND_STEP_BACKWARD '<'
#define COMMAND_JUMP_TO_START '['
#define COMMAND_JUMP_TO_END ']'
#define COMMAND_EDIT_ARRAY 'P'
#define COMMAND_ADD_DELAY 'D'
#define COMMAND_MOVE_HOME 'h'
#define COMMAND_PRINT_FILE 'f'
#define COMMAND_PING_PONG 'o'
#define COMMAND_SERVO_CALIBRATION 'C'
#define COMMAND_SET_SERVO 'S'

//EEPROM commands
#define Ecomand_SET_LINK_2 'L'
#define Ecomand_SET_END_EFFECTOR_TYPE 'E'
#define Ecomand_SET_AXIS_DIRECTION 'A'
#define Ecomand_SET_HOME_X 'X'
#define Ecomand_SET_HOME_Y 'Y'
#define Ecomand_SET_HOME_Z 'Z'
#define Ecomand_SET_HOME_GRIPPER 'G'
#define Ecomand_SET_GRIPPER_MIN_MAX 'M'

//EEPROM addresses for the delta robots configuration
#define EEPROM_ADDRESS_LINK_2 0
#define EEPROM_ADDRESS_END_EFFECTOR_TYPE 1
#define EEPROM_ADDRESS_AXIS_DIRECTION 2
#define EEPROM_ADDRESS_Z_OFFSET 3
#define EEPROM_ADDRESS_HOME_X 7
#define EEPROM_ADDRESS_HOME_Y 11
#define EEPROM_ADDRESS_HOME_Z 15
#define EEPROM_ADDRESS_HOME_GRIPPER 19
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MIN 21
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MAX 23
#define EEPROM_ADDRESS_GRIPPER_CLAW_MIN 25
#define EEPROM_ADDRESS_GRIPPER_CLAW_MAX 27
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MIN 29
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MAX 31

//Delta1 -----
Servo mg90s_1; // cria objeto servo

```

```

Servo mg90s_2; // cria objeto servo
Servo mg90s_3; // cria objeto servo

//Delta2 -----
Servo mg90s_4; // cria objeto servo
Servo mg90s_5; // cria objeto servo
Servo mg90s_6; // cria objeto servo

volatile boolean start_comand = 0;

Coordinate Lerp_Respi_OUT;
Coordinate Lerp_Batim_OUT;
Coordinate IK2_IN;
Coordinate home_position;

float servo_offset_z = SERVO_OFFSET_Z;

//-----Declaração da localização dos pinos para cada objeto -----
//-----Servos-----
#define SERVO_PIN_1 12 //Servo_1 -> theta3
#define SERVO_PIN_2 11 //Servo_2 -> theta1
#define SERVO_PIN_3 10 //Servo_3 -> theta2

#define SERVO_PIN_4 36 // Servo_4 -> theta3
#define SERVO_PIN_5 37 // Servo_5 -> theta1
#define SERVO_PIN_6 38 // Servo_6 -> theta2

//-----Led_Informação-----
#define LED_PIN 48
#define LED_COUNT 1

//-----
bool stepDirection = false;
Adafruit_NeoPixel pixel(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

char Mode = 'S'; // 'S' = Single, 'M' = Manual
char CH_Command_Out = '\0';
char CH_Command_IN = '\0';
char ST_Command_Out[100];
char ST_Command_IN[100];
int command_counter = 0;
bool LineReady_IN = false;
bool LineReady_Out = false;

boolean trig_display_fsm(float at, short int Td){
static short int state; //state internal to the state machine
static float pt; //var. internal to the state machine
boolean trig; // output

```

```

switch (state) {
case 0:
if (at-pt >= Td) {state = 1;}
trig = 0; // output
return trig;
case 1:
if (1) {state = 0;}
trig = 1; pt = at; // output
return trig;
}
return trig;
}

int setup_counter = 0;

int state_test = 0;
int state_test2 = 0;
float Training_Exame_Starttime = 0;
float Training_Exame_Finishingtime = 0;
float Training_Exame_Durantiontime = 0;
int Safe_comand = 0;

void FSM_Serial_reader(unsigned long nowMs){
static short int state;
static int counter = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
counter = 0;
CH_Command_Out = '\0';

if(Serial.available() > 0 && Mode == 'S'){state = 1;}
if(Serial.available() > 0 && Mode == 'M'){state = 6;}
break;

```

```

case 1:
    CH_Bus = Serial.read();
    counter++;

if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

case 8:

int maxCopy = counter-1; // aqui definimos a quantidade de caracteres a copiar, não em termos
do indice mas sim a quantidade
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {

```

```

    maxCopy = sizeof(ST_Command_Out) - 1;
}
strncpy(ST_Command_Out, ST_BUS, maxCopy);
ST_Command_Out[maxCopy] = '\0';
counter = 0;

break;
}
return;
}

```

```

void FSM_Serial_Command(unsigned long nowMs){

```

```

    static short int state;

```

```

    state_test2 = state;

```

```

    static int n = 0;

```

```

    static int m = 0;

```

```

    static char ST_copy[100];

```

```

    /*

```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar

- Condições de transição por hierarquia

```

    */

```

```

    //Serial.print("Estado atual: ");

```

```

    //Serial.println(inst.state);

```

```

    switch (state) {

```

case 0:

```

        Serial.println("Escolha um comando:");

```

```

    if(1){state = 1;}

```

```

    break;

```

case 1:

```

        LineReady_Out = false;

```

```

    if(CH_Command_IN == 'S' || CH_Command_IN == 's'){state = 2;}

```

```

        if(CH_Command_IN == 'P' || CH_Command_IN == 'p'){state = 3;}

```

```

        if(CH_Command_IN == 'L' || CH_Command_IN == 'l'){state = 5;}

```

```

        if(CH_Command_IN == 'M' || CH_Command_IN == 'm'){state = 14;}

```

```

        if(CH_Command_IN == 'W' || CH_Command_IN == 'w'){state = 23;}

```

```

    break;

```

case 2:

```

        Serial.println("Ativo");

```

```

        start_comand = 1;

```

```

        Training_Exame_Starttime = nowMs;

```

```

    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 0, 0));
    pixel.show();

if (1) { state = 4;}
    break;

case 3:
    Serial.println("Desativo");
    start_comand = 0;
    Training_Exame_Finishingtime = nowMs;
    Training_Exame_Durantiontime = Training_Exame_Finishingtime -
Training_Exame_Starttime;
    if(Training_Exame_Durantiontime > 0){
        Serial.print("Duração do treino: ");
        Serial.print(Training_Exame_Durantiontime/1000.0f);
        Serial.println(" Segundos");
    }
    Training_Exame_Finishingtime = 0;
    Training_Exame_Starttime = 0;
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 0, 0));
    pixel.show();

    if (1) { state = 4;}
    break;

case 4:
    Serial.println("Escolha um comando:");
    LineReady_Out = false;

if(1){state = 1;}
    break;

case 5:
    Serial.println("ler");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 0));
    pixel.show();

if(1){state = 6;}
    break;

case 6:
    LineReady_Out = false;

```

```

if(CH_Command_IN == '1'){state = 7;}
    if(CH_Command_IN == '2'){state = 8;}
    if(CH_Command_IN == '3'){state = 9;}
    if(CH_Command_IN == 'r'){state = 10;}
    if(CH_Command_IN == 'b'){state = 11;}
    if(CH_Command_IN == 't'){state = 12;}
    if(CH_Command_IN == 'a'){state = 13;}
    break;

case 7:
    Serial.println("Comando L1");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 255));
    pixel.show();
    Safe_comand = 1;

if(1){state = 4;}
    break;

case 8:
    Serial.println("Comando L2");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 255, 255));
    pixel.show();
    Safe_comand = 0;

if(1){state = 4;}
    break;

case 9:
    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Serial.println("Sem nada"); //Retirar quando estiver implementado

if(1){state = 4;}
    break;

case 10:
    Serial.println("Comando Lr");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    debugPrintMove(Resp_move, "Resp_move");
    debugPrintInstance(Resp_inst, "Resp_inst");

```



```
    if(1){state = 4;}
```

```
    break;
```

case 11:

```
    Serial.println("Comando Lb");
```

```
    LineReady_Out = true;
```

```
    CH_Command_Out = '\0';
```

```
    debugPrintMove(Bat_move, "Bat_move");
```

```
    debugPrintInstance(Bat_inst, "Bat_inst");
```

```
    if(1){state = 4;}
```

```
    break;
```

case 12:

```
    Serial.println("Comando Lt");
```

```
    LineReady_Out = true;
```

```
    CH_Command_Out = '\0';
```

```
    //debugPrintMove(Bat_move, "Tosse_move"); //ainda é preciso implementar
```

```
    //debugPrintInstance(Bat_inst, "Tosse_inst"); //ainda é preciso implementar
```

```
    Serial.println("Sem nada"); //Retirar quando estiver implementado
```

```
    if(1){state = 4;}
```

```
    break;
```

case 13:

```
    Serial.println("Comando LA");
```

```
    LineReady_Out = true;
```

```
    CH_Command_Out = '\0';
```

```
    debugPrintMove(Bat_move, "Bat_move");
```

```
    debugPrintMove(Resp_move, "Resp_move");
```

```
    debugPrintInstance(Bat_inst, "Bat_inst");
```

```
    debugPrintInstance(Resp_inst, "Resp_inst");
```

```
    Serial.println("Sem Tosse"); //Retirar quando estiver implementado a tosse
```

```
    if(1){state = 4;}
```

```
    break;
```

case 14:

```
    Serial.println("Comando Mode");
```

```
    start_comand = 0;
```

```
    Training_Exame_Finishingtime = nowMs;
```

```
    Training_Exame_Durantiontime = Training_Exame_Finishingtime -
```

```
    Training_Exame_Starttime;
```

```
    if(Training_Exame_Durantiontime > 0){
```

```
        Serial.print("Duração do treino: ");
```

```
        Serial.print(Training_Exame_Durantiontime/1000.0f);
```

```
        Serial.println(" Segundos");
```

```

    }
    Training_Exame_Finishingtime = 0;
    Training_Exame_Starttime = 0;
    LineReady_Out = true;

if(1){state = 15;}
    break;

case 15:
    LineReady_Out = false;

if(CH_Command_IN == 'a'){state = 16;}
    if(CH_Command_IN == 'h'){state = 17;}
    if(CH_Command_IN == 'r'){state = 18;}
    if(CH_Command_IN == 'b'){state = 19;}
    if(CH_Command_IN == 's'){state = 20;}
    break;

case 16:
    Serial.println("Comando Mode ALL");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = true;

if(1){state = 21;}
    break;

case 17:
    Serial.println("Comando Mode Human (Bat+Resp)");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 18:
    Serial.println("Comando Mode Respiração");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

```

```
if(1){state = 21;}
    break;

case 19:
    Serial.println("Comando Mode Batimento");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 20:
    Serial.println("Comando Mode Static");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 21:
    Serial.println("want to start de cicle? y/n");
    LineReady_Out = false;

if(1){state = 22;}
    break;

case 22:
    if(CH_Command_IN == 'y'){state = 2;}
    if(CH_Command_IN == 'n'){state = 3;}
    break;

case 23:
    Mode = 'M';
    Serial.println("Comando Mode multiple characters");
    LineReady_Out = true;

if(1){state = 24;}
    break;

case 24:
    LineReady_Out = false;
    n = 0;
```

```

if(ST_Command_IN[n] != 'D'){state = 25;}
    if(ST_Command_IN[n] != 'd'){state = 26;}
    break;

case 25:
    start_comand = 0;
    n++;

if(1){state = 27;}
    break;

case 26:
    n++;

if(1){state = 28;}
    break;

case 27:
    if(ST_Command_IN[n] == 'R' || ST_Command_IN[n] == 'r'){state = 28;}
    if(ST_Command_IN[n] == 'B' || ST_Command_IN[n] == 'b'){state = 29;}
    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 30;}
    break;

case 28:
    currentMove = &Resp_move;
    currentInst = &Resp_inst;

if(1){state = 31;}
    break;

case 29:
    currentMove = &Bat_move;
    currentInst = &Bat_inst;

if(1){state = 31;}
    break;

case 30:
    currentMove = &Tosse_move;
    currentInst = &Tosse_inst;

if(1){state = 31;}
    break;

case 31:
    n++;

if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;} //Periodo
    if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;} //Pausa

```

```

        if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;} //Movimento
        break;

case 32:
    n++;

if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
    break;

case 33:
    //L
    break;

case 34:
    //L
    break;

case 35:
    //L
    n++;
    ST_copy[m]=ST_Command_IN[n];
    m++;

if(ST_Command_IN[n] != 'I'){state = 35;}
    if(ST_Command_IN[n] == 'I'){state = 36;}
    break;

case 36:
    //L
    currentMove->period=atof(ST_copy);
    memset(ST_copy,0,sizeof(ST_copy));
    LineReady_Out = true;

if(1){state = 4;}
    break;
}

return;
}

void setup() {

Serial.begin(115200);

setup_counter++;
Serial.print("setup chamado: ");
Serial.println(setup_counter);

Serial.println("Teste LED RGB");

```

```

pixel.begin();
pixel.clear();
pixel.show();

ESP32PWM::allocateTimer(0);
ESP32PWM::allocateTimer(1);
ESP32PWM::allocateTimer(2);
ESP32PWM::allocateTimer(3);

bool servo_ok = attach_servos();

delay(1000);
if (servo_ok) {
    Serial.println("Servos attached successfully");
} else {
    Serial.println("Error attaching servos");
}

//-----Inicialização do movimento de teste-----
    Iniciate_Bat_Move();
    Iniciate_Resp_Move();
    Iniciate_Bat_Instance();
    Iniciate_Resp_Instance();

    //Serial.println("Setup completo, pronto para correr Sequence.");

}

void loop() {
    //Serial.println("beep");
    //delay(1000);

    static boolean trig_serial_IN; static boolean trig_serial_OUT;
    float actual_time = millis();
    float tempo_inicial = millis();

    unsigned long now1 = millis();
        trig_serial_OUT = trig_display_fsm(actual_time,1000);
        FSM_Serial_reader(now1);
        FSM_Serial_Command(now1);

    unsigned long now2 = millis();
        // --- Apenas um comentario temporario, -> tenho que ativar denovo isto mais tarde!!
        FSM_Motion_Update(start_comand , Bat_inst, now2);
        unsigned long now3 = millis();
        Intermed_position(start_comand ,Bat_inst, now3, Lerp_Batim_OUT, trig_serial_IN);
        FSM_Motion_Update(start_comand , Resp_inst, now2);
        Intermed_position(start_comand ,Resp_inst, now3, Lerp_Respi_OUT, trig_serial_IN);

```

```

bool ik1 = Fusao_plus_IK_D1(Lerp_Respi_OUT, Lerp_Batim_OUT);
bool ik2 = Fusao_plus_IK_D2(Lerp_Respi_OUT, Lerp_Batim_OUT);

if (ik1 && ik2) {
    move_servos(delta_1_Cfg, delta_2_Cfg);
}

float tempo_final = millis();
float demora = tempo_final - tempo_inicial;
Clock_loop++;
//Serial.print("Clock loop: ");
// Serial.println(Clock_loop);
if(trig_serial_IN == 1 && Safe_comand == 1){

    Serial.print("Estado da leitura:");
    Serial.println(state_test);
    Serial.print("Estado da Interpretação:");
    Serial.println(state_test2);

}
trig_serial_IN = trig_serial_OUT;
LineReady_IN=LineReady_Out;
CH_Command_IN=CH_Command_Out;
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
//ST_Command_IN=ST_Command_Out;
}

```

O que é que achas agora das duas FSM conforme a estrutura das imagens que te tinha enviado?

O que tens agora está muito próximo do que desenhaste nas FSMs, mas há alguns pontos concretos que vão dar problemas se não ajustares. Vou focar só no que já implementaste (modo w/strings) e nas duas FSMs, não na cinemática.

Vou dividir em 3 partes: leitura (FSM_Serial_reader), comandos (FSM_Serial_Command), e depois perguntar-te um caso de teste.

1. FSM_Serial_reader – modo M

Pontos fortes:

- Modo S continua simples e independente, como antes. ^[125]
- Modo M lê a linha completa para ST_BUS e copia para ST_Command_Out, com maxCopy = counter-1 para evitar o \n, o que funciona porque counter-1 é exatamente a posição do \n. ^[125]

Coisas a ajustar ligeiramente:

- Atualmente guardas o \n dentro de ST_BUS e depois ignoras na cópia; isso é aceitável, mas:

- Se quiseres que ST_BUS seja uma string C válida, podes pôr:

```
ST_BUS[counter-1] = '\0';    // substitui o '\n'
```

antes do strncpy, e depois usar maxCopy = counter-1 na mesma.

- Falta só garantir terminação em ST_Command_IN depois de copiares no loop:

```
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';
```

Com isso, tens as duas strings bem formadas para a FSM de comandos.

2. FSM_Serial_Command – ramo w com estados 23–36

A estrutura geral bate bem com o teu diagrama:^[125]

- 23: entra no modo múltiplo (Mode = 'M')
- 24: lê ST_Command_IN[^41_0] para ver se é 'D' ou 'd'
- 25/26: ajustam n e param movimento (start_comand = 0) para D
- 27–30: decidem Resp/Bat/Tosse e apontam currentMove, currentInst
- 31: escolhe que variável vais alterar (T, P, M)
- 32: tenta procurar '['
- 35: copia o conteúdo entre [e] para ST_copy
- 36: converte e escreve em currentMove->period com atof.

Há alguns erros lógicos/sintáticos que convém corrigir:

a) Condições em 24 estão invertidas

```
case 24:
    LineReady_Out = false;
    n = 0;

    if(ST_Command_IN[n] != 'D'){state = 25;}
    if(ST_Command_IN[n] != 'd'){state = 26;}
    break;
```

- Se ST_Command_IN[^41_0] for 'd', então:
 - 'd' != 'D' → primeira condição é verdadeira → state = 25
 - 'd' != 'd' → segunda é falsa → não vais a 26
- Isto faz com que **nunca entres em 26 para 'd'**.

O que querias é:

- se for 'D' → 25

- se for 'd' → 26

Logo:

```
if (ST_Command_IN[n] == 'D') { state = 25; }
else if (ST_Command_IN[n] == 'd') { state = 26; }
else {
    // comando inválido
    state = 4;
}
```

b) Estado 32 não procura '[' corretamente

```
case 32:
    n++;

    if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
break;
```

- Se ST_Command_IN[n] for '[', entras em 32 de novo, fazes n++ outra vez, e ficas num loop estranho.
- Em vez disso, o fluxo que queres é:
 - antes de 32, já leste a letra da variável (T, P, M)
 - 32 deve andar em frente até encontrar '[', e quando encontrar avançar para o 35.

Algo como:

```
case 32:
    // andar até '['
    while (ST_Command_IN[n] != '[' && ST_Command_IN[n] != '\0') {
        n++;
    }

    if (ST_Command_IN[n] == '[') {
        n++;          // salta o '['
        state = 35;    // vai copiar o valor
    } else {
        // não encontrou '['
        state = 4;
    }
break;
```

c) Estado 35 precisa de reinicializar m e proteger limites

```
case 35:
    n++;
    ST_copy[m]=ST_Command_IN[n];
    m++;
```

```
if(ST_Command_IN[n] != ' '){state = 35;}
if(ST_Command_IN[n] == ' '){state = 36;}
break;
```

Problemas:

- `m` não é reinicializado quando entras neste fluxo; se já tiver lixo de antes, escreves fora da parte que queres.
- Estás a copiar o carácter antes de verificar se é ' ' – ou seja, acabas a pôr ' ' dentro de `ST_copy`.
- Não limitas `m` ao tamanho de `ST_copy`.

Uma versão mais robusta (FSM-friendly) seria:

```
case 35:
    m = 0; // começar sempre do zero quando entras aqui
    // não incrementas n aqui; assume que já estás no primeiro char depois de '['
    while (ST_Command_IN[n] != ' ' &&
           ST_Command_IN[n] != '\0' &&
           m < (int)sizeof(ST_copy) - 1) {

        ST_copy[m] = ST_Command_IN[n];
        m++;
        n++;
    }
    ST_copy[m] = '\0';

    if (ST_Command_IN[n] == ' ') {
        n++; // salta o ' '
        state = 36;
    } else {
        state = 4; // não encontrou ' ' -> aborta
    }
    break;
```

Se quiseres manter o estilo “incrementa `n` e volta ao mesmo state”, podes fazer com um `if/else`, mas o ponto chave é:

- inicializar `m`
- não escrever o ' ' no `ST_copy`
- terminar com `'\0'`.

d) Estado 36: cuidado com `atof(ST_copy)` sem garantir `ST_copy` limpa

Com o ajuste no 35, ficas com:

```
case 36:
    currentMove->period = atof(ST_copy);
    memset(ST_copy, 0, sizeof(ST_copy));
```

```
LineReady_Out = true;
```

```
if(1){state = 4;}  
break;
```

Isto é ok, desde que tenhas a certeza que:

- `currentMove` não é `nullptr` (o estado 28/29/30 tem de ter sido chamado),
- `ST_copy` é uma string C válida terminada com `'\0'`.

3. Pergunta de teste

Para validar e depurar a FSM, vale a pena pegares numa string concreta, por exemplo:

```
W d r t[4.0]
```

e pensar:

- Depois da leitura (`FSM_Serial_reader` em modo M), o que fica exatamente em `ST_Command_Out` e depois em `ST_Command_IN`?
- Passando pelos estados 23–36, quais valores vão ter `n`, `m` e `ST_copy` quando o estado 36 corre?

Se quiseses, podemos fazer esse “dry-run” juntos: dizes qual string exata vais mandar (por exemplo `"W d r t[4.0]\n"`), e seguimos passo a passo pela FSM até ver se o `period` de `Resp_move` é atualizado como queres.

Que string concreta estás a planear usar primeiro para alterar só o `period` da respiração com um único valor?

✱

`strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));`

`ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';`

mas se eu faço isto vou substituir um dos caracteres por `\0`
sendo que quando passo para `ST_comand` a string já está limpa,

Boa observação; vamos separar duas coisas diferentes aqui:

1. O que o `strncpy` faz
2. Por que esse `'\0'` extra no fim normalmente não te estraga a string

1. O que acontece com esse código

```
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));  
ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';
```

- `strncpy` copia **no máximo** `sizeof(ST_Command_IN)` caracteres de `ST_Command_Out` para `ST_Command_IN`. [\[126\]](#) [\[127\]](#)
- Se `ST_Command_Out` for menor que esse tamanho, o resto de `ST_Command_IN` pode ser preenchido com `'\0'` ou lixo, dependendo da implementação. [\[128\]](#) [\[127\]](#)
- Quando fazes `ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';`, estás a garantir que a **última posição** do array destino é `'\0'`.

Sim: se nessa última posição tivesse um carácter útil copiado, ele é substituído por `'\0'`.

Mas isto só acontece se:

- a tua origem (`ST_Command_Out`) tiver **comprimento $\geq$ tamanho do buffer destino**, o que não devia acontecer no teu caso, porque já limitaste o tamanho da string antes (modo M não passa dos 100 chars).

Na prática:

- Se a tua string real tem, por exemplo, 20 chars, e o array tem 100,
 - `strncpy` copia 20 chars,
 - o resto é irrelevante,
 - o `'\0'` que colocas na posição 99 **não substitui nada que te interesse**, porque o fim lógico da string já é numa posição muito antes.

2. No teu caso específico

Tu próprio disseste:

- “quando passo para `ST_Command_Out` a string já está limpa”
- e garantes que **nunca** envias mais de 100 caracteres.

Então, tens duas opções:

1. Mais simples, suficiente para ti

Se tens a certeza absoluta que `ST_Command_Out` vem sempre com `'\0'` antes de 100 caracteres:

```
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));  
// não precisas da linha extra, podes tirar
```

Porque a origem já é string bem terminada e o destino é pelo menos do mesmo tamanho.

2. Versão defensiva (caso geral)

Manténs a linha do `'\0'` no fim para te proteger contra bugs futuros, aceitando que só perdes o último char se alguém, no futuro, meter uma string maior do que devia:

```
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';
```

Aqui, só “substituis um carácter por '\0'” no cenário-limite em que a origem é demasiado grande; é precisamente para evitar que o resto do código leia lixo para além do buffer. ^[129]
^[130]

Portanto:

- em condições normais (strings pequenas, já limpas) esse '\0' extra não te estraga nada;
- se quiseses ser perfeccionista e não tocar em posição nenhuma a mais, podes simplesmente confiar em strncpy porque já limitaste o tamanho da origem lá atrás.

Para fechar: tens preferência por manter a versão “segura” (com o '\0' na última posição, para prevenir bugs futuros) ou queres mesmo ficar com a versão minimalista sem essa linha, confiando no teu limite de 100 caracteres?

✱

Se ST_Command_IN[n] for '[', entras em 32 de novo, fazes n++ outra vez, e ficas num loop estranho.

Em vez disso, o fluxo que queres é:

antes de 32, já leste a letra da variável (T, P, M)

32 deve andar em frente até encontrar '[', e quando encontrar avançar para o 35.

Não o que eu quero naquele estado é chegar até ao '['

por isso ele vai continuar a seguir em frente na string até passar não encontrar '['

eu aqui estou a assumir que a string está toda junta, e como sou eu que controlo as coisas consigo definir que vai haver sempre um '[' depois dos caracteres iniciais todos

Ok, então estás a usar o estado 32 como “scanner” até encontrar o primeiro '[', assumindo que a string é bem formada e que **vai sempre haver** um '[' depois dos caracteres iniciais. Isso faz sentido, e é exatamente isso que o estado deve fazer.

O problema não é a ideia; é só a ordem das operações no código atual:

```
case 32:
    n++;

    if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
    break;
```

Repara no que isto faz:

- Primeiro fazes n++.
- Se o char em ST_Command_IN[n] for '[', ficas em state = 32 outra vez → na próxima iteração voltas a fazer n++ e saltas por cima do '[' sem nunca ir ao 35.

- Só vais ao 35 quando o char **não** é '[', ou seja, exatamente o inverso do que queres.

Se a ideia é “andar até ao '[' e quando o encontrares passas para o 35”, então o comportamento certo é:

- enquanto **não** for '[', vais incrementando n
- quando **for** '[', saís desse estado e vais copiar no 35.

Em estilo FSM “um passo por vez”, fica assim:

```
case 32:
    // avançar um char de cada vez
    n++;

    if (ST_Command_IN[n] == '[') {
        // parámos no '[', podemos ir copiar o valor
        n++;          // opcional: salta o '[' para ficar já no primeiro dígito
        m = 0;        // começar ST_copy do zero
        state = 35;
    } else if (ST_Command_IN[n] == '\0') {
        // chegou ao fim da string sem encontrar '[', aborta
        state = 4;
    } else {
        // ainda não encontraste '[', continua neste estado
        state = 32;
    }
    break;
```

Este faz exatamente o que descreveste:

- “vai continuar a seguir em frente na string até encontrar '['”
- e, como tu assumas que a string é bem formada, eventualmente encontra o '[' e segue para o 35.

A diferença para o teu código original é só esta inversão:

- no teu: '[' → volta ao 32; “não '['” → vai ao 35
- no esperado: “não '['” → volta ao 32; '[' → vai ao 35

Queres tentar ajustar o teu estado 32 desta forma e, a seguir, testarmos mentalmente com uma string concreta tipo `double t[0.4]` para ver como `n` e `m` avançam até chegar ao `atof`?

case 32:

```
// avançar um char de cada vez
n++;

if (ST_Command_IN[n] == '[') {
    // parámos no '[', podemos ir copiar o valor
    n++; // opcional: salta o '[' para ficar já no primeiro dígito
    m = 0; // começar ST_copy do zero
```

```

state = 35;
} else if (ST_Command_IN[n] == '\0') {
// chegou ao fim da string sem encontrar '[', aborta
state = 4;
} else {
// ainda não encontraste '[', continua neste estado
state = 32;
}
break;

```

mas isso é exatamente igual do que definir que caso ST_comand[n]=='[' volta a repetir o estado (pois o estado faz n++, posso também colocar m=0;) e depois no estado seguinte já não vou encontrar '[' por isso avanço para state=35

Boa, estás a pensar bem na mecânica da FSM. A diferença é subtil, mas importante.

O que tens agora (versão tua original, lógica invertida) é:

```

case 32:
    n++;

    if (ST_Command_IN[n] == '[') { state = 32; }
    if (ST_Command_IN[n] != '['){ state = 35; }
    break;

```

Vamos simular com uma string simples (ignorando espaços antes):

```

... t[0.4]
      ^
      n

```

1. Entraste em 32 com n a apontar para o t (por exemplo).
2. n++ → passa a apontar para '['.
3. if (ST_Command_IN[n] == '[') → é verdade → state = 32.
4. Sai do switch, volta ao switch (state) de novo em 32.
5. De novo n++ → agora aponta para '0'.
6. if (ST_Command_IN[n] == '[') → agora é falso.
if (ST_Command_IN[n] != '[') → verdadeiro → state = 35.

Ou seja: na prática, com esta lógica, só passas para 35 quando já estás no primeiro dígito, não no '['. Acontece que isto até pode funcionar, *desde que*:

- chegues ao estado 32 com n a apontar para o carácter anterior ao '[',
- não tenhas mais nenhum '[' na string depois,
- e aceites que entras uma vez em 32, fazes n++ (chegas ao '['), voltas ao mesmo estado, fazes n++ outra vez (chegas ao primeiro dígito) e só aí saltas para o 35.

A grande diferença para a versão que sugeri é:

- no teu raciocínio atual, o momento do salto para 35 é “quando deixas de ver '['”,
- na versão que te propus, o momento do salto para 35 é “assim que vês o '['”.

Funciona? Pode funcionar, sim, mas é bem mais frágil e menos legível:

- depende de “já estar no char anterior a [quando entras em 32”;
- depende de “não haver mais [depois”;
- e mistura duas coisas: “procurar o [” e “saltar para o primeiro dígito” de forma pouco explícita.

Se quiseses manter o *conceito* que descreveste (“continuar até não encontrar '['”), o que precisas mesmo é só inverter as condições:

```
case 32:
    n++;

    if (ST_Command_IN[n] != '[') {
        // ainda não chegaste ao '[' → continua a varrer
        state = 32;
    } else {
        // apanhaste o '[' → prepara para copiar
        n++;          // salta o '['
        m = 0;        // reset do buffer de destino
        state = 35;
    }
    break;
```

Repara:

- Agora o fluxo coincide com a tua descrição verbal: “andar para a frente até encontrar '['”.
- Fica claro que o salto 32→35 acontece *ao ver o '['*, e não ao deixar de vê-lo.

Se quiseses, podemos testar mentalmente com uma string concreta (por exemplo "W d r t[4.0]\n") e seguir *n* passo a passo para ver exatamente em que índice estás quando saís de 32 e entras em 35. Como explicarias tu, passo a passo, o valor de *n* desde que saís de 31 até chegar ao `atof`?

#include <Arduino.h>

```
#include <Adafruit_NeoPixel.h>
```

```
#include <ESP32Servo.h>
```

```
#include <math.h>
```

```
// declarar no topo do ficheiro (globais)
```

```
int last_delta2_s1 = 0;
```

```
int last_delta2_s2 = 0;
```

```
int last_delta2_s3 = 0;
```



```

struct Manipulador_Config {
    // geometria
    float L1;
    float L2;
    float L3;
    float servoOffsetX;
    float servoOffsetZ;
    float angleMin;
    float angleMax;
    float rotationOffsetZ; // offset global de rotação da base

    // calibração de PWM
    int servo1MinPulse;
    int servo1MaxPulse;

    int servo2MinPulse;
    int servo2MaxPulse;

    int servo3MinPulse;
    int servo3MaxPulse;

    int theta1MinPulse;
    int theta1MaxPulse;

    int theta2MinPulse;
    int theta2MaxPulse;

    int theta3MinPulse;
    int theta3MaxPulse;

    // estado atual (pulsos gerados pela IK)
    int servo1Pulse;
    int servo2Pulse;
    int servo3Pulse;
};

Manipulador_Config delta_1_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -15.6f * PI / 180.0f, //-75 degrees in radians

    // calibração de PWM
    .servo1MinPulse = 520,

```

```

        .servo1MaxPulse = 2520,

        .servo2MinPulse = 550,
        .servo2MaxPulse = 2550,

        .servo3MinPulse = 560,
        .servo3MaxPulse = 2560,

        // estado atual (pulsos gerados pela IK)
        .servo1Pulse = 0,
        .servo2Pulse = 0,
        .servo3Pulse = 0
    };

Manipulador_Config delta_2_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -44.4f * PI / 180.0f, //-45 degrees in radians

    // calibração de PWM
    .servo1MinPulse = 460,
    .servo1MaxPulse = 2460,

    .servo2MinPulse = 470,
    .servo2MaxPulse = 2470,

    .servo3MinPulse = 450,
    .servo3MaxPulse = 2450,

    // estado atual (pulsos gerados pela IK)
    .servo1Pulse = 0,
    .servo2Pulse = 0,
    .servo3Pulse = 0
};

//Motion_test -> Arrays of points for testing the motion functions
float X_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 5.0, 10.0, 15.0, 0.0, 0.0};
float Y_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 5.0, 15.0};    // -> teste de
movimento linear dos 3 eixos
float Z_test_calibration[] = {40.0, 40.0, 50.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};
const int N_test_calibration = 9;

```

```

//bat
float X_test[] = {15.607, 14.923, 13.995, 12.845, 11.503, 10.000, 6.665, 1.464, -2.965, -5.000,
-5.659, -6.056, -6.180, -6.029, -5.607, -4.923, -3.995, -2.845, -1.503, 0.000, 3.335, 8.536, 12.965,
15.000, 15.659, 16.056, 16.180, 16.029};
//float Y_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
float Y_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = { 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0};
float Z_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
const int N_test = 28;
float T_period = 0.3f;
float T_pausedt = 0.6f;

//resp
//float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000, 10.667, 9.333,
8.000, 6.667, 5.333, 4.000, 2.667, 1.333 };
//float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0 };
//float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000, 15.802,
12.099, 8.889, 6.173, 3.951, 2.222, 0.988, 0.247};
float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000};
float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0 };
float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000};

//const int N_test2 = 18;
const int N_test2 = 10;
float T_period2 = 4.0f;
float T_pausedt2 = 0.8f;
//Link lengths (mm)
#define L1 35
#define L2 60
#define L3 15

#define END_EFFECTOR_Z_OFFSET 0
#define SERVO_OFFSET_X 32
#define SERVO_OFFSET_Y 0

```

```

#define SERVO_OFFSET_Z (0.0 + END_EFFECTOR_Z_OFFSET)
// #define SERVO_OFFSET_Z_INVERTED -293

// #define SERVO_ANGLE_MIN (100.0f * PI / 180.0f)
// #define SERVO_ANGLE_MAX (190.0f * PI / 180.0f)

#define SERVO_ANGLE_MIN 0.78539816339744830961566084581988f //45 degrees
#define SERVO_ANGLE_MAX 3.9269908169872415480783042290994f //225 degrees

#define ROTATION_OFFSET_Z (-75.0f * PI / 180.0f) //0.26179938779914943f (15° em radianos =
π/12):

#define ROTATION_OFFSET_Z_Delta1 (-75.0f * PI / 180.0f)
#define ROTATION_OFFSET_Z_Delta2 (-45.0f * PI / 180.0f)

#define MIN 'm'
#define MAX 'M'

// Servo microsecond pulse limits (Calibration)-----Valores normais: Min = 500 ; Max = 2500]-----
-----

#define SERVO_1_MIN 620
#define SERVO_1_MAX 2520
#define SERVO_2_MIN 615
#define SERVO_2_MAX 2480
#define SERVO_3_MIN 620
#define SERVO_3_MAX 2550
// Servo_1 -> theta3 min 550, max 2450
// Servo_2 -> theta1 min 545, max 2410
// Servo_3 -> theta2 min 520, max 2450

// variáveis globais para valores suavizados
int servo_1_pulse_smooth = 0;
int servo_2_pulse_smooth = 0;
int servo_3_pulse_smooth = 0;

// fator de suavização [0 - 1]. tipo 0.2 para começar
const float SERVO_SMOOTH_ALPHA = 0.2f;

#define SERVO_4_MIN 550
#define SERVO_4_MAX 2400
#define SERVO_5_MIN 550
#define SERVO_5_MAX 2400
#define SERVO_6_MIN 550
#define SERVO_6_MAX 2400

// variáveis globais para valores suavizados
int servo_4_pulse_smooth = 0;
int servo_5_pulse_smooth = 0;

```

```

int servo_6_pulse_smooth = 0;
//-----

#define INVERTED -1

// Máximo de pontos por movimento (ajusta mais tarde se precisares)
const int MAX_POINTS = 100;

// Armazenamento Bruto: dados completos de um movimento - Será utilizada para armazenar
os movimentos que o Pi enviar, ou movimentos pré-definidos, etc.
struct MotionStorage {
    int  n_Points;           // nº de pontos válidos em X/Y/Z/easing
    float X[MAX_POINTS];     // coordenadas X dos pontos do movimento
    float Y[MAX_POINTS];     // coordenadas Y dos pontos do movimento
    float Z[MAX_POINTS];     // coordenadas Z dos pontos do movimento
    float period;
    float T_pause;
    float dtMs;              // duração do ciclo em segundos
    float dt_pauseMs;        // duração da pausa entre ciclos em segundos
    int  startIndex; // índice de ponto que será o "0,0,0" lógico
    bool bidirectional; // true = vai-e-vem, false = ciclo fechado
    int  N_pontos ; // nº de segmentos por ciclo

float getDtMs(){
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_pontos = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinhas
            N_pontos = n_Points;
        }
        return (period * 1000.0f) / (float)N_pontos;
    }
    return 0.0f;
}

void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}

float getD_PauseMs() const {
    if (n_Points > 1 && T_pause > 0.0f) {
        return (T_pause * 1000.0f); // só lê nPoints e T_pause
    }
    return 0.0f;
}

```

```

void updateDt_PauseMs() {
    dt_pauseMs = getD_PauseMs(); // aqui sim, mudas dt_pauseMs
}

};

// Armazenamento temporário: runtime que executa um MotionStorage
struct MotionInstance {
    MotionStorage* def;    // ponteiro para MotionStorage (a definição do Armazenamento Bruto
)
    int      currentIndex;// índice i (início do segmento i -> i+1) -> índice atual no movimento
    unsigned int  segmentStartMs; // timestamp do início do segmento atual
                        // -> é uma variavel importante para controlar a velocidade do movimento,
ou seja, quando é que deve avançar para o próximo ponto do movimento
    bool      active;    // se este movimento está ativo
    bool      Executing;  // se este movimento está em pausa (entre ciclos)
    short int  state;
    unsigned long elapsed_1;
    unsigned long elapsed_2;
    unsigned long elapsed_3;

int iIndex; // novo: índice i para a interpolação
    int jIndex; // novo: índice j para a interpolação
    int Direction; // novo: direção do movimento (1 ou -1)
};

// -----Temporario-----
MotionStorage Bat_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Bat_inst;
MotionStorage Resp_move; //Declaração de objetos para utilizar com as structures de
movimento
MotionInstance Resp_inst;
MotionStorage Tosse_move; //Declaração de objetos para utilizar com as structures de
movimento
MotionInstance Tosse_inst;
MotionStorage* currentMove = nullptr;
MotionInstance* currentInst = nullptr;

void Iniciate_Bat_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Bat_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Bat_move cada vez que quiseses testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Bat_move.n_Points = N_test;
    Bat_move.period = T_period;
    Bat_move.T_pause = T_pausedt;
    for (int i = 0; i < N_test; i++) {

```

```

    Bat_move.X[i] = X_test[i];
    Bat_move.Y[i] = -1 * Y_test[i];
    Bat_move.Z[i] = Z_test[i];
}
Bat_move.startIndex = 0;
Bat_move.bidirectional = false; // Define o movimento como vai-e-vem
Bat_move.updateDtMs();
Bat_move.updateDt_PauseMs();
}

void Inicieate_Resp_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Resp2_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Resp2_move cada vez que quiseses testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Resp_move.n_Points = N_test2;
    Resp_move.period = T_period2;
    Resp_move.T_pause = T_pausedt2;
    for (int i = 0; i < N_test2; i++) {
        Resp_move.X[i] = X_test2[i];
        Resp_move.Y[i] = -1 * Y_test2[i];
        Resp_move.Z[i] = Z_test2[i];
    }
    Resp_move.startIndex = 5;
    Resp_move.bidirectional = true; // Define o movimento como vai-e-vem
    Resp_move.updateDtMs();
    Resp_move.updateDt_PauseMs();
}

void Inicieate_Bat_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Bat_inst.def = &Bat_move;
    Bat_inst.currentIndex = 0;
    Bat_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Bat_inst.active = true;
    Bat_inst.Executing = 0; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Bat_inst.state = 0;
    Bat_inst.elapsed_1 = 0;
    Bat_inst.elapsed_2 = 0;
    Bat_inst.elapsed_3 = 0;
    Bat_inst.iIndex = 0;
    Bat_inst.jIndex = 0;
    Bat_inst.Direction = 1; // novo: inicializa a direção como positiva
}

```

```

void Inicie_Resp_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Resp_inst.def      = &Resp_move;
    Resp_inst.currentIndex = 0;
    Resp_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Resp_inst.active     = true;
    Resp_inst.Executing   = false; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Resp_inst.state      = 0;
    Resp_inst.elapsed_1  = 0;
    Resp_inst.elapsed_2  = 0;
    Resp_inst.elapsed_3  = 0;
    Resp_inst.iIndex     = 0;
    Resp_inst.jIndex     = 0;
    Resp_inst.Direction   = 1; // novo: inicializa a direção como positiva
}

```

//-----Fim Temporario-----

```

struct Coordinate {
    float X;
    float Y;
    float Z;
};

```

```

int Clock_loop = 0;
const float pi = 3.141592653;

```

// -----Declaração das variaveis-----

```

void LED_LOOP(); //Declaração de funções

```

```

void Servo_test();

```

//-----Novas-----

```

float boundFloat(float, float, float);

```

```

bool attach_servos(void);

```

```

bool inverse_kinematics_1(float, float, float, float, float&);

```

```

bool inverse_kinematics_2(float, float, float, float, float&);

```

```

bool inverse_kinematics_3(float, float, float, float, float&);

```

```

bool inverse_kinematics(Manipulador_Config& cfg, float, float, float);

```

```

void move_servos(const Manipulador_Config& d1, const Manipulador_Config& d2);

```

```

double mapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

int roundMapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

double degToRads(double deg);

```

```

double radsToDeg(double rads);

```

```

void debugPrintMove(const MotionStorage& move, const char* name);

```

```

void debugPrintInstance(const MotionInstance& inst, const char* name);

```

```

float lerp(float a, float b, float t);

```



```

float applyEasing(float alpha, char mode);
float spline3(float p0, float p1, float p2, float t);
void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs,
Coordinate& xyz, boolean trig_serial);
bool Fusao_plus_IK_D1(const Coordinate& respi, const Coordinate& batim);
bool Fusao_plus_IK_D2(const Coordinate& respi, const Coordinate& batim);
//-----FSM-----
void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs);
void FSM_Serial_reader(unsigned long nowMs);
void FSM_Serial_Command(unsigned long nowMs);
void FSM_Main(boolean start, MotionInstance& inst, unsigned long nowMs);
//-----

//--Declaração dos comandos de leitura
//char commands
#define Scomand_start_Program 's'
#define Scomand_stop_Program 'p'
#define Scomand_GRIPPER 3
#define Scomand_ABSOLUTE_CARTESIAN_LINEAR 4
#define Scomand_SET_PROGRAM_ARRAY 5
#define Scomand_REQUEST_READY_FLAG 6

//String commands
#define Mcomand_JOG_X 'x'
#define Mcomand_JOG_Y 'y'
#define COMMAND_JOG_Z 'z'
#define COMMAND_GRIPPER_ASCII 'g'
#define COMMAND_STATUS 's'
#define COMMAND_REPORT_COMMANDS 'r'
#define COMMAND_ADD_POSITION 'p'
#define COMMAND_SET_STEP_DELAY 'd'
#define COMMAND_SET_STEP_INCREMENT 'i'
#define COMMAND_CLEAR_ARRAY 'c'
#define COMMAND_EXECUTE 'e'
#define COMMAND_EXECUTE_JOINT 'j'
#define COMMAND_EXECUTE_TIME 't'
#define COMMAND_SET_US_INCREMENT_LINEAR 'u'
#define COMMAND_SET_US_INCREMENT_JOINT 'U'
#define COMMAND_STEP_FORWARD '>'
#define COMMAND_STEP_BACKWARD '<'
#define COMMAND_JUMP_TO_START '['
#define COMMAND_JUMP_TO_END ']'
#define COMMAND_EDIT_ARRAY 'P'
#define COMMAND_ADD_DELAY 'D'
#define COMMAND_MOVE_HOME 'h'
#define COMMAND_PRINT_FILE 'f'
#define COMMAND_PING_PONG 'o'

```

```

#define COMMAND_SERVO_CALIBRATION 'C'
#define COMMAND_SET_SERVO 'S'

//EEPROM commands
#define Ecomand_SET_LINK_2 'L'
#define Ecomand_SET_END_EFFECTOR_TYPE 'E'
#define Ecomand_SET_AXIS_DIRECTION 'A'
#define Ecomand_SET_HOME_X 'X'
#define Ecomand_SET_HOME_Y 'Y'
#define Ecomand_SET_HOME_Z 'Z'
#define Ecomand_SET_HOME_GRIPPER 'G'
#define Ecomand_SET_GRIPPER_MIN_MAX 'M'

//EEPROM addresses for the delta robots configuration
#define EEPROM_ADDRESS_LINK_2 0
#define EEPROM_ADDRESS_END_EFFECTOR_TYPE 1
#define EEPROM_ADDRESS_AXIS_DIRECTION 2
#define EEPROM_ADDRESS_Z_OFFSET 3
#define EEPROM_ADDRESS_HOME_X 7
#define EEPROM_ADDRESS_HOME_Y 11
#define EEPROM_ADDRESS_HOME_Z 15
#define EEPROM_ADDRESS_HOME_GRIPPER 19
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MIN 21
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MAX 23
#define EEPROM_ADDRESS_GRIPPER_CLAW_MIN 25
#define EEPROM_ADDRESS_GRIPPER_CLAW_MAX 27
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MIN 29
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MAX 31

//Delta1 -----
Servo mg90s_1; // cria objeto servo
Servo mg90s_2; // cria objeto servo
Servo mg90s_3; // cria objeto servo

//Delta2 -----
Servo mg90s_4; // cria objeto servo
Servo mg90s_5; // cria objeto servo
Servo mg90s_6; // cria objeto servo

volatile boolean start_comand = 0;

Coordinate Lerp_Respi_OUT;
Coordinate Lerp_Batim_OUT;
Coordinate IK2_IN;
Coordinate home_position;

float servo_offset_z = SERVO_OFFSET_Z;

```

```

//-----Declaração da localização dos pinos para cada objeto -----
//-----Servos-----
#define SERVO_PIN_1 12 //Servo_1 -> theta3
#define SERVO_PIN_2 11 //Servo_2 -> theta1
#define SERVO_PIN_3 10 //Servo_3 -> theta2

#define SERVO_PIN_4 36 // Servo_4 -> theta3
#define SERVO_PIN_5 37 // Servo_5 -> theta1
#define SERVO_PIN_6 38 // Servo_6 -> theta2

//-----Led_Informação-----
#define LED_PIN 48
#define LED_COUNT 1

//-----
bool stepDirection = false;
Adafruit_NeoPixel pixel(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);

char Mode = 'S'; // 'S' = Single, 'M' = Manual
char CH_Command_Out = '\0';
char CH_Command_IN = '\0';
char ST_Command_Out[100];
char ST_Command_IN[100];
int command_counter = 0;
bool LineReady_IN = false;
bool LineReady_Out = false;

boolean trig_display_fsm(float at, short int Td){
static short int state; //state internal to the state machine
static float pt; //var. internal to the state machine
boolean trig; // output

switch (state) {
case 0:
if (at-pt >= Td) {state = 1;}
trig = 0; // output
return trig;
case 1:
if (1) {state = 0;}
trig = 1; pt = at; // output
return trig;
}
return trig;
}

int setup_counter = 0;

int state_test = 0;
int state_test2 = 0;

```

```

float Training_Exame_Starttime = 0;
float Training_Exame_Finishingtime = 0;
float Training_Exame_Durantiontime = 0;
int Safe_comand = 0;

void FSM_Serial_reader(unsigned long nowMs){
static short int state;
static int counter = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

```

```

if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;

case 6:
    ST_BUS[counter] = Serial.read();
    counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;

case 8:

int maxCopy = counter-1; // aqui definimos a quantidade de caracteres a copiar, não em termos
do indice mas sim a quantidade
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    ST_BUS[counter-1] = '\0'; // substitui o '\n'
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';
    counter = 0;

    break;
}
return;
}

void FSM_Serial_Command(unsigned long nowMs){
static short int state;
state_test2 = state;
static int n = 0;
static int m = 0;
static char ST_copy[100];
/*

```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar
- Condições de transição por hierarquia

*/

```
//Serial.print("Estado atual: ");
```

```
//Serial.println(inst.state);
```

```
switch (state) {
```

```
case 0:
```

```
    Serial.println("Escolha um comando:");
```

```
if(1){state = 1;}
```

```
break;
```

```
case 1:
```

```
    LineReady_Out = false;
```

```
if(CH_Command_IN == 'S' || CH_Command_IN == 's'){state = 2;}
```

```
    if(CH_Command_IN == 'P' || CH_Command_IN == 'p'){state = 3;}
```

```
    if(CH_Command_IN == 'L' || CH_Command_IN == 'l'){state = 5;}
```

```
    if(CH_Command_IN == 'M' || CH_Command_IN == 'm'){state = 14;}
```

```
    if(CH_Command_IN == 'W' || CH_Command_IN == 'w'){state = 23;}
```

```
    break;
```

```
case 2:
```

```
    Serial.println("Ativo");
```

```
    start_comand = 1;
```

```
    Training_Exame_Starttime = nowMs;
```

```
    LineReady_Out = true;
```

```
    CH_Command_Out = '\0';
```

```
    pixel.setPixelColor(0, pixel.Color(255, 0, 0));
```

```
    pixel.show();
```

```
if (1) { state = 4;}
```

```
    break;
```

```
case 3:
```

```
    Serial.println("Desativo");
```

```
    start_comand = 0;
```

```
    Training_Exame_Finishingtime = nowMs;
```

```
    Training_Exame_Durantiontime = Training_Exame_Finishingtime -  
Training_Exame_Starttime;
```

```
if(Training_Exame_Durantiontime > 0){
```

```
    Serial.print("Duração do treino: ");
```

```
    Serial.print(Training_Exame_Durantiontime/1000.0f);
```

```
    Serial.println(" Segundos");
```

```

    }
    Training_Exame_Finishingtime = 0;
    Training_Exame_Starttime = 0;
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 0, 0));
    pixel.show();

    if (1) { state = 4;}
    break;

case 4:
    Serial.println("Escolha um comando:");
    LineReady_Out = false;

if(1){state = 1;}
    break;

case 5:
    Serial.println("ler");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 0));
    pixel.show();

if(1){state = 6;}
    break;

case 6:
    LineReady_Out = false;

if(CH_Command_IN == '1'){state = 7;}
    if(CH_Command_IN == '2'){state = 8;}
    if(CH_Command_IN == '3'){state = 9;}
    if(CH_Command_IN == 'r'){state = 10;}
    if(CH_Command_IN == 'b'){state = 11;}
    if(CH_Command_IN == 't'){state = 12;}
    if(CH_Command_IN == 'a'){state = 13;}
    break;

case 7:
    Serial.println("Comando L1");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 255));
    pixel.show();
    Safe_comand = 1;

```

```
if(1){state = 4;}
```

```
break;
```

```
case 8:
```

```
Serial.println("Comando L2");
```

```
LineReady_Out = true;
```

```
CH_Command_Out = '\0';
```

```
pixel.setPixelColor(0, pixel.Color(0, 255, 255));
```

```
pixel.show();
```

```
Safe_comand = 0;
```

```
if(1){state = 4;}
```

```
break;
```

```
case 9:
```

```
Serial.println("Comando L3");
```

```
LineReady_Out = true;
```

```
CH_Command_Out = '\0';
```

```
Serial.println("Sem nada"); //Retirar quando estiver implementado
```

```
if(1){state = 4;}
```

```
break;
```

```
case 10:
```

```
Serial.println("Comando Lr");
```

```
LineReady_Out = true;
```

```
CH_Command_Out = '\0';
```

```
debugPrintMove(Resp_move, "Resp_move");
```

```
debugPrintInstance(Resp_inst, "Resp_inst");
```

```
if(1){state = 4;}
```

```
break;
```

```
case 11:
```

```
Serial.println("Comando Lb");
```

```
LineReady_Out = true;
```

```
CH_Command_Out = '\0';
```

```
debugPrintMove(Bat_move, "Bat_move");
```

```
debugPrintInstance(Bat_inst, "Bat_inst");
```

```
if(1){state = 4;}
```

```
break;
```

```
case 12:
```

```
Serial.println("Comando Lt");
```

```
LineReady_Out = true;
```

```
CH_Command_Out = '\0';
```



```
//debugPrintMove(Bat_move, "Tosse_move"); //ainda é preciso implementar
//debugPrintInstance(Bat_inst, "Tosse_inst"); //ainda é preciso implementar
Serial.println("Sem nada"); //Retirar quando estiver implementado
```

```
if(1){state = 4;}
break;
```

case 13:

```
Serial.println("Comando LA");
LineReady_Out = true;
CH_Command_Out = '\0';
debugPrintMove(Bat_move, "Bat_move");
debugPrintMove(Resp_move, "Resp_move");
debugPrintInstance(Bat_inst, "Bat_inst");
debugPrintInstance(Resp_inst, "Resp_inst");
Serial.println("Sem Tosse"); //Retirar quando estiver implementado a tosse
```

```
if(1){state = 4;}
break;
```

case 14:

```
Serial.println("Comando Mode");
start_comand = 0;
Training_Exame_Finishingtime = nowMs;
Training_Exame_Durantiontime = Training_Exame_Finishingtime -
```

Training_Exame_Starttime;

```
if(Training_Exame_Durantiontime > 0){
    Serial.print("Duração do treino: ");
    Serial.print(Training_Exame_Durantiontime/1000.0f);
    Serial.println(" Segundos");
}
```

```
Training_Exame_Finishingtime = 0;
Training_Exame_Starttime = 0;
LineReady_Out = true;
```

```
if(1){state = 15;}
break;
```

case 15:

```
LineReady_Out = false;
```

```
if(CH_Command_IN == 'a'){state = 16;}
if(CH_Command_IN == 'h'){state = 17;}
if(CH_Command_IN == 'r'){state = 18;}
if(CH_Command_IN == 'b'){state = 19;}
if(CH_Command_IN == 's'){state = 20;}
break;
```

```

case 16:
    Serial.println("Comando Mode ALL");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = true;

if(1){state = 21;}
    break;

case 17:
    Serial.println("Comando Mode Human (Bat+Resp)");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 18:
    Serial.println("Comando Mode Respiração");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = true;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 19:
    Serial.println("Comando Mode Batimento");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 20:
    Serial.println("Comando Mode Static");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;

```

```

    Resp_inst.active = false;
    //Tosse_inst.active = false;

if(1){state = 21;}
    break;

case 21:
    Serial.println("want to start de cicle? y/n");
    LineReady_Out = false;

if(1){state = 22;}
    break;

case 22:
    if(CH_Command_IN == 'y'){state = 2;}
    if(CH_Command_IN == 'n'){state = 3;}
    break;

case 23:
    Mode = 'M';
    Serial.println("Comando Mode multiple characters");
    LineReady_Out = true;

if(1){state = 24;}
    break;

case 24:
    LineReady_Out = false;
    n = 0;

if(ST_Command_IN[n] == 'D'){state = 25;}
    if(ST_Command_IN[n] == 'd'){state = 26;}
    break;

case 25:
    start_comand = 0;
    n++;

if(1){state = 27;}
    break;

case 26:
    n++;

if(1){state = 28;}
    break;

case 27:
    if(ST_Command_IN[n] == 'R' || ST_Command_IN[n] == 'r'){state = 28;}
    if(ST_Command_IN[n] == 'B' || ST_Command_IN[n] == 'b'){state = 29;}

```

```

    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 30;}
    break;

case 28:
    currentMove = &Resp_move;
    currentInst = &Resp_inst;

if(1){state = 31;}
    break;

case 29:
    currentMove = &Bat_move;
    currentInst = &Bat_inst;

if(1){state = 31;}
    break;

case 30:
    currentMove = &Tosse_move;
    currentInst = &Tosse_inst;

if(1){state = 31;}
    break;

case 31:
    n++;

if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;} //Periodo
    if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;} //Pausa
    if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;} //Movimento
    break;

case 32:
    n++;
    m = 0;

if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
    break;

case 33:
    //L
    break;

case 34:
    //L
    break;

case 35:
    //L

```

```

    n++;
    ST_copy[m]=ST_Command_IN[n];
    m++;

if(ST_Command_IN[n] != ' '){state = 35;}
    if(ST_Command_IN[n] == ' '){state = 36;}
    break;

case 36:
    //L
    currentMove->period=atof(ST_copy);
    memset(ST_copy,0,sizeof(ST_copy));
    LineReady_Out = true;

if(1){state = 4;}
    break;
}

    return;
}

void setup() {

Serial.begin(115200);

setup_counter++;
    Serial.print("setup chamado: ");
    Serial.println(setup_counter);

    Serial.println("Teste LED RGB");

pixel.begin();
    pixel.clear();
    pixel.show();

ESP32PWM::allocateTimer(0);
    ESP32PWM::allocateTimer(1);
    ESP32PWM::allocateTimer(2);
    ESP32PWM::allocateTimer(3);

bool servo_ok = attach_servos();

delay(1000);
    if (servo_ok) {
        Serial.println("Servos attached successfully");
    } else {
        Serial.println("Error attaching servos");
    }
}

```

```

//-----Inicialização do movimento de teste-----
  Iniciate_Bat_Move();
  Iniciate_Resp_Move();
  Iniciate_Bat_Instance();
  Iniciate_Resp_Instance();

  //Serial.println("Setup completo, pronto para correr Sequence.");

}

void loop() {
  //Serial.println("beep");
  //delay(1000);

  static boolean trig_serial_IN; static boolean trig_serial_OUT;
  float actual_time = millis();
  float tempo_inicial = millis();

  unsigned long now1 = millis();
    trig_serial_OUT = trig_display_fsm(actual_time,1000);
    FSM_Serial_reader(now1);
    FSM_Serial_Command(now1);

  unsigned long now2 = millis();
    // --- Apenas um comentario temporario, -> tenho que ativar denovo isto mais tarde!!
    FSM_Motion_Update(start_comand , Bat_inst, now2);
    unsigned long now3 = millis();
    Intermed_position(start_comand ,Bat_inst, now3, Lerp_Batim_OUT, trig_serial_IN);
    FSM_Motion_Update(start_comand , Resp_inst, now2);
    Intermed_position(start_comand ,Resp_inst, now3, Lerp_Respi_OUT, trig_serial_IN);

  bool ik1 = Fusao_plus_IK_D1(Lerp_Respi_OUT, Lerp_Batim_OUT);
    bool ik2 = Fusao_plus_IK_D2(Lerp_Respi_OUT, Lerp_Batim_OUT);

  if (ik1 && ik2) {
    move_servos(delta_1_Cfg, delta_2_Cfg);
  }

  float tempo_final = millis();
  float demora = tempo_final - tempo_inicial;
  Clock_loop++;
  //Serial.print("Clock loop: ");
  // Serial.println(Clock_loop);
  if(trig_serial_IN == 1 && Safe_comand == 1){

    Serial.print("Estado da leitura:");
    Serial.println(state_test);
  }
}

```

```

Serial.print("Estado da Interpretação:");
Serial.println(state_test2);

}
trig_serial_IN = trig_serial_OUT;
LineReady_IN=LineReady_Out;
CH_Command_IN=CH_Command_Out;
strncpy(ST_Command_IN, ST_Command_Out, sizeof(ST_Command_IN));
ST_Command_IN[sizeof(ST_Command_IN)-1] = '\0';
}

```

Fiz algumas das alterações que recomendas-te,
outras não fiz pq acho que não faz sentido

A versão nova está bem mais limpa e próxima do que tinhas desenhado; onde não mexeste, em muitos casos faz sentido mesmo manter simples. Vou focar só na parte crítica: o ramo W (estados 23–36) e, em particular, o tal estado 32 e a cópia do valor.

1. O que já está bem

- A leitura em modo M agora termina a string em ST_BUS e ST_Command_Out corretamente: substituis o '\n' por '\0' e copias com limite de maxCopy, o que te garante uma string bem formada sem estouro de buffer. ^[136]
- No loop, copias ST_Command_Out para ST_Command_IN com strncpy, o que é aceitável porque estás a trabalhar sempre com buffers de mesmo tamanho e já limitaste antes. ^[136]
- A parte 24–31 da FSM de comandos (W, D/d, R/B/T, escolha de Resp/Bat/Tosse, e escolha de T/P/M) está alinhada com a lógica que explicaste. ^[136]

2. Onde a FSM ainda se pode “tropeçar”

O ponto delicado continua a ser a sequência 31 → 32 → 35 → 36. ^[136]

Neste momento tens:

```

case 31:
    n++;

    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;} //Periodo
    if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;} //Pausa
    if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;} //Movimento
    break;

case 32:
    n++;
    m = 0;

    if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
    break;

```

```

case 35:
    n++;
    ST_copy[m]=ST_Command_IN[n];
    m++;

    if(ST_Command_IN[n] != ' '){state = 35;}
    if(ST_Command_IN[n] == ' '){state = 36;}
    break;

case 36:
    currentMove->period=atof(ST_copy);
    memset(ST_copy,0,sizeof(ST_copy));
    LineReady_Out = true;

    if(1){state = 4;}
    break;

```

A tua ideia para o 32 é:

“quero andar na string até encontrar o ' ' ; como controlo a formatação, sei que há um [algures a seguir”

O problema é a forma como estás a usar o `if` duplo:

- No 32:
 - incrementas `n`
 - se `ST_Command_IN[n] == ' '` → `state = 32` de novo
 - se `ST_Command_IN[n] != ' '` → vais para 35
- No 35:
 - incrementas `n` de novo
 - copias o char em `ST_Command_IN[n]` para `ST_copy[m]`
 - só paras quando `ST_Command_IN[n] == ' '`

Ou seja, na prática:

- Nunca entras em 35 quando estás em cima do ' ' ;
- 35 arranca sempre já no **primeiro dígito**, e vai copiar até incluir o ' ' (porque só verificas depois de copiar).

Isto até pode funcionar “por sorte” com uma string bem comportada tipo:

```
d r t[4.0]
```

mas tens dois riscos:

1. **Inclui] no buffer:** o `ST_copy` vai ficar com `'4', '.', '0', ']', '\0'`. O `atof` por acaso lida bem com isso (para no `' ] '`), mas fica uma dependência implícita na função da `libc`.

2. Não tens proteção de limites em `m`: se um dia a string tiver valores maiores, podes escrever para lá do fim de `ST_copy`.

Se queres mesmo manter a ideia “o 32 é só o scanner até ao [”, o mínimo que eu mudaria seria:

- inverter a lógica do 32
- limpar `m` só uma vez, quando saís do 32 para o 35
- garantir que o '[' não entra no `ST_copy` e que o buffer fica terminado corretamente.

Uma versão bem próxima da tua, mas mais robusta, seria:

```
case 32:
    // andar para a frente até achar '['
    n++;
    if (ST_Command_IN[n] != '[') {
        // ainda não é '[', continua neste estado
        state = 32;
    } else {
        // apanhaste o '[' -> prepara cópia
        n++;          // salta o '['
        m = 0;        // reset do buffer de destino
        state = 35;
    }
    break;

case 35:
    // copiar até ']' ou até encher o buffer
    if (ST_Command_IN[n] != ']' && m < (int)sizeof(ST_copy) - 1) {
        ST_copy[m] = ST_Command_IN[n];
        m++;
        n++;
        state = 35;    // continua a copiar
    } else {
        ST_copy[m] = '\0'; // termina a string
        state = 36;       // mesmo que tenhas parado por causa do ']' ou do tamanho
    }
    break;
```

E o 36 fica exatamente igual:

```
case 36:
    currentMove->period = atof(ST_copy);
    memset(ST_copy, 0, sizeof(ST_copy));
    LineReady_Out = true;
    state = 4;
    break;
```

Com isto, manténs:

- o “scanner” que tu queres no 32
- a suposição de que há um '[' algures

- mas garantes que ST_copy recebe só o valor numérico e está sempre bem terminada.

Para consolidar: se tiveres a string exatamente como pretendes, por exemplo "d r t[3.5]", consegues escrever, de cabeça, quais são os índices que n percorre desde que saís do state 31 (no t) até ao fim do state 35 (logo antes do ' ')?

**

eu escrevi só Wd

e ele está a dar-me isto

da leitura:255

Esp32: Estado da Interpertação:24

Esp32: Estado da leitura:255

Esp32: Estado da Interpertação:24

Esp32: Estado da leitura:255

Esp32: Estado da Interpertação:24

eu puz ativo o Safe_comand como 1, para ele escrever os estados das duas FSM

```
if(trig_serial_IN == 1 && Safe_comand == 1){
```

```
    Serial.print("Estado da leitura:");
    Serial.println(state_test);
    Serial.print("Estado da Interpretação:");
    Serial.println(state_test2);
```

```
}
```

Aqui o comportamento que estás a ver faz sentido com o código atual:

- state_test (FSM_Serial_reader) = 255 → significa que a variável state local ainda não foi inicializada explicitamente, por isso começa com o valor default da memória (255) até o primeiro switch correr e acertar num case com `if(...) state = X;`.^[172]
- state_test2 (FSM_Serial_Command) = 24 → a FSM de comandos está a ficar presa no case 24, o que bate certo com o facto de teres mandado só Wd.^[172]

Vamos destrinçar isso com o comando que enviaste.

1. O que acontece quando escreves só Wd

Entrada no Serial:

```
'W' 'd' '\n'
```

Fluxo:

1. FSM_Serial_reader em state 0 vê Mode == 'S', passa a state 1 e apanha o 'W' como CH_Command_Out (modo single).^[172]
2. FSM_Serial_Command em state 1 vê CH_Command_IN == 'W' e vai para state 23.^[172]

3. state 23: muda Mode = 'M', escreve “Comando Mode multiple characters” e vai para state 24.^[172]
4. A partir daqui, a FSM de leitura já está em modo M e vai começar a encher ST_BUS com os caracteres seguintes da mesma linha: 'd' e depois '\n'. Isso vai parar em state 8 da leitura, copiar "d" para ST_Command_Out e pôr counter = 0 de novo.^[172]
5. No fim do loop, copias ST_Command_Out para ST_Command_IN com strncpy. Neste caso, ST_Command_IN[^46_0] = 'd', ST_Command_IN[^46_1] = '\0'.^[172]
6. FSM_Serial_Command no state 24 faz:

```
LineReady_Out = false;
n = 0;

if(ST_Command_IN[n] == 'D'){state = 25;}
if(ST_Command_IN[n] == 'd'){state = 26;}
```

Como ST_Command_IN[^46_0] == 'd', vai para state 26.^[172]

7. state 26 faz:

```
n++;
if(1){state = 28;}
```

Ou seja, n passa a 1 e saltas para o state 28.^[172]

8. state 28 define currentMove = &Resp_move; e vai para state 31.^[172]
9. state 31 faz:

```
n++;
if(ST_Command_IN[n] == 'T' || ...) { ... }
```

Aqui está o problema:

- n tinha ficado em 1
- ST_Command_IN[^46_1] == '\0'
- Depois de n++, n = 2, e ST_Command_IN[^46_2] é lixo/memória, não há 'T', 'P' nem 'M', por isso **nenhuma condição é verdadeira** e ficas preso no state 31 sem sair dali.^[172]

O que estás a ver:

- Quando ligas Safe_comand = 1, o log mostra Estado da Interpretação:24, o que indica que, no momento em que disparas o print, ainda não chegaste a reprocessar aquele ciclo até passar por 26, 28, 31 — ou estás a ver estados antigos acumulados. Mas, mesmo que o print apanhe 24, o bug estrutural é esse: Wd não tem mais nada para além de 'd', portanto estás a pedir à FSM para interpretar um comando incompleto (faltam r/b/t, T/P/M e o valor entre []).

2. Duas coisas a fazer aqui

1. Detecção de comando incompleto

No state 24, devias testar se a string tem comprimento suficiente antes de avançar:

```
case 24:
    LineReady_Out = false;
    n = 0;

    if (ST_Command_IN[n] == 'D' || ST_Command_IN[n] == 'd') {
        // verifica se há pelo menos mais 1 carácter (R/B/T)
        if (ST_Command_IN[^46_2] == '\0') {
            Serial.println("Erro: comando 'D' incompleto");
            LineReady_Out = true;
            state = 4;    // volta ao menu principal
        } else {
            state = (ST_Command_IN[n] == 'D') ? 25 : 26;
        }
    }
    break;
```

2. Só entra na parte T/P/M se houver caracteres suficientes

Em state 31, antes de fazer n++, convém garantir que não estás já no '\0':

```
case 31:
    // se já não houver mais chars, aborta
    if (ST_Command_IN[n+1] == '\0') {
        Serial.println("Erro: faltam T/P/M no comando");
        LineReady_Out = true;
        state = 4;
        break;
    }

    n++;

    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;}
    else if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;}
    else if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;}
    else {
        Serial.println("Erro: letra desconhecida em D...?T/P/M");
        LineReady_Out = true;
        state = 4;
    }
    break;
```

Assim, quando escreves só Wd, a FSM já não tenta interpretar como comando completo: ela detecta que só veio o 'd' sem mais nada e devolve-te uma mensagem de erro em vez de ir parar a estados estranhos.

Para garantir que entendeste a causa: como resumias, nas tuas palavras, o que está a faltar na string Wd para que a FSM consiga chegar até ao state 36 sem se perder?

o problema não é o estado 24, e sim o estado 255 que não existe na FSM reader

e volto a dizer antes quando introduzo alguma coisa ela está a funcionar corretamente, mas quando coloco `w d r t[0.4]` ou `wdr[t[0.4]` ou `wd` (só para ver em que estado é que vai dar é quando dá o erro no reader)

Perfeito, então vamos atacar diretamente o que te está a intrigar: o tal estado 255 na FSM de leitura.

1. O que significa `state = 255` na `FSM_Serial_reader`

Na tua `FSM_Serial_reader` tens:

```
void FSM_Serial_reader(unsigned long nowMs){
    static short int state;
    static int counter = 0;
    static char CH_Bus;
    static char ST_BUS[^47_100];
    state_test = state;
    ...
    switch (state) {
        case 0:
            ...
            break;
        case 1:
            ...
            break;
        ...
    }
}
```

- `state` é `static` mas **nunca é inicializada explicitamente**.
- No C/C++ em ambiente embarcado, variáveis `static` de função são inicializadas a 0 por default, mas às vezes, dependendo do toolchain / otimizações / memória lida antes do primeiro uso, podes ver lixo até o código correr o `switch` pela primeira vez. [\[182\]](#)
- Quando fazes o teu `Serial.print("Estado da leitura:"); Serial.println(state_test);`, estás a imprimir `state` logo no início da função, **antes** de qualquer `case` o alterar. Se a função ainda não tinha corrido no reset, esse valor pode aparecer como 255 (0xFF), que é o que estás a ver. [\[183\]](#)

Repara em duas coisas importantes:

1. Não tens nenhum default: no `switch`.
2. Se `state` tiver um valor diferente de 0, 1, 2, 3, 4, 5, 6, 8, o `switch` não entra em caso nenhum e **não altera o `state`**, por isso ele fica eternamente nesse valor “fantasma” até ocorrer um reset ou algo semelhante. [\[183\]](#)

Por isso, faz todo o sentido veres:

```
Estado da leitura:255
Estado da Interpretação:24
```

Assim que a tua FSM de comandos entrou no state 24 (por causa do W), a de leitura ainda não tinha sido “inicializada” corretamente, por isso continua com state = 255.

2. Por que só acontece com Wd, wdrt[0.4], etc.

Quando dizes:

“antes, quando introduzo alguma coisa ela está a funcionar corretamente, mas quando coloco w d r t[0.4] ou wdrt[0.4] ou wd [...] é quando dá o erro no reader”

Isso encaixa nisto:

- Nos casos em que envias um comando simples (por exemplo, só 'S' + enter), a FSM de leitura percorre rapidamente os cases 0, 1, 2, etc., e o state chega a 0 ou 1 antes de tu imprimires state_test.
- Nos casos W... , a sequência de eventos entre as duas FSMs muda o timing: a FSM de comandos entra em 23 → 24, a de leitura muda Mode para 'M' e fica à espera da linha completa, mas o teu print pode acontecer num ciclo em que state ainda não foi tocado (ou foi deixado num valor inválido por um comando anterior). A ausência de default: agrava isto, porque um valor estranho nunca é corrigido.

Ou seja: não é que a FSM tenha “ido para um estado 255”; é que **nunca a inicializaste explicitamente** para 0, e não tens default para a recuperar caso apanhe lixo.

3. Como corrigir isso de forma simples

Há duas medidas pequenas e muito eficazes:

1. Inicializa explicitamente o state em 0

Logo na declaração:

```
void FSM_Serial_reader(unsigned long nowMs){
    static short int state = 0;    // garante arranque em 0
    ...
}
```

2. Adiciona um default no switch para recuperar de valores inválidos:

```
switch (state) {
    case 0:
        ...
        break;
    case 1:
        ...
        break;
    ...
    default:
        // estado inválido -> reset
```

```

    state = 0;
    counter = 0;
    CH_Bus = '\0';
    break;
}

```

Isto garante:

- Mesmo que algum glitch te deixe `state` com 255, no ciclo seguinte o `default` apanha e volta a pôr 0.
- O `state_test` deixa de ficar preso em 255, porque nunca mais vais ficar indefinidamente num estado não mapeado.

Se quiseres confirmar o raciocínio: olhando só para a `FSM_Serial_reader`, onde é que adicionarias tu um `default` e que variáveis achas que vale a pena limpar nesse caso para poderes recomeçar a ler um comando do zero sem lixo de antes?

✱

ent basicamente estás me a dizer que da mesma forma que eu não sabes qual é o problema no reader

Neste caso específico, dá para apontar uma causa provável no reader, não é só “não sei”.

Há três pistas importantes no teu código e no comportamento:

1. `state` da FSM reader nunca é inicializado explicitamente a 0 dentro da função.
2. Não tens `default`: no `switch (state)` da `FSM_Serial_reader`.
3. O valor 255 aparece exatamente como “estado da leitura” quando comesas a usar o modo W com comandos mais compridos (`wd`, `wdrt[0.4]`, etc.).^[193]

Isto é típico de duas coisas combinadas:

- `state` começa com um valor qualquer na RAM (255), e como não há `default`, o `switch` não entra em case nenhum, logo o valor nunca é corrigido.
- Em alguns cenários (dependendo do que envias e do timing), a FSM não chega a passar por `case 0` para reinicializar, então ficas sempre a ver 255 quando fazes o `print`.

Ou seja: o problema no reader é estrutural, não é “misterioso”.

Sugestão direta para validares:

```

void FSM_Serial_reader(unsigned long nowMs){
    static short int state = 0;    // 1) inicializar
    static int counter = 0;
    static char CH_Bus;
    static char ST_BUS[^48_100];
    state_test = state;

    switch (state) {

```

```

    case 0:
        ...
        break;
    case 1:
        ...
        break;
    ...
    default:                                // 2) recuperar de estado inválido
        state = 0;
        counter = 0;
        CH_Bus = '\0';
        break;
}
}

```

Se fizeres só estas duas mudanças e voltares a testar com `Wd` e `Wdrt[0.4]`, vês logo se o 255 desaparece e se a reader passa a cair outra vez em estados válidos.

Para garantir que entendeste a ideia: se tiveres de explicar a outra pessoa por que é que um `state` pode ficar “preso” num valor que não existe na FSM (como 255), como é que explicarias isso em termos de inicialização e de falta de `default` no `switch`?

✱✱

eu fiz umas alterações no código e corri de novo.

fiz uns testes com os comandos simples e funcionou e depois enviei isto `"wdB[0.4]"`
este foi o que recebi no monitor serie

PI_4B: wdB[0.4]

Esp32: Comando Mode multiple characters

Esp32: ST_Busd

Esp32: ST_BusB

Esp32: ST_Bus[

Esp32: ST_Bus0

Esp32: ST_Bus.

Esp32: ST_Bus4

Esp32: ST_Bus]

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

Esp32: ST_Bus

```
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: Estado da leitura:6  
Esp32: Estado da Interpretacao:24  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ST_Bus  
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan  
Esp32: ST_Bus  
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan  
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan  
Esp32: ST_Bus  
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan  
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
```

```
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: Estado da leitura:6
Esp32: Estado da Interpretao:24
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
```

Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: Estado da leitura:6
Esp32: Estado da Interpretacao:24
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Serv= -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ERROR: Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Se Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Se Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus
Esp32: ERROR: Se Servo angle 1 out of range: Angle = -nan
Esp32: ST_Bus

[illegible]

será que ele não está a encontrar o `\n`?

mas antes com as outras mensagens ele encontrava sempre que eu envia uma mensagem parecida

Pelo log dá para ver duas coisas distintas a acontecerem ao mesmo tempo:

1. A FSM de leitura está, de facto, a ler o `"wdB[0.4]"` carácter a carácter.
2. Depois disso, a FSM de comandos não está a fechar o comando nem a parar o movimento, e a cinemática começa a cuspir NaN, o que enche o Serial de erros e de prints de debug (ST_Bus).^[194]

Repara nesta parte do que mostraste:

- Esp32: Comando Mode multiple characters → state 23 da FSM de comandos.
- Depois vêm:
 - ST_Busd
 - ST_BusB
 - ST_Bus[
 - ST_Bus0
 - ST_Bus.
 - ST_Bus4
 - ST_Bus]

Isto mostra que, no reader, o case 6 está realmente a ler cada carácter e a fazer o `Serial.println("ST_Bus...")` (parece que adicionaste esse print dentro do case 6). Ou seja: o `\n` está a chegar, porque senão não entravas no case 8 que faz a cópia para `ST_Command_Out/ST_Command_IN`.^[194]

Depois disso, o que vês é:

- Estado da leitura: 6 → a FSM reader ficou no state 6 (modo M, a ler caracteres).
- Estado da Interpretação: 24 → a FSM de comandos ficou presa no state 24.
- Muitos ST_Bus vazios → tu continuas a imprimir o buffer mesmo depois de teres terminado a leitura efetiva da linha.
- Muitos ERROR: Servo angle 1 out of range: Angle = -nan → a FSM principal continua a correr ganhos de IK com valores inválidos enquanto as FSMs de serial não “fecham” o comando.^[194]

Portanto, o problema não é o `\n` desaparecer; é:

- a FSM_Serial_reader não voltar ao state 0 depois de terminar a linha,
- e/ou a FSM_Serial_Command nunca sair do state 24 porque não há condição que a leve a 25/26, ou porque ST_Command_IN ainda não está preenchida quando 24 é executado.

Para perceberes exatamente onde se perde, o próximo passo bom é fazer um “dry run” com o comando `"wdB[0.4]\n"`:

- No reader:
 - em que momento o state passa de 6 para 8 e depois para 0?
- No command:
 - quando o state está em 24, o que há dentro de ST_Command_IN (primeiros 5 caracteres)?

Se quiseres, para avançarmos, podes dizer o que esperas que esteja em ST_Command_IN logo depois de enviar "wdb[0.4]" (por exemplo: dB[0.4] ou wdb[0.4]?), e qual destes estados queres que a FSM_Serial_Command atinja no fim (28 → 31 → 32 → 36 ou outro caminho).

✱

if(ST_BUS[counter-1] == ' '){state = 6;}

```
if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
```

```
if(ST_BUS[counter-1] == '\n'){state = 8;}
```

encontrei isto

Secalhar é esta parte `if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}`

Boa caça, é exatamente aí que está o bug principal da leitura.

Olha para estas três linhas juntas no case 6:

```
if(ST_BUS[counter-1] == ' '){state = 6;}
if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
if(ST_BUS[counter-1] == '\n'){state = 8;}
```

O que isto faz na prática:

1. Lê um char e metes em ST_BUS[counter-1].
2. Se for espaço, ficas em 6 (ok).
3. Se for '\n' e ainda houver dados disponíveis, também pões state = 6.
4. Logo a seguir, como ainda é '\n', o terceiro if tenta pôr state = 8.

Mas como estas condições são independentes, o resultado depende da ordem e do `Serial.available()` naquele ciclo. Em particular:

- Se ainda houver dados no buffer quando lê o '\n', a segunda linha força state = 6 e pode “anular” o salto para 8.
- Ficas preso no state 6, a ler “nada” e a imprimir ST_Bus vazio.

Na tua FSM, só precisas de duas coisas no case 6:

- Ignorar espaços
- Avançar para o estado 8 quando encontrares '\n'

Uma versão estável:

```

case 6:
    ST_BUS[counter] = Serial.read();
    Serial.print("ST_Bus");
    Serial.println(ST_BUS[counter]);    // o teu debug
    counter++;

    if (ST_BUS[counter-1] == ' ') {
        // ignora espaços e continua neste estado
        state = 6;
    } else if (ST_BUS[counter-1] == '\n') {
        // fim da linha -> vai preparar cópia
        state = 8;
    } else {
        // carácter normal -> continua a ler
        state = 6;
    }
    break;

```

Repara:

- Retirei completamente a condição `if(Serial.available() > 0 && ... '\n')`.
- O `'\n'` é sempre tratado como “fim da linha”, independentemente de ainda haver coisas na serial ou não (que é o que queres, porque a tua linha termina ali).

Depois, no case 8, tu já fazes:

```

int maxCopy = counter-1;
if (maxCopy >= (int)sizeof(ST_Command_Out)) {
    maxCopy = sizeof(ST_Command_Out) - 1;
}
ST_BUS[counter-1] = '\0';    // substitui o '\n'
strncpy(ST_Command_Out, ST_BUS, maxCopy);
ST_Command_Out[maxCopy] = '\0';
counter = 0;

```

Isto fecha bem a linha e limpa o counter, portanto, se o state chegar de forma consistente ao 8, ST_Command_Out e ST_Command_IN passam a ter exatamente o que escreveste no Pi.

Se aplicares esta correção, o próximo passo é testar outra vez com `"wdb[0.4]"` e ver:

- O que é impresso em ST_Command_Out (podes fazer um `Serial.println(ST_Command_Out)`; logo no fim do case 8).
- Em que estado fica a FSM de comandos depois (devias ver 24 → 25/26 → 27/28 → 31 → 32/35/36).

Para fechar: se reescreveres essa parte só com dois casos – “espaço” e “newline” – como explicarias tu próprio, em palavras simples, o comportamento desejado do case 6?

mas não parece estar a detetar mesmo nenhum \n

pq será?

nos caracteres simples ele parece detetar e receber o \n

o que eu estou a falar é no `Serial.Read` ele não deveria mostrar o \n se eu faço logo print em seguida?

Boa pergunta, e sim: se fizeres `Serial.read()` e logo a seguir `Serial.println(...)`, o \n também devia aparecer, só que normalmente não o vês como “\n”, mas como uma linha vazia ou um carácter estranho, dependendo de como imprimes. ^[205] ^[206]

Vamos por partes.

1. O que o Serial Monitor realmente envia

No monitor série da Arduino IDE (ou VSCode), tens uma combo “No line ending / Newline / Carriage return / Both”. ^[207] ^[205]

- Se estiver em **Newline**, ao carregares Enter, o PC envia:
 - todos os caracteres que escreveste
 - seguido de um byte 10 (LF, `'\n'`).
- Se estiver em **Both**, envia `'\r'` (13) e `'\n'` (10).

Portanto, sim: no fim do `"wdB[0.4]"` deverá vir um byte 10 que o `Serial.read()` consegue ler. ^[208] ^[205]

2. Porque “não vês” o \n no teu debug

No teu case 6 estás a fazer algo do género:

```
ST_BUS[counter] = Serial.read();
Serial.print("ST_Bus");
Serial.println(ST_BUS[counter]);
counter++;
```

Quando `ST_BUS[counter]` é um carácter normal (d, B, [, 0, ., 4,],) o `println` mostra:

```
ST_Busd
ST_BusB
ST_Bus[
ST_Bus0
...
```

Quando `ST_BUS[counter]` é `'\n'` (10):

- `Serial.println(ST_BUS[counter]);` imprime o carácter 10, que é “new line”, por isso o que vês é:

ST_Bus

com a linha a “mudar de sítio”, mas não aparece um símbolo visível para o `\n`.^[206]

Na prática, está a ser lido — a tua evidência de que vês várias linhas ST_Bus vazias confirma isso.

Se quiseres ver explicitamente o código ASCII, faz um teste rápido:

```
int c = Serial.read();
Serial.print("ST_Bus (dec) = ");
Serial.println(c);    // isto mostra 10 para '\n'
```

Isso confirma de vez se o 10 está a chegar.

3. O bug não é “não detectar `\n`”, é o `if` sobre o `\n`

O pedaço que encontrei:

```
if(ST_BUS[counter-1] == ' '){state = 6;}
if(Serial.available() > 0 && ST_BUS[counter-1] == '\n'){state = 6;}
if(ST_BUS[counter-1] == '\n'){state = 8;}
```

faz com que, em muitos casos, o estado volte para 6 mesmo quando o carácter é `\n`. A sequência é:

- lê `\n`
- `ST_BUS[counter-1] == '\n' → a segunda if pode fazer state = 6`
- logo a seguir a terceira if também vê `\n` e tenta fazer `state = 8`, mas a execução e o timing podem fazer com que fiques em 6 na iteração seguinte.

Se simplificares para:

```
if (ST_BUS[counter-1] == ' ') {
    state = 6;          // ignora espaços
} else if (ST_BUS[counter-1] == '\n') {
    state = 8;          // fim da linha
} else {
    state = 6;          // carácter normal
}
```

garantes que:

- sempre que lê `\n`, vais ao `state 8`
- nunca deixas o `state` em 6 “por cima” de um newline.

Portanto, respondendo diretamente à tua pergunta:

- Sim, o `Serial.read()` está a receber o `\n`.

- O que parece é que o teu debug print o mostra como linha vazia, e a tua lógica de `if(Serial.available() > 0 && ... '\n')` está a impedir o salto limpo para o state 8.

Se fizeres esse `Serial.println` com o valor decimal (como sugerido acima), achas que consegues confirmar se o 10 ('\n') aparece exatamente uma vez quando envias `"wdB[0.4]"`?

**

o problema está na deteção do \n

por algum motivo ele não está mesmo a ler, pois os outros erros são pq o ST_bus chegou ao tamanho maximo e deu overflow, o que nunca devia acontecer visto que enviei uma mensagem com 5 caracteres

Concordo contigo que, pelo comportamento, o \n não está a ser tratado como “fim da linha”, e isso faz o ST_BUS encher até ao limite. Mas repare que, com o código atual, há uma explicação simples: a FSM não está a sair do estado 6, logo continua a ler tudo o que chega (incluindo lixo posterior), em vez de parar no \n que fecha a tua mensagem.

Vamos isolar só o case 6 e garantir duas coisas:

1. Ver **objetivamente** se o \n está a ser lido.
2. Fazer a transição para o state 8 de forma determinística sempre que vier um \n.

1. Ver se o \n está a chegar mesmo

Troca temporariamente o case 6 para isto:

```
case 6: {
    int c = Serial.read();
    ST_BUS[counter] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[counter]); // pode aparecer “vazio” para \n
    Serial.print("   code = ");
    Serial.println(c);             // MOSTRA 10 para '\n'

    counter++;

    if (ST_BUS[counter-1] == ' ') {
        state = 6;
    } else if (ST_BUS[counter-1] == '\n') {
        state = 8;
    } else {
        state = 6;
    }
    break;
}
```

Com isto:

- Se enviares `wdB[0.4]` com “Newline” selecionado na IDE, deverás ver no monitor algo tipo:

- `char = 'w' code = 119`
- `char = 'd' code = 100`
- `char = 'B' code = 66`
- `char = '[' code = 91`
- `char = '0' code = 48`
- `char = '.' code = 46`
- `char = '4' code = 52`
- `char = ']' code = 93`
- `char = '' code = 10` (este é o `\n`)

Se neste print **nunca** aparece um `code = 10`, então o problema está no lado do PC: ou a IDE está em “No line ending” ou “CR only”, ou a tua string não leva `\n`. Se aparece `code = 10` e mesmo assim o state não vai para 8, então é 100% problema da FSM.

2. Evitar overflow do ST_BUS

Para não deixares o buffer rebentar quando, por qualquer razão, o `\n` não chegar, podes ainda proteger o counter:

```
if (counter >= (int)sizeof(ST_BUS) - 1) {
    // buffer cheio sem ter encontrado '\n' -> aborta linha
    ST_BUS[sizeof(ST_BUS) - 1] = '\0';
    state = 8;          // força processamento com o que tens
    break;
}
```

Colocas isto logo depois do `counter++`. Assim, mesmo que o newline falhe, nunca ficas a escrever para fora do array.

Então, resumindo a ideia:

- Primeiro, confirmar com o `code = 10` se o `\n` está a ser lido.
- Depois, com a lógica `if (...) else if ( == '\n') ...`, garantir que o state passa sempre para 8 quando esse 10 aparece.

Como exercício: se fizeres esse print do `code` e ele mostrar 13 (`'\r'`) mas nunca 10 (`'\n'`), qual seria o ajuste mais simples que farias na condição para tratar esse caso também?

`void FSM_Serial_reader(unsigned long nowMs){`

```
static short int state;
static int counter = 0;
static char CH_Bus;
static char ST_BUS[100];
```

```

state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

    if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

    if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

    if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

    if(1){state = 0;}
    break;

```

```

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;
/*
    case 6:
        ST_BUS[counter] = Serial.read();
        Serial.print("ST_Bus: ");
        Serial.println(ST_BUS[counter]);
        counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    //if(Serial.available() > 0 && ST_BUS[counter-1] != '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;
*/
    case 6:
        int c = Serial.read();
        ST_BUS[counter] = (char)c;

Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[counter]); // pode aparecer "vazio" para \n
    Serial.print(" code = ");
    Serial.println(c);          // MOSTRA 10 para '\n'

counter++;

if (ST_BUS[counter-1] == ' ') {
    state = 6;
} else if (ST_BUS[counter-1] == '\n') {
    state = 8;
} else {
    state = 6;
}
break;

case 8:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = counter-1; // aqui definimos a quantidade de caracteres a copiar, não em
termos do índice mas sim a quantidade
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    ST_BUS[counter-1] = '\0'; // substitui o '\n'
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    //ST_Command_Out[maxCopy] = '\0';

```

```

counter = 0;

break;
}
return;
}

```

deu-me este erro:

Compiling .pio\build\freenove_esp32_s3_wroom\src\main.cpp.o
src/main.cpp: In function 'void FSM_Serial_reader(long unsigned int)':
src/main.cpp:639:10: error: jump to case label [-fpermissive]
case 8:
^
src/main.cpp:620:9: note: crosses initialization of 'int c'
int c = Serial.read();
^
*** [.pio\build\freenove_esp32_s3_wroom\src\main.cpp.o] Error 1

Esse erro do compilador faz todo o sentido: é uma regra de C/C++ sobre variáveis declaradas dentro de switch.

Dentro de case 6 fizeste:

```

case 6:
    int c = Serial.read();    // <--- variável local declarada aqui
    ST_BUS[counter] = (char)c;
    ...
    break;

case 8:
    ...
    break;

```

Em C/C++, o switch não cria automaticamente um “bloco” novo por case.

Como o compilador vê isto como um único bloco contínuo, entende que existe uma variável `int c` que começa a existir em case 6 e continua “visível” até depois, passando por case 8. Isso viola as regras de saltos para dentro de escopos com variáveis não inicializadas (por isso a mensagem “jump to case label crosses initialization of 'int c'”). ^[220]

Para corrigir, há duas opções simples:

1. Usar um bloco {} à volta do case 6 (melhor aqui)

```

case 6: {
    int c = Serial.read();
    ST_BUS[counter] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[counter]);
    Serial.print("'   code = ");

```

```

    Serial.println(c);

    counter++;

    if (ST_BUS[counter-1] == ' ') {
        state = 6;
    } else if (ST_BUS[counter-1] == '\n') {
        state = 8;
    } else {
        state = 6;
    }
    break;
}

case 8:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = counter - 1;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    ST_BUS[counter-1] = '\0';
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    counter = 0;
    break;

```

O bloco { ... } em redor do case 6 garante que a variável int c só existe dentro desse bloco, e o salto para case 8 já não “atravessa” a inicialização dela. [\[220\]](#)

2. Declarar c antes do switch

Outra alternativa é:

```

void FSM_Serial_reader(unsigned long nowMs){
    static short int state;
    static int counter = 0;
    static char CH_Bus;
    static char ST_BUS[^53_100];
    int c = 0;                // <--- aqui

    switch (state) {
        ...
        case 6:
            c = Serial.read();    // só atribuis
            ST_BUS[counter] = (char)c;
            ...
            break;
        ...
    }
}

```

Aqui não há “cruzamento” de escopo, porque c é visível para todos os casos desde o início da função.

Depois de corrigires isso, volta a compilar e testa:

- O que o `Serial.println("ST_Bus char = ... code = ...")` imprime para cada carácter quando envias `wdB[0.4]`.
- Se aparece um `code = 10` ou `code = 13` no fim (LF ou CR).

Com base no que o print mostrar (10, 13 ou nenhum), consegues ajustar facilmente a condição do if para tratar '\n', '\r' ou ambos.

O que achas de, depois desta correção, mandares o output dos `code = ...` para a string `wdB[0.4]` para ver exatamente que bytes o ESP32 está a receber no fim da linha?

Esp32: Comando Mode multiple characters

[illegible]

[illegible]

[illegible]

escreveu isto

Isto agora explicou tudo: o problema não é o `\n`, é que não está a vir nenhum byte de fim de linha e tu estás a chamar `Serial.read()` mesmo quando já não há dados.

Repara no log:

```
d code = 100
r code = 114
t code = 116
[ code = 91
0 code = 48
. code = 46
4 code = 52
] code = 93
' code = -1
' code = -1
...
```

- Os primeiros 8 caracteres são exatamente `drt[0.4]` (porque o `w` já foi consumido no modo S), todos com códigos ASCII normais.
- A seguir, `Serial.read()` começa a devolver `-1`, que é o valor de retorno quando **não há mais nada no buffer** (`Serial.read()` retorna `-1` se `Serial.available() == 0`).^[221]

Ou seja:

- Não está a chegar nenhum `\n` (nem 10, nem 13).
- A tua FSM está no state 6 a chamar `Serial.read()` em todos os loops, mesmo com `Serial.available() == 0`, por isso vai enchendo `ST_BUS` com `-1` até rebentar.

Por isso:

1. Ou o lado do Pi **não está a enviar** `\n` (nem nenhum “line ending”).
2. Ou o Monitor Série / script está configurado para “No line ending”.

1. Proteger o case 6 contra `available() == 0`

Primeiro, precisas de evitar ler quando não há dados:

```
case 6: {
    if (Serial.available() == 0) {
        // nada para ler ainda, mantém o estado e sai
        break;
    }

    int c = Serial.read();
    ST_BUS[counter] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[counter]);
    Serial.print("'   code = ");
    Serial.println(c);

    counter++;

    if (c == ' ') {
        state = 6;
    } else if (c == '\n' || c == '\r') {    // já a pensar no futuro
        state = 8;
    } else {
        state = 6;
    }

    // proteção contra overflow
    if (counter >= (int)sizeof(ST_BUS) - 1) {
        ST_BUS[counter] = '\0';
        state = 8;    // força processamento do que veio
    }
    break;
}
```

Assim:

- Deixas de meter -1 no buffer.
- Vais para o state 8 só quando receberes realmente um terminador de linha.

2. Garantir que o Pi envia um terminador

No lado que está a enviar "wdB[0.4]", tens de garantir que, no fim, mandas \n (ou \r\n). Por exemplo, em Python:

```
ser.write(b"wdB[0.4]\n")
```

ou na IDE, garantir que a dropdown está em “Newline” ao carregar Enter. [\[222\]](#) [\[223\]](#)

No Arduino Serial Monitor:

- Abre o monitor.
- Em baixo à direita, muda “No line ending” para “Newline” (ou “Both NL & CR”). [\[222\]](#)

Sem isso, a tua FSM nunca vai ver um '\n' e vai ficar eternamente no state 6.

Portanto, há dois pontos:

- Confirmaste que o ESP32 não está a receber nenhum \n (só dados normais + -1).
- Agora precisas de:
 - proteger o case 6 para não ler quando Serial.available() == 0, e
 - garantir que o emissor envia mesmo um byte de fim de linha.

Como resumo: se ajustares o emissor para mandar \n no fim, que condição exata colocarias tu no if do case 6 para tratar ao mesmo tempo '\n' e '\r', de forma a suportar tanto “Newline” como “Both NL & CR”?

✱✱

eu agora onde tinha a situação para sair caso encontrasse '\n' pus tb caso detete ';

para sair, mas do nada deu-me este erro e ao que parece um reset do esp32

```
Esp32: ST_Bus char = 'd' code = 100
Esp32: ST_Bus char = 'r' code = 114
Esp32: ST_Bus char = 't' code = 116
Esp32: ST_Bus char = '[' code = 91
Esp32: ST_Bus char = '0' code = 48
Esp32: ST_Bus char = '!' code = 46
Esp32: ST_Bus char = '4' code = 52
Esp32: ST_Bus char = ']' code = 93
```

Esp32: ST_Bus char = ';' code = 59
Esp32: ST_Bus completo:
Esp32: ST_Bus completo:
Esp32: Guru Meditation Error: Core 0 panic'ed (Unhandled debug exception).
Esp32: Debug exception reason:
Esp32: assert failed: pcTaskGetName tasks.c:2803 (pxTCB)
Esp32: Backtrace: 0x403778ca:0x3fcf41b0 0x4037ac41:0x3fcf41d0 0x4038059d:0x3fcf41f0
0x4037ca02:0x3fcf4320 0x4201015d:0x3fcf4340 0x4200fade:0x3fcf4360 0x4200ff12:0x3fcf43d0
0x40377961:0x3fcf4420 0x4037912d:0x3fcf4440 0x00040021:0x3fcf4500 |<-CORRUPTED
Esp32: ELF file SHA256: 0000000000000000
Esp32: E (8921) esp_core_dump_flash: Core dump flash config is corrupted! CRC=0x7bd5c66f
instead of 0x0
Esp32: Rebooting...
Esp32:

Boa, agora deu para ver que:

- A leitura de ';' está a funcionar (código 59 aparece certinho).
- O crash não é da FSM em si, é um **Guru Meditation** típico de stack/corruption de memória no ESP32, logo depois do case 8 onde tratas ST_BUS completo. ^[224] ^[225]

O ponto crítico está aqui:

```
case 8:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = counter-1;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    ST_BUS[counter-1] = '\0';
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    //ST_Command_Out[maxCopy] = '\0';
    counter = 0;
    break;
```

Repara no que acontece no teu teste:

- Leste: `d r t [ 0 . 4 ] ;` → 8 caracteres + 1 terminador (';').
- counter foi incrementado para 9 depois de leres o ';'.
- Entraste no case 8 e fizeste:
 - `maxCopy = counter-1` → 8
 - `ST_BUS[counter-1] = '\0'` → `ST_BUS[^55_8] = '\0'`

Só que:

1. **Nunca limitaste o counter antes.** Se por acaso counter já foi crescendo em loops anteriores (devido a `Serial.read()` com -1, por exemplo), `counter-1` pode apontar para fora da parte válida da string e espetar '\0' num sítio aleatório da RAM.

2. Estás a chamar `Serial.println(ST_BUS)` ; antes de garantir que `ST_BUS` está terminada com `'\0'`. Se counter estivesse em 0 (ou cheio de lixo de antes), `println` vai procurar um `'\0'` algures na memória e isso, mais tarde, pode corromper estruturas do FreeRTOS, causando esse assert failed: `pcTaskGetName pxTCB.` [\[225\]](#) [\[224\]](#)

Para arrumar isto, faz três coisas simples:

1. Garante que counter nunca passa do tamanho do buffer

Logo no case 6:

```
case 6: {
    if (Serial.available() == 0) {
        break;
    }

    int c = Serial.read();
    if (counter >= (int)sizeof(ST_BUS) - 1) {
        // buffer cheio sem terminador -> força fecho
        ST_BUS[counter] = '\0';
        state = 8;
        break;
    }

    ST_BUS[counter] = (char)c;
    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[counter]);
    Serial.print("   code = ");
    Serial.println(c);

    counter++;

    if (c == ' ') {
        state = 6;
    } else if (c == '\n' || c == '\r' || c == ';') { // inclui ';'
        state = 8;
    } else {
        state = 6;
    }
    break;
}
```

2. Só usa `println(ST_BUS)` depois de terminar a string

No case 8:

```
case 8:
    // se não recebeste nada, não faças nada
    if (counter == 0) {
        state = 0;
        break;
    }
```

```
// garante que há espaço e termina a string
if (counter >= (int)sizeof(ST_BUS)) {
    ST_BUS[sizeof(ST_BUS) - 1] = '\0';
} else {
    ST_BUS[counter] = '\0';    // mete o '\0' a seguir ao último char válido
}

Serial.print("ST_Bus completo: ");
Serial.println(ST_BUS);

int maxCopy = counter;
if (maxCopy >= (int)sizeof(ST_Command_Out)) {
    maxCopy = sizeof(ST_Command_Out) - 1;
}

strncpy(ST_Command_Out, ST_BUS, maxCopy);
ST_Command_Out[maxCopy] = '\0';

counter = 0;
state = 0;    // volta ao estado inicial da FSM reader
break;
```

Repara que aqui:

- Não mexemos em `ST_BUS[counter-1]`, só garantimos `ST_BUS[counter] = '\0'` (a seguir ao último char útil).
- Copiamos exatamente `counter` caracteres válidos.
- As duas strings (`ST_BUS` e `ST_Command_Out`) ficam sempre terminadas.

3. Inicializa `state` e trata estados inválidos

No topo da função:

```
static short int state = 0;
```

E no fim do `switch` (se quiseses ser robusto):

```
default:
    state = 0;
    counter = 0;
    break;
```

Isto evita que o reader fique perdido em estados esquisitos se algo correr mal.

Esse Guru Meditation normalmente acontece quando alguma escrita fora de limites apanha estruturas de tarefas do FreeRTOS; neste caso, é quase certo que o `ST_BUS` ou `ST_Command_Out` pisaram memória de alguém ao mexer em `counter-1` e ao usar `println` antes de pôr o `'\0'` no sítio certo. ^[224] ^[225]

Se fizeres estas três correções, queres tentar de novo com `wdB[0.4]`; e depois mostrar só:

- o ST_Bus completo: ...
- e o que fica em ST_Command_Out (podes pôr um `Serial.println(ST_Command_Out);` logo a seguir no case 8)?

Isso vai mostrar se a linha está a fechar exatamente onde queres.

✱✱

void FSM_Serial_reader(unsigned long nowMs){

```
static short int state;
static int counter = 0;
static int n = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

    if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

    if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;
```

```

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

    if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

    if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

    if(LineReady_IN == true){state = 0;}
    break;
/*
    case 6:
        ST_BUS[counter] = Serial.read();
        Serial.print("ST_Bus: ");
        Serial.println(ST_BUS[counter]);
        counter++;

    if(ST_BUS[counter-1] == ' '){state = 6;}
    //if(Serial.available() > 0 && ST_BUS[counter-1] != '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;
*/
    case 6: {
        int c = Serial.read();
        ST_BUS[n] = (char)c;

        Serial.print("ST_Bus char = ");
        Serial.print(ST_BUS[n]);
        Serial.print(" code = ");
        Serial.println(c);

        if(ST_BUS[n] == ' '){state = 6;}
        if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
        if(ST_BUS[n] == '\n'){state = 8;}
        break;
    }
}

```

```

case 7:
    n++;

if(Serial.available() > 0){state = 6;}
    if(Serial.available() > 1){state = 9;}
    break;

case 8:
    if(1){state = 9;}
    break;

case 9:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = n; // aqui definimos a quantidade de caracteres a copiar, não em termos do
indice mas sim a quantidade
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    ST_BUS[n] = '\0'; // substitui o '\n'
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    //ST_Command_Out[maxCopy] = '\0';
    n = 0;

if(LineReady_IN == true){state = 0;}
    break;

}
return;
}

```

Mudei algumas coisas no modo 'M'
para ficar mais parecido ao funcionamento do modo 'S'
pois esse estava a funcionar bem antes
o que achas?

A lógica geral ficou mais parecida com o modo S e isso ajuda a pensar, mas do jeito que está agora há três problemas concretos que vão causar comportamentos estranhos:

1. **n** nunca é reinicializado fora do case 9
2. Não proteges `Serial.read()` com `Serial.available()` no case 6
3. A condição em 7 (`available() > 0` e depois `> 1`) é frágil para linhas pequenas

Vamos olhar só para o fluxo M (estados 6, 7, 8, 9) com a string:

```
wdB[0.4]\n
```

1. O que a tua FSM nova faz (6–9)

- Começa no state 6 com n inicialmente 0.
- Lê um char:

```
int c = Serial.read();
ST_BUS[n] = (char)c;
...
if(ST_BUS[n] == ' '){state = 6;}
if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
if(ST_BUS[n] == '\n'){state = 8;}
```

Portanto:

- Se for espaço → fica em 6 e não incrementa n.
- Se for normal (d, r, t, [, 0, ., 4,]) → vai para 7 sem terminar a string e sem incrementar n.
- Se for \n → vai para 8.

No state 7:

```
n++;

if(Serial.available() > 0){state = 6;}
if(Serial.available() > 1){state = 9;}
```

- Incrementa n e decide:
 - se ainda há 1 byte → volta a 6 para ler mais
 - se já há mais de 1 byte no buffer → salta para 9 diretamente (sem passar pelo 8).

No state 8:

```
if(1){state = 9;}
```

Ou seja, o 8 é só “ponte” para 9.

No state 9:

```
Serial.print("ST_Bus completo: ");
Serial.println(ST_BUS);
int maxCopy = n;
...
ST_BUS[n] = '\0';
strncpy(ST_Command_Out, ST_BUS, maxCopy);
n = 0;

if(LineReady_IN == true){state = 0;}
```

Aqui:

- Usas `n` como comprimento da string.
- Metes `'\0'` em `ST_BUS[n]`.
- Copias `maxCopy = n` chars.
- **Só voltas ao state 0 quando** `LineReady_IN == true`.

Então, problemas:

- Se `LineReady_IN` ainda não estiver `true` (vem da FSM de comandos), o reader fica preso em 9 a imprimir `ST_Bus` completo: ... em todos os loops.
- Se a linha for curta e `Serial.available()` estiver a oscilar entre 0 e 1 num timing esquisito, podes ir para 9 com `n` ainda pequeno ou até 0.
- Se em algum momento o state 6 for chamado com `Serial.available() == 0`, `Serial.read()` devolve -1 e metes isso em `ST_BUS[n]`, enchendo com lixo.

2. Como deixar esta versão robusta mantendo o teu estilo

A ideia que queres é:

- 6: ler um carácter
- 7: avançar índice
- 8/9: processar string completa

Sugestão mínima, aproveitando o que já tens:

```
case 6: {
    if (Serial.available() == 0) {
        // nada para ler ainda
        break;
    }

    int c = Serial.read();
    ST_BUS[n] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[n]);
    Serial.print("'   code = ");
    Serial.println(c);

    if (ST_BUS[n] == ' ') {
        // ignora espaços, não avança n
        state = 6;
    } else if (ST_BUS[n] == '\n' || ST_BUS[n] == ';' ) {
        // fim de comando -> fecha string
        ST_BUS[n] = '\0';
        state = 9;    // salta direto para o processamento
    } else {
        // carácter normal -> avança índice
        n++;
        if (n >= (int)sizeof(ST_BUS) - 1) {
            // evita overflow: força terminação
            ST_BUS[n] = '\0';
        }
    }
}
```

```

        state = 9;
    } else {
        state = 6;
    }
}
break;
}

case 9:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);

    int maxCopy = n; // n agora é o comprimento da string
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';

    n = 0; // prepara para próxima linha
    if (LineReady_IN == true) {
        state = 0;
    } else {
        // se ainda não te deixaram continuar, ficas aqui parado
        state = 9;
    }
    break;
}

```

E no topo da função:

```

static short int state = 0;
static int n = 0;
static char ST_BUS[100] = {0};

```

Assim:

- Não lês se `Serial.available() == 0`.
- Não dependes de `Serial.available() > 1` para decidir se já acabou a linha.
- Consideras `'\n'` ou `','` como “fim de comando”.
- `n` é sempre o tamanho real da string, e a terminação está garantida (`ST_BUS[n] = '\0'`).

A tua ideia de “modo M funcionar parecido com o S” é boa; para ficar mesmo fiável, o ponto-chave é **não chamar** `Serial.read()` **sem olhar para** `Serial.available()`, e **usar sempre um terminador claro** (no teu caso, `\n` ou `;`) em vez de depender da contagem de bytes disponíveis.

Como exercício rápido: olhando para este fluxo novo, como descreverias tu a diferença de papel entre as variáveis `counter` e `n` na FSM de leitura em modo M?

Se em algum momento o state 6 for chamado com `Serial.available() == 0`, `Serial.read()` devolve -1 e metes isso em `ST_BUS[n]`, enchendo com lixo

isto nunca pode acontecer, pois ele quando entra no 6 pela primeira vez, têm lá um valor, e quando vai repetir a segunda tentativa (no ponto de vista do 7) ele vê se ainda existe alguma coisa, por isso nunca vai passar do nada a 0 o bus

em relação a isto

Se a linha for curta e `Serial.available()` estiver a oscilar entre 0 e 1 num timing esquisito, podes ir para 9 com `n` ainda pequeno ou até 0.

Este modo é só destinado a ler algo grande, para valores pequenos ele vai logo para o outro modo

Faz sentido o que dizes sobre a intenção, e é bom sinal que estejas a pensar no “timing” entre 6 e 7. Mas, mesmo com essa intenção, o código atual não garante isso em 100% dos ciclos, por causa de duas coisas:

- `FSM_Serial_reader` corre a cada `loop()`, várias vezes por segundo.
- Entre uma chamada e outra, o estado pode ficar em 6 sem novos bytes terem chegado.

1. Por que `Serial.read()` com `available()==0` ainda pode acontecer

Olha para a sequência 6–7:

```
case 6: {
    int c = Serial.read();
    ST_BUS[n] = (char)c;
    ...
    if(ST_BUS[n] == ' '){state = 6;}
    if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
    if(ST_BUS[n] == '\n'){state = 8;}
    break;
}

case 7:
    n++;
    if(Serial.available() > 0){state = 6;}
    if(Serial.available() > 1){state = 9;}
    break;
```

Cenário realista com `"drt[0.4];"`:

- Primeiro loop:
 - `state = 6` → `read()` lê 'd', vai para 7.
- Segundo loop:
 - `state = 7` → `n++` → `Serial.available() > 0`? Sim → `state = 6`.

- Terceiro loop:
 - `state = 6` outra vez → assumes que ainda há dados.

Agora imagina que, entre o segundo e o terceiro loop, não chegou ainda o próximo byte (por exemplo, porque o lado do Pi está ligeiramente atrasado, ou porque a UART está a 115200 e o teu loop é muito rápido). Nesse instante:

- No terceiro loop, `state == 6`, mas `Serial.available() == 0`.
- O teu código entra no `case 6` na mesma, chama `Serial.read()`, que devolve `-1`, e esse `-1` vai para `ST_BUS[n]`.

Ou seja: o “nunca pode acontecer” que descreves é uma suposição sobre o tempo entre 7 e 6, mas o código não tem nenhum `if (Serial.available() == 0) break;` para o garantir.

Mesmo que na prática o Pi esteja a mandar tudo muito rápido e isso raramente aconteça, basta um ciclo em que o ESP32 esteja ocupado (por exemplo, a mexer nos servos ou na IK) para apanhar um instante com 0 bytes disponíveis.

2. “Este modo é só para ler algo grande”

Percebo: queres usar o modo M para comandos mais longos e deixar os curtyinhos para o modo S. Mas o problema não é o tamanho da linha; é:

- a sincronia entre o `loop()` e a chegada de bytes
- a ausência de proteção no `case 6`
- e a dependência de `Serial.available() > 1` no 7.

Mesmo num comando grande, o envio pode vir em “rajadas”: alguns bytes, pequena pausa, mais alguns bytes. Durante essa pausa, a tua FSM pode ficar em 6 com `Serial.available() == 0` e cair no `read()`.

3. Pequeno ajuste que resolve isso sem mudar a tua ideia

Mantendo a tua estrutura 6–7–8–9, basta pôr uma guarda no 6:

```
case 6: {
    if (Serial.available() == 0) {
        // ainda não há nada novo, espera
        break;
    }

    int c = Serial.read();
    ST_BUS[n] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[n]);
    Serial.print("    code = ");
    Serial.println(c);

    if (ST_BUS[n] == ' ') {
        state = 6;
    }
}
```



```

    } else if (ST_BUS[n] == '\n' || ST_BUS[n] == ';') {
        state = 8;    // fim do comando
    } else {
        state = 7;    // carácter normal, vai para 7 incrementar n
    }
    break;
}

```

E no 7:

```

case 7:
    n++;
    if (n >= (int)sizeof(ST_BUS) - 1) {
        ST_BUS[n] = '\0'; // evita overflow
        state = 9;        // força processamento
    } else if (Serial.available() > 0) {
        state = 6;        // há mais dados, volta a ler
    } else {
        // não há mais dados ainda; fica aqui, não corre para 6 nem 9
        // até aparecer algo novo
    }
    break;

```

Assim:

- Nunca lê com `available() == 0`.
- Nunca ultrapassa o tamanho de `ST_BUS`.
- A linha só fecha quando detectas `\n` ou `;`, não só porque “agora o `available` ficou `>1`”.

A tua ideia base continua: 6 lê, 7 avança, 8/9 processam. Só adicionas guard-rails para o mundo real do ESP32.

Para verificar que estas guardas fazem sentido para ti: se tivesses de explicar a alguém “porque é que é perigoso chamar `Serial.read()` sem testar `Serial.available()` num loop rápido como o do ESP32”, como é que o explicavas em duas frases?

void FSM_Serial_reader(unsigned long nowMs){

```

static short int state;
static int counter = 0;
static int n = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*

```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar
- Condições de transição por hierarquia

```

*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

    if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

    if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;

case 3:
    CH_Bus = '\0';

    if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

    if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

    if(LineReady_IN == true){state = 0;}
    break;
/*

```

```

    case 6:
        ST_BUS[counter] = Serial.read();
        Serial.print("ST_Bus: ");
        Serial.println(ST_BUS[counter]);
        counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    //if(Serial.available() > 0 && ST_BUS[counter-1] != '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;
*/
    case 6: {
        int c = Serial.read();
        ST_BUS[n] = (char)c;

Serial.print("ST_Bus char = ");
        Serial.print(ST_BUS[n]);
        Serial.print(" code = ");
        Serial.println(c);

if(ST_BUS[n] == ' '){state = 6;}
        if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
        if(ST_BUS[n] == '\n'){state = 8;}
        break;
    }

    case 7:
        n++;

if(Serial.available() > 0){state = 6;}
        if(Serial.available() > 1){state = 8;}
        break;

    case 8:
        Serial.print("ST_Bus completo: ");
        Serial.println(ST_BUS);
        int maxCopy = n; // aqui definimos a quantidade de caracteres a copiar, não em termos do
indice mas sim a quantidade
        if (maxCopy >= (int)sizeof(ST_Command_Out)) {
            maxCopy = sizeof(ST_Command_Out) - 1;
        }
        strncpy(ST_Command_Out, ST_BUS, maxCopy);

if(1){state = 9;}
        break;

    case 9:
        ST_BUS[n] = '\0'; // substitui o '\n'

```

```
//ST_Command_Out[maxCopy] = '\0';
n = 0;

if(LineReady_IN == true){state = 0;}
    break;

}
    return;
}
```

"Agora imagina que, entre o segundo e o terceiro loop, não chegou ainda o próximo byte (por exemplo, porque o lado do Pi está ligeiramente atrasado, ou porque a UART está a 115200 e o teu loop é muito rápido). Nesse instante:"

onde é que isso faz sentido?

as mensagens são enviadas quase de uma só vez, cada mensagem alterei um outro cenário no código

Há duas coisas diferentes aqui:

1. A tua intuição sobre “as mensagens vêm quase de uma só vez”.
2. O que o teu código faz *independentemente* da forma como os bytes chegam.

Mesmo que o Pi mande tudo de uma só vez, o ESP32:

- lê a UART em “pacotes” conforme o `loop()` corre
- o hardware UART e o driver têm o seu próprio buffer interno
- e `Serial.available()` pode ser >0, depois 0, depois >0 de novo se o teu loop for muito rápido em relação ao hardware.

Ou seja: **não é garantido** que cada vez que entras no case 6 haja sempre pelo menos 1 byte disponível, a menos que tu próprio o garantas com um `if (Serial.available() == 0) break;`.

[226]

No teu código atual para modo M:

```
case 6: {
    int c = Serial.read();          // lê SEM testar available
    ST_BUS[n] = (char)c;
    ...
    if(ST_BUS[n] == ' '){state = 6;}
    if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
    if(ST_BUS[n] == '\n'){state = 8;}
    break;
}

case 7:
    n++;
    if(Serial.available() > 0){state = 6;}
    if(Serial.available() > 1){state = 8;}
    break;
```

Mesmo com mensagens “rápidas”, o que pode acontecer:

- No momento T, ainda há bytes $\rightarrow$ 6 lê, 7 vê `available() > 0` e aponta de novo para 6.
- No momento $T+\Delta$, o loop volta, o state ainda é 6, mas os bytes que o hardware tinha no buffer já foram todos lidos (inclusive por este próprio código) $\rightarrow$ `Serial.available()` é 0, mas tu **não testas** isso, chamas `Serial.read()` e recebes -1, que vai para `ST_BUS[n]`.^[226]

É por isso que tinhas visto antes linhas com `code = -1` a encher o `ST_BUS`: exatamente esse cenário.

Além disso, no fluxo novo:

- Nunca terminas `ST_BUS` com `'\0'` antes do `Serial.println(ST_BUS)`; no case 8.
- Alteraste 8 para:

```
Serial.print("ST_Bus completo: ");
Serial.println(ST_BUS);
int maxCopy = n;
...
strncpy(ST_Command_Out, ST_BUS, maxCopy);

if(1){state = 9;}
```

e só no 9 é que fazes `ST_BUS[n] = '\0'`;

Isso significa que, se `n` não tiver sido gerido exatamente como esperas, podes estar a imprimir lixo ou a pisar memória inadvertidamente.

Se queres manter esta estrutura 6-7-8-9, mas sem surpresas, o mínimo seguro (sem mexer na tua ideia base) é:

```
case 6: {
    if (Serial.available() == 0) {
        // ainda não há nada novo, não faças read
        break;
    }

    int c = Serial.read();
    ST_BUS[n] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[n]);
    Serial.print("   code = ");
    Serial.println(c);

    if (ST_BUS[n] == ' ') {
        state = 6;
    } else if (ST_BUS[n] == '\n' || ST_BUS[n] == ';' ) {
        // fim do comando
        ST_BUS[n] = '\0';
        state = 8;
    } else {
        state = 7;
    }
}
```

```

        break;
    }

    case 7:
        n++;
        if (n >= (int)sizeof(ST_BUS) - 1) {
            ST_BUS[n] = '\0';
            state = 8;    // evita overflow, força fechar
        } else if (Serial.available() > 0) {
            state = 6;    // há mais dados para ler
        } else {
            // espera mais dados; não volta a 6 sem available>0
        }
        break;

    case 8:
        Serial.print("ST_Bus completo: ");
        Serial.println(ST_BUS);

        int maxCopy = strlen(ST_BUS);    // agora é string bem terminada
        if (maxCopy >= (int)sizeof(ST_Command_Out)) {
            maxCopy = sizeof(ST_Command_Out) - 1;
        }
        strncpy(ST_Command_Out, ST_BUS, maxCopy);
        ST_Command_Out[maxCopy] = '\0';

        n = 0;
        state = 9;
        break;

    case 9:
        if (LineReady_IN == true) {
            state = 0;
        }
        break;

```

Assim:

- Nunca lês com `available()==0`.
- Sempre que decides que o comando acabou (`\n` ou `;`), marcas o `'\0'` logo ali.
- O `println(ST_BUS)` em 8 passa a ser sempre sobre uma string bem formada.

Mesmo que as mensagens venham “todas de uma vez”, estas guardas não te prejudicam, só te protegem nos ciclos onde o ESP32 chega à FSM antes de a UART ter enchido.

Para fechar: como explicarias tu a alguém a diferença entre “eu acho que o emissor manda tudo de uma vez” e “o meu código garante que nunca tento ler quando não há dados”?

✱✱

Compiling

.pio\build\freenove_esp32_s3_wroom\src\main.cpp.o

src/main.cpp: In function 'void FSM_Serial_reader(long unsigned int)':

src/main.cpp:656:10: error: jump to case label [-fpermissive]

case 9:

^

src/main.cpp:646:9: note: crosses initialization of 'int maxCopy'

int maxCopy = n; // aqui definimos a quantidade de caracteres a copiar, não em termos do índice mas sim a quantidade

~~~~~

\*\*\* [.pio\build\freenove\_esp32\_s3\_wroom\src\main.cpp.o] Error 1

estava a dar-me este erro no meu código

Esse erro é o mesmo padrão de há pouco: variável declarada dentro de um case sem bloco {} e o compilador reclama que o salto para o case seguinte “atravessa” a inicialização dessa variável.

Tens isto (resumindo):

```
case 8:
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = n; // ...
    ...
    if(1){state = 9;}
    break;

case 9:
    ST_BUS[n] = '\0';
    n = 0;
    ...
    break;
```

C/C++ vê o switch como um único bloco contínuo. A variável `int maxCopy` “nasce” em case 8 e continua viva até ao fim do switch. O salto para case 9 pode entrar “no meio” da sua existência, então o compilador não deixa. <sup>[227]</sup>

Há duas formas simples de resolver:

### 1. Envolver o case 8 num bloco {}

```
case 8: {
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = n;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
```

```

    state = 9;
    break;
}

case 9:
    ST_BUS[n] = '\0';
    n = 0;
    if (LineReady_IN == true) { state = 0; }
    break;

```

Repara no { logo a seguir ao case 8: e no } antes do case 9:. Isso limita o escopo de maxCopy só ao case 8.

## 2. Declarar maxCopy antes do switch

```

void FSM_Serial_reader(unsigned long nowMs){
    static short int state;
    static int counter = 0;
    static int n = 0;
    static char CH_Bus;
    static char ST_BUS[59_100];
    int maxCopy = 0;    // aqui

    switch (state) {
        ...
        case 8:
            Serial.print("ST_Bus completo: ");
            Serial.println(ST_BUS);
            maxCopy = n;
            if (maxCopy >= (int)sizeof(ST_Command_Out)) {
                maxCopy = sizeof(ST_Command_Out) - 1;
            }
            strncpy(ST_Command_Out, ST_BUS, maxCopy);
            state = 9;
            break;

        case 9:
            ST_BUS[n] = '\0';
            n = 0;
            if (LineReady_IN == true) { state = 0; }
            break;
    }
}

```

Aqui maxCopy já existe antes do switch, por isso o compilador deixa usar nos dois casos.

Qual destas duas abordagens te parece mais limpa para continuares a organizar os teus outros cases também (por exemplo, sempre pôr {} nos casos que declaram variáveis locais)?



**eu agora escrevi isto:**

`wdrt[0.4]`

e ele pos-me isto no monitor:

[illegible]

não consigo entender como é que ele entrou no 34, já que só devia entrar no 34 caso eu tivesse colocado: wdrM[0.4]

código:

```
void FSM_Serial_reader(unsigned long nowMs){
static short int state;
static int counter = 0;
static int n = 0;
static char CH_Bus;
static char ST_BUS[100];
state_test = state;
/*
Têm em atenção constituição de cada caso:
case x:
- Ações a executar
- Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    counter = 0;
    CH_Command_Out = '\0';

if(Serial.available() > 0 && Mode == 'S'){state = 1;}
    if(Serial.available() > 0 && Mode == 'M'){state = 6;}
    break;

case 1:
    CH_Bus = Serial.read();
    counter++;

if(CH_Bus == ' '){state = 1;}
    if(CH_Bus != '\n' && CH_Bus != ' '){state = 2;}
    if(CH_Bus == '\n'){state = 4;}
    break;

case 2:
    CH_Command_Out= CH_Bus;
    command_counter++;

    if (Serial.available() > 0) { state = 3;}
    else {state = 5;}
    break;
```

```

case 3:
    CH_Bus = '\0';

if(LineReady_IN == true && Mode == 'S'){state = 1;}
    if(LineReady_IN == true && Mode == 'M'){state = 6;}
    break;

case 4:
    CH_Bus = '\0';

if(1){state = 0;}
    break;

case 5:
    CH_Bus = '\0';

if(LineReady_IN == true){state = 0;}
    break;
/*
    case 6:
        ST_BUS[counter] = Serial.read();
        Serial.print("ST_Bus: ");
        Serial.println(ST_BUS[counter]);
        counter++;

if(ST_BUS[counter-1] == ' '){state = 6;}
    //if(Serial.available() > 0 && ST_BUS[counter-1] != '\n'){state = 6;}
    if(ST_BUS[counter-1] == '\n'){state = 8;}
    break;
*/
    case 6: {
        int c = Serial.read();
        ST_BUS[n] = (char)c;

Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[n]);
    Serial.print(" code = ");
    Serial.println(c);

if(ST_BUS[n] == ' '){state = 6;}
    if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
    if(ST_BUS[n] == '\n'){state = 8;}
    break;
}

case 7:
    n++;

```

```

if(Serial.available() > 0){state = 6;}
    if(Serial.available() > 1){state = 8;}
    break;

case 8: {
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = n;
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    strncpy(ST_Command_Out, ST_BUS, maxCopy);

    if(1){state = 9;}
    break;
}

case 9:
    ST_BUS[n] = '\0'; // substitui o '\n'
    //ST_Command_Out[maxCopy] = '\0';
    n = 0;

if(LineReady_IN == true){state = 0;}
    break;

}
    return;
}

void FSM_Serial_Command(unsigned long nowMs){
static short int state;
state_test2 = state;
static int n = 0;
static int m = 0;
static char ST_copy[100];
/*
Têm em atenção constituição de cada caso:
case x:
    - Ações a executar
    - Condições de transição por hierarquia
*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (state) {

case 0:
    Serial.println("Escolha um comando:");

```

```

if(1){state = 1;}

break;

case 1:
    LineReady_Out = false;

if(CH_Command_IN == 'S' || CH_Command_IN == 's'){state = 2;}
    if(CH_Command_IN == 'P' || CH_Command_IN == 'p'){state = 3;}
    if(CH_Command_IN == 'L' || CH_Command_IN == 'l'){state = 5;}
    if(CH_Command_IN == 'M' || CH_Command_IN == 'm'){state = 14;}
    if(CH_Command_IN == 'W' || CH_Command_IN == 'w'){state = 23;}
    break;

case 2:
    Serial.println("Ativo");
    start_comand = 1;
    Training_Exame_Starttime = nowMs;
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 0, 0));
    pixel.show();

if (1) { state = 4;}
    break;

case 3:
    Serial.println("Desativo");
    start_comand = 0;
    Training_Exame_Finishingtime = nowMs;
    Training_Exame_Durantiontime = Training_Exame_Finishingtime -
Training_Exame_Starttime;
    if(Training_Exame_Durantiontime > 0){
        Serial.print("Duração do treino: ");
        Serial.print(Training_Exame_Durantiontime/1000.0f);
        Serial.println(" Segundos");
    }
    Training_Exame_Finishingtime = 0;
    Training_Exame_Starttime = 0;
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 0, 0));
    pixel.show();

    if (1) { state = 4;}
    break;

```

```

case 4:
    Serial.println("Escolha um comando:");
    LineReady_Out = false;

if(1){state = 1;}
    break;

case 5:
    Serial.println("ler");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 0));
    pixel.show();

if(1){state = 6;}
    break;

case 6:
    LineReady_Out = false;

if(CH_Command_IN == '1'){state = 7;}
    if(CH_Command_IN == '2'){state = 8;}
    if(CH_Command_IN == '3'){state = 9;}
    if(CH_Command_IN == 'r'){state = 10;}
    if(CH_Command_IN == 'b'){state = 11;}
    if(CH_Command_IN == 't'){state = 12;}
    if(CH_Command_IN == 'a'){state = 13;}
    break;

case 7:
    Serial.println("Comando L1");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(255, 255, 255));
    pixel.show();
    Safe_comand = 1;

if(1){state = 4;}
    break;

case 8:
    Serial.println("Comando L2");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    pixel.setPixelColor(0, pixel.Color(0, 255, 255));
    pixel.show();
    Safe_comand = 0;

```

```

if(1){state = 4;}
    break;

case 9:
    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Serial.println("Sem nada"); //Retirar quando estiver implementado

if(1){state = 4;}
    break;

case 10:
    Serial.println("Comando Lr");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    debugPrintMove(Resp_move, "Resp_move");
    debugPrintInstance(Resp_inst, "Resp_inst");


    if(1){state = 4;}
    break;

case 11:
    Serial.println("Comando Lb");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    debugPrintMove(Bat_move, "Bat_move");
    debugPrintInstance(Bat_inst, "Bat_inst");


    if(1){state = 4;}
    break;

case 12:
    Serial.println("Comando Lt");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    //debugPrintMove(Bat_move, "Tosse_move"); //ainda é preciso implementar
    //debugPrintInstance(Bat_inst, "Tosse_inst"); //ainda é preciso implementar
    Serial.println("Sem nada"); //Retirar quando estiver implementado


    if(1){state = 4;}
    break;

case 13:
    Serial.println("Comando LA");
    LineReady_Out = true;
    CH_Command_Out = '\0';

```



```
debugPrintMove(Bat_move, "Bat_move");
debugPrintMove(Resp_move, "Resp_move");
debugPrintInstance(Bat_inst, "Bat_inst");
debugPrintInstance(Resp_inst, "Resp_inst");
Serial.println("Sem Tosse"); //Retirar quando estiver implementado a tosse
```

```
if(1){state = 4;}
break;
```

case 14:

```
Serial.println("Comando Mode");
start_comand = 0;
Training_Exame_Finishingtime = nowMs;
Training_Exame_Durantiontime = Training_Exame_Finishingtime -
Training_Exame_Starttime;
if(Training_Exame_Durantiontime > 0){
    Serial.print("Duração do treino: ");
    Serial.print(Training_Exame_Durantiontime/1000.0f);
    Serial.println(" Segundos");
}
Training_Exame_Finishingtime = 0;
Training_Exame_Starttime = 0;
LineReady_Out = true;
```

```
if(1){state = 15;}
break;
```

case 15:

```
LineReady_Out = false;
```

```
if(CH_Command_IN == 'a'){state = 16;}
if(CH_Command_IN == 'h'){state = 17;}
if(CH_Command_IN == 'r'){state = 18;}
if(CH_Command_IN == 'b'){state = 19;}
if(CH_Command_IN == 's'){state = 20;}
break;
```

case 16:

```
Serial.println("Comando Mode ALL");
LineReady_Out = true;
CH_Command_Out = '\0';
Bat_inst.active = true;
Resp_inst.active = true;
//Tosse_inst.active = true;
```

```
if(1){state = 21;}
break;
```

```
case 17:
    Serial.println("Comando Mode Human (Bat+Resp)");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = false;
```

```
if(1){state = 21;}
    break;
```

```
case 18:
    Serial.println("Comando Mode Respiração");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = true;
    //Tosse_inst.active = false;
```

```
if(1){state = 21;}
    break;
```

```
case 19:
    Serial.println("Comando Mode Batimento");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = false;
    //Tosse_inst.active = false;
```

```
if(1){state = 21;}
    break;
```

```
case 20:
    Serial.println("Comando Mode Static");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = false;
    Resp_inst.active = false;
    //Tosse_inst.active = false;
```

```
if(1){state = 21;}
    break;
```

```
case 21:
    Serial.println("want to start de cicle? y/n");
    LineReady_Out = false;
```

```
if(1){state = 22;}
    break;

case 22:
    if(CH_Command_IN == 'y'){state = 2;}
    if(CH_Command_IN == 'n'){state = 3;}
    break;

case 23:
    Mode = 'M';
    Serial.println("Comando Mode multiple characters");
    LineReady_Out = true;

if(1){state = 24;}
    break;

case 24:
    LineReady_Out = false;
    n = 0;

if(ST_Command_IN[n] == 'D'){state = 25;}
    if(ST_Command_IN[n] == 'd'){state = 26;}
    break;

case 25:
    start_comand = 0;
    n++;

if(1){state = 27;}
    break;

case 26:
    n++;

if(1){state = 28;}
    break;

case 27:
    if(ST_Command_IN[n] == 'R' || ST_Command_IN[n] == 'r'){state = 28;}
    if(ST_Command_IN[n] == 'B' || ST_Command_IN[n] == 'b'){state = 29;}
    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 30;}
    break;

case 28:
    currentMove = &Resp_move;
    currentInst = &Resp_inst;

if(1){state = 31;}
    break;
```

```

case 29:
    currentMove = &Bat_move;
    currentInst = &Bat_inst;

if(1){state = 31;}
    break;

case 30:
    currentMove = &Tosse_move;
    currentInst = &Tosse_inst;

if(1){state = 31;}
    break;

case 31:
    n++;

if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;} //Periodo
    if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;} //Pausa
    if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;} //Movimento
    break;

case 32:
    n++;
    m = 0;

if(ST_Command_IN[n] == '['){state = 32;}
    if(ST_Command_IN[n] != '['){state = 35;}
    break;

case 33:
    //L
    break;

case 34:
    //L
    break;

case 35:
    ST_copy[m]=ST_Command_IN[n];
    n++;
    m++;

if(ST_Command_IN[n] != 'I'){state = 35;}
    if(ST_Command_IN[n] == 'I'){state = 36;}
    break;

case 36:
    //L
    currentMove->period=atof(ST_copy);

```

```

    memset(ST_copy,0,sizeof(ST_copy));
    LineReady_Out = true;

if(1){state = 4;}
    break;
}

    return;
}

```

O que está a acontecer é coerente com o teu código atual, mesmo que não seja o que queres.

## 1. O que é que o reader está a passar para o FSM\_Serial\_Command

Pelo log:

```

Esp32: ST_Bus char = 'd'  code = 100
Esp32: ST_Bus completo: d
Esp32: Estado da leitura:9
Esp32: Estado da Interpretao:31

```

Isto diz:

- O modo M leu apenas o 'd' e já considerou o comando completo.
- Em case 8 da reader, ST\_Bus completo: d → logo ST\_Command\_Out passa a "d" e ST\_Command\_IN também.
- Ou seja, para a tua FSM de comandos, o comando é só "d", não "drt[0.4]".

Porquê?

No reader:

```

case 6: {
    int c = Serial.read();
    ST_BUS[n] = (char)c;

    if(ST_BUS[n] == ' '){state = 6;}
    if(ST_BUS[n] != '\n' && ST_BUS[n] != ' '){state = 7;}
    if(ST_BUS[n] == '\n'){state = 8;}
    break;
}

case 7:
    n++;

    if(Serial.available() > 0){state = 6;}
    if(Serial.available() > 1){state = 8;}
    break;
}

```

Fluxo para "wdrt[0.4]\n":

- `w` foi lido pelo modo `S`, o reader começa a `M` com `'d'`.
- Em 6: lê `'d'` → é normal → `state = 7`.
- Em 7:
  - `n++` → `n` passa a 1.
  - Se `Serial.available() > 0`, volta a 6 e lê `r`.
  - Mas se num certo instante `Serial.available() > 1` (por exemplo, porque a UART ainda tem `r` e `t` no buffer), entra no segundo `if` e `state = 8` sem ler esses bytes.

Ou seja: é perfeitamente possível ir para o case 8 com `n == 1` e `ST_BUS` contendo só `'d'`, e o resto da linha ainda no buffer da `Serial`. A seguir:

```
case 8: {
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = n;    // = 1
    ...
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    if(1){state = 9;}
    break;
}
```

- `maxCopy = n = 1` → copias só o `'d'`.
- Nunca metes `'\0'` em `ST_BUS[n]` antes do `println`, por isso podes mostrar lixo ou só o `'d'`.
- O comando que chega ao `FSM_Serial_Command` é `"d"`.

## 2. Porque é que o FSM de comandos chega ao estado 34

A partir daqui, a FSM de comandos vê:

```
ST_Command_IN == "d"
```

Fluxo:

- `state 23`: recebe o `'W'` → vai para 23 → define `Mode='M'` → 24.
- `state 24`:

```
n = 0;
if(ST_Command_IN[n] == 'D'){state = 25;}
if(ST_Command_IN[n] == 'd'){state = 26;}
```

Como `ST_Command_IN[0] == 'd'`, vai para 26.

- `state 26`: faz `n++` (`n=1`) e vai para 28.
- `state 28`: seleciona `Resp_move` e `Resp_inst`, vai para 31.
- `state 31`:

```
n++;
if(ST_Command_IN[n] == 'T' ...){state = 32;}
if(ST_Command_IN[n] == 'P' ...){state = 33;}
if(ST_Command_IN[n] == 'M' ...){state = 34;}
```

- Antes deste case, n já era 1.
- `n++` → `n = 2`.
- `ST_Command_IN[2]` é memória não inicializada / lixo, porque a string era `"d\0"`.
- Se, por acaso, um ciclo apanhar um valor que coincide com `'M'` (77) nessa posição de memória, a condição `== 'M'` torna-se verdadeira e vais para 34.

É por isso que vês:

```
Estado da Interpretacao:31
...
Estado da Interpretacao:34
```

Quando deveria ir só para 32 ou 33.

Não é que tenhas escrito `'M'`; é que estás a ler fora da parte válida da string, e eventualmente o conteúdo da RAM nessa posição é igual ao código ASCII de `'M'`.

### 3. Como evitar isto, mantendo a tua ideia

Há dois problemas a resolver:

1. O reader está a fechar a linha demasiado cedo (só `"d"`).
2. A FSM de comandos está a ler `ST_Command_IN[n]` sem verificar se está dentro dos limites da string.

Sugestões mínimas:

#### A) Reader – não usar `Serial.available() > 1` para decidir fim de linha

Substituir o case 6/7 por algo mais determinístico, por exemplo:

```
case 6: {
    if (Serial.available() == 0) break;

    int c = Serial.read();
    ST_BUS[n] = (char)c;

    Serial.print("ST_Bus char = ");
    Serial.print(ST_BUS[n]);
    Serial.print("'   code = ");
    Serial.println(c);

    if (ST_BUS[n] == ' ') {
        // ignora espaços, não avança
```

```

        state = 6;
    } else if (ST_BUS[n] == '\n') {
        // fim da linha, termina string
        ST_BUS[n] = '\0';
        state = 8;
    } else {
        // carácter normal
        n++;
        if (n >= (int)sizeof(ST_BUS) - 1) {
            ST_BUS[n] = '\0';
            state = 8;
        } else {
            state = 6;
        }
    }
    break;
}

```

E depois case 8 pode simplesmente usar `strlen(ST_BUS)`:

```

case 8: {
    Serial.print("ST_Bus completo: ");
    Serial.println(ST_BUS);
    int maxCopy = strlen(ST_BUS);
    if (maxCopy >= (int)sizeof(ST_Command_Out)) {
        maxCopy = sizeof(ST_Command_Out) - 1;
    }
    strncpy(ST_Command_Out, ST_BUS, maxCopy);
    ST_Command_Out[maxCopy] = '\0';
    n = 0;
    state = 9;
    break;
}

```

Assim, o reader só fecha a linha quando lê mesmo `'\n'`.

## B) FSM comandos – proteger acesso a `ST_Command_IN[n]`

Antes de ler `ST_Command_IN[n]` em 27 e 31, verifica se `n` ainda está dentro do comprimento da string:

```

case 27:
    if (n >= (int)strlen(ST_Command_IN)) {
        // comando incompleto
        Serial.println("Erro: comando D... incompleto (faltam R/B/T)");
        state = 4;
        break;
    }
    if(ST_Command_IN[n] == 'R' || ST_Command_IN[n] == 'r'){state = 28;}
    else if(ST_Command_IN[n] == 'B' || ST_Command_IN[n] == 'b'){state = 29;}
    else if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 30;}
    else {
        Serial.println("Erro: letra desconhecida depois de D");
    }
}

```



```

        state = 4;
    }
    break;

case 31:
    if (n+1 >= (int)strlen(ST_Command_IN)) {
        Serial.println("Erro: comando D?T/P/M incompleto");
        state = 4;
        break;
    }
    n++;
    if(ST_Command_IN[n] == 'T' || ST_Command_IN[n] == 't'){state = 32;}
    else if(ST_Command_IN[n] == 'P' || ST_Command_IN[n] == 'p'){state = 33;}
    else if(ST_Command_IN[n] == 'M' || ST_Command_IN[n] == 'm'){state = 34;}
    else {
        Serial.println("Erro: esperado T/P/M");
        state = 4;
    }
    break;

```

Assim, mesmo que ST\_Command\_IN seja só "d", nunca vais ler fora da string nem cair em 34 por acaso.

Resumindo:

- O reader, como está, está a passar só "d" em vez de "drt[0.4]".
- A FSM de comandos, como está, lê ST\_Command\_IN[n] fora dos limites, e isso explica o salto “misterioso” para o 34.

Se primeiro corrigisses o reader para só fechar a linha ao ver '\n' e depois adicionasses estas verificações de strlen(...) na FSM, qual dos dois achas que queres implementar primeiro para ver logo um comportamento mais previsível?

## vamos voltar á parte da FSM do movimento

que tínhamos deixado a meio,

lembras-te quais eram os novos requisitos que eu queria introduzir na FSM?

Lembras-te bem: tínhamos estacionado na FSM do movimento quando começaste a atacar a parte do serial. Pelas notas do que já tinhas dito antes, os “novos requisitos” para a FSM de movimento eram estes: <sup>[228]</sup>

### 1. Separar bem definição vs. execução

- Tudo o que é “definição do movimento” (pontos X/Y/Z, período, pausa, bidirecional, startIndex, etc.) fica no MotionStorage.
- Tudo o que é “estado em runtime” (índice atual, timestamps, estado da FSM, dtMs, direção, se está ativo, se está em pausa) fica no MotionInstance.

### 2. Calcular dtMs (e derivados) fora do update

- `dtMs` e `dt_pauseMs` são calculados uma vez na definição (`MotionStorage::updateDtMs()` e `updateDt_PauseMs()`), sempre que mudas `period` ou `T_pause`, e não dentro da FSM de movimento.
- A FSM `FSM_Motion_Update` recebe `inst` e só usa os `dtMs` já calculados para decidir quando avançar segmentos, sem recalculá-los tudo a cada ciclo.

### 3. Uma FSM de movimento por instância (sem `static global`)

- O “estado” da FSM (o antigo `static short int state`) devia passar a um campo, por exemplo `inst.state`, dentro de `MotionInstance`, para poderes ter `Bat_inst`, `Resp_inst`, etc. a correr em paralelo sem interferirem um com o outro. <sup>[228]</sup>
- O mesmo para coisas como `elapsed_1`, `elapsed_2`, `elapsed_3`, `iIndex`, `jIndex`, `Direction`.

### 4. Fluxo da FSM de movimento bem definido

A sequência que tinhas em mente era algo do género: <sup>[228]</sup>

- Estado 0: `idle` / à espera de `start` e de `inst.active`.
- Estado 1: início de ciclo → configurar `currentIndex`, `Direction`, marcar `segmentStartMs`.
- Estado 2: execução do segmento atual → esperar até `nowMs - segmentStartMs >= dtMs`.
- Estado 3: avançar `currentIndex` (e/ou `Direction` se bidirecional), detetar fim do ciclo.
- Estado 4: gerir pausa entre ciclos (`dt_pauseMs`) antes de voltar ao estado 1.

### 5. Interligação com a fusão e IK

- A FSM de movimento só tinha de fornecer “ponto atual” (via `Intermed_position + lerp/spline3`) para as duas estruturas: `batimento` e `respiração`.
- A fusão (`Fusao_plus_IK_D1/D2`) recebia depois as duas `Coordinate` interpoladas e fazia a combinação + cinemática inversa, sem guardar estado próprio.

Se fizer sentido para ti, o próximo passo pode ser:

- Pegar na tua `FSM_Motion_Update` atual (a que estás a usar para `Bat_inst` e `Resp_inst`)
- Reescrevê-la contigo, passo a passo, para que:
  - use `inst.state` em vez de `static state`
  - use `inst.def->dtMs` e `inst.def->dt_pauseMs`
  - trate explicitamente o “fim do ciclo + pausa” com base nos `N_pontos` já calculados na `MotionStorage`.

Para avançarmos de forma mais focada: poderes colar só a versão atual da `FSM_Motion_Update` (a que estás a chamar no loop), para retomarmos exatamente na mesma função onde tínhamos parado?

# ok vou te contar os meus requisitos para o movimento desta FSM

ela basicamente têm que fazer parecido á antiga, mas com novas alterações:  
todos os requisitos são:

- Andar para a frente quando em ciclo fechado (até ao ponto final demarcado)
- Andar para a frente e para trás quando em ciclo aberto
- Parar por x tempo no início e fim, caso os valores de paragem sejam superiores a 0 (quando em ciclo fechado só interessa o tempo de paragem no final)

Em ciclo aberto, permitir escolher um ponto para inicial (o Index pode começar em 0 ou num outro ponto do movimento)

-Alterar o sentido do ciclo (para a frente e para trás) rapidamente caso exista um sinal (apenas em ciclo aberto), isto vai ser útil para o acontecimento da tosse (o sinal que estava a falar é esse)

fiz a seguinte estrutura da maquinas de estados, diz-me o que achas, (ainda não fiz a parte do sinal para a tosse)

Pela foto dá para perceber bem a lógica geral da FSM de movimento que desenhaste :

- Estado inicial (0 / INI) com  $i = 0$ ,  $j = 0$ ,  $\delta = 0$ .
- Estados onde calculas `N_Points` e decides se é ciclo fechado (`bidirectional == false`) ou aberto (`bidirectional == true`).
- Ramos para:
  - avançar  $i$  até `PointEnd` em ciclo fechado,
  - andar  $i$  para a frente e para trás com  $\delta = +1$  ou  $\delta = -1$  em ciclo aberto.
- Estados específicos para pausas: `D_Pause_Init` e `D_Pause_Fin`, com condições tipo `D_Pause_Init != 0` e `D_Pause_Fin != 0`.
- Transições que mudam  $j$  e  $\delta$  quando chegas ao início/fim e o ciclo é bidirecional.

No contexto dos requisitos que listaste, a estrutura está bem alinhada:

## 1. Andar para a frente em ciclo fechado

- Tens um ramo em que `bidirectional == false`, `N_Points = m_Points` e  $\delta = +1$ , e o índice  $i$  avança até `PointEnd = m_Points`. Isso encaixa com “andar até ao ponto final demarcado” em ciclo fechado.

## 2. Andar para a frente e para trás em ciclo aberto

- Nos círculos do meio vês bem  $\delta = -1$  e  $\delta = +1$  com condições tipo `Bidirectional == True & Direction == +1`, e depois  $i + \delta = j$  para inverter. Isso satisfaz “andar para a frente e para trás” em vai-e-vem.

## 3. Parar X tempo no início/fim

- No canto inferior esquerdo tens estados para `D_Pause_Init` e `D_Pause_Fin`, com set/reset e transições condicionadas a `D_Pause_Init != 0` e `D_Pause_Fin != 0`, que permitem esperar antes de arrancar e depois de terminar. Em ciclo fechado, só usas `D_Pause_Fin`, o que bate com o requisito.

#### 4. Escolher ponto inicial em ciclo aberto

- Vê-se um estado onde `Index > Point_Ini` e `j = i`, que permite começar num ponto diferente de 0 quando `bidirectional == true`. Isso dá-te exatamente “Index pode começar em 0 ou num outro ponto”.

#### 5. Mudar o sentido rapidamente para tosse

- A parte do sinal da tosse ainda não está desenhada, mas já tens o mecanismo essencial ( $\delta$  e `Direction`) bem separado. Isso vai facilitar ter um estado extra que, ao receber o sinal, faz algo como  $\delta = -\delta$  e talvez força `j = i` para inverter no ponto atual.

O que eu sugeria como próximo passo, para ficar mesmo limpo na implementação:

- Manter exatamente esta estrutura de índices:
  - `i` = índice atual dentro do ciclo.
  - `j` = índice “espelho” quando tens de inverter no bidirecional.
  - $\delta$  = incrementador (+1 para frente, -1 para trás).
- Pôr o “estado da FSM” (`state`) e estes três (`i`, `j`,  $\delta$ ) dentro do `MotionInstance`, como tínhamos falado, para poderes usar a mesma FSM para Bat, Resp e Tosse sem variáveis estáticas globais.
- Quando fores adicionar o sinal de tosse, criar um estado curto que:
  - lê um flag externo (por exemplo, `inst.tosseSignal`),
  - se ativo, altera  $\delta$  e reconfigura `PointIni` / `PointEnd` para um ciclo mais curto de “vai e vem” rápido, sem quebrar o resto da lógica.

Para validar que entendeste bem a relação entre índices e requisitos: como descreverias, nas tuas palavras, o papel de  $\delta$  e de `j` na tua FSM quando o movimento está em modo bidirecional (vai-e-vem)?



na verdade, no estado zero só tenho  $I=0$ ,  $i=0$  e  $j=0$  ( $i$  e  $j$  são as variáveis como tínhamos antes para andar para a frente e para trás, o  $I$  grande é apenas um contador global dos pontos, para saber quando o ciclo acabou, este é o meu novo modo de saída, isto serve mais para o novo funcionamento da respiração que começa de um ponto que não é 0, mas á partida têm que funcionar da mesma forma que antes)

Ok, então a foto ganha mais sentido com essa distinção:

- $I$  (maiúsculo) = contador global de segmentos/pontos percorridos no ciclo.
- $i$  = índice atual no array de pontos (vai mudando com  $\delta$ ).
- $j$  = índice auxiliar/espelho para gerir a inversão (frente  $\leftrightarrow$  trás) no ciclo aberto.

Isso encaixa bem com o que queres:

### Papel de $I$ , $i$ e $j$ na FSM

- Estado 0 (INI)
  - $I = 0$ : ainda não percorreste nenhum ponto do ciclo.
  - $i = 0, j = 0$ : posição lógica inicial, mas podes depois redefinir o ponto de início com base em `startIndex` ou algo semelhante quando arrancas.
- Contador global  $I$ 
  - Vai de 0 até  $N\_Pontos$  (o número total de segmentos por ciclo que já calculas em `MotionStorage`), independentemente de  $i$  começar em 0 ou noutro ponto.
  - Serve como “novo modo de saída”: quando  $I \geq N\_Pontos$ , sabes que o ciclo acabou e podes disparar:
    - pausa final (`D_Pause_Fin`),
    - ou recomeçar o ciclo (respiração, batimento, etc.).
  - Isto é especialmente útil para a respiração, onde o movimento começa num ponto intermédio mas, ainda assim, queres contar o ciclo completo; o início visual não é `X[63_0]`, mas o ciclo lógico continua a ter os mesmos  $N\_Pontos$ .
- Índices  $i$  e  $j$  (como antes)
  - $i$  anda para a frente ou para trás com base em  $\delta$  (+1 frente, -1 trás).
  - $j$  guarda “para onde estás a ir” quando invertiores:
    - por exemplo, se estás a subir (frente) e chegas ao fim, guardas  $j = i$  e depois passas a descer (trás), usando  $j$  para saber de onde vieste.

- No ciclo aberto (bidirecional),  $i$  e  $j$  gerem o vaivém, enquanto  $I$  continua a contar “quantos segmentos já fiz neste ciclo” para saber quando parar.

Se a respiração começa num ponto que não é 0:

- No arranque do ciclo, podes pôr  $i = startIndex$ ,  $j = startIndex$ , mas manter  $I = 0$ .
- Cada vez que fazes um segmento (interpolas de  $i$  para  $i + \delta$ ), incrementas  $I$ .
- O ciclo acaba quando  $I$  chega ao  $N\_Pontos$  calculado (com base no número de pontos e se é vai-e-vem ou fechado).

Isso mantém o comportamento antigo (usar  $i/j/\delta$  para andar para a frente e para trás) e, ao mesmo tempo, dá-te um “odómetro” global  $I$  para controlar respirações que começam no meio.

Para garantir que estás a usar o contador global do jeito que queres: como imaginas implementar, na FSM, a transição “fim de ciclo” baseada em  $I$  em vez de só olhar para  $i$  igual ao último índice?

\*\*

## diz-me o que achas do código da FSM:

```
#include <Arduino.h>
#include <Adafruit_NeoPixel.h>
#include <ESP32Servo.h>
#include <math.h>

// declarar no topo do ficheiro (globais)
int last_delta2_s1 = 0;
int last_delta2_s2 = 0;
int last_delta2_s3 = 0;

struct Manipulador_Config {
    // geometria
    float L1;
    float L2;
    float L3;
    float servoOffsetX;
    float servoOffsetZ;
    float angleMin;
    float angleMax;
    float rotationOffsetZ; // offset global de rotação da base

    // calibração de PWM
    int servo1MinPulse;
    int servo1MaxPulse;

    int servo2MinPulse;
    int servo2MaxPulse;
```

```

int servo3MinPulse;
    int servo3MaxPulse;

int theta1MinPulse;
    int theta1MaxPulse;

int theta2MinPulse;
    int theta2MaxPulse;

int theta3MinPulse;
    int theta3MaxPulse;

// estado atual (pulsos gerados pela IK)
    int servo1Pulse;
    int servo2Pulse;
    int servo3Pulse;
};

Manipulador_Config delta_1_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -15.6f * PI / 180.0f, //-75 degrees in radians

// calibração de PWM
    .servo1MinPulse = 520,
    .servo1MaxPulse = 2520,

    .servo2MinPulse = 550,
    .servo2MaxPulse = 2550,

    .servo3MinPulse = 560,
    .servo3MaxPulse = 2560,

// estado atual (pulsos gerados pela IK)
    .servo1Pulse = 0,
    .servo2Pulse = 0,
    .servo3Pulse = 0
};

Manipulador_Config delta_2_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,

```







```
//Servo microsecond pulse limits (Calibration)-----Valores normais: Min = 500 ; Max = 2500]-----
```

```
-----
```

```
#define SERVO_1_MIN 620
```

```
#define SERVO_1_MAX 2520
```

```
#define SERVO_2_MIN 615
```

```
#define SERVO_2_MAX 2480
```

```
#define SERVO_3_MIN 620
```

```
#define SERVO_3_MAX 2550
```

```
//Servo_1 -> theta3 min 550, max 2450
```

```
//Servo_2 -> theta1 min 545, max 2410
```

```
//Servo_3 -> theta2 min 520, max 2450
```

```
// variáveis globais para valores suavizados
```

```
int servo_1_pulse_smooth = 0;
```

```
int servo_2_pulse_smooth = 0;
```

```
int servo_3_pulse_smooth = 0;
```

```
// fator de suavização [0 - 1]. tipo 0.2 para começar
```

```
const float SERVO_SMOOTH_ALPHA = 0.2f;
```

```
#define SERVO_4_MIN 550
```

```
#define SERVO_4_MAX 2400
```

```
#define SERVO_5_MIN 550
```

```
#define SERVO_5_MAX 2400
```

```
#define SERVO_6_MIN 550
```

```
#define SERVO_6_MAX 2400
```

```
// variáveis globais para valores suavizados
```

```
int servo_4_pulse_smooth = 0;
```

```
int servo_5_pulse_smooth = 0;
```

```
int servo_6_pulse_smooth = 0;
```

```
//-----
```

```
#define INVERTED -1
```

```
// Máximo de pontos por movimento (ajusta mais tarde se precisares)
```

```
const int MAX_POINTS = 100;
```

```
// Armazenamento Bruto: dados completos de um movimento - Será utilizada para armazenar os movimentos que o Pi enviar, ou movimentos pré-definidos, etc.
```

```
struct MotionStorage {
```

```
    int n_Points;           // nº de pontos do ciclo
```

```
    int N_points;           // nº de pontos do movimento completo
```

```
    float X[MAX_POINTS];    // coordenadas X dos pontos do movimento
```

```
    float Y[MAX_POINTS];    // coordenadas Y dos pontos do movimento
```

```
    float Z[MAX_POINTS];    // coordenadas Z dos pontos do movimento
```

```
    float period;
```

```
    float T_pause_Ini;
```

```

float T_pause_End;
float dtMs;          // duração do ciclo em segundos
float Dt_pauseMs_Ini;    // duração da pausa entre ciclos em segundos
float Dt_pauseMs_End;
int start_Index; // índice de ponto que será o "0,0,0" lógico
int Begginig_point;
int End_Point;
bool bidirectional; // true = vai-e-vem, false = ciclo fechado

float getDtMs(){
    if (n_Points > 1 && period > 0.0f) {
        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_points = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinha
            N_points = n_Points-1;
        }
        return (period * 1000.0f) / (float)N_points;
    }
    return 0.0f;
}

void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}

float getD_PauseMs(float T_pause) const {
    if (n_Points > 1 && T_pause > 0.0f) {
        return (T_pause * 1000.0f); // só lê nPoints e T_pause
    }
    return 0.0f;
}

void updateDt_PauseMs() {
    Dt_pauseMs_Ini = getD_PauseMs(T_pause_Ini); // aqui sim, mudas dt_pauseMs
    Dt_pauseMs_End = getD_PauseMs(T_pause_End); // aqui sim, mudas dt_pauseMs
}

};

// Armazenamento temporário: runtime que executa um MotionStorage
struct MotionInstance {
    MotionStorage* def;    // ponteiro para MotionStorage (a definição do Armazenamento Bruto
)
    int      currentIndex; // índice i (início do segmento i -> i+1) -> índice atual no movimento
    unsigned int  segmentStartMs; // timestamp do início do segmento atual
    // -> é uma variavel importante para controlar a velocidade do movimento,

```

ou seja, quando é que deve avançar para o próximo ponto do movimento

```
bool    active;    // se este movimento está ativo
bool    Executing; // se este movimento está em pausa (entre ciclos)
short int state;
float Dt_pause;
unsigned long elapsed_1;
unsigned long elapsed_2;
unsigned long elapsed_3;

int iIndex; // novo: índice i para a interpolação
int jIndex; // novo: índice j para a interpolação
int I_Index;
int Direction; // novo: direção do movimento (1 ou -1)
};

// -----Temporario-----
MotionStorage Bat_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Bat_inst;
MotionStorage Resp_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Resp_inst;
MotionStorage Tosse_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Tosse_inst;
MotionStorage* currentMove = nullptr;
MotionInstance* currentInst = nullptr;

void Iniciate_Bat_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Bat_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Bat_move cada vez que quiseres testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Bat_move.n_Points = N_test;
    Bat_move.period = T_period;
    Bat_move.T_pause_End = T_pausedt;
    for (int i = 0; i < N_test; i++) {
        Bat_move.X[i] = X_test[i];
        Bat_move.Y[i] = -1 * Y_test[i];
        Bat_move.Z[i] = Z_test[i];
    }
    Bat_move.start_Index = 0;
    Bat_move.Beggining_point = 0;
    Bat_move.End_Point = 0;
    Bat_move.bidirectional = false; // Define o movimento como vai-e-vem
    Bat_move.updateDtMs();
```

```

    Bat_move.updateDt_PauseMs();
}

```

void Inicieate\_Resp\_Move() { //Serve para copiar os valores definidos mais acima para o objeto Resp2\_move, que é do tipo MotionStorage.

// -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter que copiar os valores manualmente para o Resp2\_move cada vez que quiseses testar algo.

// -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM ou do PI

```

    Resp_move.n_Points = N_test2;
    Resp_move.period = T_period2;
    Resp_move.T_pause_End = T_pausedt2;
    for (int i = 0; i < N_test2; i++) {
        Resp_move.X[i] = X_test2[i];
        Resp_move.Y[i] = -1 * Y_test2[i];
        Resp_move.Z[i] = Z_test2[i];
    }
    Resp_move.start_Index = 5;
    Resp_move.Beggining_point = 0;
    Resp_move.End_Point = 0;
    Resp_move.bidirectional = true; // Define o movimento como vai-e-vem
    Resp_move.updateDtMs();
    Resp_move.updateDt_PauseMs();
}

```

void Inicieate\_Bat\_Instance() { //Serve para arrancar a structure de temporaria de runtime

```

    Bat_inst.def = &Bat_move;
    Bat_inst.currentIndex = 0;
    Bat_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Bat_inst.active = true;
    Bat_inst.Executing = 0; // novo: inicia o movimento em pausa, ou seja, não começa imediatamente
    Bat_inst.state = 0;
    Bat_inst.elapsed_1 = 0;
    Bat_inst.elapsed_2 = 0;
    Bat_inst.elapsed_3 = 0;
    Bat_inst.I_Index = 0;
    Bat_inst.iIndex = 0;
    Bat_inst.jIndex = 0;
    Bat_inst.Direction = 1; // novo: inicializa a direção como positiva
    Bat_inst.Dt_pause = 0;
}

```

void Inicieate\_Resp\_Instance() { //Serve para arrancar a structure de temporaria de runtime

```

    Resp_inst.def = &Resp_move;
    Resp_inst.currentIndex = 0;
    Resp_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update

```

```

    Resp_inst.active    = true;
    Resp_inst.Executing = false; // novo: inicia o movimento em pausa, ou seja, não começa
    imediatamente
    Resp_inst.state      = 0;
    Resp_inst.elapsed_1  = 0;
    Resp_inst.elapsed_2  = 0;
    Resp_inst.elapsed_3  = 0;
    Resp_inst.I_Index    = 0;
    Resp_inst.iIndex     = 0;
    Resp_inst.jIndex     = 0;
    Resp_inst.Direction  = 1; // novo: inicializa a direção como positiva
    Resp_inst.Dt_pause   = 0;
}

```

```

//-----Fim Temporario-----

```

```

struct Coordinate {
    float X;
    float Y;
    float Z;
};

```

```

int Clock_loop = 0;
const float pi = 3.141592653;

```

```

// -----Declaração das variaveis-----

```

```

void LED_LOOP(); //Declaração de funções

```

```

void Servo_test();

```

```

//-----Novas-----

```

```

float boundFloat(float, float, float);

```

```

bool attach_servos(void);

```

```

bool inverse_kinematics_1(float, float, float, float, float&);

```

```

bool inverse_kinematics_2(float, float, float, float, float&);

```

```

bool inverse_kinematics_3(float, float, float, float, float&);

```

```

bool inverse_kinematics(Manipulador_Config& cfg, float, float, float);

```

```

void move_servos(const Manipulador_Config& d1, const Manipulador_Config& d2);

```

```

double mapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

int roundMapNumber(double x, double in_min, double in_max, double out_min, double
out_max);

```

```

double degToRads(double deg);

```

```

double radsToDeg(double rads);

```

```

void debugPrintMove(const MotionStorage& move, const char* name);

```

```

void debugPrintInstance(const MotionInstance& inst, const char* name);

```

```

float lerp(float a, float b, float t);

```

```

float applyEasing(float alpha, char mode);

```

```

float spline3(float p0, float p1, float p2, float t);

```

```

void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs,
Coordinate& xyz, boolean trig_serial);
bool Fusao_plus_IK_D1(const Coordinate& respi, const Coordinate& batim);
bool Fusao_plus_IK_D2(const Coordinate& respi, const Coordinate& batim);
//-----FSM-----
void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs);
void FSM_Serial_reader(unsigned long nowMs);
void FSM_Serial_Command(unsigned long nowMs);
void FSM_Main(boolean start, MotionInstance& inst, unsigned long nowMs);
//-----

//--Declaração dos comandos de leitura
//char commands
#define Scomand_start_Program 's'
#define Scomand_stop_Program 'p'
#define Scomand_GRIPPER 3
#define Scomand_ABSOLUTE_CARTESIAN_LINEAR 4
#define Scomand_SET_PROGRAM_ARRAY 5
#define Scomand_REQUEST_READY_FLAG 6

//String commands
#define Mcomand_JOG_X 'x'
#define Mcomand_JOG_Y 'y'
#define COMMAND_JOG_Z 'z'
#define COMMAND_GRIPPER_ASCII 'g'
#define COMMAND_STATUS 's'
#define COMMAND_REPORT_COMMANDS 'r'
#define COMMAND_ADD_POSITION 'p'
#define COMMAND_SET_STEP_DELAY 'd'
#define COMMAND_SET_STEP_INCREMENT 'i'
#define COMMAND_CLEAR_ARRAY 'c'
#define COMMAND_EXECUTE 'e'
#define COMMAND_EXECUTE_JOINT 'j'
#define COMMAND_EXECUTE_TIME 't'
#define COMMAND_SET_US_INCREMENT_LINEAR 'u'
#define COMMAND_SET_US_INCREMENT_JOINT 'U'
#define COMMAND_STEP_FORWARD '>'
#define COMMAND_STEP_BACKWARD '<'
#define COMMAND_JUMP_TO_START '['
#define COMMAND_JUMP_TO_END ']'
#define COMMAND_EDIT_ARRAY 'P'
#define COMMAND_ADD_DELAY 'D'
#define COMMAND_MOVE_HOME 'h'
#define COMMAND_PRINT_FILE 'f'
#define COMMAND_PING_PONG 'o'
#define COMMAND_SERVO_CALIBRATION 'C'
#define COMMAND_SET_SERVO 'S'

```

```

//EEPROM commands
#define Ecomand_SET_LINK_2 'L'
#define Ecomand_SET_END_EFFECTOR_TYPE 'E'
#define Ecomand_SET_AXIS_DIRECTION 'A'
#define Ecomand_SET_HOME_X 'X'
#define Ecomand_SET_HOME_Y 'Y'
#define Ecomand_SET_HOME_Z 'Z'
#define Ecomand_SET_HOME_GRIPPER 'G'
#define Ecomand_SET_GRIPPER_MIN_MAX 'M'

//EEPROM addresses for the delta robots configuration
#define EEPROM_ADDRESS_LINK_2 0
#define EEPROM_ADDRESS_END_EFFECTOR_TYPE 1
#define EEPROM_ADDRESS_AXIS_DIRECTION 2
#define EEPROM_ADDRESS_Z_OFFSET 3
#define EEPROM_ADDRESS_HOME_X 7
#define EEPROM_ADDRESS_HOME_Y 11
#define EEPROM_ADDRESS_HOME_Z 15
#define EEPROM_ADDRESS_HOME_GRIPPER 19
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MIN 21
#define EEPROM_ADDRESS_GRIPPER_ROTATION_MAX 23
#define EEPROM_ADDRESS_GRIPPER_CLAW_MIN 25
#define EEPROM_ADDRESS_GRIPPER_CLAW_MAX 27
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MIN 29
#define EEPROM_ADDRESS_GRIPPER_VACUUM_MAX 31

//Delta1 -----
Servo mg90s_1; // cria objeto servo
Servo mg90s_2; // cria objeto servo
Servo mg90s_3; // cria objeto servo

//Delta2 -----
Servo mg90s_4; // cria objeto servo
Servo mg90s_5; // cria objeto servo
Servo mg90s_6; // cria objeto servo

volatile boolean start_comand = 0;

Coordinate Lerp_Respi_OUT;
Coordinate Lerp_Batim_OUT;
Coordinate IK2_IN;
Coordinate home_position;

void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs){
MotionStorage& m = (inst.def);
/
Têm em atenção constituição de cada caso:
case x:

```



- Ações a executar
- Condições de transição por hierarquia

\*/

```
//Serial.print("Estado atual: ");
```

```
//Serial.println(inst.state);
```

```
switch (inst.state) {
```

```
case 0:
```

```
    inst.Executing = 0;
```

```
    inst.I_Index=0;
```

```
    inst.iIndex = 0;
```

```
    inst.jIndex = 0;
```

```
if (start0) {inst.state = 0;}
```

```
    if (!inst.active || inst.def == nullptr) {inst.state = 0;}
```

```
    if(start1 && inst.active && inst.def != nullptr){inst.state = 1;}
```

```
    break;
```

```
case 1:
```

```
    inst.I_Index=0;
```

```
if (m.n_Points <= 1 || m.period <= 0.0f) {inst.state = 1;}
```

```
    if(inst.Direction==1){inst.state = 2;}
```

```
    //else {inst.state = 2;}
```

```
    break;
```

```
case 2:
```

```
    inst.Executing = 1;
```

```
    inst.segmentStartMs = nowMs;
```

```
    inst.iIndex = m.start_Index;
```

```
    m.End_Point = m.n_Points-1;
```

```
    inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;
```

```
if (1) {inst.state = 3;}
```

```
    break;
```

```
case 3:
```

```
    inst.elapsed_1 = nowMs - inst.segmentStartMs;
```

```
if(inst.elapsed_1 >= m.dtMs) {inst.state = 4;}
```

```
    break;
```

```
case 4:
```

```
    inst.I_Index++;
```

```
    inst.iIndex = (inst.iIndex + inst.Direction);
```

```
    inst.jIndex = (inst.iIndex + inst.Direction);
```

```
    inst.elapsed_1 = 0;
```

```
    inst.segmentStartMs = nowMs;
```

```

if(inst.iIndex < m.End_Point) {inst.state = 3;}
    if(inst.iIndex >= m.End_Point ) {inst.state = 5;}
    if(inst.I_Index == m.N_points && m.bidirectional){inst.state = 0;}
    break;

case 5: //função trás
    inst.segmentStartMs = nowMs;
    inst.jIndex = inst.iIndex;

if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {inst.state = 7;}
    if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {inst.state = 8;}
    else {
        if (m.bidirectional == true) { inst.state = 6; }
        if (m.bidirectional != true && m.Dt_pauseMs_Ini != 0 && m.Dt_pauseMs_End != 0) { inst.state
= 1; }
    }
    break;

case 6:
    inst.Direction = inst.Direction * (-1);

if(inst.Direction == -1) {inst.state = 11;}
    if(inst.Direction == 1) {inst.state = 12;}
    break;

case 7:
    inst.Dt_pause = m.Dt_pauseMs_Ini;

if(1) {inst.state = 9;}
    break;

case 8: //função trás
    inst.Dt_pause = m.Dt_pauseMs_End;

if(1) {inst.state = 9;}
    break;

case 9: //função trás
    inst.elapsed_3 = nowMs - inst.segmentStartMs;

    if(inst.elapsed_3 >= m.dtMs) {inst.state = 10;}

break;

case 10: //função trás
    inst.segmentStartMs = nowMs;
    inst.elapsed_3 = 0;

if(start==0 || inst.active != true) {inst.state = 0;}
    if(m.bidirectional == true) {inst.state = 6;}

```

```

        break;

case 11:
    inst.iIndex = m.n_Points;
    inst.jIndex = inst.iIndex + inst.Direction;
    m.End_Point = 0;

if(1){inst.state = 13;}
    break;

case 12:
    inst.iIndex = 0;
    inst.jIndex = inst.iIndex + inst.Direction;
    m.End_Point = m.n_Points-1;

if(1){inst.state = 13;}
    break;

case 13:
    inst.segmentStartMs = nowMs;

if(1){inst.state=3;}
    break;

}
return;
}

```

Se bate certo com o que tínhamos definido

No geral está bem alinhado com o que tinhas definido na FSM do quadro: tens separação MotionStorage/MotionInstance, usas `inst.state` em vez de `static`, geres `Direction`, pausas e ciclo bidirecional. Mas há alguns detalhes que convém afinar, senão vais ter comportamentos estranhos.

Vou apontar só os pontos mais importantes, para não ficar pesado.

## 1. Entrada e saída do estado 0

No case 0:

```

inst.Executing = 0;
inst.I_Index=0;
inst.iIndex = 0;
inst.jIndex = 0;

if (start==0) {inst.state = 0;}
if (!inst.active || inst.def == nullptr) {inst.state = 0;}
if(start==1 && inst.active && inst.def != nullptr){inst.state = 1;}

```

Isto funciona, mas:

- Essas três if não são mutuamente exclusivas; se `start == 0` e `inst.active == false`, vais chamar as duas, mas como ambas dão `state=0`, está ok.
- A transição “de 0 para 1” depende de `start == 1`, `active == true` e `def != nullptr`, o que é o que querias; está bem.

Só lembra-te de que aqui estás sempre a reiniciar `iIndex/jIndex` para 0. Para a respiração começar em `start_Index`, vais corrigir isso no estado 2 (como fazes).

## 2. Estado 1 – validação do movimento

```
if (m.n_Points <= 1 || m.period <= 0.0f) {inst.state = 1;}
if(inst.Direction==1){inst.state = 2;}
```

- Se o movimento não for válido (menos de 2 pontos ou período  $\leq 0$ ), ficas preso em 1 para sempre.
- Se `Direction == 1`, saltas para 2 mesmo que o movimento seja inválido, porque não há else.

Sugestão:

```
if (m.n_Points <= 1 || m.period <= 0.0f) {
    // movimento inválido -> volta ao idle
    inst.state = 0;
} else if (inst.Direction == 1) {
    inst.state = 2;
} else {
    // se quiseses, podes tratar Direction != 1 aqui
}
```

## 3. Estado 2 – início do ciclo

```
inst.Executing = 1;
inst.segmentStartMs = nowMs;
inst.iIndex = m.start_Index;
m.End_Point = m.n_Points-1;
inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;
```

Isto bate bem com a respiração que começa no `start_Index` e com o batimento que começa em 0. Só cuidado:

- Estás a guardar `End_Point` dentro de `MotionStorage` (`m.End_Point`), mas depois usas esse campo só na FSM. Tudo bem, mas lembra-te que `End_Point` agora é parte da definição; se quiseses manter tudo relativo à execução, podias mover esse conceito para `inst`, por exemplo `inst.endIndex`. Não é obrigatório, só estilo.

## 4. Estado 4 – avanço de índices e fim de ciclo

```
inst.I_Index++;
inst.iIndex = (inst.iIndex + inst.Direction);
inst.jIndex = (inst.iIndex + inst.Direction);

if(inst.iIndex < m.End_Point) {inst.state = 3;}
if(inst.iIndex >= m.End_Point ) {inst.state = 5;}
if(inst.I_Index == m.N_points && m.bidirectional){inst.state = 0;}
```

Aqui há três coisas chave:

1. jIndex é igual a iIndex + Direction sempre; para a tua FSM do quadro, pensavas em usar j como “índice espelho” quando invertes, mas aqui estás a mantê-lo sempre acoplado. Não é errado, mas perdes um pouco da semântica original.
2. Tens três if independentes:
  - Se iIndex >= End\_Point, vais para 5, mas se ao mesmo tempo I\_Index == N\_points e bidirectional, também pões state=0.
  - O último if não é else if, por isso pode sobrepor state=5 com state=0.
3. No fim do ciclo bidirecional, voltar direto a 0 pode ser o que queres (parar completamente) ou não (recomeçar), dependendo da respiração/batimento.

Sugestão:

```
inst.I_Index++;
inst.iIndex += inst.Direction;
inst.jIndex = inst.iIndex + inst.Direction;

if (m.bidirectional && inst.I_Index >= m.N_points) {
    // fim de ciclo global
    inst.state = 0;    // ou 1, se quiseres repetir automaticamente
} else if (inst.iIndex < m.End_Point) {
    inst.state = 3;
} else {
    inst.state = 5;    // chegaste ao fim local
}
```

## 5. Estado 5–10 – pausas e inversão

A ideia geral está boa:

- 5: decidir se há pausa ou não e se é bidirecional.
- 6: inverter Direction.
- 7/8: escolher Dt\_pause inicial ou final.
- 9: esperar essa pausa.
- 10: sair da pausa e decidir se vais para 6 (bidirecional) ou 0.

Mas há alguns detalhes:

- Em 5:

```
if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {inst.state = 7;}
if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {inst.state = 8;}
else {
    if (m.bidirectional == true) { inst.state = 6; }
    if (m.bidirectional != true && m.Dt_pauseMs_Ini != 0 && m.Dt_pauseMs_End != 0)
}
```

Aqui o else está ligado só ao segundo if. Se Direction == -1 e Dt\_pauseMs\_Ini != 0, vais a 7, mas o segundo if é falso; o else corre e pode mandar-te para 6 ou 1 logo em seguida. O comportamento não fica claro.

Devias estruturar como if/else if/else:

```
if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {
    inst.state = 7;
} else if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {
    inst.state = 8;
} else {
    if (m.bidirectional) {
        inst.state = 6;
    } else if (m.Dt_pauseMs_End != 0) {
        // ciclo fechado, só pausa final importa
        inst.state = 8; // pausa final e depois recomeça ou pára
    } else {
        inst.state = 1; // sem pausas, recomeça ciclo
    }
}
```

- Em 9:

```
inst.elapsed_3 = nowMs - inst.segmentStartMs;
if(inst.elapsed_3 >= m.dtMs) {inst.state = 10;}
```

Aqui estás a usar m.dtMs (tempo por segmento) para a pausa, mas tens inst.Dt\_pause guardado (7/8). Faz mais sentido comparar com inst.Dt\_pause:

```
if(inst.elapsed_3 >= inst.Dt_pause) {inst.state = 10;}
```

## 6. Estados 11-13 – arranque para trás / frente

```
case 11:
    inst.iIndex = m.n_Points;
    inst.jIndex = inst.iIndex + inst.Direction;
    m.End_Point = 0;

case 12:
    inst.iIndex = 0;
```

```
inst.jIndex = inst.iIndex + inst.Direction;  
m.End_Point = m.n_Points-1;
```

- Se queres começar a andar “para trás”, pôr `iIndex = m.n_Points` vai sair fora do array (último índice válido é `m.n_Points - 1`).

- Devia ser:

```
inst.iIndex = m.n_Points - 1; // último segmento  
m.End_Point = 0;             // vamos até ao índice 0
```

- O state 12 está correto para o sentido frente: começa em 0 e vai até `End_Point = n_Points - 1`.

## 7. Coisas boas que já estão a seguir os requisitos

- `MotionStorage.updateDtMs()` e `updateDt_PauseMs()` calculam `dtMs` e pausas fora da FSM, como querias.
- `MotionInstance` tem `state`, `iIndex`, `jIndex`, `I_Index`, `Direction`, `segmentStartMs`, que te deixam ter Bat/Resp/Tosse em paralelo sem partilha de estado estático.
- A FSM consegue:
  - ciclo fechado: `bidirectional == false`, `End_Point = n_Points-1`, sem inversão.
  - ciclo aberto: `bidirectional == true`, mexe em `Direction` e `End_Point` nos estados 11/12.

Se quiseres, podemos pegar num cenário concreto, por exemplo:

- Movimento de respiração: `n_Points = 10`, `start_Index = 5`, `bidirectional = true`, `T_pause_End = 0.8`, `T_pause_Ini = 0`.

e seguir à mão:

- valores de `iIndex`, `I_Index`, `Direction` e `state` desde o momento em que `start_comand` passa a 1 até acabar um ciclo.

Que tal tentares tu descrever, passo a passo, o que esperas que aconteça nesses estados (0 → 1 → 2 → 3 → 4 → ...) para esse movimento de respiração, e vemos se coincide com o código atual?

"jIndex é igual a iIndex + Direction sempre; para a tua FSM do quadro, pensavas em usar j como “índice espelho” quando invertes, mas aqui estás a mantê-lo sempre acoplado. Não é errado, mas perdes um pouco da semântica original."

como assim?

Quando falei disso, estava a comparar o que desenhaste na FSM do quadro com o que o código está a fazer.

Na FSM do quadro, a ideia de ter dois índices  $i$  e  $j$  era mais ou menos esta (pela tua descrição):

- $i$ : posição atual no array de pontos, que vai andando para a frente ou para trás com  $\delta$ .
- $j$ : índice auxiliar/“espelho”, usado quando tens de inverter ou quando queres saber “de onde vim / para onde vou” no ciclo bidirecional (vai-e-vem).

Ou seja:  $j$  não precisava de ser sempre, em todos os estados, igual a  $i + \delta$ . Em certos momentos podias guardar  $j = i$  ou outro valor para marcar o ponto de inversão, o início de um novo ramo, etc.

No código atual, em vários estados fazes sempre:

```
inst.jIndex = (inst.iIndex + inst.Direction);
```

ou

```
inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;
```

Isto significa:

- Em todos esses estados,  $j$  é calculado diretamente a partir de  $i$  e da `Direction`.
- $j$  nunca guarda informação “extra” além de um passo à frente ou atrás em relação a  $i$ .

Por isso disse que estás a mantê-lo sempre “acoplado”:  $j$  não tem vida própria, é só “o próximo índice” relativo a  $i$ . A semântica de “índice espelho” que tinhas desenhado (por exemplo: guardar o ponto onde invertes, distinguir fronteiras de ciclo, etc.) perde-se, porque no código  $j$  está sempre preso a  $i$ .

Não é errado — podes perfeitamente implementar o movimento só com  $i$  e `Direction` — mas:

- Se quiseses usar  $j$  como na FSM do quadro (para marcar explicitamente pontos especiais: início, fim, espelho da inversão), então em alguns estados convém definir  $j$  de forma diferente, por exemplo:
  - no momento de inverter: `jIndex = iIndex`; (guardar o ponto atual como referência);
  - em transições de ciclo: usar  $j$  para marcar o “novo” início.



Para te ajudar a pensar: num ciclo aberto de respiração, quando chegas ao máximo e comesas a voltar para trás, que valor gostarias que  $j$  guardasse — o mesmo que  $i$ , o próximo índice, ou o ponto em que inverteste?



## a unica cena é que eu nunca disse que j era um espelho de i

o  $j$  constinua a ter a mesma função que em todas as outra versões, acompanhar o  $i$ , para depois permitir o funcionamento do `intermed_position()`

o  $i$  sempre foi o ponto atual e o meu limite/barreira dos pontos

Percebe-se melhor agora, obrigado pelo esclarecimento.

Então, pela tua descrição:

- $i$  = ponto atual, e também quem faz de “barreira” (limite) dos pontos.
- $j$  = ponto seguinte (ou anterior) em relação a  $i$ , que acompanha  $i$  e é usado pelo `Intermed_position()` para interpolar entre  $i$  e  $j$ .

Ou seja, a lógica é:

- `Intermed_position()` olha para `inst.iIndex` e `inst.jIndex`.
- Para cada segmento, interpola entre o ponto  $i$  e o ponto  $j$ , usando `nowMs`, `segmentStartMs` e `m.dtMs` para calcular o  $\alpha$ .
- Quando o segmento acaba, a FSM atualiza  $i$  e  $j$  para o próximo par de pontos.

Dentro dessa função,  $j$  sempre foi “o outro ponto do segmento”, nunca um espelho especial. O que tens agora no código:

```
// no início do ciclo
inst.iIndex = m.start_Index;
m.End_Point = m.n_Points-1;
inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;

// depois, a cada segmento
inst.iIndex = inst.iIndex + inst.Direction;
inst.jIndex = inst.iIndex + inst.Direction;
```

combina bem com essa ideia de:

- $i$  = ponto de origem do segmento atual,
- $j$  = ponto de destino do segmento atual,
- `Direction` decide se vais para a frente (+1) ou para trás (-1).

Portanto, não há “erro” conceitual aí; a única coisa que eu estava a sugerir é:

- Se, no futuro, quiseses guardar mais contexto (por exemplo, “ponto de inversão” para tosse ou respiração assimétrica), poderias usar *j* para isso.
- Mas, com o uso atual, a função de *j* como “ponto seguinte para interpolar” está consistente com o que tinhas antes.

Para garantir mesmo que estamos alinhados: se tiveres um segmento qualquer, como é que imaginas que `Intermed_position()` está a usar concretamente `iIndex` e `jIndex` por dentro para gerar a posição interpolada ao longo do tempo?

## fiz, certas das modificações que sugeriste

diz-e agora o que achas

Olhando para a versão nova da `FSM_Motion_Update`, está bem mais consistente com aquilo que tinhas desenhado e com os requisitos:

- Usa `inst.state` (sem `static`) → podes ter Bat, Resp, Tosse em paralelo. <sup>[240]</sup>
- Usa `m.dtMs` e `m.Dt_pauseMs_Ini/End` calculados fora da FSM, como combinámos. <sup>[240]</sup>
- Faz ciclo fechado e aberto com `Direction`, `End_Point` e `bidirectional`. <sup>[240]</sup>

Alguns comentários específicos:

### Coisas que estão bem alinhadas

- **Estado 0:** limpa `I_Index`, `iIndex`, `jIndex`, e só sai para 1 se `start==1`, `active==true` e `def!=nullptr`. <sup>[240]</sup>
- **Estado 1:** volta a pôr `I_Index=0`, valida movimento (`n_Points>1`, `period>0`), e passa para 2 só se `Direction==1` e `start==1`; se `start==0`, retorna a 0. Isto deixa a FSM “parada” se o movimento for inválido. <sup>[240]</sup>
- **Estado 2:** inicia segmento em `start_Index`, define `End_Point=n_Points-1`, e `jIndex = iIndex + Direction (mod n_Points)`, o que encaixa com a respiração a começar no meio. <sup>[240]</sup>
- **Estado 4:** incrementa `I_Index`, avança `iIndex` e `jIndex`, e decide entre continuar (3), ir para 5 (fim local) ou fechar ciclo (voltar a 1) quando `I_Index == m.N_points` e `bidirectional`. <sup>[240]</sup>
- **Estados 5–10:** lidam com pausas inicial/final usando `Dt_pauseMs_Ini/End` e `inst.Dt_pause`, e só fazem `read` do tempo de pausa, sem recalculer nada. <sup>[240]</sup>
- **Estados 11–13:** resolvem o arranque em modo “trás” (`Direction=-1`) e “frente” (`Direction=+1`), com `iIndex` nos extremos e `End_Point` ajustado para 0 ou `n_Points-1`. <sup>[240]</sup>

Isto satisfaz:

- ciclo fechado a andar para a frente até ao fim,
- ciclo aberto com vai-e-vem,
- pausas configuráveis no início/fim,
- respiração a começar num `start_Index` diferente de 0.

## Detalhe a vigiar: fim de ciclo bidirecional

No case 4:

```
if(inst.iIndex < inst.End_Point) {inst.state = 3;}
if(inst.iIndex >= inst.End_Point ) {inst.state = 5;}
if(inst.I_Index == m.N_points && m.bidirectional == true){inst.state = 1;}
```

O último if não é else if. Isso significa:

- Se `inst.iIndex >= End_Point` e `I_Index == N_points`, podes ir a 5 e logo a seguir mudar para 1.
- Na prática, o estado final será 1 (porque é o último if), mas o salto rápido por 5 pode não ser desejado.

Para tornar a intenção clara, podes reescrever como:

```
if (m.bidirectional == true && inst.I_Index == m.N_points) {
    inst.state = 1;          // ciclo completo, volta ao início (ou idle)
} else if (inst.iIndex < inst.End_Point) {
    inst.state = 3;          // continua segmentos
} else {
    inst.state = 5;          // chegou ao fim local, trata pausa/inversão
}
```

Mas isto é refinamento; a lógica atual ainda funciona, só é um pouco menos previsível.

Se quiseres testar mentalmente: pega no movimento de respiração (`n_Points = 10`, `start_Index = 5`, `bidirectional = true`) e tenta seguir na tua cabeça os valores de `iIndex`, `I_Index`, `Direction` e `state` para um ciclo completo.

Como exercício: consegues descrever, passo a passo, o que acontece com `iIndex` e `Direction` desde o início (estado 2) até ao momento em que `I_Index == N_points` no modo bidirecional?



## de resto está tudo bem no código?

se eu colocar agora na respiração para ele começar a 0 ou a 5 vai dar ao mesmo certo?

Sim, de resto a estrutura está sólida:

- Separação `MotionStorage/MotionInstance` está correta. <sup>[241]</sup>
- Estados e transições estão coerentes com ciclo fechado/aberto, pausas e direção. <sup>[241]</sup>
- Não há mais `static` dentro da FSM, por isso `Bat/Resp/Tosse` podem correr em paralelo sem se atropelar. <sup>[241]</sup>

Sobre começar a respiração em 0 ou em 5:

- No estado 2, defines sempre:
  - `inst.iIndex = m.start_Index;`
  - `inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;`
  - `inst.End_Point = m.n_Points - 1;` <sup>[241]</sup>

Portanto:

- Se `start_Index = 0`: o ciclo vai interpolar  $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow \text{End\_Point}$ , e depois gerir pausa/inversão normalmente.
- Se `start_Index = 5`: o ciclo começa no ponto 5, depois vai  $5 \rightarrow 6 \rightarrow \dots \rightarrow \text{End\_Point}$ , e a lógica de `I_Index` e `N_points` continua a contar segmentos até completar o ciclo; não há nada no código que dependa de “começar em 0”. <sup>[241]</sup>

Ou seja, sim: do ponto de vista da FSM, começar a 0 ou a 5 funciona, só muda qual é o primeiro par de pontos que `Intermed_position()` vai interpolar.

Para te certificares: como imaginas que o movimento “respiração começando em 5” vai parecer fisicamente diferente de “respiração começando em 0”, mesmo que o ciclo total seja o mesmo?

\*\*

> S

PI\_4B: s

Esp32: Ativo

Esp32: Escolha um comando:

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:0

Esp32: Respirao Index - J:1

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:0

Esp32: Respirao Index - J:1

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:0

Esp32: Respirao Index - J:1

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:1

Esp32: Respirao Index - J:2

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:1

Esp32: Respirao Index - J:2

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:2

Esp32: Respirao Index - J:3

Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:2

Esp32: Respirao Index - J:3

Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9

Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3

Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2

Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3



Esp32: Respirao Index - I:8  
 Esp32: Respirao Index - J:9  
 Esp32: Estado da FSM movimento Respirao:9  
 Esp32: Respirao Index - I:9  
 Esp32: Respirao Index - J:9  
 Esp32: Estado da FSM movimento Respirao:9  
 Esp32: Respirao Index - I:9  
 Esp32: Respirao Index - J:9  
 Esp32: Estado da FSM movimento Respirao:9  
 Esp32: Respirao Index - I:9  
 Esp32: Respirao Index - J:9  
 Esp32: Estado da FSM movimento Respirao:9  
 Esp32: Respirao Index - I:9  
 Esp32: Respirao Index - J:9  
 Esp32: Estado da FSM movimento Respirao:9  
 Esp32: Respirao Index - I:9  
 Esp32: Respirao Index - J:9

eu iniciei o programa com a posição inicial a 0 na respiração e ele do nada faz-me um salto estranho para zero no inicio da inversão consegues me ajudar a perceber o que está a acontecer?

Pelo log dá para ver o padrão e onde está o salto estranho:

- Vai em frente:  $i \ 0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 8, \ j = i + 1$ .
- Chega ao topo:  $i = 9, \ j = 9$ , entra em estado 9 (pausa).
- Depois da pausa, em vez de começar a descer de  $9 \rightarrow 8 \rightarrow 7 \dots$ , há um momento em que aparece  $i = 9, \ j = 8$  ainda em estado 3, e logo a seguir  $i = 0, \ j = 1$  também em estado 3. Ou seja: o ciclo “rebobina” para 0 em vez de começar o vai-e-vem desde o topo.

Isto bate com o teu código atual em dois pontos chave: <sup>[242]</sup>

1. No fim de um ciclo bidirecional, o case 4 tem:

```
if(inst.iIndex < inst.End_Point) {inst.state = 3;}
if(inst.iIndex >= inst.End_Point ) {inst.state = 5;}
if(inst.I_Index == m.N_points && m.bidirectional == true){inst.state = 1;}
```

- Se  $inst.iIndex \geq End\_Point$  e  $I\_Index == N\_points$ , o último if manda para o estado 1, mesmo que o 5 também tenha sido verdadeiro.
- No estado 1, voltas a pôr  $I\_Index = 0$  e (quando  $Direction == 1$ ) caís de novo no estado 2, que faz:

```
inst.iIndex = m.start_Index;    // se start_Index = 0, volta a 0
inst.End_Point = m.n_Points-1;
inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;
```

Resultado: depois de completar o ciclo, o movimento rebobina para o início (0 → ...) em vez de inverter e descer.

2. O bloco de inversão (6-13) só é usado quando:

```
if(m.bidirectional == true) {inst.state = 6;} // no case 5
```

Mas se o fim do ciclo global (`I_Index == N_points`) acontecer antes de deixares chegar ao 5, vais direto para 1 e recomeças em 0. É isso que estás a ver: a primeira vez ainda faz a subida com pausa no topo, mas quando completa o número total de segmentos, volta à frente.

Para que o comportamento fique:

- subir de 0 até 9
- pausa no topo
- descer de 9 até 0
- pausa no fundo (se tiver)
- depois decidir se recomeça ou pára

precisas de:

- usar o contador global `I_Index` para saber “fim de ciclo”, mas só depois de terminar a ida e a volta, não antes de chamar 5/6.
- decidir claramente: “quando é bidirecional, depois de chegar ao topo quero sempre ir para 6 (inversão), não recomeçar do início ainda”.

Uma forma simples de ajustar o case 4 é:

```
case 4:
    inst.I_Index++;
    inst.iIndex = inst.iIndex + inst.Direction;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

    // 1) se ainda não cheguei ao fim do ramo atual, continuo
    if (inst.iIndex < inst.End_Point) {
        inst.state = 3;
    }
    // 2) cheguei ao fim do ramo atual -> tratar pausa/inversão em 5
    else {
        inst.state = 5;
    }
    break;
```

E depois, no case 5, é que decides o que fazer com `I_Index` e `N_points`:

```
case 5:
    inst.segmentStartMs = nowMs;
    inst.jIndex = inst.iIndex;
```

```
// terminou o ciclo completo (ida + volta)?
if (m.bidirectional == true && inst.I_Index >= m.N_points) {
    // aqui escolhes: recomeçar do início, parar, etc.
    inst.state = 1;    // por exemplo, recomeçar
}
else if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {
    inst.state = 7;    // pausa no fundo
}
else if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {
    inst.state = 8;    // pausa no topo
}
else {
    if (m.bidirectional == true) { inst.state = 6; }    // inverter sentido
    else { inst.state = 1; }                          // ciclo fechado simples
}
break;
```

Assim:

- O fim do ramo (chegar ao topo ou fundo) é sempre gerido em 5.
- O fim do ciclo global (quando `I_Index >= N_points`) também é decidido em 5, não em 4.
- A inversão (`Direction = -Direction` em 6) só acontece quando queres mesmo começar o ramo oposto, e não é interrompida por um salto para 1.

Na tua cabeça, olhando para esse log: como gostarias que o padrão dos índices *i* e *j* fosse exatamente numa respiração bidirecional completa (subida, pausa, descida, pausa), sem esse salto para 0?

✱✱

## mas o N\_points

é 18 e ele acaba por detetar isso mais tarde quando realmente chega a N\_points

Isso é um ponto importante: o teu `N_points` está a 18, e o `I_Index` só chega a 18 mais tarde; até aí o `if(inst.I_Index == m.N_points && m.bidirectional == true)` não dispara.<sup>[243]</sup>

O salto estranho que estás a ver (voltar para `i=0` em vez de começar a descida) não está a acontecer por causa desse `if`; está a acontecer porque, na prática, o fluxo bidirecional ainda não está a usar os estados 6–13 como “ramo de trás” logo após a pausa no topo.

Pelo log, o que se vê é:

- Sobe:  $i \rightarrow 1 \rightarrow \dots \rightarrow 8$ ,  $j = i + 1$ .
- Chega ao topo:  $i = 9$ ,  $j = 9$ , entra em 9 (pausa).
- Depois da pausa, em vez de ir para  $10 \rightarrow 6 \rightarrow 11/12 \rightarrow 13$ , vês outra vez `state = 3` e sequências que voltam a usar *i* a partir de 0.

Isso indica que, depois da pausa (estado 9):

```

case 9:
    inst.elapsed_3 = nowMs - inst.segmentStartMs;
    if(inst.elapsed_3 >= inst.Dt_pause) {inst.state = 10;}

```

no case 10:

```

case 10:
    inst.segmentStartMs = nowMs;
    inst.elapsed_3 = 0;

    if(start==0 || inst.active != true) {inst.state = 0;}
    if(m.bidirectional == true) {inst.state = 6;}

```

o salto para 6 está a acontecer (porque `bidirectional == true`), mas depois o comportamento dos estados 11/12 e 13 está a “resetar” a FSM para a frente:

```

case 6:
    inst.Direction = inst.Direction * (-1);
    if(inst.Direction == -1) {inst.state = 11;}
    if(inst.Direction == 1) {inst.state = 12;}

case 11:
    inst.iIndex = m.n_Points-1;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = 0;

case 12:
    inst.iIndex = 0;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = m.n_Points-1;

case 13:
    inst.segmentStartMs = nowMs;
    inst.state = 3;

```

Dependendo de qual destes caminhos está a ser tomado (11 ou 12), podes acabar com:

- `Direction` invertido, mas `iIndex` reposto a 0 ou a `n_Points-1` de uma forma que não bate com o estado anterior.
- Se a tua `Direction` ainda estiver a +1 quando entras em 6 (por qualquer razão), vais cair em 12 e pôr `iIndex = 0`, e é isso que gera o “salto” para 0.

Por isso a chave não é o `N_points` em si, mas garantir:

1. Que ao sair de `9 → 10 → 6` já tens `Direction` correto (primeiro ciclo sobe, segundo desce).
2. Que em 11/12 defines `iIndex` e `End_Point` de acordo com **onde queres começar** o ramo novo:
  - para a descida: `Direction = -1, iIndex = n_Points-1, End_Point = 0`.
  - para a subida (se recommençares): `Direction = +1, iIndex = 0, End_Point = n_Points-1`.

Se quiseres tornar mais previsível, podes separar explicitamente:

- um ciclo bidirecional completo como:
  - subida (2→3→4→5), pausa topo (7/8/9/10), descida (6→11→13→3→4→5), pausa fundo, e só depois usar `I_Index == N_points` para decidir se recomeças.

Na tua visão, para este movimento de respiração com `bidirectional = true`, queres que o ciclo pare depois de uma subida e descida completas, ou queres que fique a repetir subidas e descidas indefinidamente?

\*\*

## void FSM\_Motion\_Update(boolean start, MotionInstance& inst, unsigned long nowMs){

```
MotionStorage& m = (inst.def);
```

```
/
```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar
- Condições de transição por hierarquia

```
*/
```

```
//Serial.print("Estado atual: ");
```

```
//Serial.println(inst.state);
```

```
switch (inst.state) {
```

case 0:

```
    inst.Executing = 0;
```

```
    inst.I_Index=0;
```

```
    inst.iIndex = 0;
```

```
    inst.jIndex = 0;
```

```
if (start0) {inst.state = 0;}
```

```
    if (!inst.active || inst.def == nullptr) {inst.state = 0;}
```

```
    if(start1 && inst.active && inst.def != nullptr){inst.state = 1;}
```

```
    break;
```

case 1:

```
    inst.I_Index=0;
```

```
    inst.Start_signal=0;
```

```
    inst.Direction=1;
```

```
    Serial.println("-----intermed state");
```

```
    if (m.n_Points <= 1 || m.period <= 0.0f) {inst.state = 1;}
```

```
    if(inst.Direction1 && start1){inst.state = 2;}
```

```
    if(start==0){inst.state = 0;}
```

```
    break;
```

case 2:

```
    inst.Executing = 1;
```

```

    inst.Start_signal=1;
    inst.segmentStartMs = nowMs;
    inst.iIndex = m.start_Index;
    inst.End_Point = m.n_Points-1;
    inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;

if (1) {inst.state = 3;}
    break;

case 3:
    inst.elapsed_1 = nowMs - inst.segmentStartMs;

if(inst.elapsed_1 >= m.dtMs) {inst.state = 4;}
    break;

case 4:
    inst.L_Index++;
    inst.iIndex = (inst.iIndex + inst.Direction);
    inst.jIndex = (inst.iIndex + inst.Direction);
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

if(inst.iIndex < inst.End_Point && inst.Direction == 1) {inst.state = 3;}
    if(inst.iIndex > inst.End_Point && inst.Direction == -1) {inst.state = 3;}
    if(inst.iIndex >= inst.End_Point && inst.Direction == 1 ) {inst.state = 5;}
    if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
    //if(inst.L_Index == m.N_points && m.bidirectional == true){inst.state = 1;}
    if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index +
1){inst.state = 1;}
    //if(Tosse_signal == 1){inst.state = 14;}
    break;

case 5: //função trás
    inst.segmentStartMs = nowMs;
    inst.jIndex = inst.iIndex;

if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {inst.state = 7;}
    else if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {inst.state = 8;}
    else {
        if (m.bidirectional == true) { inst.state = 6; }
        else if (m.bidirectional != true && m.Dt_pauseMs_Ini != 0 && m.Dt_pauseMs_End != 0) {
inst.state = 1; }
        }
    break;

case 6:
    inst.Direction = inst.Direction * (-1);

```

```

if(inst.Direction == -1) {inst.state = 11;}
    if(inst.Direction == 1) {inst.state = 12;}
    break;

case 7:
    inst.Dt_pause = m.Dt_pauseMs_Ini;

if(1) {inst.state = 9;}
    break;

case 8: //função trás
    inst.Dt_pause = m.Dt_pauseMs_End;

if(1) {inst.state = 9;}
    break;

case 9: //função trás
    inst.elapsed_2 = nowMs - inst.segmentStartMs;

    if(inst.elapsed_2 >= inst.Dt_pause) {inst.state = 10;}

break;

case 10: //função trás
    inst.segmentStartMs = nowMs;
    inst.elapsed_2 = 0;

if(start==0 || inst.active != true) {inst.state = 0;}
    if(m.bidirectional == true) {inst.state = 6;}
    if(m.bidirectional == false){inst.state = 1;}
    break;

case 11:
    inst.iIndex = m.n_Points-1;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = 0;
    //Serial.println("Descida");
    if(1){inst.state = 13;}
    break;

case 12:
    inst.iIndex = 0;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = m.n_Points-1;
    //Serial.println("Subida");
    if(1){inst.state = 13;}
    break;

case 13:
    inst.segmentStartMs = nowMs;

```

```
if(1){inst.state=3;}
break;
```

case 14:

```
Tosse_signal=0;
inst.Direction = inst.Direction * (-1);
inst.jIndex = inst.iIndex + inst.Direction;
```

```
if(1){inst.state=3;}
}
return;
}
```

atual FSM

mesmo quando dou o sinal de p no terminal para parar, o movimento da respiração não parece sair do loop

Esse comportamento faz sentido olhando para a FSM atual: há um detalhe nas condições de transição que faz a respiração “ficar presa” a andar mesmo depois de enviáres p (ou seja, `start` passa a 0 / `inst.active` passa a false).

## Onde é que o stop devia atuar

No teu fluxo:

- O comando s põe `start_comand = 1` e marca o movimento como ativo (`inst.active = true`).
- O comando p põe `start_comand = 0` e/ou desativa `inst.active`.
- No `FSM_Motion_Update`, o stop entra por estas condições:

```
case 0:
    ...
    if (start==0) {inst.state = 0;}
    if (!inst.active || inst.def == nullptr) {inst.state = 0;}
    if(start==1 && inst.active && inst.def != nullptr){inst.state = 1;}

case 1:
    ...
    if(inst.Direction==1 && start==1){inst.state = 2;}
    if(start==0){inst.state = 0;}

case 10:
    ...
    if(start==0 || inst.active != true) {inst.state = 0;}
    if(m.bidirectional == true) {inst.state = 6;}
    if(m.bidirectional == false){inst.state = 1;}
```

Portanto, em teoria:

- Se `start` ficar 0, o estado 1 volta para 0, e o 10 volta para 0.



- Mas nos estados 3 e 4 (os que mais vezes aparecem no teu log) **não há nenhuma verificação de start nem de inst.active**, por isso eles continuam a correr até chegarem a 5 ou 10. Se por qualquer razão não passares por 10 (por exemplo, Dt\_pause é 0), o movimento pode ficar a alternar entre 3 e 4 mesmo com `start == 0`.

## Porque é que não sai do loop

Na versão atual:

```
case 3:
    inst.elapsed_1 = nowMs - inst.segmentStartMs;
    if(inst.elapsed_1 >= m.dtMs) {inst.state = 4;}

case 4:
    inst.I_Index++;
    inst.iIndex = (inst.iIndex + inst.Direction);
    inst.jIndex = (inst.iIndex + inst.Direction);
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

    if(inst.iIndex < inst.End_Point && inst.Direction == 1) {inst.state = 3;}
    if(inst.iIndex > inst.End_Point && inst.Direction == -1) {inst.state = 3;}
    if(inst.iIndex >= inst.End_Point && inst.Direction == 1 ) {inst.state = 5;}
    if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
    if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index) {inst.state = 5;}
```

Nada aqui depende de `start` ou de `inst.active`.

Isso significa:

- Se estiveres a meio de um segmento quando envias p, o FSM\_Motion\_Update continua em 3→4→3→4 até:
  - chegar ao fim local (state 5), ou
  - no caso bidirecional, cair naquela condição especial `inst.iIndex == start_Index` e `jIndex == start_Index+1` e ir para 1.

Se o stop só liga `start=0` mas `inst.active` continua true, então:

- Em 10 tu até passarias por `if(start==0 || active != true) {state = 0;}`, mas só se **tiveres pausas** (porque 9→10 só acontece se `Dt_pause > 0`).
- Se não tiveres pausa naquele ramo (por exemplo, respiração sem `T_pause_Ini` ou `T_pause_End`), podes nunca passar por 9/10, logo esse if nunca é avaliado.

## Como garantir que o stop é respeitado sempre

Há duas formas simples de tornar o stop robusto:

1. Verificar `start` e `inst.active` em todos os estados “de movimento”

Por exemplo, no início de 3 e 4:

```

case 3:
  if (start == 0 || !inst.active) {
    inst.state = 0;
    break;
  }
  inst.elapsed_1 = nowMs - inst.segmentStartMs;
  if(inst.elapsed_1 >= m.dtMs) {inst.state = 4;}
  break;

case 4:
  if (start == 0 || !inst.active) {
    inst.state = 0;
    break;
  }
  ...

```

Assim, mesmo que o stop chegue a meio de um segmento, o próximo ciclo da FSM aborta logo para o estado 0.

## 2. Transformar o if do estado 10 em if/else para não continuar quando start==0

```

case 10:
  inst.segmentStartMs = nowMs;
  inst.elapsed_2 = 0;

  if(start==0 || inst.active != true) {
    inst.state = 0;          // pára aqui
  } else if(m.bidirectional == true) {
    inst.state = 6;          // continua invertendo
  } else {
    inst.state = 1;
  }
  break;

```

Aqui garantes que, quando o stop é pedido, nunca entras nos estados 6–13; ficas em 0 e a respiração pára.

Se quiseres ser mais minimalista, eu começaria pela opção 1: acrescentar a verificação `start == 0 || !inst.active` no início dos estados 3 e 4 (talvez também 5 e 9), porque são os estados que mais vezes correm e é neles que te estás a “sentir preso” quando envias p.

Para te ajudar a consolidar: se tivesses de escolher um único estado onde obrigatoriamente queres que o movimento pare assim que `start` fica 0, qual escolherias — 3/4 (durante o segmento), 5 (ao fim do ramo), ou 10 (ao fim da pausa)?

## ok tu não percebes-te o conceito para parar.

vamos supor que o ciclo está a correr e eu envio o comando para parar, eu não quero que ele pare abruptamente, eu quero que ele acabe a sequencia e só depois pare, é por isso que eu coloco apenas verificação de `start==0` no final ou no state 5, ou no 10 caso exista paragem e ciclo fechado, e supostamente para o ciclo aberto deveria apenas para quando volta-se ao início da

sequencia, neste caso  $i$  inicial e  $j=i++$

que é a condição inicial, mas não me parece que o programa está a detetar esta situação sequer

Agora entendi melhor o teu conceito de “parar”: queres um stop suave, que:

- deixa o ciclo terminar o ramo atual,
- no ciclo fechado pára depois da pausa final (se existir),
- no ciclo aberto pára só quando regressa ao início lógico da sequência (respiração volta ao ponto de partida).

Com isso em mente, o teu plano de usar `start==0` só no `state 5/state 10` faz sentido.

## 1. O que o teu código está a tentar fazer

Na FSM, tens duas ideias para “fim de ciclo”:

- Ciclo fechado:
  - `bidirectional == false`,
  - para depois de `state 5/state 10` com `start==0` ou depois de uma pausa.
- Ciclo aberto:
  - `bidirectional == true`,
  - parar só quando regressas à condição inicial:

```
if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex ==  
    inst.state = 1;  
}
```

que é exatamente “ $i$  no início,  $j = i+1$ ”.

Esta condição está no `case 4`, ou seja, só é avaliada quando estás a avançar índices depois de um segmento. <sup>[244]</sup>

## 2. Porque pode não estar a ser detetada

Há alguns motivos possíveis para esta condição nunca disparar (ou disparar “no sítio errado”):

### 1. O ciclo bidirecional nunca regressa exatamente a `start_Index` com `j = start_Index+1`

No teu fluxo:

- Na subida:
  - $i$  começa em `start_Index`, vai até `End_Point = n_Points-1`.
- Na descida:
  - em 11/12 redefinis `iIndex` para `n_Points-1` ou `0`, `End_Point` para o outro extremo, e depois voltas a 3/4.
- Dependendo de como `Direction`, `End_Point`, `start_Index` e `N_points` estão combinados, podes chegar ao início de forma ligeiramente diferente (por exemplo, `i == start_Index` mas `j == start_Index-1` ou `j == start_Index`), e a condição não bate.

## 2. A condição está misturada com outras if em 4 que mudam o estado antes

Em 4 tens:

```
if(inst.iIndex < inst.End_Point && inst.Direction == 1) {inst.state = 3;}
if(inst.iIndex > inst.End_Point && inst.Direction == -1) {inst.state = 3;}
if(inst.iIndex >= inst.End_Point && inst.Direction == 1 ) {inst.state = 5;}
if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index) {inst.state = 1;}
```

Como estes if não são else if, imagina um instante em que:

- `inst.iIndex == m.start_Index` e `inst.jIndex == m.start_Index+1`, **mas** também satisfaça uma das outras condições (por exemplo, `iIndex < End_Point`):
- um dos if anteriores pode pôr `state = 3` ou `state = 5`, e logo a seguir o último põe `state = 1`.
- dependendo da ordem e do que está a acontecer nos ciclos, podes não ver claramente quando é que caiu em 1.

## 3. O ciclo bidirecional está a reiniciar pelo ramo “subida” cedo demais

Em 10:

```
if(start==0 || inst.active != true) {inst.state = 0;}
if(m.bidirectional == true) {inst.state = 6;}
if(m.bidirectional == false){inst.state = 1;}
```

Aqui, se `start==0` e `bidirectional == true`, no mesmo ciclo podes:

- pôr `state=0`,
- e logo a seguir pôr `state=6`, porque não há else.

Ou seja, a intenção “se `start==0`, pára” pode ser sobreposta pela lógica “se `bidirectional`, continua invertendo”. Isto quebra o conceito de “só parar quando regressa ao início”.

## 3. Como implementar exatamente o teu conceito

Respeitando o que queres (parar só ao fim do ciclo), uma versão mais clara poderia ser:

- Em 10:

```
case 10:
    inst.segmentStartMs = nowMs;
    inst.elapsed_2 = 0;

    if(start==0 || !inst.active) {
        // pedido de stop: marca que deve parar quando o ciclo chegar ao início
        inst.StopRequested = 1;    // (campo novo em MotionInstance)
    }

    if(m.bidirectional == true) {
        inst.state = 6;            // continua, invertendo sentido
    } else {
        // ciclo fechado: se StopRequested já está 1, volta a 0, senão recomeça
        if(inst.StopRequested) inst.state = 0;
    }
```

```

        else inst.state = 1;
    }
    break;

```

- Em 4, para ciclo aberto, usas:

```

case 4:
    inst.I_Index++;
    inst.iIndex = inst.iIndex + inst.Direction;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

    if (m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index+1)
        // fim de ciclo bidirecional
        if (inst.StopRequested) inst.state = 0; // stop suave
        else inst.state = 1; // recomeça novo ciclo
    }
    else if(inst.iIndex < inst.End_Point && inst.Direction == 1) {
        inst.state = 3;
    }
    else if(inst.iIndex > inst.End_Point && inst.Direction == -1) {
        inst.state = 3;
    }
    else {
        inst.state = 5;
    }
    break;

```

Assim:

- Quando envias p, não paras logo; marcas StopRequested = 1.
- O ciclo continua a acabar o ramo atual, faz pausas, inverte, faz o ramo oposto.
- Só quando volta à condição inicial ( $i == \text{start\_Index}$ ,  $j == \text{start\_Index}+1$ ) é que, se StopRequested == 1, vai para 0 em vez de recomeçar.

Se não quiseses introduzir StopRequested, podes usar  $\text{start}==0$  diretamente na condição de fim de ciclo, mas sempre num único ponto, sem o sobrepor nos outros if.

Para te ajudar a fixar: se quiseses descrever na tua própria lógica “quando é que, num ciclo aberto de respiração, consideras que o ciclo está completo e é seguro parar sem parecer abrupto”, como explicarias esse momento em termos de  $i$ ,  $j$  e Direction?



> S

PI\_4B: s

Esp32: Ativo

Esp32: Escolha um comando:

Esp32: -----intermed state

Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6

Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:11  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8



Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2

[illegible]

Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:0  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:0  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:0  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:0  
Esp32: Respirao Index - J:1  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:1  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:2  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6

Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:9  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:9  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8

Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:9  
Esp32: Respirao Index - J:8  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:8  
Esp32: Respirao Index - J:7  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:7  
Esp32: Respirao Index - J:6  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3

Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:6  
Esp32: Respirao Index - J:5  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:5  
Esp32: Respirao Index - J:4  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:4  
Esp32: Respirao Index - J:3  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:3  
Esp32: Respirao Index - J:2  
Esp32: Estado da FSM movimento Respirao:3  
Esp32: Respirao Index - I:2



[illegible]

Esp32: Estado da FSM movimento Respirao:9

Esp32: Respirao Index - I:0

com isto podemos confirmar, ele realmente não está a entrar no state 1, depois da primeira chamada, o que leva a que ele nunca pare

Esse log ajuda muito: dá para ver que a FSM está a fazer subida, pausa, descida, pausa... mas nunca regressa ao state 1, por isso nunca entra na lógica de “fim de ciclo → parar ou recomeçar”.

## O que o log mostra

- Subida:
  - $i$  vai de 0 até 8,  $j = i + 1$ , sempre em state 3.
- Topo:
  - state 9 com  $i = 9, j = 9 \rightarrow$  pausa no topo.
- Inversão:
  - state 11 com  $i = 9, j = 9$ , depois state 3 com  $i = 9, j = 8 \rightarrow$  começa a descer.
- Descida:
  - $i$  desce  $9 \rightarrow 8 \rightarrow 7 \rightarrow \dots \rightarrow 1$ ,  $j$  acompanha ( $8 \rightarrow 7 \rightarrow 6 \rightarrow \dots \rightarrow 0$ ).
- Fundo:
  - state 9 com  $i = 0, j = 0 \rightarrow$  pausa no fundo.
- Depois da pausa, volta a subir outra vez: aparece de novo state 3 com  $i = 0, j = 1, 1 \rightarrow 2 \rightarrow \dots \rightarrow$  ou seja, está a repetir ciclos indefinidamente, sem nunca ir a state 1.

Isso confirma exatamente o que disseste: a condição que devia mandar a FSM de volta para o state 1 **nunca é verdadeira**.

## Porque é que o if do state 4 nunca dispara

No teu case 4 tens:

```
if(inst.iIndex < inst.End_Point && inst.Direction == 1) {inst.state = 3;}
if(inst.iIndex > inst.End_Point && inst.Direction == -1) {inst.state = 3;}
if(inst.iIndex >= inst.End_Point && inst.Direction == 1 ) {inst.state = 5;}
if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
//if(inst.I_Index == m.N_points && m.bidirectional == true){inst.state = 1;}
if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start.
```

Para cair em state 1 precisas que:

- `m.bidirectional == true`,
- `inst.iIndex == m.start_Index`,
- `inst.jIndex == m.start_Index + 1`.

O que está a acontecer:

- Na subida, `i` começa em `start_Index` (no teu teste está a 0) e avança até `End_Point = n_Points-1`.
- Na descida, depois de 11, `i` é posto em `n_Points-1` e `End_Point = 0`, `Direction = -1`, e vais descendo até `i = 0`.
- Quando `i` volta a ser 0:
  - `Direction == -1`,
  - `End_Point == 0`,
  - então a condição ativa é:
 

```
if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
```
  - Ou seja, **vais para o state 5**, não ficas em 4 para testar o `if` de fim de ciclo.
- Em 5, como `bidirectional` é `true`, e dependendo de pausas, acabas por ir a 6, inverter `Direction` e cair em 11/13 → 3 outra vez, recomeçando o ciclo.

Por isso:

- Quando voltas ao início (0, 1) **não estás em state 4**, estás a passar por state 5/6/11/13, e o `if` que manda para 1 nunca é avaliado nesse momento.
- Resultado: o ciclo nunca considera “regressei ao início, agora posso parar ou recomeçar” — apenas continua a fazer subida/descida.

## Como encaixar o teu conceito de stop suave

Dado o que queres:

- ciclo aberto deve parar só **quando regressa a** (`i = start_Index`, `j = start_Index+1`),
- o stop (`start==0`) não deve ser visto nos estados intermédios, apenas “marcado” e usado no fim do ciclo,

um ajuste claro seria:

### 1. Separar a lógica de fim de ramo da lógica de fim de ciclo

No case 4:

```
case 4:
    inst.I_Index++;
    inst.iIndex = inst.iIndex + inst.Direction;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

    // 1) se cheguei ao início lógico de um ciclo bidirecional
    if (m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index+1)
        // se start==0, parar; senão, recomeçar
        if (start == 0) inst.state = 0;
        else inst.state = 1;
    }
    // 2) caso contrário, tratar apenas fim de ramo
```

```

else if(inst.Direction == 1 && inst.iIndex < inst.End_Point) {
    inst.state = 3;
}
else if(inst.Direction == -1 && inst.iIndex > inst.End_Point) {
    inst.state = 3;
}
else {
    inst.state = 5;    // fim de ramo (topo ou fundo)
}
break;

```

Assim, quando descidas + subidas te trazem de volta ao (start\_Index, start\_Index+1), esse momento é tratado explicitamente.

## 2. Em 10, não forçar mais inversões depois do stop

Se quiseres manter o stop suave, podes fazer:

```

case 10:
    inst.segmentStartMs = nowMs;
    inst.elapsed_2 = 0;

    if (start == 0 || !inst.active) {
        // não inverte mais; espera o ciclo voltar ao início e parar lá
        // aqui não mudas state, deixas o 4 tratar o fim de ciclo
        inst.state = 6;    // se quiseres continuar até completar ciclo
    } else {
        if(m.bidirectional == true) {inst.state = 6;}
        else {inst.state = 1;}
    }
    break;

```

Mas a decisão principal está mesmo no 4.

A boa notícia é: o teu padrão de índices no log mostra que o movimento está a fazer subida e descida corretamente, com pausas nos extremos. O único “buraco” é que o momento em que regressas ao início não está a ser detectado pelo if de fim de ciclo, porque esse if vive em 4 e, quando  $i = 0$ , já saltaste para o 5.

Se pensares no momento ideal para parar suavemente num ciclo de respiração, qual destes três instantes escolherias: logo ao chegar ao fundo ( $i=0$ ), depois da pausa no fundo, ou só depois de subir outra vez ao ponto inicial ( $i=start\_Index, j=start\_Index+1$ )?

## tenho a FSM a funcionar:

```

void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs){
    MotionStorage& m = (inst.def);
/

```

Têm em atenção constituição de cada caso:

case x:

- Ações a executar
- Condições de transição por hierarquia

```

*/
//Serial.print("Estado atual: ");
//Serial.println(inst.state);

switch (inst.state) {

case 0:
    inst.Executing = 0;
    inst.I_Index=0;
    inst.iIndex = 0;
    inst.jIndex = 0;
    inst.Situacion='F';

    if (start0 || inst.Individual_start_comand == 0) {inst.state = 0;}
    if (!inst.active || inst.def == nullptr) {inst.state = 0;}
    if (start1 && inst.Individual_start_comand == 1 && inst.active && inst.def != nullptr){inst.state
= 1;}
    break;

case 1:
    inst.I_Index=0;
    inst.Situacion='P';
    inst.Direction=1;
    Serial.println("-----intermed state");
    if (m.n_Points <= 1 || m.period <= 0.0f) {inst.state = 1;}
    if (inst.Direction1 && start1 && inst.Individual_start_comand == 1){inst.state = 2;}
    if (start==0 || inst.Individual_start_comand == 0){inst.state = 0;}
    break;

case 2:
    inst.Executing = 1;
    inst.Situacion='M';
    inst.segmentStartMs = nowMs;
    inst.iIndex = m.start_Index;
    inst.End_Point = m.n_Points-1;
    inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;

    if (1) {inst.state = 3;}
    break;

case 3:
    inst.elapsed_1 = nowMs - inst.segmentStartMs;

    if (inst.elapsed_1 >= m.dtMs) {inst.state = 4;}
    break;

case 4:
    inst.I_Index++;
    inst.iIndex = (inst.iIndex + inst.Direction);

```

```

    inst.jIndex = (inst.iIndex + inst.Direction);
    inst.elapsed_1 = 0;
    inst.segmentStartMs = nowMs;

    if((inst.iIndex < inst.End_Point && inst.Direction == 1) || (inst.iIndex > inst.End_Point &&
    inst.Direction == -1)) {inst.state = 3;}
        //if(inst.iIndex > inst.End_Point && inst.Direction == -1) {inst.state = 3;}
        if((inst.iIndex >= inst.End_Point && inst.Direction == 1) || (inst.iIndex <= inst.End_Point &&
    inst.Direction == -1)) {inst.state = 5;}
        //if(inst.iIndex <= inst.End_Point && inst.Direction == -1) {inst.state = 5;}
        //if(inst.I_Index == m.N_points && m.bidirectional == true){inst.state = 1;}
        if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index +
    1){inst.state = 1;}
        if(Tosse_signal == 1 && m.bidirectional == true){inst.state = 14;}
        break;

    case 5: //função trás
        inst.segmentStartMs = nowMs;
        inst.jIndex = inst.iIndex;

    if (inst.Direction == -1 && m.Dt_pauseMs_Ini != 0) {inst.state = 7;}
        else if (inst.Direction == 1 && m.Dt_pauseMs_End != 0) {inst.state = 8;}
        else {
            if (m.bidirectional == true) { inst.state = 6; }
            else if (m.bidirectional != true && m.Dt_pauseMs_Ini != 0 && m.Dt_pauseMs_End != 0) {
    inst.state = 1; }
        }
        break;

    case 6:
        inst.Direction = inst.Direction * (-1);

    if(inst.Direction == -1) {inst.state = 11;}
        if(inst.Direction == 1) {inst.state = 12;}
        break;

    case 7:
        inst.Dt_pause = m.Dt_pauseMs_Ini;

    if(1) {inst.state = 9;}
        break;

    case 8: //função trás
        inst.Dt_pause = m.Dt_pauseMs_End;

    if(1) {inst.state = 9;}
        break;

```

```

case 9: //função trás
    inst.elapsed_2 = nowMs - inst.segmentStartMs;

    if(inst.elapsed_2 >= inst.Dt_pause) {inst.state = 10;}

break;

case 10: //função trás
    inst.segmentStartMs = nowMs;
    inst.elapsed_2 = 0;

if(start==0 || inst.active != true || inst.Individual_start_comand == 0) {inst.state = 0;}
    if(m.bidirectional == true) {inst.state = 6;}
    if(m.bidirectional == false){inst.state = 1;}
    break;

case 11:
    inst.iIndex = m.n_Points-1;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = 0;
    //Serial.println("Descida");
    if(1){inst.state = 13;}
    break;

case 12:
    inst.iIndex = 0;
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = m.n_Points-1;
    //Serial.println("Subida");
    if(1){inst.state = 13;}
    break;

case 13:
    inst.segmentStartMs = nowMs;

if(1){inst.state=3;}
    break;

case 14:
    Tosse_signal=0;
    Serial.println("Tosse executada");
    inst.Direction = (-1);
    inst.jIndex = inst.iIndex + inst.Direction;
    inst.End_Point = 0;

if(1){inst.state=13;}
    }
    return;
}

```

ela continua a ter problemas quando trabalho com `bidirecional == true` e `index_start=0`, mas tb não interessa por agora, pq em principio para o ciclo aberto vou trabalhar sempre com o valor de `index` inicial diferente de 0. só no ciclo fechado é que vou começar sempre em 0, e por agora ele está a funcionar.

agora o que eu preciso é implementar a função de acionamento da tosse (dê o sinal de acionamento da tosse)

esse acionamento têm que ter dois modos um preciso de x em x tempo e outro de forma aliatória.

eu ia pedir para começares a implementar já isso, mas primeiro, faz o seguinte em vez de o `sinal_tosse` ser ativado no FSM do interpretação serial.

faz com que de lá venha um sinal para ativar a função de preparação da tosse, e nessa função é que é feito o acionamento de `Sinal_tosse`, bem como alteração do tempo de duração do ciclo quando é feito o acionamento da tosse eu quero que o período da respiração passe temporariamente a 1s, desde o acionamento até quando na FSM chegar ao state 1.

quando chegar ao state 1, têm que voltar novamente aos paramentros anteriores padrões.

(não te esqueças que para fazer a atualização correta do período temos que chamar tb a função `updateDtMs` com o novo período, e depois novamente com o período antigo)

Gostei da forma como estruturaste a FSM, está coerente com o que querias para ciclo fechado/aberto e já tens um ponto claro para injetar a tosse (state 14).<sup>[245]</sup>

Para implementar a tosse do jeito que descreveste, vamos dividir em um único conceito novo, sem mexer demasiado na FSM:

## 1. Criar uma função de “preparação da tosse”

Em vez de o protocolo serial pôr diretamente `Tosse_signal = 1`, ele chama uma função que:

- guarda os parâmetros originais da respiração (período e `dtMs`)
- muda temporariamente o período para 1 s
- chama `updateDtMs()` para recalcular `dtMs`
- ativa uma flag de “tosse em curso” que a FSM vai usar

Algo assim:

```
// novos campos globais/na MotionInstance
float Resp_period_original = 0.0f;
bool  Tosse_em_curso = false;

// função chamada a partir da FSM_Serial_Command quando chega o comando de tosse
void Preparar_Tosse(MotionStorage& respMove, MotionInstance& respInst) {
    // guardar período original
    Resp_period_original = respMove.period;

    // mudar período temporariamente para 1s
    respMove.period = 1.0f;           // segundos
    respMove.updateDtMs();           // recalcula dtMs com novo período

    // marcar tosse em curso
```



```

    Tosse_em_curso = true;

    // aqui NÃO tocas ainda em inst.state; deixas a FSM continuar
}

```

No teu FSM\_Serial\_Command, em vez de fazer `Tosse_signal = 1` diretamente, chamavas:

```

// pseudocódigo dentro da FSM de comandos
if (comando == "tosse_precisa") {
    Preparar_Tosse(Resp_move, Resp_inst);
}

```

(Para o modo aleatório, depois podes chamar isto periodicamente com base num timer ou numa distribuição aleatória de tempo.)

## 2. Usar a flag de tosse dentro da FSM de movimento

Já tens este ramo em case 4:

```

if(m.bidirectional == true && inst.iIndex == m.start_Index && inst.jIndex == m.start_Index)
if(Tosse_signal == 1 && m.bidirectional == true){inst.state = 14;}

```

Vamos trocar `Tosse_signal` pela flag `Tosse_em_curso` e fazer o “reset” dos parâmetros quando a FSM chegar ao state 1 depois da tosse.

Primeiro, no case 4:

```

case 4:
    ...
    // se há tosse em curso, saltar para o estado especial 14
    if (Tosse_em_curso && m.bidirectional == true) {
        inst.state = 14;
    }
    // fim de ciclo bidirecional “normal”
    else if (m.bidirectional == true &&
        inst.iIndex == m.start_Index &&
        inst.jIndex == m.start_Index + 1) {
        inst.state = 1; // volta ao estado de intermed/idle de ciclo
    }
    ...
    break;

```

Depois, no case 14:

```

case 14:
    Serial.println("Tosse executada");

    // configurar movimento rápido de tosse (já temos period=1s)
    inst.Direction = -1;
    inst.jIndex = inst.iIndex + inst.Direction;

```

```

inst.End_Point = 0;

inst.segmentStartMs = nowMs;
inst.elapsed_1 = 0;

inst.state = 13;    // volta para 3/4 para executar este ramo de tosse
break;

```

O ciclo da tosse vai então descer rapidamente e subir de novo, usando o novo dtMs de 1s por ciclo (porque mudaste o período para 1s antes).

### 3. Restaurar o período original quando a FSM voltar ao state 1

Finalmente, no case 1 da FSM, podes pôr a lógica de “reset”:

```

case 1:
    inst.I_Index = 0;
    inst.Situacion = 'P';
    inst.Direction = 1;

    // se a tosse estava em curso, restaurar parâmetros
    if (Tosse_em_curso) {
        Resp_move.period = Resp_period_original;
        Resp_move.updateDtMs();    // recalcula dtMs com período original

        Tosse_em_curso = false;
    }

    Serial.println("-----intermed state");

    if (m.n_Points <= 1 || m.period <= 0.0f) {inst.state = 1;}
    else if(inst.Direction == 1 && start==1 && inst.Individual_start_comand == 1){inst
    else if(start==0 || inst.Individual_start_comand == 0){inst.state = 0;}
    break;

```

Assim:

- A tosse é preparada fora da FSM (mudança de período e updateDtMs).
- A FSM, ao ver `Tosse_em_curso` em case 4, entra no ramo 14 para fazer o “espasmo” rápido.
- Quando a FSM regressa ao state 1 depois do ciclo de tosse, restaura automaticamente o período original e o dtMs.
- O ciclo respiratório volta ao comportamento normal sem precisarem de mais comandos seriais.

Para o modo “x em x tempo” vs “aleatório”, a diferença estará na função `Preparar_Tosse`:

- No modo preciso, chamas `Preparar_Tosse` com um timer fixo (por exemplo, a cada N segundos).
- No modo aleatório, decides se chamas `Preparar_Tosse` com base numa distribuição aleatória ou num limiar.

Se quiseres, o próximo passo pode ser: escreveres tu uma versão simplificada dessa função Preparar\_Tosse em código e dizeres onde preferes guardar Resp\_period\_original (global ou dentro da MotionInstance), e eu ajusto contigo para ficar bem integrado.

✱✱

## // novos campos globais/na MotionInstance

```
float Resp_period_original = 0.0f;  
bool Tosse_em_curso = false;
```

```
// função chamada a partir da FSM_Serial_Command quando chega o comando de tosse  
void Preparar_Tosse(MotionStorage& respMove, MotionInstance& respInst) {  
// guardar período original  
Resp_period_original = respMove.period;
```

```
// mudar período temporariamente para 1s  
respMove.period = 1.0f;           // segundos  
respMove.updateDtMs();           // recalcula dtMs com novo período  
  
// marcar tosse em curso  
Tosse_em_curso = true;  
  
// aqui NÃO tocas ainda em inst.state; deixas a FSM continuar
```

```
}
```

não foi nada disto que eu pedi,

o que eu pedi foi uma função chamada preparação\_tosse, que vai servir como intermedio entre o comando para ativar a tosse e a execução da tosse e comando para a FSM, tenta alterar o minimo possivel a FSM do movimento.

por isso todas as coisas tenta fazer diretamente nesta função, estou falando na copia temporaria do periodo padrão, e alteração para o periodo tosse, uma verificação inicial com o start\_comando == 1. e a leitura a ver se o state já está a um, quando o comando da tosse for iniciado,

tenta ao maximo deixar estas coisas na função e tenta não alterar a FSM do movimento e as structures do storage e instance, o maximo possivel, faz também com que não comprometa o tempo de execução dos ciclos

Percebeu: queres uma função “entre” o comando de tosse e a FSM, que mexa nos períodos de forma temporária, quase sem tocar na FSM nem nas structures.

Vamos desenhar essa função de preparação como tu pediste, mantendo a FSM intacta:

## Ideia geral da Preparacao\_Tosse

A função vai:

- Só agir se o programa estiver a correr (`start_comand == 1`).
- Ler o state atual da FSM de respiração (`Resp_inst.state`).
- Se a tosse ainda não começou:
  - guardar o período padrão (`Resp_move.period`) numa variável temporária local/estática.
  - pôr o período de tosse (1 s) em `Resp_move.period` e chamar `updateDtMs()`.
  - levantar o sinal de tosse (`Tosse_signal = 1`), que a tua FSM já sabe usar no case 4.
- Se a tosse já foi executada e a FSM voltou ao state 1:
  - restaurar o período antigo em `Resp_move.period`.
  - chamar `updateDtMs()` de novo.
  - limpar o sinal de tosse.

Tudo isto acontece **fora** da FSM de movimento. A FSM continua como está: ela vê `Tosse_signal == 1 && bidirectional == true` no case 4 e entra no state 14.

## Código da função (tocando o mínimo possível na FSM)

Pressupondo que já tens:

- `MotionStorage Resp_move;`
- `MotionInstance Resp_inst;`
- `volatile boolean start_comand;`
- `int Tosse_signal;` (ou `bool`), usado na FSM em case 4 e case 14.

Podemos fazer:

```
// variáveis temporárias para guardar/restaurar período
static float Resp_period_padrao = 0.0f;
static bool  Tosse_preparada = false;

// chamada sempre no loop, depois da FSM_Serial_Command, antes da FSM_Motion_Update
void Preparacao_Tosse() {
    // 1) só age se o programa estiver ativo
    if (start_comand == 0) {
        // se o programa parar, também limpamos qualquer preparação
        Tosse_preparada = false;
        Tosse_signal     = 0;
        return;
    }

    // 2) leitura do estado atual da FSM de respiração
    short int estadoResp = Resp_inst.state;

    // 3) se houve comando externo para iniciar tosse (por exemplo, vindo da FSM_Ser:
    //     e ainda não preparámos nada, ajustamos período e levantamos o sinal.
```

```

//    Aqui estou a assumir que a FSM de comandos te põe um flag externo,
//    por exemplo: 'Comando_tosse_ativo == 1'.
extern bool Comando_tosse_ativo; // ou outra variável que já tenhas

if (Comando_tosse_ativo && !Tosse_preparada) {
    // garantir que o movimento respiração está em curso
    if (!Resp_inst.active || Resp_inst.def == nullptr) {
        return; // não faz nada se o movimento não estiver configurado
    }

    // guardar período padrão
    Resp_period_padrao = Resp_move.period;

    // alterar para período de tosse (1s) e atualizar dtMs
    Resp_move.period = 1.0f; // segundos
    Resp_move.updateDtMs(); // recalcula dtMs com novo período

    // levantar sinal de tosse para a FSM de movimento
    Tosse_signal = 1;
    Tosse_preparada = true;

    // a FSM continua normalmente; quando chegar ao case 4 com Tosse_signal==1,
    // ela própria vai cair no state 14 (como já tens).
    return;
}

// 4) se a tosse já foi executada e a FSM voltou ao state 1, restaurar período pa
if (Tosse_preparada && estadoResp == 1) {
    // restaurar período original
    Resp_move.period = Resp_period_padrao;
    Resp_move.updateDtMs(); // volta a dtMs padrão

    // limpar sinais
    Tosse_signal = 0;
    Tosse_preparada = false;
    Comando_tosse_ativo = false; // reset do pedido externo

    // daqui para a frente, respiração volta ao ciclo normal
}
}

```

#### Notas importantes:

- Esta função **não mexe** em `Resp_inst.state`, `iIndex`, `Direction`, etc.; só observa `Resp_inst.state`.
- Não altera a FSM de movimento; apenas usa a API que tu já forneceste:
  - `Resp_move.period`
  - `Resp_move.updateDtMs()`
  - `Tosse_signal`, que a FSM já usa em case 4/case 14.
- O impacto no tempo de execução é mínimo: são só uns `if` e, ocasionalmente, duas chamadas a `updateDtMs()`, que já existem.

## Onde chamar Preparacao\_Tosse no loop

No teu loop() principal, a ordem típica ficaria:

```
void loop() {
    unsigned long nowMs = millis();

    FSM_Serial_reader(nowMs);
    FSM_Serial_Command(nowMs);

    Preparacao_Tosse();           // aqui, entre comandos e movimento

    FSM_Motion_Update(start_comand, Resp_inst, nowMs);
    FSM_Motion_Update(start_comand, Bat_inst, nowMs);
    // etc...
}
```

Assim, qualquer comando de tosse que venha pela FSM de serial é traduzido em preparação (alterar período, levantar Tosse\_signal) antes de o movimento ser atualizado.

Para te certificar que entendeste bem o papel desta função: se tivesses de explicar em uma frase quando exatamente o período volta ao valor padrão, como dirias isso em termos do state da FSM de respiração?

## acho que me enganei em alguma coisa a modificar na função:

// chamada sempre no loop, depois da FSM\_Serial\_Command, antes da FSM\_Motion\_Update

```
void Preparacao_Tosse() {
    static float Resp_period_padrao = 0.0f;
    // 1) só age se o programa estiver ativo
    if (start_comand == 0) {
        // se o programa parar, também limpamos qualquer preparação
        Tosse_preparar = false;
        Tosse_signal = 0;
        return;
    }

    // 2) leitura do estado atual da FSM de respiração
    short int estadoResp = Resp_inst.state;

    if (Tosse_inst.active == true && Tosse_preparar == 0) {
        // garantir que o movimento respiração está em curso
        if (!Resp_inst.active || Resp_inst.def == nullptr) {
            return; // não faz nada se o movimento não estiver configurado
        }

        // guardar período padrão
        Resp_period_padrao = Resp_move.period;
    }
}
```

```

// alterar para período de tosse (1s) e atualizar dtMs
    Resp_move.period = 1.0f;    // segundos
    Resp_move.updateDtMs();    // recalcula dtMs com novo período

// levantar sinal de tosse para a FSM de movimento
    Tosse_signal = 1;
    Tosse_preparar = true;

// a FSM continua normalmente; quando chegar ao case 4 com Tosse_signal==1,
// ela própria vai cair no state 14 (como já tens).
    return;
}

// 4) se a tosse já foi executada e a FSM voltou ao state 1, restaurar período padrão
if (Tosse_preparar && estadoResp == 1) {
    // restaurar período original
    Resp_move.period = Resp_period_padrao;
    Resp_move.updateDtMs(); // volta a dtMs padrão

// limpar sinais
    Tosse_signal = 0;
    Tosse_preparar = false;
    // Comando_tosse_ativo = false; // reset do pedido externo

// daqui para a frente, respiração volta ao ciclo normal
}
}

```

agora vou te mandar o código todo:

e faltou dizeres-me qual a variável que tenho que alterar e qual a variável que tenho que adicionar no FSM de interpretação serial onde antes estava:

case 9:

```

    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Tosse_signal=1;
    //Serial.println("Sem nada"); //Retirar quando estiver implementado
    pixel.setPixelColor(0, pixel.Color(255, 0, 255));
    if(1){state = 4;}
    break;

```

Dá para aproveitar quase tudo o que já tens; só precisas de alinhar dois pontos:

1. Quem ativa a preparação da tosse (antes era `Tosse_signal = 1` direto na FSM de comandos).
2. A lógica dentro da `Preparacao_Tosse` (há um detalhe no uso de `Tosse_inst.active` e `Tosse_preparar`).

Vou separar em duas partes.

## 1. Alteração mínima na FSM de interpretação serial

Antes tinhas:

```
case 9:
    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Tosse_signal = 1;
    pixel.setPixelColor(0, pixel.Color(255, 0, 255));
    if(1){state = 4;}
    break;
```

Como agora queres que a FSM de comandos **não mexa diretamente em** `Tosse_signal`, mas apenas sinalize que a tosse deve ser preparada, o mínimo é:

- Criar uma variável global simples, por exemplo:

```
bool Tosse_preparar = false;    // já estás a usar no Preparacao_Tosse
```

- E no case 9 substituir:

```
case 9:
    Serial.println("Comando L3");
    LineReady_Out = true;
    CH_Command_Out = '\0';

    Tosse_preparar = false;           // limpamos qualquer estado antigo
    Tosse_inst.active = true;         // dizemos que o modo tosse está ativo
    pixel.setPixelColor(0, pixel.Color(255, 0, 255));

    if(1){state = 4;}
    break;
```

Não precisas de tocar em `Tosse_signal` aqui; quem o vai levantar é a `Preparacao_Tosse()`.

Os outros cases que definem `Tosse_inst.active` (16–20) já estão coerentes com isto:

- em Mode Human, Resp, Bat, Static, tens `Tosse_inst.active = false`, o que impede a tosse. <sup>[246]</sup>

## 2. Ajustar a `Preparacao_Tosse` (tens quase tudo certo)

A tua versão atual:

```
void Preparacao_Tosse() {
    static float Resp_period_padrao = 0.0f;
    if (start_comand == 0) {
        Tosse_preparar = false;
```



```

        Tosse_signal    = 0;
        return;
    }

    short int estadoResp = Resp_inst.state;

    if (Tosse_inst.active == true && Tosse_preparar == 0) {
        if (!Resp_inst.active || Resp_inst.def == nullptr) {
            return;
        }

        Resp_period_padrao = Resp_move.period;

        Resp_move.period = 1.0f;
        Resp_move.updateDts();

        Tosse_signal    = 1;
        Tosse_preparar   = true;
        return;
    }

    if (Tosse_preparar && estadoResp == 1) {
        Resp_move.period = Resp_period_padrao;
        Resp_move.updateDts();

        Tosse_signal    = 0;
        Tosse_preparar   = false;
    }
}

```

Isto está bem alinhado com o que pediste:

- Só age se `start_comand == 1`.
- Só prepara tosse se `Tosse_inst.active == true` e ainda não foi preparada (`Tosse_preparar == 0`).
- Guarda o período padrão, altera para 1s, chama `updateDts()`, levanta `Tosse_signal`.
- Quando a FSM da respiração volta ao state 1, restaura período e limpa sinais.

Dois detalhes para garantir que tudo se comporta como queres:

#### 1. Certificar que `Tosse_preparar` é um bool global inicializado a false

No topo do ficheiro:

```

bool Tosse_preparar = false;
int  Tosse_signal    = 0;    // ou bool, se preferires

```

Assim, o teste `Tosse_preparar == 0` funciona como esperado.

#### 2. Tratar o caso de “modo tosse desligado”

Se saíres de um modo onde `Tosse_inst.active == true` para um modo onde

`Tosse_inst.active == false` (por exemplo, mudas de Mode Human para Mode Resp), é boa ideia garantir que não ficas com coisas penduradas:

No início da função, logo depois de `start_comand`:

```

if (!Tosse_inst.active) {
    // tosse não deve estar ativa neste modo
    Tosse_preparar = false;
    Tosse_signal    = 0;
    return;
}

```

Isso evita preparar tosse se o modo atual não a suporta.

A FSM de movimento continua praticamente intocada:

- Ela já usa `Tosse_signal == 1 && m.bidirectional == true` para saltar para o state 14.<sup>[247]</sup>
- E no state 1 já tens a lógica que funciona como “reset de ciclo”, onde a tua função de preparação agora detecta `estadoResp == 1` para restaurar o período.

Portanto, recapitulando o fluxo do que tens agora:

1. FSM de comandos (state 9) ativa `Tosse_inst.active = true`.<sup>[246]</sup>
2. `Preparacao_Tosse()` vê que:
  - `start_comand == 1`
  - `Tosse_inst.active == true`
  - `Tosse_preparar == false`
 → guarda período, poe 1s, chama `updateDtMs()`, levanta `Tosse_signal`.
3. FSM de movimento vê `Tosse_signal == 1` em case 4 e entra em state 14 (spasmo de tosse).<sup>[247]</sup>
4. Quando a FSM regressa ao state 1, `Preparacao_Tosse()` restaura o período original e limpa os flags.

Para verificar que captaste bem o papel de cada módulo: como resumirias, em uma frase, o que a FSM de comandos faz em relação à tosse e o que a `Preparacao_Tosse()` faz em seguida?

✱✱

## ent como que cacete ele faz isso, se tu me acabas-te de dizer que no FSM de comando eu devo colocar false?!?!?

eu devo mandar sinal no monitor serie com l3, o l3 manda sinal para a função de `preparação_tosse()`

`signal=true` -> copia o valor padrão de periodo para uma copia, altera o valor do periodo para 1s, e ativa o `getDTms()` (isto uma unica vez, visto que a função vai estar no loop. só pode executar esta sequencia 1 vez por signal), quando `state resp == 1`, volta ao valor padrão do periodo, ativa `getDtms`, e fica aguardado novamente pelo signal do monitor serie (tb só pode executar uma vez por ativação de tosse e `state resp == 1`)

Tens razão, a forma como expliquei antes ficou confusa em relação a quem põe true e quem põe false. Vamos alinhar com o que tu acabaste de dizer, passo a passo, sem tocar quase na FSM.

## 1. Qual é o fluxo que tu queres

Pelas tuas palavras:

- No monitor série, mandas L3.
- O L3 deve mandar **um sinal** para a função `Preparacao_Tosse()`.
- `Preparacao_Tosse()` faz, uma única vez por sinal:
  - `signal == true` → copia o período padrão, mete `period = 1s`, chama `updateDts()`, levanta `Tosse_signal`.
  - Depois espera a FSM da respiração chegar a `state == 1`.
  - Quando `stateResp == 1`, restaura o período padrão, chama `updateDts()` outra vez, baixa `Tosse_signal` e volta a ficar à espera de novo sinal.

Ou seja:

- FSM de comandos só dispara o sinal.
- `Preparacao_Tosse` faz todo o trabalho, e só uma vez por ciclo de tosse.

Vamos traduzir isso diretamente.

## 2. Variáveis que precisas

No topo do ficheiro (globais):

```
bool Tosse_sinal = false;          // vem da FSM de comandos
bool Tosse_em_preparacao = false; // controlo interno da Preparacao_Tosse
float Resp_period_padrao = 0.0f;
```

A tua FSM de movimento continua a usar `Tosse_signal` (podes manter esse nome) ou mudá-lo para `Tosse_sinal`, desde que seja consistente.

## 3. FSM de comandos: L3 apenas levanta o sinal

No `FSM_Serial_Command`, no comando L3 (onde antes punhas `Tosse_signal = 1`), fica assim:

```
case 9:
  Serial.println("Comando L3");
  LineReady_Out = true;
  CH_Command_Out = '\0';

  Tosse_sinal = true;          // DISPARA uma tosse
  pixel.setPixelColor(0, pixel.Color(255, 0, 255));
```

```
if(1){state = 4;}
break;
```

Nada de false aqui; L3 é só “ligar o sinal”.

#### 4. Função Preparacao\_Tosse: faz tudo, uma vez por sinal

Agora sim, escreve a função quase como descreveste:

```
void Preparacao_Tosse() {
    // só trabalha se o programa estiver ativo
    if (start_comand == 0) {
        Tosse_sinal = false;
        Tosse_em_preparacao = false;
        return;
    }

    short int estadoResp = Resp_inst.state;

    // 1) Se chegou um novo sinal de tosse e ainda não preparámos nada
    if (Tosse_sinal && !Tosse_em_preparacao) {
        // garantir que a respiração está configurada
        if (!Resp_inst.active || Resp_inst.def == nullptr) {
            // não faz nada se o movimento não estiver ativo
            Tosse_sinal = false; // descarta este sinal
            return;
        }

        // copiar período padrão uma vez
        Resp_period_padrao = Resp_move.period;

        // alterar período para tosse (1s) e atualizar dtMs
        Resp_move.period = 1.0f; // segundos
        Resp_move.updateDtMs(); // recalcula dtMs com novo período

        // levantar sinal para a FSM de movimento (ela usa isto no case 4/14)
        Tosse_sinal = 1;

        // marcar que esta tosse já foi preparada
        Tosse_em_preparacao = true;

        // NÃO mexe na FSM; ela própria vai reagir ao Tosse_sinal
        return;
    }

    // 2) Se já está em preparação e a FSM da respiração voltou ao state 1,
    // então restauramos o período padrão e limpamos tudo
    if (Tosse_em_preparacao && estadoResp == 1) {
        // voltar ao período original
        Resp_move.period = Resp_period_padrao;
        Resp_move.updateDtMs(); // volta a dtMs padrão

        // limpar flags
        Tosse_sinal = 0;
    }
}
```

```

        Tosse_em_preparacao = false;
        Tosse_sinal          = false;    // este ciclo de tosse acabou

        // pronto para receber novo L3
    }
}

```

Repara:

- `Tosse_sinal` vem da FSM de comandos (L3).
- `Tosse_em_preparacao` impede que a mesma tosse seja preparada várias vezes enquanto a FSM ainda está a executar.
- A FSM de movimento continua intacta: só vê `Tosse_signal == 1` e entra em state 14 quando achar que deve (como já tens).
- A restauração do período acontece **exclusivamente** quando `estadoResp == 1`, como pediste.

No `loop()`:

```

void loop() {
    unsigned long nowMs = millis();

    FSM_Serial_reader(nowMs);
    FSM_Serial_Command(nowMs);

    Preparacao_Tosse();                // intermedio tosse

    FSM_Motion_Update(start_comand, Resp_inst, nowMs);
    FSM_Motion_Update(start_comand, Bat_inst, nowMs);
}

```

Se tudo isso corresponder ao teu esquema, o comportamento esperado é:

- Envias L3 → levanta `Tosse_sinal`.
- No próximo loop, `Preparacao_Tosse` faz a cópia de período, muda para 1s, dá `Tosse_signal = 1`.
- A FSM de respiração vê `Tosse_signal` no case 4 e entra no ramo 14 de tosse.
- Quando a FSM de respiração terminar esse ciclo e voltar ao state 1, `Preparacao_Tosse` repõe o período original e limpa os sinais.

Para fecharmos: como descreverias, em termos da tua FSM de movimento, em que momento exato queres que a tosse “comece” (logo ao detetar o `Tosse_signal`, num ponto específico do ciclo, etc.)?

## assim?

Sim, essa versão da `Preparacao_Tosse()` está alinhada com o comportamento que descreveste e integra bem com a FSM de movimento e a FSM de comandos. <sup>[248]</sup>

Recapitulando o que ela faz agora:

- Só trabalha se `start_comand == 1`.
- Usa `senal_tossir` como input vindo da FSM de comandos (por exemplo, no L3).
- Prepara a tosse uma única vez por sinal:
  - copia `Resp_move.period` para `Resp_period_padrao`,
  - muda `Resp_move.period` para 1.0,
  - chama `Resp_move.updateDtMs()`,
  - levanta `Tosse_signal = 1`, que a FSM de movimento já usa no case 4/14. <sup>[248]</sup>
- Depois, espera a FSM da respiração voltar ao `state == 1` para:
  - restaurar o período original,
  - chamar `updateDtMs()` novamente,
  - limpar `Tosse_signal`, `Tosse_em_preparacao` e `senal_tossir`. <sup>[248]</sup>

Isso bate exatamente com: “1 vez por sinal L3, período passa a 1s; quando a FSM volta ao estado 1, regressa ao período padrão”.

Só garante estas duas coisas no resto do código:

- `senal_tossir` e `Tosse_em_preparacao` estão declarados como `bool` globais (e inicializados a `false`).
- Na FSM de comandos, no lugar onde antes tinhas `Tosse_signal = 1` para L3, agora colocaste `senal_tossir = true` (ou algo equivalente).

Para confirmar a tua compreensão: se tu explicasses o que acontece, passo a passo, desde o momento em que envias o comando L3 no monitor até ao momento em que a respiração volta ao período normal, como descreverias esse fluxo com as variáveis `senal_tossir`, `Tosse_signal` e `Resp_inst.state`?

\*\*

## adiciona mais uma modificação á função da tosse,

que é para também alterares o tempo desta variavel `float T_pausedt2_Ini = 0.6f`; para tipo 0.3f e depois chamares `Resp_move.updateDt_PauseMs()`; no final fazes exatamente igual ao periodo, voltas ao valor padrão e chamas as funções para executar as alterações novamente

Boa, isso encaixa bem: durante a tosse queres período mais curto e também pausa inicial mais curta, depois tudo volta ao normal quando a FSM regressa ao estado 1.

Com o que tens agora, dá para acrescentar isto à `Preparacao_Tosse()` sem mexer na FSM.

Assumindo:

- `Resp_move.T_pause_Ini` contém o valor atual da pausa inicial (no teu exemplo, 0.6f).
- `Resp_move.updateDt_PauseMs()` já existe e atualiza `Dt_pauseMs_Ini` e `Dt_pauseMs_End` com base nos `T_pause_*`.<sup>[249]</sup>

Adiciona dois valores padrão globais:

```
float Resp_period_padrao      = 0.0f;
float Resp_T_pause_Ini_padrao = 0.0f;
bool  Tosse_em_preparacao     = false;
bool  sinal_tossir            = false;
int    Tosse_signal           = 0;
```

E ajusta a função assim:

```
void Preparacao_Tosse() {
    // só trabalha se o programa estiver ativo
    if (start_comand == 0) {
        sinal_tossir      = false;
        Tosse_em_preparacao = false;
        Tosse_signal      = 0;
        return;
    }

    short int estadoResp = Resp_inst.state;

    // 1) Se chegou um novo sinal de tosse e ainda não preparámos nada
    if (sinal_tossir && !Tosse_em_preparacao) {
        // garantir que a respiração está configurada
        if (!Resp_inst.active || Resp_inst.def == nullptr) {
            sinal_tossir = false;    // descarta este sinal
            return;
        }

        // copiar valores padrão uma vez
        Resp_period_padrao      = Resp_move.period;
        Resp_T_pause_Ini_padrao = Resp_move.T_pause_Ini;

        // alterar período para tosse (1s) e pausa inicial para 0.3s
        Resp_move.period      = 1.0f;    // segundos
        Resp_move.T_pause_Ini = 0.3f;    // segundos

        // atualizar dtMs e pausas com os novos valores
        Resp_move.updateDtMs();
        Resp_move.updateDt_PauseMs();

        // levantar sinal para a FSM de movimento (ela usa isto no case 4/14)
        Tosse_signal = 1;

        // marcar que esta tosse já foi preparada
        Tosse_em_preparacao = true;
    }
}
```

```

        // NÃO mexe na FSM; ela própria vai reagir ao Tosse_signal
        return;
    }

    // 2) Se já está em preparação e a FSM da respiração voltou ao state 1,
    // então restauramos os valores padrão e limpamos tudo
    if (Tosse_em_preparacao && estadoResp == 1) {
        // voltar aos valores originais
        Resp_move.period      = Resp_period_padrao;
        Resp_move.T_pause_Ini = Resp_T_pause_Ini_padrao;

        // atualizar dtMs e pausas com os valores padrão
        Resp_move.updateDtMs();
        Resp_move.updateDt_PauseMs();

        // limpar flags
        Tosse_signal          = 0;
        Tosse_em_preparacao   = false;
        sinal_tossir          = false;    // este ciclo de tosse acabou
    }
}

```

Assim:

- Durante a tosse, respiração usa `period = 1.0` e `T_pause_Ini = 0.3`.
- Quando a FSM volta ao state 1, recuperas `period` e `T_pause_Ini` originais e voltas a chamar `updateDtMs()` e `updateDt_PauseMs()` para aplicar esses valores.

Para verificar que te sentes confortável com isto: como resumirias, em termos de comportamento físico do movimento, o efeito combinado de reduzir o período e a `T_pause_Ini` durante a tosse?

✱✱

**void Intermed\_position(boolean start,  
MotionInstance& inst, unsigned long nowMs,  
Coordinate& xyz, boolean trig\_serial) { //Interpolação  
linear entre dois pontos a e b com t em [0,1] -> de  
momento não está a ser utilizada**

```
MotionStorage& m = *(inst.def);
```

```

if (!inst.active || m.n_Points <= 1 || m.period <= 0.0f || (start == 0 && inst.Executing == 0)) {
    xyz.X = 0.0f;
    xyz.Y = 0.0f;
    xyz.Z = 0.0f;
    return;
}

```



```

// calcula alpha com base em elapsed e dtMs
//inst.elapsed = nowMs - inst.segmentStartMs;

int i = inst.iIndex;
int j = inst.jIndex;
float elapsed;

// Se estiveres a ir para trás, inverte os extremos do segmento
if(inst.Direction == 1){
    elapsed = (float)inst.elapsed_1;
}
if (inst.Direction == -1) {
    elapsed = (float)inst.elapsed_1;
}

float alpha = elapsed / m.dtMs;
if (alpha < 0.0f) alpha = 0.0f;
if (alpha > 1.0f) alpha = 1.0f;

xyz.X = lerp(m.X[i], m.X[j], alpha);
xyz.Y = lerp(m.Y[i], m.Y[j], alpha);
xyz.Z = lerp(m.Z[i], m.Z[j], alpha);

return;
}

```

agora aqui na função implementa aquilo que falamos uma vez, que caso o index de iniciação fosse diferente de 0, ele iria adicionar na fusão as coordenadas do ponto desse index na zona junto do home, e que todos os outros pontos seriam subtraídos pelos valores de x,y e z do ponto inicial. os pontos não devem ser todos subtraídos de uma só vês na inicialização do programa e sim aqui na função de interpolação á medida que vai sendo atualizado o valor do lerp

diz-me se consegues perceber o que estou a pedir

Sim, dá para perceber: queres fazer a respiração “centrada” num ponto de referência que pode não ser o ponto 0. A ideia é:

- O ponto `start_Index` passa a ser o “zero local” da respiração.
- Na fusão, esse ponto é adicionado à posição base (home).
- Todos os outros pontos são usados **relativamente** a esse ponto (subtraindo  $X_{start}$ ,  $Y_{start}$ ,  $Z_{start}$  a cada valor interpolado), mas essa subtração é feita em tempo real, dentro da `Intermed_position`, não na inicialização.

Vamos implementar isso direto na função, mexendo o mínimo.

## Passo 1: obter o ponto de referência

Dentro da `Intermed_position`, recuperas o índice de início:

```
int startIdx = m.start_Index;
```

E os valores do ponto de referência:

```
float X0 = m.X[startIdx];  
float Y0 = m.Y[startIdx];  
float Z0 = m.Z[startIdx];
```

## Passo 2: fazer o lerp como já fazes

Mantém o que já tens:

```
int i = inst.iIndex;  
int j = inst.jIndex;  
float elapsed;  
  
if (inst.Direction == 1) {  
    elapsed = (float)inst.elapsed_1;  
}  
if (inst.Direction == -1) {  
    elapsed = (float)inst.elapsed_1;  
}  
  
float alpha = elapsed / m.dtMs;  
if (alpha < 0.0f) alpha = 0.0f;  
if (alpha > 1.0f) alpha = 1.0f;  
  
// posição bruta entre os pontos i e j  
float X_lerp = lerp(m.X[i], m.X[j], alpha);  
float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);  
float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);
```

## Passo 3: aplicar a transformação relativa ao ponto inicial

Agora vem o que pediste:

- Todos os pontos são **subtraídos** pelo ponto inicial, para ficarem “centrados” nesse ponto.
- Na fusão/home, vais usar mais à frente o `home_position` + esses deltas, mas aqui na função já devolves o delta relativo.

Então, em vez de atribuir direto:

```
xyz.X = lerp(m.X[i], m.X[j], alpha);  
xyz.Y = lerp(m.Y[i], m.Y[j], alpha);  
xyz.Z = lerp(m.Z[i], m.Z[j], alpha);
```

fazes:

```
// coordenadas relativas ao ponto inicial (start_Index)
xyz.X = X_lerp - X0;
xyz.Y = Y_lerp - Y0;
xyz.Z = Z_lerp - Z0;
```

Se quiseres já incluir a parte “junto do home”, podes deixar isso para a fusão (Fusao\_plus\_IK\_D1/D2), onde somas:

```
Coordinate respi_rel; // vindo da Intermed_position

float X_total = home_position.X + respi_rel.X + ...;
```

Mas a transformação “subtrair o ponto inicial” fica toda concentrada na Intermed\_position, como querias, e é aplicada em cada passo do lerp, sem alterar o array original nem a inicialização.

Portanto, a função completa fica, em essência:

```
void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs, Coord
    MotionStorage& m = *(inst.def);

    if (!inst.active || m.n_Points <= 1 || m.period <= 0.0f || (start == 0 && inst.E
        xyz.X = 0.0f;
        xyz.Y = 0.0f;
        xyz.Z = 0.0f;
        return;
    }

    int i = inst.iIndex;
    int j = inst.jIndex;
    float elapsed = (float)inst.elapsed_1;

    float alpha = elapsed / m.dtMs;
    if (alpha < 0.0f) alpha = 0.0f;
    if (alpha > 1.0f) alpha = 1.0f;

    // ponto inicial de referência
    int startIdx = m.start_Index;
    float X0 = m.X[startIdx];
    float Y0 = m.Y[startIdx];
    float Z0 = m.Z[startIdx];

    // posição bruta interpolada
    float X_lerp = lerp(m.X[i], m.X[j], alpha);
    float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);
    float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);

    // posição relativa ao ponto inicial
    xyz.X = X_lerp - X0;
    xyz.Y = Y_lerp - Y0;
```

```
xyz.Z = Z_lerp - Z0;

return;
}
```

Isto corresponde ao que pediste: o array original fica intacto, o `start_Index` define o “zero local”, e a correção é feita a cada atualização do lerp.

Para garantir que está claro para ti: como explicarias, em termos físicos do movimento, o que muda quando a respiração começa num `start_Index` diferente de 0 e todas as posições são calculadas relativas a esse ponto?

## o que achas assim?

Está bem implementado e bate com o que querias fazer. <sup>[250]</sup>

Pontos positivos do que tens agora na `Intermed_position`:

- Vais buscar `startIdx = m.start_Index` logo no início e calculas um `Index0` com as coordenadas do ponto inicial se `startIdx != 0`; se for 0, usas (0,0,0) como referência, o que mantém o comportamento antigo para movimentos que começam em 0. <sup>[250]</sup>
- Fazes o lerp normal entre `m.X[i]` e `m.X[j]` (e Y, Z) para ter `X_lerp`, `Y_lerp`, `Z_lerp`, e só depois subtraís `Index0`, transformando tudo em coordenadas relativas ao ponto inicial. <sup>[250]</sup>
- Não mexes no array original nem na FSM; a correção é feita em tempo real, em cada chamada da `Intermed_position`, como tinhas pedido.

Só um pequeno detalhe de estilo/performance que podes considerar mais tarde (não é obrigatório):

- `Index0` está a ser recalculado em todas as chamadas, mas na prática só muda quando muda `m.start_Index` ou o próprio movimento. Se quiseres otimizar, podias guardar esse offset pré-calculado numa variável do `MotionStorage` quando defines o movimento. Mas para já, como a função é simples e chamada com frequência, o custo extra é muito pequeno no ESP32.

Em termos de efeito físico: agora, se `start_Index` for 5, o ponto 5 passa a ser o “centro” da respiração, e o manipulador vai oscilar em torno desse ponto, independentemente de como a trajetória foi desenhada no array.

Para ter a certeza que o raciocínio ficou claro: se escolheres `start_Index = 5` numa trajetória onde o ponto 5 está deslocado do home, como imaginas que isso vai aparecer no movimento do braço em comparação com `start_Index = 0`?



**Index0 está a ser recalculado em todas as chamadas, mas na prática só muda quando muda m.start\_Index ou o próprio movimento. Se quiseses otimizar, podias guardar esse offset pré-calculado numa variável do MotionStorage quando defines o movimento. Mas para já, como a função é simples e chamada com frequência, o custo extra é muito pequeno no ESP32.**

faz isso e diz-me o que tenho que alterar

Dá para otimizar isso de forma limpa, sem mexer na FSM nem na lógica do lerp. A ideia é:

- Guardar o offset do ponto inicial (Index0) dentro do próprio MotionStorage.
- Calcular esse offset apenas quando configuras o movimento ou quando mudas o start\_Index.
- Na Intermed\_position, só lê esse offset, em vez de o recalcular a cada chamada.

Vou dividir em 3 alterações pequenas.

## 1. Adicionar o offset ao MotionStorage

No sítio onde tens o struct MotionStorage (onde já estão X, Y, Z, start\_Index, etc.), adiciona três campos:

```
struct MotionStorage {
    ...
    short int start_Index;
    ...
    // offset do ponto inicial (coordenadas do ponto start_Index)
    float X_startOffset;
    float Y_startOffset;
    float Z_startOffset;
};
```

[Confirma o nome exato da tua struct, mas é esta a ideia.]<sup>[251]</sup>

## 2. Calcular o offset quando defines o movimento ou mudas o start\_Index

Tens uma função onde carregas/defines o movimento (por ex. quando lê de série ou inicializas Resp\_move). Sempre que fixas m.start\_Index e o array de pontos já está preenchido, faz:

```
void AtualizarStartOffset(MotionStorage& m) {
    if (m.start_Index >= 0 && m.start_Index < m.n_Points) {
        m.X_startOffset = m.X[m.start_Index];
        m.Y_startOffset = m.Y[m.start_Index];
    }
}
```

```

        m.Z_startOffset = m.Z[m.start_Index];
    } else {
        // fallback seguro
        m.X_startOffset = 0.0f;
        m.Y_startOffset = 0.0f;
        m.Z_startOffset = 0.0f;
    }
}

```

Depois:

- Quando inicializas Resp\_move (logo após definires start\_Index e os pontos), chama:

```
AtualizarStartOffset(Resp_move);
```

- Se em algum comando série permites mudar start\_Index em runtime, depois de alterar o start\_Index chamas esta função de novo.

Assim, o offset fica pré-calculado e coerente com o movimento atual.

### 3. Usar o offset na Intermed\_position

Agora simplificas a Intermed\_position. A versão atual recalcula Index0 em cada chamada; vamos trocar por leitura do offset:

```

void Intermed_position(boolean start,
                       MotionInstance& inst,
                       unsigned long nowMs,
                       Coordinate& xyz,
                       boolean trig_serial) {

    MotionStorage& m = *(inst.def);

    if (!inst.active || m.n_Points <= 1 || m.period <= 0.0f || (start == 0 && inst.E)
        xyz.X = 0.0f;
        xyz.Y = 0.0f;
        xyz.Z = 0.0f;
        return;
    }

    // calcula alpha com base em elapsed e dtMs
    int i = inst.iIndex;
    int j = inst.jIndex;
    float elapsed = (float)inst.elapsed_1;

    float alpha = elapsed / m.dtMs;
    if (alpha < 0.0f) alpha = 0.0f;
    if (alpha > 1.0f) alpha = 1.0f;

    // posição bruta entre os pontos i e j
    float X_lerp = lerp(m.X[i], m.X[j], alpha);
    float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);
    float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);

```

```

    // coordenadas relativas ao ponto inicial (start_Index), usando offset pré-calculado
    xyz.X = X_lerp - m.X_startOffset;
    xyz.Y = Y_lerp - m.Y_startOffset;
    xyz.Z = Z_lerp - m.Z_startOffset;

    return;
}

```

Repara que:

- Já não usas mais `Index0` nem recalculas nada por chamada.
- Só precisas garantir que `AtualizarStartOffset(m)` foi chamado depois de definires `start_Index` e os arrays de pontos. <sup>[251]</sup>

Se quiseres consolidar: em que ponto do teu código (função ou zona de inicialização) te parece mais natural chamar `AtualizarStartOffset(Resp_move)` para garantir que o offset está sempre sincronizado com o `start_Index`?

\*\*

## não

faz uma função á parte que utiliza a structure coordenadas (como eu já tinha feito, e utiliza a variavel já preesistente de `start_Index` no `move`, para definir a posição inicial, não precisas de mais variaveis, e chama depois essas função no `setup`

Percebe-se, queres algo ainda mais simples:

- Não queres novos campos em `MotionStorage`.
- Queres uma função separada, que usa a struct `Coordinate` (como o teu `Index0`) e o `start_Index` já existente, para preencher essa posição inicial uma vez no `setup()`.
- Depois, na `Intermed_position`, usas só esse `Index0` pré-calculado, sem voltar a olhar para `m.X[start_Index]` em cada chamada.

### 1. Função para calcular a posição inicial

Assumindo que tens algo como:

```

struct Coordinate {
    float X;
    float Y;
    float Z;
};

Coordinate Index0;    // já usaste este nome

```

Cria esta função:

```

void Calcular_Index0(MotionStorage& m, Coordinate& idx0) {
    // usa o start_Index já existente no movimento
    int startIdx = m.start_Index;

    if (startIdx >= 0 && startIdx < m.n_Points) {
        idx0.X = m.X[startIdx];
        idx0.Y = m.Y[startIdx];
        idx0.Z = m.Z[startIdx];
    } else {
        // fallback seguro se start_Index estiver fora do intervalo
        idx0.X = 0.0f;
        idx0.Y = 0.0f;
        idx0.Z = 0.0f;
    }
}

```

Nenhuma variável nova na `MotionStorage`, tudo usa o que já tens. <sup>[252]</sup>

## 2. Chamar essa função no `setup()`

No teu `setup()`, depois de definires o movimento de respiração (`Resp_move`), com os pontos e o `start_Index` corretos, chamas:

```

void setup() {
    ...

    // depois de preencher Resp_move.X/Y/Z e Resp_move.start_Index
    Calcular_Index0(Resp_move, Index0);

    ...
}

```

Se, a meio da execução, mudares `start_Index` por comandos série, faz sentido voltares a chamar `Calcular_Index0(Resp_move, Index0)` logo a seguir a essa alteração.

## 3. Usar o `Index0` na `Intermed_position` (sem recalcular)

Agora simplificas a `Intermed_position` para apenas ler o `Index0`:

```

void Intermed_position(boolean start, MotionInstance& inst, unsigned long nowMs, Coordinate& xyz,
    MotionStorage& m = *(inst.def));

if (!inst.active || m.n_Points <= 1 || m.period <= 0.0f || (start == 0 && inst.E)) {
    xyz.X = 0.0f;
    xyz.Y = 0.0f;
    xyz.Z = 0.0f;
    return;
}

int i = inst.iIndex;
int j = inst.jIndex;

```



```

float elapsed = (float)inst.elapsed_1;

float alpha = elapsed / m.dtMs;
if (alpha < 0.0f) alpha = 0.0f;
if (alpha > 1.0f) alpha = 1.0f;

// posição bruta entre os pontos i e j
float X_lerp = lerp(m.X[i], m.X[j], alpha);
float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);
float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);

// coordenadas relativas ao ponto inicial (Index0 pré-calculado no setup)
xyz.X = X_lerp - Index0.X;
xyz.Y = Y_lerp - Index0.Y;
xyz.Z = Z_lerp - Index0.Z;

return;
}

```

Agora:

- O `start_Index` é usado uma vez no `setup` para preencher `Index0`.
- `Intermed_position` usa sempre o mesmo `Index0`, sem recalculá-lo por chamada. [\[252\]](#)

Se quiseres, podes testar mentalmente: se mudares o `start_Index` a meio do programa via comando série, onde te parece mais natural voltar a chamar `Calcular_Index0` para manter o `Index0` sincronizado?

✱

## codigo com as alterações que disses-te:

assim que adicionei as coisas, apareceram uns erros estranhos:

src/main.cpp:533:1: error: expected initializer before 'Servo'

Servo mg90s\_1; // cria objeto servo

~~~~~

src/main.cpp: In function 'void setup()':

src/main.cpp:1180:5: error: 'Calcular_Index0' was not declared in this scope

Calcular_Index0(Resp_move, Index0);

~~~~~

src/main.cpp: In function 'bool attach\_servos()':

src/main.cpp:1277:3: error: 'mg90s\_1' was not declared in this scope

mg90s\_1.setPeriodHertz(50); // Frequência PWM 50 Hz

~~~~~

src/main.cpp:1277:3: note: suggested alternative: 'mg90s_6'

mg90s_1.setPeriodHertz(50); // Frequência PWM 50 Hz

~~~~~

mg90s\_6

src/main.cpp: In function 'void move\_servos(const Manipulador\_Config&, const Manipulador\_Config&):'

```

src/main.cpp:1500:5: error: 'mg90s_1' was not declared in this scope
mg90s_1.writeMicroseconds(d1.servo1Pulse);
~~~~~
src/main.cpp:1500:5: note: suggested alternative: 'mg90s_6'
mg90s_1.writeMicroseconds(d1.servo1Pulse);
~~~~~
mg90s_6
src/main.cpp: In function 'void Servo_test()':
src/main.cpp:2071:3: error: 'mg90s_1' was not declared in this scope
mg90s_1.write(5);
~~~~~
src/main.cpp:2071:3: note: suggested alternative: 'mg90s_6'
mg90s_1.write(5);
~~~~~
mg90s_6
*** [.pio\build\freenove_esp32_s3_wroom\src\main.cpp.o] Error 1

```

Os erros que apareceram não são por causa da lógica da respiração em si, mas por detalhes de posição/ordem do código e de declarações globais. <sup>[253]</sup>

Vamos por partes.

## 1. Erro no Servo mg90s\_1; expected initializer

Esse erro acontece normalmente quando:

- Há algum código “solto” ou função mal fechada antes dessa linha, de forma que o compilador já não está mais no contexto global. <sup>[253]</sup>
- Ou declaraste Servo mg90s\_1; a meio de outra função, onde não é permitido declarar objetos globais.

No teu ficheiro, a função Calcular\_Index0 e o bloco de FSM antigo estão imediatamente antes da zona dos servos. <sup>[253]</sup>

Verifica:

- Todos os { e } da Intermed\_position e da Calcular\_Index0 estão a fechar corretamente.
- O bloco comentado com /\* ... \*/ não está a “comer” o } final de alguma função.

Uma forma segura é garantir que tens isto, nesta ordem:

```

// lerp
float lerp(float a, float b, float t) {
    return a + t * (b - a);
}

// Intermed_position
void Intermed_position(...) {
    ...
}

```

```
// Calcular_Index0
void Calcular_Index0(MotionStorage& m, Coordinate& idx0) {
    ...
}

// depois disto, já fora de qualquer função:
// Servo mg90s_1;
Servo mg90s_1;
Servo mg90s_2;
// etc...
```

Se por engano puseste `Calcular_Index0` **depois** de algum `return` fora de função ou dentro de outro bloco, o compilador interpreta mal e a seguir já não aceita `Servo mg90s_1`; como declaração global.

## 2. Erro: `Calcular_Index0` was not declared in this scope

Isso é só ordem de declaração:

- Estás a chamar `Calcular_Index0(Resp_move, Index0)`; no `setup()` antes de o compilador conhecer a assinatura da função. <sup>[253]</sup>

Há duas formas de resolver:

### 1. Mover a definição completa de `Calcular_Index0` para cima do `setup()`:

```
// protótipos e funções utilitárias
void Calcular_Index0(MotionStorage& m, Coordinate& idx0) {
    ...
}

// depois vem setup()
void setup() {
    ...
    Calcular_Index0(Resp_move, Index0);
    ...
}
```

### 2. Ou declarar só o **protótipo** antes do `setup()`, e deixar a definição onde está:

```
// protótipo no topo
void Calcular_Index0(MotionStorage& m, Coordinate& idx0);

// mais abaixo, setup()
void setup() {
    ...
    Calcular_Index0(Resp_move, Index0);
    ...
}

// ainda mais abaixo, definição
void Calcular_Index0(MotionStorage& m, Coordinate& idx0) {
```

```
...  
}
```

Qualquer uma destas opções resolve esse erro.

### 3. Erros com `mg90s_1` não declarado

Depois de corrigires o erro “expected initializer”, é provável que estes desapareçam, porque:

- Se a linha `Servo mg90s_1;` falhar, todas as utilizações de `mg90s_1` em `attach_servos`, `move_servos`, etc., passam a “símbolo não declarado”.<sup>[253]</sup>

Então:

1. Garante que a declaração global `Servo mg90s_1;` está fora de qualquer função, logo depois dos includes e structs.
2. Garante que não há nenhum erro de sintaxe entre o fim da tua nova função `Calcular_Index0` e essa declaração.

Resumindo as alterações concretas que deves fazer:

- Confirma que `Intermed_position` e `Calcular_Index0` estão bem fechadas com `}`.
- Move o **protótipo** ou a definição de `Calcular_Index0` para cima do `setup()`.
- Deixa `Servo mg90s_1;` e os outros `Servo` fora de qualquer função e depois de todas essas definições.

Se quiseres, podes copiar aqui só o bloco desde o final de `Intermed_position` até à declaração do primeiro `Servo`, e analisamos linha a linha onde está o desequilíbrio de chaves.

✱

## não percebi ao certo o que aconteceu, mas agora já não está a dar erros de compilação

podes verificar se está tudo em ordem com o que tínhamos definido?

Está coerente com o que tínhamos combinado, e a forma como integraste as funções está correta.<sup>[254]</sup>

### `Intermed_position`

- Continua a verificar as condições de saída (`inst.active`, `n_Points`, `period`, `start/Executing`) antes de fazer contas.<sup>[254]</sup>
- Usa `inst.iIndex`, `inst.jIndex` e `inst.elapsed_1` para calcular `alpha = elapsed / m.dtMs`, com `clamp` entre 0 e 1.<sup>[254]</sup>
- Faz o lerp entre `m.X[i]` e `m.X[j]` (e Y/Z) para obter a posição bruta.<sup>[254]</sup>

- Subtrai `Index0.X/Y/Z`, que agora é pré-calculado no setup, para devolver coordenadas relativas ao ponto inicial (`start_Index`).<sup>[254]</sup>

Isto cumpre exatamente: “usar o `start_Index` como centro, sem mexer no array original, e aplicar a subtração em tempo real na interpolação”.

## Calcular\_Index0

- Lê `m.start_Index` e verifica se está dentro de `[0, m.n_Points)`.<sup>[254]</sup>
- Se estiver, copia `m.X[startIdx]`, `m.Y[startIdx]`, `m.Z[startIdx]` para `idx0` (o teu `Index0`).<sup>[254]</sup>
- Se não estiver, usa `(0,0,0)` como fallback.<sup>[254]</sup>

Isto bate exatamente com a ideia de ter uma função à parte que calcula a posição inicial usando só `start_Index` e a struct `Coordinate`, sem novos campos no `MotionStorage`.

O único ponto a garantir fora deste excerto é:

- No `setup()`, chamaste `Calcular_Index0(Resp_move, Index0)`; depois de teres preenchido `Resp_move.X/Y/Z` e `Resp_move.start_Index`.
- Se em algum comando série alterares `Resp_move.start_Index`, seria bom voltar a chamar `Calcular_Index0(Resp_move, Index0)`; logo a seguir, para manter o offset sincronizado.

Do ponto de vista de lógica, está tudo alinhado com o que definimos. Para te ajudar a consolidar: se agora quiseses testar no robô a diferença entre `start_Index = 0` e `start_Index = 5`, que tipo de padrão de movimento esperas ver na ponta do manipulador em cada caso?

✱

## agora adiciona na preparação da tosse uma variavel de multiplicação que vai passar a 1.25

essa variavel serve para aumentar ligeiramente a amplitude no eixo dos z na respiração durante a tosse, podes guardar isso n storage instance

depois quando a FSM chegar ao state =1 o multiplicador volta a 1

A forma mais limpa de fazer isso é:

- Guardar um multiplicador de amplitude em Z na `MotionInstance` da respiração.
- A `Preparacao_Tosse()` passa esse multiplicador para 1.25 quando prepara a tosse.
- A `Intermed_position()` multiplica a componente Z pelo fator atual.
- Quando a FSM volta ao `state == 1` (fim da tosse), a `Preparacao_Tosse()` volta o multiplicador para 1.0.

Vou sugerir alterações mínimas.

## 1. Campo novo na MotionInstance

Na tua struct `MotionInstance`, adiciona um campo (a 1.0 por defeito quando inicializas):

```
struct MotionInstance {  
    ...  
    float Z_amp_mult;    // multiplicador de amplitude em Z  
};
```

Na inicialização do `Resp_inst` (onde limpas o resto), garante que:

```
Resp_inst.Z_amp_mult = 1.0f;
```

(Se já tens um sítio onde limpas a instance, mete aí também.)

## 2. Usar o multiplicador em Intermed\_position

Na função `Intermed_position` atual: [\[255\]](#)

```
// posição bruta entre os pontos i e j  
float X_lerp = lerp(m.X[i], m.X[j], alpha);  
float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);  
float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);  
  
// coordenadas relativas ao ponto inicial (Index0 pré-calculado no setup)  
xyz.X = X_lerp - Index0.X;  
xyz.Y = Y_lerp - Index0.Y;  
xyz.Z = Z_lerp - Index0.Z;
```

basta incorporares o multiplicador em Z:

```
// coordenadas relativas ao ponto inicial (Index0 pré-calculado no setup)  
xyz.X = X_lerp - Index0.X;  
xyz.Y = Y_lerp - Index0.Y;  
  
// aplica multiplicador de amplitude em Z  
float Z_rel = Z_lerp - Index0.Z;  
xyz.Z = Z_rel * inst.Z_amp_mult;
```

Assim, em funcionamento normal `inst.Z_amp_mult = 1.0` (sem alteração); na tosse, passa a 1.25.

### 3. Atualizar Preparacao\_Tosse para mexer no multiplicador

Na tua Preparacao\_Tosse() atual [file:985/987], já guardas período padrão e T\_pause\_Ini. Vamos acrescentar também o multiplicador:

Primeiro, guarda o valor padrão num global:

```
float Resp_period_padrao      = 0.0f;
float Resp_T_pause_Ini_padrao = 0.0f;
float Resp_Z_amp_mult_padrao  = 1.0f;
bool  Tosse_em_preparacao     = false;
bool  sinal_tossir            = false;
int    Tosse_signal           = 0;
```

Agora, dentro da função:

```
void Preparacao_Tosse() {
    if (start_comand == 0) {
        sinal_tossir      = false;
        Tosse_em_preparacao = false;
        Tosse_signal      = 0;
        Resp_inst.Z_amp_mult = 1.0f;    // garantir reset se programa parar
        return;
    }

    short int estadoResp = Resp_inst.state;

    // 1) novo sinal de tosse, ainda não preparado
    if (sinal_tossir && !Tosse_em_preparacao) {
        if (!Resp_inst.active || Resp_inst.def == nullptr) {
            sinal_tossir = false;
            return;
        }

        // copiar valores padrão
        Resp_period_padrao      = Resp_move.period;
        Resp_T_pause_Ini_padrao = Resp_move.T_pause_Ini;
        Resp_Z_amp_mult_padrao = Resp_inst.Z_amp_mult;    // geralmente 1.0

        // alterar para tosse
        Resp_move.period      = 1.0f;    // segundos
        Resp_move.T_pause_Ini = 0.3f;    // segundos
        Resp_inst.Z_amp_mult  = 1.25f;    // aumentar amplitude em Z

        Resp_move.updateDtMs();
        Resp_move.updateDt_PauseMs();

        Tosse_signal      = 1;
        Tosse_em_preparacao = true;

        return;
    }

    // 2) fim da tosse: FSM voltou ao state 1
```

```

    if (Tosse_em_preparacao && estadoResp == 1) {
        Resp_move.period      = Resp_period_padrao;
        Resp_move.T_pause_Ini = Resp_T_pause_Ini_padrao;
        Resp_inst.Z_amp_mult  = Resp_Z_amp_mult_padrao;    // volta a 1.0

        Resp_move.updateDtMs();
        Resp_move.updateDt_PauseMs();

        Tosse_signal          = 0;
        Tosse_em_preparacao = false;
        sinal_tossir           = false;
    }
}

```

Com isto:

- Quando a tosse é preparada:
  - período → 1 s
  - pausa inicial → 0.3 s
  - amplitude em Z → 1.25 vezes
- Quando a respiração regressa ao state == 1:
  - período e pausa voltam ao valor padrão
  - multiplicador de Z volta ao valor padrão (normalmente 1.0)
  - flags de tosse são limpos.

Se fores testar: como esperas que a “silhueta” da respiração mude quando `Z_amp_mult` passa de 1.0 para 1.25 durante a tosse, comparando com a respiração normal?

\*\*

## o multiplicador não parece estar a fazer muito efeito,

supostamente se eu multiplicar  $-6 \times 2$  deveria dar -12

e em termos de movimento isso deveria ser bastante grande

Há três pontos a verificar para perceber porque é que quase não notas efeito no Z:

1. Onde exatamente o multiplicador está a ser aplicado.
2. Se o valor de `Z_amp_mult` está mesmo a ir para 2.0 (ou 1.25) durante a tosse.
3. Se a trajetória de respiração em Z tem amplitude suficiente para se notar.



## 1. Confirmar a aplicação do multiplicador

Na versão atual da `Intermed_position`, a lógica devia estar assim (com multiplicador só no Z relativo):<sup>[256]</sup>

```
// posição bruta entre os pontos i e j
float X_lerp = lerp(m.X[i], m.X[j], alpha);
float Y_lerp = lerp(m.Y[i], m.Y[j], alpha);
float Z_lerp = lerp(m.Z[i], m.Z[j], alpha);

// coordenadas relativas ao ponto inicial (Index0 pré-calculado no setup)
xyz.X = X_lerp - Index0.X;
xyz.Y = Y_lerp - Index0.Y;

float Z_rel = Z_lerp - Index0.Z;
xyz.Z = Z_rel * inst.Z_amp_mult;
```

Se em vez disso estiveres a multiplicar o `Z_lerp` antes de subtrair o `Index0.Z`, o efeito pode ser diferente do que estás a imaginar. Garante que:

- O multiplicador é aplicado **depois** da subtração do ponto inicial, como acima.
- Não estás a multiplicar X ou Y por engano.

## 2. Ver se `Z_amp_mult` está mesmo a mudar

Dentro da tosse, confirma com um `Serial.println` rápido (só para debug) em `Intermed_position` ou na `Preparacao_Tosse`:

```
Serial.print("Z_amp_mult = ");
Serial.println(inst.Z_amp_mult);
```

Espera ver:

- Normal: `Z_amp_mult = 1.0`.
- Durante tosse: `Z_amp_mult = 2.0` (se puseste 2) ou 1.25.

Se nunca muda de 1.0, então:

- Ou a `Preparacao_Tosse()` não está a ser chamada.
- Ou está a ser chamada mas `senal_tossir` nunca fica true.
- Ou `Resp_inst` que estás a alterar não é o mesmo `inst` que passa em `Intermed_position` da respiração.

### 3. Amplitude real da respiração

Mesmo que tenhas um ponto a -6, convém lembrar:

- O Z que chegas à IK é algo do tipo:

```
float Z_total = 1.0f * respi.Z + 0.2f * batim.Z + 50.0f; // exemplo
```

Se o offset base é 50, passar de -6 para -12 em `respi.Z` significa:

- Antes:  $50 + (-6) = 44$
- Depois:  $50 + (-12) = 38$

Ou seja, a diferença no espaço real é 6 mm (ou unidades de Z), que pode não parecer “gigante”, dependendo da escala mecânica do teu braço.

Se quiseres testar de forma mais agressiva:

- Mete temporariamente `Z_amp_mult = 3.0f` na tosse e vê se notas claramente.
- Ou faz um log de `respi.Z` antes de entrar na IK para ver valores numéricos.

Para te orientar no próximo passo: preferes que olhemos juntos para a sequência completa onde `respi.Z` é calculado (a fusão + IK) para ver exatamente que valores entram e onde multiplicar Z teria mais impacto percebido?

✱✱

## este é o código atual,

eu quero agora implementar algo a mais na parte do acionamento da tosse, quero que deixes a parte com quando escrevo l3 ele manda um sinal para executar a tosse.

eu agora quero algo para além disso, quero um sistema que quando a structure da tosse está ativa fica constantemente a tentar executar a tosse, mas de maneira diferente, eu quero que seja algo aliatorio, não sei ainda como definir esse sistema de aliatorio eu quero que tenha é o seguinte, caso o sinal para executar seja enviado mas o index seja igual ou abaixo de 2 ele não executa e espera até que volte a subir o movimento e index seja superior a 3.

quero também que caso esteja a subir e o index seja igual ou superior a `n_points-3` ele espera até o movimento chegar a `n_points-1` e começar a descer (`ddirection = -1`)

quero que ele tenha essas regras e isto deve estar implementado na preparação para tosse.

a parte ddo aliatorio faz numa outra função separada da preparação para tosse().

vê o que é que é melhor para tornar a tosse aliatoria, secalhar até pode-se utilizar uma variável para calcular um x tempo aliatorio até á próxima tosse, (uma coisa que eu quero dizer é quando ocorre um acontecimento de tosse ele têm uma probabilidade um pouco maior de acontecer logo em seguida mais uma ou 4 tosses de seguida, tipo logo no segundo a seguir) achas que consegues-me ajudar com isso?

Dá perfeitamente para fazer isso mantendo a arquitetura que já tens:

- L3 continua a acionar uma tosse “manual” via `sinal_tossir`.

- Em paralelo, quando `Tosse_inst.active == true`, um sistema aleatório vai, de tempos a tempos, pôr `senal_tossir = true`, mas respeitando as regras de índice e direção que definiste.

Vou propor a lógica em blocos pequenos.

## 1. Regras de quando PODE tossir

As regras que queres impor para o momento da tosse:

- Se `Resp_inst.iIndex <= 2`, nunca inicia tosse (muito perto do início).
- Se estiver a subir (`Direction == 1`) e `iIndex >= m.n_Points - 3`, também não inicia; espera chegar a `iIndex == m.n_Points-1` e começar a descer (`Direction == -1`).
- Ou seja, a tosse só dispara quando:
  - `iIndex > 2`
  - e (subida com “folga” antes do topo, ou descida):

Em código:

```
bool PosicaoValidaParaTosse(const MotionInstance& inst, const MotionStorage& m) {
    int i = inst.iIndex;

    // demasiado perto do início
    if (i <= 2) return false;

    // a subir perto demais do topo
    if (inst.Direction == 1 && i >= m.n_Points - 3) return false;

    // todos os outros casos são válidos
    return true;
}
```

## 2. Sistema aleatório para decidir “quando”

Vamos fazer uma função separada, chamada no loop, que:

- Só atua se:
  - `start_comand == 1`
  - `Resp_inst.active == true`
  - `Tosse_inst.active == true` (modo em que tosse aleatória está ligada).
- Mantém um “tempo alvo” para a próxima tosse, em ms (`proximaTosseMs`).
- Quando o `nowMs` passa esse alvo, tenta disparar tosse:
  - se a posição for válida (`PosicaoValidaParaTosse`), ativa `senal_tossir = true`;
  - se não for válida, “empurra” o tempo alvo um pouco para a frente e volta a tentar noutro ciclo.

- Depois de uma tosse, escolhe um novo intervalo aleatório até à próxima, com probabilidade maior de haver tosses logo a seguir (1–4 tosses em sequência).

Variáveis globais (no topo):

```
bool  sinal_tossir          = false;
bool  Tosse_em_preparacao   = false;
float Resp_period_padrao    = 0.0f;
float Resp_T_pause_Ini_padrao = 0.0f;
float Resp_Z_amp_mult_padrao = 1.0f;

// para tosse aleatória
unsigned long proximaTosseMs = 0;
int  tossesRestantesNoCluster = 0;    // quantas tosses ainda faltam neste “cluster”
```

Função de aleatoriedade (simples, usando random() do Arduino):

```
void AtualizarTosseAleatoria(unsigned long nowMs) {
    // só funciona se programa e respiração estiverem ativos
    if (start_comand == 0) return;
    if (!Resp_inst.active || Resp_inst.def == nullptr) return;
    if (!Tosse_inst.active) return;    // só quando o modo de tosse estiver ligado

    MotionStorage& m = Resp_move;

    // se já há um sinal de tosse em curso ou em preparação, não marcamos outra
    if (sinal_tossir || Tosse_em_preparacao) return;

    // se ainda não há alvo definido, cria um
    if (proximaTosseMs == 0) {
        // intervalo inicial aleatório entre 3 e 8 segundos, por exemplo
        unsigned long intervalo = random(3000, 8000);
        proximaTosseMs = nowMs + intervalo;
        return;
    }

    // ainda não chegámos ao momento alvo
    if (nowMs < proximaTosseMs) return;

    // chegámos ao momento alvo -> ver se a posição atual permite tosse
    if (!PosicaoValidaParaTosse(Resp_inst, m)) {
        // posição má; empurra alvo um pouco para a frente (por ex. 200 ms)
        proximaTosseMs = nowMs + 200;
        return;
    }

    // posição boa -> disparar tosse
    sinal_tossir = true;

    // gerir clusters de 1-4 tosses
    if (tossesRestantesNoCluster <= 0) {
        // novo cluster: probabilidade de 1-4 tosses
        tossesRestantesNoCluster = random(1, 5); // 1,2,3 ou 4
    } else {
```

```

        tossesRestantesNoCluster--;
    }

    // definir próximo alvo:
    if (tossesRestantesNoCluster > 0) {
        // próxima tosse dentro de 0.5-1.5s (mais provável logo a seguir)
        unsigned long intervalo = random(500, 1500);
        proximaTosseMs = nowMs + intervalo;
    } else {
        // depois de acabar o cluster, maior intervalo até novo cluster (3-10s)
        unsigned long intervalo = random(3000, 10000);
        proximaTosseMs = nowMs + intervalo;
    }
}

```

Repara: a função só marca `senal_tossir = true` quando:

- chegou ao tempo alvo,
- posição é válida,
- não há tosse em preparação.

### 3. Preparacao\_Tosse integra tudo (manual + aleatória)

A tua `Preparacao_Tosse()` já trata:

- copiar período e pausas padrão,
- pôr período para 1s, `T_pause_Ini` para 0.3f, multiplicador Z,
- levantar `Tosse_signal`,
- restaurar tudo quando `Resp_inst.state == 1`.

Ela não precisa de saber se `senal_tossir` veio de L3 ou do sistema aleatório; basta continuar como está, usando `senal_tossir` como gatilho. <sup>[257]</sup>

A única coisa a acrescentar é chamar `AtualizarTosseAleatoria(nowMs)` no `loop()`, antes da `Preparacao_Tosse()`:

```

void loop() {
    unsigned long nowMs = millis();

    FSM_Serial_reader(nowMs);
    FSM_Serial_Command(nowMs);

    AtualizarTosseAleatoria(nowMs); // gera senal_tossir aleatório
    Preparacao_Tosse();             // converte senal_tossir em tosse real (período,

    FSM_Motion_Update(start_comand, Resp_inst, nowMs);
    FSM_Motion_Update(start_comand, Bat_inst, nowMs);
    ...
}

```

O comando L3 continua como está: em vez de alterar diretamente `Tosse_signal`, ele deve pôr `senal_tossir = true;` (disparo manual). A lógica da preparação trata ambos os casos da mesma maneira.

Resumindo o comportamento:

- `L3 → senal_tossir = true → Preparacao_Tosse` inicia tosse assim que as condições forem boas.
- Modo tosse ativo (`Tosse_inst.active == true`) → `AtualizarTosseAleatoria` vai, de tempos em tempos, marcar `senal_tossir = true`, mas só em posições válidas e com clusters de 1–4 tosses.

Para te fazer pensar no design: dado este esquema, preferes que uma tosse aleatória só possa começar enquanto a respiração está a subir, ou também te faz sentido permitir tosses quando está a descer, desde que respeitando as regras de índice?

✱

## verifica só se está tudo certo com o que tínhamos falado

Está consistente com o que combinámos e a integração parece correta. Vou passar ponto a ponto para ver se bate tudo com os requisitos. <sup>[258]</sup>

### Preparação da tosse

Na `Preparacao_Tosse()`:

- Se `start_comand == 0`, limpas `senal_tossir`, `Tosse_em_preparacao`, `Tosse_signal` e voltas `Resp_inst.Z_amp_mult` para 1.0. <sup>[258]</sup>
- Quando `senal_tossir && !Tosse_em_preparacao`:
  - Confirma que a respiração está ativa (`Resp_inst.active` e `Resp_inst.def`).
  - Guardas `Resp_period_padrao = Resp_move.period` e `Resp_T_pause_Ini_padrao = Resp_move.T_pause_Ini`.
  - Pões:
    - `Resp_move.period = 0.50f` (meio segundo)
    - `Resp_move.T_pause_Ini = 0.1f`
    - `Resp_inst.Z_amp_mult = 1.50f` (amplitude Z 50% maior)
  - Chamas `Resp_move.updateDtMs()` e `Resp_move.updateDt_PauseMs()`.
  - Levantas `Tosse_signal = 1` e marcas `Tosse_em_preparacao = true`. <sup>[258]</sup>
- Quando `Tosse_em_preparacao && estadoResp == 1`:
  - Restaura `Resp_move.period` e `Resp_move.T_pause_Ini` pelos valores padrão.
  - Volta `Resp_inst.Z_amp_mult` a 1.0.

- Chamas de novo `updateDtMs()` e `updateDt_PauseMs()`.
- Limpa `Tosse_signal`, `Tosse_em_preparacao` e `sinal_tossir`.<sup>[258]</sup>

Isto respeita exatamente a ideia: período e pausa encurtados + Z maior durante a tosse, e tudo volta ao normal quando a FSM regressa ao estado 1.

## Multiplicador Z

Na `Intermed_position()` (da versão anterior), já estavas a fazer `xyz.Z` como interpolação em Z relativa ao `Index0`. Confirmando que ajustaste para:

- calcular `Z_rel = Z_lerp - Index0.Z`
- aplicar `xyz.Z = Z_rel * Resp_inst.Z_amp_mult`

se está assim, o multiplicador atua apenas sobre a variação do movimento em Z, que é o que querias.

## Comportamento aleatório da tosse

Não está visível neste excerto, mas assumindo que:

- Tens uma função `AtualizarTosseAleatoria(nowMs)` que:
  - respeita as regras de índice (não dispara com `iIndex <= 2`, não dispara a subir perto do topo),
  - decide momentos aleatórios com `random()` e produz `sinal_tossir = true` quando chegar a hora e a posição é válida.
- Chamas essa função no `loop()` antes da `Preparacao_Tosse()`.

Então:

- Tosse manual (L3) continua a funcionar via `sinal_tossir = true`.
- Tosse aleatória usa o mesmo caminho (`sinal_tossir` → `Preparacao_Tosse` → `Tosse_signal` → FSM).

Do ponto de vista de lógica de movimento e tosse, está tudo alinhado com o que definiste até agora.

Para testar o teu entendimento: se quiseses distinguir na prática quando a tosse foi disparada manualmente (L3) e quando foi aleatória, que variável ou impressão de debug achas mais simples adicionar para ver isso no terminal?

✱✱

# no modo all

não me parece estar a fazer nenhuma tosse

No modo ALL, pelas tuas FSMs, a tosse aleatória só acontece se **todas** as condições forem verdade ao mesmo tempo:

1. `start_comand == 1` (ciclo a correr).
2. `Resp_inst.active == true` e `Resp_inst.def != nullptr`.
3. `Tosse_inst.active == true` (modo tosse ligado).
4. `AtualizarTosseAleatoria(nowMs)` está a ser chamada no `loop()`.
5. A posição é válida segundo as regras de índice/direção.

[259]

No teu código atual, no modo ALL (case 16 da FSM de comandos), tens:

```
case 16:
    Serial.println("Comando Mode ALL");
    ...
    Bat_inst.active = true;
    Resp_inst.active = true;
    //Tosse_inst.active = true;
```

Ou seja, a tosse está comentada aqui, fica false nesse modo. [259]

Se a função `AtualizarTosseAleatoria` faz `if (!Tosse_inst.active) return;`, ela sai logo e nunca marca `senal_tossir = true`.

Para o comportamento que queres (modo ALL = batimento + respiração + tosse aleatória ligada), precisas de:

```
case 16:
    Serial.println("Comando Mode ALL");
    LineReady_Out = true;
    CH_Command_Out = '\0';
    Bat_inst.active = true;
    Resp_inst.active = true;
    Tosse_inst.active = true;      // <--- descomentar / acrescentar

    if(1){state = 21;}
    break;
```

Depois confirma estes dois pontos no resto do código:

- No `loop()` a ordem está algo como:

```
unsigned long nowMs = millis();
FSM_Serial_reader(nowMs);
FSM_Serial_Command(nowMs);

AtualizarTosseAleatoria(nowMs);
```



```
Preparacao_Tosse();
```

```
FSM_Motion_Update(start_comand, Resp_inst, nowMs);  
FSM_Motion_Update(start_comand, Bat_inst, nowMs);
```

- Em AtualizarTosseAleatoria, a primeira coisa **não** é descartar se Resp\_inst.active ou Tosse\_inst.active estiverem falsos:

```
if (start_comand == 0) return;  
if (!Resp_inst.active || Resp_inst.def == nullptr) return;  
if (!Tosse_inst.active) return;
```

Com estas linhas, no modo ALL passam a estar ativas respiração + tosse, e a função aleatória começa mesmo a tentar gerar sinal\_tossir.

Para garantir que está a disparar, um truque simples é pôr um `Serial.println("Tosse aleatória trigger");` logo antes de `sinal_tossir = true;` na `AtualizarTosseAleatoria` e ver se aparece no terminal quando corres no modo ALL.

Se fores olhar para a função aleatória, consegues identificar qual é a primeira condição de saída (return) que pode estar a impedir as tosses no teu teste atual?

✱✱

## ok está a funcionar, tinha sido eu que me tinha esquecido de descomentar uma linha antes na FSM de serial command

agora temos que introduzir as vibrações.

vais criar umas variaveis parecidas a como colocamos nos movimentos da respiração e batimentos cardiacos e vais colocar para a structure da tosse estas posições

X: 2.000, 1.618, 0.618, -0.618, -1.618, -2.000, -1.618, -0.618, 0.618, 1.618

Y: 0.000, -1.176, -1.902, -1.902, -1.176, 0.000, 1.176, 1.902, 1.902, 1.176

Z: 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000

periodo = 0.5s

e as pausas a 0s o ponto inicial é a 0 e a tosse nesta parte poem como `bidirecional=false` (pois a structure da tosse na verdade é a structure da vibração da tosse, pois a tosse em si é feita ao acelerar a FSM da respiração)

vai tb adicionar a FSM para a tosse da mesma forma que temos para a respiração e para o batimento cardiaco a unica diferenca é que este movimento vai ser controlado pela `preparacao_tosse()` na fsm e structure instance existe uma variavel chamada:

`Tosse_inst.Individual_start_comand = 0;`

que é a unica das structures que está a 0, é só preciso colocares a 1 quando for dado o sinal para executar a tosse na FSM de respiração e colocares a 0 quando desativares a outra tb

adiciona tb o seguinte, caso start passe a 0 para reescrever novamente o

`Resp_move.period = Resp_period_padrao;`

`Resp_move.T_pause_Ini = Resp_T_pause_Ini_padrao;`

Resp\_inst.Z\_amp\_mult = 1.0f; // volta a 1.0  
pois eu reparei se o timing bater certo e eu desativar o modo all quando a tosse aconteceu no instante a respiração depois passa a ficar com aquele periodo para sempre e eu não quero isso

O que já tens está bem encaminhado; vamos por partes para encaixar estas novas coisas sem estragar o que funciona. Vou focar em três pontos: movimento da vibração (Tosse\_move), FSM da tosse, e ajustes na Preparacao\_Tosse. <sup>[260]</sup>

## 1. Definir o movimento da vibração (Tosse\_move)

Na configuração inicial (onde defines Resp\_move e Bat\_move), cria o Tosse\_move com os pontos que deste:

```
// Exemplo no setup(), depois de teres Tosse_move declarado
Tosse_move.n_Points    = 10;
Tosse_move.bidirectional = false;
Tosse_move.start_Index  = 0;
Tosse_move.period       = 0.5f;      // segundos

// Z todos a zero
Tosse_move.X[^91_0] = 2.000f; Tosse_move.Y[^91_0] = 0.000f; Tosse_move.Z[^91_0] =
Tosse_move.X[^91_1] = 1.618f; Tosse_move.Y[^91_1] = -1.176f; Tosse_move.Z[^91_1] =
Tosse_move.X[^91_2] = 0.618f; Tosse_move.Y[^91_2] = -1.902f; Tosse_move.Z[^91_2] =
Tosse_move.X[^91_3] = -0.618f; Tosse_move.Y[^91_3] = -1.902f; Tosse_move.Z[^91_3] =
Tosse_move.X[^91_4] = -1.618f; Tosse_move.Y[^91_4] = -1.176f; Tosse_move.Z[^91_4] =
Tosse_move.X[^91_5] = -2.000f; Tosse_move.Y[^91_5] = 0.000f; Tosse_move.Z[^91_5] =
Tosse_move.X[^91_6] = -1.618f; Tosse_move.Y[^91_6] = 1.176f; Tosse_move.Z[^91_6] =
Tosse_move.X[^91_7] = -0.618f; Tosse_move.Y[^91_7] = 1.902f; Tosse_move.Z[^91_7] =
Tosse_move.X[^91_8] = 0.618f; Tosse_move.Y[^91_8] = 1.902f; Tosse_move.Z[^91_8] =
Tosse_move.X[^91_9] = 1.618f; Tosse_move.Y[^91_9] = 1.176f; Tosse_move.Z[^91_9] =

// pausas 0
Tosse_move.T_pause_Ini = 0.0f;
Tosse_move.T_pause_End = 0.0f;

// atualiza dtMs e pausas
Tosse_move.updateDtMs();
Tosse_move.updateDt_PauseMs();
```

Como a vibração é rápida, faz sentido ficar com bidirectional = false e start\_Index = 0 como disseste. <sup>[260]</sup>

## 2. FSM da tosse (estrutura igual à Resp/Bat)

Já tens uma FSM genérica FSM\_Motion\_Update(start, inst, nowMs) que usas para Resp e Bat. A boa notícia: podes reaproveitar a **mesma** FSM para Tosse\_inst, porque o padrão é o mesmo (sequência de pontos, pausa, etc.). <sup>[261]</sup> <sup>[260]</sup>

O que precisas é:

- Garantir que `Tosse_inst.def = &Tosse_move`; na inicialização.
- Garantir que `Tosse_inst.Individual_start_comand` começa a 0 (como já tens) e é respeitado na FSM (já está: só arranca se `inst.Individual_start_comand == 1`).<sup>[261]</sup>

No `loop()`, acrescenta:

```
FSM_Motion_Update(start_comand, Resp_inst, nowMs);
FSM_Motion_Update(start_comand, Bat_inst, nowMs);
FSM_Motion_Update(start_comand, Tosse_inst, nowMs);
```

Como a FSM é a mesma, a vibração da tosse vai usar `Tosse_move` e os estados normais, e só corre quando puseres `Tosse_inst.Individual_start_comand = 1`.

### 3. Ligar/desligar a vibração a partir da Preparacao\_Tosse

Agora ligamos a vibração ao sistema de tosse, sem mexer muito na FSM de movimento.

Na `Preparacao_Tosse()`:

1. Quando a tosse começa (no bloco `if (sinal_tossir && !Tosse_em_preparacao)`), além de mexer no período da respiração, ativa a vibração:

```
// dentro do if (sinal_tossir && !Tosse_em_preparacao)
Tosse_inst.Individual_start_comand = 1; // arranca vibração
Tosse_inst.active = true;              // garantir que está ativa
```

2. Quando a tosse termina (no bloco `if (Tosse_em_preparacao && estadoResp == 1)`), desliga a vibração:

```
Tosse_inst.Individual_start_comand = 0;
Tosse_inst.active = false;
```

Assim, a FSM da tosse só corre durante a fase em que a tosse está “em curso”.

### 4. Reset seguro quando `start` passa a 0

Também pediste para, se `start_comand` passar a 0, restaurar imediatamente período, pausa e ganho de Z da respiração, para evitar ficar “preso” nos valores da tosse.

Já tens um bloco no início da `Preparacao_Tosse()`:

```
if (start_comand == 0) {
    sinal_tossir      = false;
    Tosse_em_preparacao = false;
    Tosse_signal      = 0;
    Resp_inst.Z_amp_mult = 1.0f;
```

```
    return;  
}
```

Aqui deves acrescentar a recuperação do período/pause, usando os valores padrão se a tosse estava em curso:

```
if (start_comand == 0) {  
    // se a tosse já tinha alterado parâmetros, repõe  
    if (Tosse_em_preparacao) {  
        Resp_move.period      = Resp_period_padrao;  
        Resp_move.T_pause_Ini = Resp_T_pause_Ini_padrao;  
        Resp_move.updateDtMs();  
        Resp_move.updateDt_PauseMs();  
    }  
  
    Resp_inst.Z_amp_mult = 1.0f;  
  
    sinal_tossir          = false;  
    Tosse_em_preparacao   = false;  
    Tosse_signal          = 0;  
  
    // e garante que a vibração pára também  
    Tosse_inst.Individual_start_comand = 0;  
    Tosse_inst.active = false;  
  
    return;  
}
```

Dessa forma, mesmo que desligues o modo ALL no “meio da tosse”, os parâmetros da respiração são repostos imediatamente.

Se quiseres solidificar: olhando para este esquema, em que momento exato achas melhor arrancar a vibração da tosse — logo quando `sinal_tossir` é detetado, ou só depois de a respiração entrar num determinado estado (por exemplo, ao começar a descer)?

✱✱

## para que é que fazes assim, é só veres os exemplos dos outros movimentos

custava muito fazer isto?:

```
#include <Arduino.h>  
#include <Adafruit_NeoPixel.h>  
#include <ESP32Servo.h>  
#include <math.h>  
  
// declarar no topo do ficheiro (globais)  
int last_delta2_s1 = 0;  
int last_delta2_s2 = 0;  
int last_delta2_s3 = 0;
```

```

struct Manipulador_Config {
    // geometria
    float L1;
    float L2;
    float L3;
    float servoOffsetX;
    float servoOffsetZ;
    float angleMin;
    float angleMax;
    float rotationOffsetZ; // offset global de rotação da base

    // calibração de PWM
    int servo1MinPulse;
    int servo1MaxPulse;

    int servo2MinPulse;
    int servo2MaxPulse;

    int servo3MinPulse;
    int servo3MaxPulse;

    int theta1MinPulse;
    int theta1MaxPulse;

    int theta2MinPulse;
    int theta2MaxPulse;

    int theta3MinPulse;
    int theta3MaxPulse;

    // estado atual (pulsos gerados pela IK)
    int servo1Pulse;
    int servo2Pulse;
    int servo3Pulse;
};

Manipulador_Config delta_1_Cfg = {
    // geometria
    .L1 = 35.0f,
    .L2 = 60.0f,
    .L3 = 15.0f,
    .servoOffsetX = 32.0f,
    .servoOffsetZ = 0.0f,
    .angleMin = 0.78539816339744830961566084581988f, //45 degrees
    .angleMax = 3.9269908169872415480783042290994f, //225 degrees
    .rotationOffsetZ = -15.6f * PI / 180.0f, //-75 degrees in radians

    // calibração de PWM
    .servo1MinPulse = 520,

```

```

        .servo1MaxPulse = 2520,

        .servo2MinPulse = 550,
        .servo2MaxPulse = 2550,

        .servo3MinPulse = 560,
        .servo3MaxPulse = 2560,

        // estado atual (pulsos gerados pela IK)
        .servo1Pulse = 0,
        .servo2Pulse = 0,
        .servo3Pulse = 0
    };

    Manipulador_Config delta_2_Cfg = {
        // geometria
        .L1 = 35.0f,
        .L2 = 60.0f,
        .L3 = 15.0f,
        .servoOffsetX = 32.0f,
        .servoOffsetZ = 0.0f,
        .angleMin = 0.78539816339744830961566084581988f, //45 degrees
        .angleMax = 3.9269908169872415480783042290994f, //225 degrees
        .rotationOffsetZ = -44.4f * PI / 180.0f, //-45 degrees in radians

        // calibração de PWM
        //Servo_1 -> theta3
        //Servo_2 -> theta1
        //Servo_3 -> theta2
        .servo1MinPulse = 460,
        .servo1MaxPulse = 2460,

        .servo2MinPulse = 470,
        .servo2MaxPulse = 2470,

        .servo3MinPulse = 420,
        .servo3MaxPulse = 2420,

        // estado atual (pulsos gerados pela IK)
        .servo1Pulse = 0,
        .servo2Pulse = 0,
        .servo3Pulse = 0
    };

    //Motion_test -> Arrays of points for testing the motion functions
    float X_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 5.0, 10.0, 15.0, 0.0, 0.0};
    float Y_test_calibration[] = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 5.0, 15.0};    // -> teste de
    movimento linear dos 3 eixos

```

```

float Z_test_calibration[] = {40.0, 40.0, 50.0, 60.0, 60.0, 60.0, 60.0, 60.0, 60.0};
const int N_test_calibration = 9;

//bat
float X_test[] = {15.607, 14.923, 13.995, 12.845, 11.503, 10.000, 6.665, 1.464, -2.965, -5.000,
-5.659, -6.056, -6.180, -6.029, -5.607, -4.923, -3.995, -2.845, -1.503, 0.000, 3.335, 8.536, 12.965,
15.000, 15.659, 16.056, 16.180, 16.029};
//float Y_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
float Y_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = {15.607, 16.029, 16.180, 16.056, 15.659, 15.000, 12.965, 8.536, 3.335, 0.000,
-1.503, -2.845, -3.995, -4.923, -5.607, -6.029, -6.180, -6.056, -5.659, -5.000, -2.965, 1.464, 6.665,
10.000, 11.503, 12.845, 13.995, 14.923};
//float Z_test[] = { 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0,
40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0, 40.0};
float Z_test[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
const int N_test = 28;
float T_period = 0.3f;
float T_pausedt = 0.6f;

//resp
//float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000, 10.667, 9.333,
8.000, 6.667, 5.333, 4.000, 2.667, 1.333 };
//float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0 };
//float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000, 15.802,
12.099, 8.889, 6.173, 3.951, 2.222, 0.988, 0.247};
float X_test2[] = {0.000, 1.333, 2.667, 4.000, 5.333, 6.667, 8.000, 9.333, 10.667, 12.000};
float Y_test2[] = { 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
float Z_test2[] = {0.000, 0.247, 0.988, 2.222, 3.951, 6.173, 8.889, 12.099, 15.802, 20.000};

//const int N_test2 = 18;
const int N_test2 = 10;
float T_period2 = 4.0f;
float T_pausedt2_Ini = 0.6f;
float T_pausedt2_End = 0.1f;

float X_tosse_vibracao[] = {2.000, 1.618, 0.618, -0.618, -1.618, -2.000, -1.618, -0.618, 0.618, 1.618};
float Y_tosse_vibracao[] = {0.000, -1.176, -1.902, -1.902, -1.176, 0.000, 1.176, 1.902, 1.902, 1.176};
float Z_tosse_vibracao[] = {0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000, 0.000};

```

```

//const int N_test2 = 18;
const int N_tosse_vibracao = 10;
float T_period_tosse_vibracao = 0.5f;
float T_pausedt_tosse_vibracao_Ini = 0.0f;
float T_pausedt_tosse_vibracao_End = 0.0f;

//Link lengths (mm)
#define L1 35
#define L2 60
#define L3 15

#define END_EFFECTOR_Z_OFFSET 0
#define SERVO_OFFSET_X 32
#define SERVO_OFFSET_Y 0
#define SERVO_OFFSET_Z (0.0 + END_EFFECTOR_Z_OFFSET)
// #define SERVO_OFFSET_Z_INVERTED -293

// #define SERVO_ANGLE_MIN (100.0f * PI / 180.0f)
// #define SERVO_ANGLE_MAX (190.0f * PI / 180.0f)

#define SERVO_ANGLE_MIN 0.78539816339744830961566084581988f //45 degrees
#define SERVO_ANGLE_MAX 3.9269908169872415480783042290994f //225 degrees

#define ROTATION_OFFSET_Z (-75.0f * PI / 180.0f) //0.26179938779914943f (15° em radianos =
π/12):

#define ROTATION_OFFSET_Z_Delta1 (-75.0f * PI / 180.0f)
#define ROTATION_OFFSET_Z_Delta2 (-45.0f * PI / 180.0f)

#define MIN 'm'
#define MAX 'M'

//Servo microsecond pulse limits (Calibration)-----Valores normais: Min = 500 ; Max = 2500]-----
-----

#define SERVO_1_MIN 620
#define SERVO_1_MAX 2520
#define SERVO_2_MIN 615
#define SERVO_2_MAX 2480
#define SERVO_3_MIN 620
#define SERVO_3_MAX 2550
//Servo_1 -> theta3 min 550, max 2450
//Servo_2 -> theta1 min 545, max 2410
//Servo_3 -> theta2 min 520, max 2450

// variáveis globais para valores suavizados
int servo_1_pulse_smooth = 0;
int servo_2_pulse_smooth = 0;
int servo_3_pulse_smooth = 0;

```



```

// fator de suavização [0 - 1]. tipo 0.2 para começar
const float SERVO_SMOOTH_ALPHA = 0.2f;

#define SERVO_4_MIN 550
#define SERVO_4_MAX 2400
#define SERVO_5_MIN 550
#define SERVO_5_MAX 2400
#define SERVO_6_MIN 550
#define SERVO_6_MAX 2400

// variáveis globais para valores suavizados
int servo_4_pulse_smooth = 0;
int servo_5_pulse_smooth = 0;
int servo_6_pulse_smooth = 0;
//-----

#define INVERTED -1
bool Tosse_signal = 0;
bool sinal_tossir = false;
bool Tosse_em_preparacao = false;
float Resp_period_padrao = 0.0f;
float Resp_T_pause_Ini_padrao = 0.0f;

// para tosse aleatória
unsigned long proximaTosseMs = 0;
int tossesRestantesNoCluster = 0; // quantas tosses ainda faltam neste "cluster"
// Máximo de pontos por movimento (ajusta mais tarde se precisares)
const int MAX_POINTS = 100;

// Armazenamento Bruto: dados completos de um movimento - Será utilizada para armazenar
os movimentos que o Pi enviar, ou movimentos pré-definidos, etc.
struct MotionStorage {
    int n_Points;           // nº de pontos do ciclo
    int N_points;           // nº de pontos do movimento completo
    float X[MAX_POINTS];    // coordenadas X dos pontos do movimento
    float Y[MAX_POINTS];    // coordenadas Y dos pontos do movimento
    float Z[MAX_POINTS];    // coordenadas Z dos pontos do movimento
    float period;
    float T_pause_Ini;
    float T_pause_End;
    float dtMs;             // duração do ciclo em segundos
    float Dt_pauseMs_Ini;   // duração da pausa entre ciclos em segundos
    float Dt_pauseMs_End;
    int start_Index; // índice de ponto que será o "0,0,0" lógico
    bool bidirectional; // true = vai-e-vem, false = ciclo fechado

float getDtMs(){
    if (n_Points > 1 && period > 0.0f) {

```

```

        if (bidirectional==true) {
            // 0..N-1..0 -> 2*(N-1) segmentos
            N_points = (n_Points * 2) - 2;
        } else {
            // ciclo normal 0..N-1..0, como já tinha
            N_points = n_Points-1;
        }
        return (period * 1000.0f) / (float)N_points;
    }
    return 0.0f;
}

void updateDtMs() {
    dtMs = getDtMs(); // aqui sim, mudas dtMs
}

float getD_PauseMs(float T_pause) const {
    if (n_Points > 1 && T_pause > 0.0f) {
        return (T_pause * 1000.0f); // só lê nPoints e T_pause
    }
    return 0.0f;
}

void updateDt_PauseMs() {
    Dt_pauseMs_Ini = getD_PauseMs(T_pause_Ini); // aqui sim, mudas dt_pauseMs
    Dt_pauseMs_End = getD_PauseMs(T_pause_End); // aqui sim, mudas dt_pauseMs
}

};

// Armazenamento temporário: runtime que executa um MotionStorage
struct MotionInstance {
    MotionStorage* def; // ponteiro para MotionStorage (a definição do Armazenamento Bruto
)
    int      currentIndex;// índice i (início do segmento i -> i+1) -> índice atual no movimento
    unsigned int  segmentStartMs; // timestamp do início do segmento atual
                // -> é uma variavel importante para controlar a velocidade do movimento,
ou seja, quando é que deve avançar para o próximo ponto do movimento
    bool      active; // se este movimento está ativo
    bool      Individual_start_comand;
    bool      Executing; // se este movimento está em pausa (entre ciclos)
    short int  state;
    float Dt_pause;
    unsigned long elapsed_1;
    unsigned long elapsed_2;
    unsigned long elapsed_3;

```

```

int iIndex; // novo: índice i para a interpolação
int jIndex; // novo: índice j para a interpolação
int I_Index;
int End_Point;
int Direction; // novo: direção do movimento (1 ou -1)
char Situacion;
float Z_amp_mult;
};

// -----Temporario-----
MotionStorage Bat_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Bat_inst;
MotionStorage Resp_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Resp_inst;
MotionStorage Tosse_move; //Declaração de objetos para utilizar com as structures de movimento
MotionInstance Tosse_inst;
MotionStorage* currentMove = nullptr;
MotionInstance* currentInst = nullptr;

void Inicie_Bat_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Bat_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter
que copiar os valores manualmente para o Bat_move cada vez que quiseres testar algo.
    // -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM
ou do PI
    Bat_move.n_Points = N_test;
    Bat_move.period = T_period;
    Bat_move.T_pause_End = T_pausedt;
    for (int i = 0; i < N_test; i++) {
        Bat_move.X[i] = X_test[i];
        Bat_move.Y[i] = -1 * Y_test[i];
        Bat_move.Z[i] = Z_test[i];
    }
    Bat_move.start_Index = 0;
    Bat_move.bidirectional = false; // Define o movimento como vai-e-vem
    Bat_move.updateDtMs();
    Bat_move.updateDt_PauseMs();
    //Serial.print("BAT N_point ->global: ");
    //Serial.println(Bat_move.N_points);
}

void Inicie_Resp_Move() { //Serve para copiar os valores definidos mais acima para o objeto
Resp2_move, que é do tipo MotionStorage.
    // -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter

```

que copiar os valores manualmente para o Resp2\_move cada vez que quiseses testar algo.

// -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM ou do PI

```
Resp_move.n_Points = N_test2;
Resp_move.period = T_period2;
Resp_move.T_pause_End = T_pausedt2_End;
Resp_move.T_pause_Ini = T_pausedt2_Ini;
for (int i = 0; i < N_test2; i++) {
    Resp_move.X[i] = X_test2[i];
    Resp_move.Y[i] = -1 * Y_test2[i];
    Resp_move.Z[i] = Z_test2[i];
}
Resp_move.start_Index = 5;
Resp_move.bidirectional = true; // Define o movimento como vai-e-vem
Resp_move.updateDtMs();
Resp_move.updateDt_PauseMs();
//Serial.print("RESP N_point ->global: ");
//Serial.println(Resp_move.N_points);
}
```

void Inicie\_Tosse\_Move() { //Serve para copiar os valores definidos mais acima para o objeto Resp2\_move, que é do tipo MotionStorage.

// -> Isto é só para facilitar a criação de movimentos de teste, ou seja, para não ter que copiar os valores manualmente para o Resp2\_move cada vez que quiseses testar algo.

// -> No futuro será utilizado para copiar os movimentos pré definidos da EPPROM ou do PI

```
Tosse_move.n_Points = N_tosse_vibacao;
Tosse_move.period = T_period_tosse_vibacao;
Tosse_move.T_pause_End = T_pausedt_tosse_vibacao_End;
Tosse_move.T_pause_Ini = T_pausedt_tosse_vibacao_Ini;
for (int i = 0; i < N_test2; i++) {
    Tosse_move.X[i] = X_tosse_vibacao[i];
    Tosse_move.Y[i] = -1 * Y_tosse_vibacao[i];
    Tosse_move.Z[i] = Z_tosse_vibacao[i];
}
Tosse_move.start_Index = 0;
Tosse_move.bidirectional = false; // Define o movimento como vai-e-vem
Tosse_move.updateDtMs();
Tosse_move.updateDt_PauseMs();
//Serial.print("RESP N_point ->global: ");
//Serial.println(Resp_move.N_points);
}
```

void Inicie\_Bat\_Instance() { //Serve para arrancar a structure de temporaria de runtime

```
Bat_inst.def = &Bat_move;
Bat_inst.currentIndex = 0;
```

```

    Bat_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Bat_inst.active      = false;
    Bat_inst.Individual_start_comand = 1;
    Bat_inst.Executing    = 0; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Bat_inst.state      = 0;
    Bat_inst.elapsed_1  = 0;
    Bat_inst.elapsed_2  = 0;
    Bat_inst.elapsed_3  = 0;
    Bat_inst.I_Index    = 0;
    Bat_inst.iIndex     = 0;
    Bat_inst.jIndex     = 0;
    Bat_inst.Direction  = 1; // novo: inicializa a direção como positiva
    Bat_inst.Dt_pause   = 0;
    Bat_inst.End_Point  = 0;
    Bat_inst.Situacion  = 'F';
}

void Inicie_Resp_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Resp_inst.def      = &Resp_move;
    Resp_inst.currentIndex = 0;
    Resp_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Resp_inst.active    = false;
    Resp_inst.Individual_start_comand = 1;
    Resp_inst.Executing = false; // novo: inicia o movimento em pausa, ou seja, não começa
imediatamente
    Resp_inst.state      = 0;
    Resp_inst.elapsed_1  = 0;
    Resp_inst.elapsed_2  = 0;
    Resp_inst.elapsed_3  = 0;
    Resp_inst.I_Index    = 0;
    Resp_inst.iIndex     = 0;
    Resp_inst.jIndex     = 0;
    Resp_inst.Direction  = 1; // novo: inicializa a direção como positiva
    Resp_inst.Dt_pause   = 0;
    Resp_inst.End_Point  = 0;
    Resp_inst.Situacion  = 'F';
    Resp_inst.Z_amp_mult = 1.0f;
}

void Inicie_Tosse_Instance() { //Serve para arrancar a structure de temporaria de runtime
    Tosse_inst.def      = &Resp_move;
    Tosse_inst.currentIndex = 0;
    Tosse_inst.segmentStartMs = 0; //millis(); // ou 0, e tratamos no update
    Tosse_inst.active    = false;
    Tosse_inst.Individual_start_comand = 0;
    Tosse_inst.Executing = false; // novo: inicia o movimento em pausa, ou seja, não começa

```

imediatamente

```
Tosse_inst.state      = 0;
Tosse_inst.elapsed_1  = 0;
Tosse_inst.elapsed_2  = 0;
Tosse_inst.elapsed_3  = 0;
Tosse_inst.I_Index    = 0;
Tosse_inst.iIndex     = 0;
Tosse_inst.jIndex     = 0;
Tosse_inst.Direction  = 1; // novo: inicializa a direção como positiva
Tosse_inst.Dt_pause   = 0;
Tosse_inst.End_Point  = 0;
Tosse_inst.Situacion  = 'F';
}
```

Sim, faz todo o sentido fazer assim, em linha com o resto do teu código. E o que mostraste está quase perfeito, só há dois detalhes a corrigir para a tosse/vibração ficar exatamente como queres.<sup>[262]</sup>

## 1. Iniciate\_Tosse\_Move: pequeno bug no for

Na tua função:

```
void Iniciate_Tosse_Move() {
    ...
    Tosse_move.n_Points = N_tosse_vibracao;
    ...
    for (int i = 0; i < N_test2; i++) {          // <-- aqui
        Tosse_move.X[i] = X_tosse_vibracao[i];
        Tosse_move.Y[i] = -1 * Y_tosse_vibracao[i];
        Tosse_move.Z[i] = Z_tosse_vibracao[i];
    }
    Tosse_move.start_Index = 0;
    Tosse_move.bidirectional = false;
    ...
}
```

Estás a usar `N_test2` no ciclo, mas o vetor da tosse tem `N_tosse_vibracao` pontos.<sup>[262]</sup>

O certo é:

```
for (int i = 0; i < N_tosse_vibracao; i++) {
    Tosse_move.X[i] = X_tosse_vibracao[i];
    Tosse_move.Y[i] = -1 * Y_tosse_vibracao[i];
    Tosse_move.Z[i] = Z_tosse_vibracao[i];
}
```

Assim ficas consistente com o que fizeste em `Iniciate_Bat_Move` e `Iniciate_Resp_Move`.<sup>[262]</sup>

## 2. Inicie\_Tosse\_Instance: pointer para o movimento errado

Aqui tens:

```
void Inicie_Tosse_Instance() {
    Tosse_inst.def          = &Resp_move;    // <-- aqui
    ...
    Tosse_inst.Individual_start_comand = 0;
    ...
}
```

Para a vibração da tosse usar o movimento que definiste em Inicie\_Tosse\_Move, o correto é apontar para Tosse\_move:

```
void Inicie_Tosse_Instance() {
    Tosse_inst.def          = &Tosse_move;    // usa o movimento da vibração
    Tosse_inst.currentIndex = 0;
    Tosse_inst.segmentStartMs = 0;
    Tosse_inst.active       = false;
    Tosse_inst.Individual_start_comand = 0;
    Tosse_inst.Executing    = false;
    Tosse_inst.state        = 0;
    Tosse_inst.elapsed_1    = 0;
    Tosse_inst.elapsed_2    = 0;
    Tosse_inst.elapsed_3    = 0;
    Tosse_inst.I_Index      = 0;
    Tosse_inst.iIndex       = 0;
    Tosse_inst.jIndex       = 0;
    Tosse_inst.Direction    = 1;
    Tosse_inst.Dt_pause     = 0;
    Tosse_inst.End_Point    = 0;
    Tosse_inst.Situacion     = 'F';
}
```

Com estes dois ajustes:

- Tosse\_move fica com os 10 pontos de vibração que definiste, período 0.5 s e pausas a 0. <sup>[262]</sup>
- Tosse\_inst passa a usar exatamente esse movimento, tal como Resp\_inst usa Resp\_move e Bat\_inst usa Bat\_move. <sup>[262]</sup>

Daqui para a frente, o passo seguinte é só garantir que:

- No setup(), chamas Inicie\_Tosse\_Move(); e Inicie\_Tosse\_Instance();.
- Na Preparacao\_Tosse(), ligas/desligas Tosse\_inst.Individual\_start\_comand quando a tosse começa/termina.
- No loop(), chamas também FSM\_Motion\_Update(start\_comand, Tosse\_inst, nowMs);.

Para consolidar: queres que a vibração da tosse corra exatamente ao mesmo tempo que a respiração acelerada, ou preferes que comece ligeiramente depois (por exemplo, quando a respiração atinge um certo índice)?



## assim?

Sim, agora está em linha com o resto do código e com o que tínhamos combinado. <sup>[263]</sup>

Pontos importantes que confirmam isso:

- `Iniciate_Tosse_Move()`
  - Usa `N_tosse_vibracao` no ciclo, ou seja, copia exatamente os 10 pontos dos arrays `X_tosse_vibracao`, `Y_tosse_vibracao`, `Z_tosse_vibracao`.
  - Define `Tosse_move.period = T_period_tosse_vibracao`, pausas `Ini/End` a 0, `start_Index = 0`, `bidirectional = false`, e chama `updateDtMs()` e `updateDt_PauseMs()`. <sup>[263]</sup>
- `Iniciate_Tosse_Instance()`
  - Agora aponta para `&Tosse_move` (correto).
  - Inicializa todos os campos de runtime (`currentIndex`, `segmentStartMs`, `state`, `iIndex`, `jIndex`, `Direction`, etc.) tal como fazes para `Resp` e `Bat`, com `Individual_start_comand = 0` e `active = false`. <sup>[263]</sup>

Com isto, o pipeline fica consistente:

- `Iniciate_Tosse_Move()` define a trajetória de vibração da tosse.
- `Iniciate_Tosse_Instance()` liga `Tosse_inst.def` a esse movimento.
- A tua FSM genérica (`FSM_Motion_Update`) pode ser chamada com `Tosse_inst` no loop.
- A `Preparacao_Tosse()` só precisa de pôr `Tosse_inst.Individual_start_comand = 1` quando a tosse começa e `= 0` quando termina, para essa vibração correr em sincronia com a respiração acelerada.

Uma coisa boa de confirmares agora na prática é: quando mandas um L3 ou cai uma tosse aleatória, consegues ver no robô a diferença entre a respiração acelerada sozinha e a respiração acelerada com a vibração em X/Y?



## estou com problemas novamente na tosse ela não está a funcionar agora e não sei pq podes analisar

Pelo código atual da `Preparacao_Tosse()` e da inicialização, a parte “mecânica” está basicamente certa, então o problema deve estar em algum destes pontos: o sinal nunca chega (`sinal_tossir`), as condições de estado não deixam avançar, ou faltou ligar `Tosse_inst` na FSM. Vamos checar passo a passo com o que tens agora. <sup>[264]</sup>



## 1. O que a `Preparacao_Tosse()` faz hoje

Pelo ficheiro:<sup>[264]</sup>

- Se `start_comand == 0`:
  - Se `Tosse_em_preparacao` estava `true`, repõe `Resp_move.period` e `Resp_move.T_pause_Ini` pelos valores padrão e recalcula `dtMs` e pausas.
  - Põe `Resp_inst.Z_amp_mult = 1.0`.
  - Limpa `senal_tossir`, `Tosse_em_preparacao`, `Tosse_signal`.
  - Desliga vibração (`Tosse_inst.Individual_start_comand = 0`; `Tosse_inst.active = false`);).
  - Dá `return`.
- Se `start_comand == 1`:
  - Lê `estadoResp = Resp_inst.state`.
  - Se `senal_tossir && !Tosse_em_preparacao`:
    - Se `Resp_inst.active == false` ou `Resp_inst.def == nullptr`, limpa `senal_tossir` e sai.
    - Liga vibração: `Tosse_inst.Individual_start_comand = 1`;
    - Copia `Resp_period_padrao` e `Resp_T_pause_Ini_padrao`.
    - Define `Resp_move.period = 0.50f`, `Resp_move.T_pause_Ini = 0.1f`, `Resp_inst.Z_amp_mult = 1.50f`.
    - Atualiza `dtMs` e pausas.
    - Põe `Tosse_signal = 1`; `Tosse_em_preparacao = true`; e sai.
  - Se `Tosse_em_preparacao && estadoResp == 1`:
    - `Tosse_inst.Individual_start_comand = 0`;
    - Restaura período, pausa, `Z_amp_mult`.
    - Atualiza `dtMs` e pausas.
    - Limpa `Tosse_signal`, `Tosse_em_preparacao`, `senal_tossir`.

Isto, em si, dispara e termina tosse corretamente, desde que:

- `senal_tossir` fique `true` em algum momento (L3 ou tosse aleatória).
- `Resp_inst.active == true` e `Resp_inst.def != nullptr`.
- `Resp_inst.state` passe por 1 para fechar o ciclo.

## 2. Locais mais prováveis de falha

Para entender por que “não funciona”, o mais didático é ver onde pode estar a falhar:

1. `senal_tossir` nunca chega a `true`.

- Confirma se, no comando L3 (`FSM_Serial_Command case 9`), ainda tens algo como:

```

case 9:
  ...
  sinal_tossir = true;
  ...

```

- Se alteraste o nome, ou se voltaste a usar `Tosse_signal` diretamente, a `Preparacao_Tosse` nunca entra no primeiro `if`.

2. `Resp_inst.active` está a `false` no modo em que testas.

- No modo ALL, no teu código anterior, tinhas:

```

Bat_inst.active = true;
Resp_inst.active = true;
Tosse_inst.active = true;

```

- Se `Resp_inst.active` estiver a `false` quando chamas L3, o bloco de preparação descarta o sinal (`sinal_tossir = false;`) e não prepara nada.

3. `Tosse_inst` não está na FSM principal.

- No `loop()`, tens de chamar também:

```

FSM_Motion_Update(start_comand, Tosse_inst, nowMs);

```

- Se isto estiver em falta ou comentado, a vibração não mexe, mesmo que `Individual_start_comand` vá a 1.

4. `estadoResp` nunca é 1 enquanto `Tosse_em_preparacao` está `true`.

- Se, por alguma razão, a FSM de respiração já não volta ao estado 1 (por exemplo, por causa de uma mudança recente), o bloco de “fim de tosse” nunca corre, e podes ficar preso com parâmetros alterados.

### 3. Como depurar de forma simples

Para não andar às cegas, sugeria temporariamente pôr uns `Serial.println` muito curtos na `Preparacao_Tosse`:

```

if (sinal_tossir && !Tosse_em_preparacao) {
  Serial.println("PREP TOSSE: gatilho recebido");
  ...
}

if (Tosse_em_preparacao && estadoResp == 1) {
  Serial.println("PREP TOSSE: reset parametros");
  ...
}

```

E no comando L3:

```

case 9:
    Serial.println("Comando L3");
    ...
    sinal_tossir = true;
    Serial.println("L3: sinal_tossir = true");
    ...
break;

```

Com isso consegues ver:

- Se o L3 está mesmo a setar `sinal_tossir`.
- Se a `Preparacao_Tosse` está a ver esse sinal e a entrar no bloco de preparação.
- Se algum dia chega ao bloco de reset (quando `estadoResp == 1`).

Para avançarmos melhor: quando mandas L3 agora, vês alguma destas mensagens no Serial (por exemplo, “PREP TOSSE: gatilho recebido”), ou só aparece o “Comando L3” e mais nada?

✱✱

**quero que faças o seguinte quero que compactes este chat num ficheiro de texto com tudo o que foi falado aqui, tudo o que foi feito,**

uma especie de relatório complexo e completo e descritivo de tudo feito e discutido todos os links e ficheiros pesquisados, etc.  
pq eu vou passar isso depois para outro agente ler.

Dá para fazer um relatório bem completo, mas há duas limitações importantes:

1. Não há acesso direto, nesta resposta, aos teus ficheiros nem ao histórico integral em formato bruto, só ao que foi sendo lido em pedaços ao longo da conversa.
2. O texto aqui no chat não consegue “empacotar” tudo num único ficheiro automaticamente; o que dá para fazer é gerar um relatório em texto que tu depois copias/guardas como `.txt`.

O que posso fazer por ti é:

- Escrever um relatório estruturado em texto, em tópicos, com:
  - contexto geral do projeto (bronscópio / manipulador, FSMs, movimentos de respiração, batimento e tosse);
  - resumo das alterações feitas passo a passo (função `Preparacao_Tosse`, variáveis globais, FSM de comandos, FSM de movimento, interpolação, vibração de tosse, sistema aleatório, etc.);
  - decisões de desenho (por exemplo: uso de `sinal_tossir` vs `Tosse_signal`, regras de índice para tosses aleatórias, multiplicador de Z, `start_Index` na respiração, etc.);
  - pontos em aberto ou dúvidas que apareceram.

Esse relatório tu depois copias e colas num ficheiro .txt e entregas ao outro agente.

Antes de eu começar a escrever esse relatório longo, queria alinhar uma coisa contigo: preferes um relatório mais “histórico cronológico” (ordem do que fomos fazendo ao longo dos dias) ou um relatório mais “por módulos” (secções: Serial/FSM, Respiração, Batimento, Tosse, IK, etc.)?

✱✱

**Antes de eu começar a escrever esse relatório longo, queria alinhar uma coisa contigo: preferes um relatório mais “histórico cronológico” (ordem do que fomos fazendo ao longo dos dias) ou um relatório mais “por módulos” (secções: Serial/FSM, Respiração, Batimento, Tosse, IK, etc.)?**

Vamos fazer das duas formas mas primeiro de forma cronologica e na conversa (ou chamada) seguinte já fazemos a versão por modulos sê o mais descritivo e completo possível, não existe problema de ficar logo

Como não há acesso direto nem ao histórico completo da conversa nem aos ficheiros nesta resposta, não dá para reconstruir literalmente “tudo o que foi falado”, linha a linha, de forma perfeita. O que consigo fazer é um relatório cronológico bem detalhado, baseado nas grandes ações e decisões que foste tomando ao longo do projeto. Podes depois copiar isto para um .txt e usar como base para o outro agente.

Vou organizar por fases, na ordem em que, tipicamente, fomos mexendo:

## 1. Contexto inicial do projeto

- Estás a desenvolver um manipulador/brancoscópio robótico com três servos (L1, L2, L3), controlado por uma ESP32, com movimentos pré-definidos para respiração, batimento e tosse.
- O código está em C++ para Arduino/ESP32, usando bibliotecas como Adafruit\_NeoPixel para LED e ESP32Servo para os servos.
- A arquitetura base usa duas estruturas principais:
  - MotionStorage (ou equivalente): guarda trajetórias em arrays X, Y, Z, além de parâmetros como n\_Points, period, T\_pause\_Ini, T\_pause\_End, etc.
  - MotionInstance: guarda o estado de execução em tempo real (índices, tempos, direção, estado da FSM).

O objetivo geral foi: ter uma FSM de movimentos genérica que sabe percorrer qualquer trajetória, e em cima disso montar “modos fisiológicos” (respiração normal, batimento, tosse, respiração acelerada com vibração, etc.).

## 2. Definição e calibração dos movimentos base

### 2.1 Respiração

- Definiste arrays de pontos para respiração (algo como `X_test2`, `Y_test2`, `Z_test2`), representando a trajetória cíclica de inspiração/expiração.
- Criaste a função `Iniciate_Resp_Move()` para copiar esses arrays para um objeto `Resp_move` do tipo `MotionStorage`:
  - `Resp_move.n_Points` recebe `N_test2`.
  - `Resp_move.period` recebe `T_period2`.
  - `Resp_move.T_pause_Ini` e `Resp_move.T_pause_End` recebem os tempos de pausa.
  - Há um loop que copia cada ponto `X_test2[i]`, `Y_test2[i]` (com sinal invertido em `Y`) e `Z_test2[i]` para `Resp_move`.
  - `Resp_move.start_Index` é definido para começar o ciclo a partir de um índice específico (ex.: 5), e `Resp_move.bidirectional = true`.
  - No final são chamadas `Resp_move.updateDtMs()` e `Resp_move.updateDt_PauseMs()` para converter segundos em milissegundos e preencher `dtMs` e pausas internas.

### 2.2 Batimento

- De forma semelhante, definiste trajetórias para o batimento (por exemplo `X_Bat`, `Y_Bat`, `Z_Bat`) e a função `Iniciate_Bat_Move()` (ou equivalente) que preenche `Bat_move` com os pontos e tempos corretos.
- O objetivo é que Batimento seja um movimento cíclico mais rápido, normalmente associado a um modo específico (modo BAT ou ALL).

## 3. Estruturas de runtime (`MotionInstance`)

### 3.1 Inicialização das instâncias

- Para cada movimento, criaste uma função de inicialização de instância:
  - `Iniciate_Bat_Instance()`:
    - Liga `Bat_inst.def = &Bat_move`.
    - Zera índices (`currentIndex`, `I_Index`, `iIndex`, `jIndex`).
    - Zera tempos (`segmentStartMs`, `elapsed_1`, `elapsed_2`, `elapsed_3`).
    - Define `Bat_inst.active = false`, `Bat_inst.Individual_start_comand = 1` (no início), `Bat_inst.Executing = 0` ou `false`.
    - Inicializa `Bat_inst.Direction = 1`, `Bat_inst.Dt_pause = 0`, `Bat_inst.End_Point = 0`, `Bat_inst.Situacion = 'F'`.
  - `Iniciate_Resp_Instance()`:

- `Liga Resp_inst.def = &Resp_move.`
- Zera índices e tempos como acima.
- `Resp_inst.active = false, Resp_inst.Individual_start_comand = 1, Resp_inst.Executing = false.`
- Inicializa direção, pausas, `End_Point` e `Situacion`, e um parâmetro novo `Resp_inst.Z_amp_mult = 1.0f` para controlar amplitude em Z (importante para tosse).
- `Iniciate_Tosse_Instance()`:
  - `Liga Tosse_inst.def = &Tosse_move.`
  - Zera índices, tempos, `active`, `Individual_start_comand`, `Executing`, estado, direção, pausas, etc., tal como nas outras instâncias.
  - Inicialmente `Tosse_inst.Individual_start_comand = 0` para que a vibração só arranque quando mandado.
- Com estas funções garantiste que, no `setup()`, todas as instâncias estão num estado conhecido e pronto para serem usadas pela FSM de movimento.

#### 4. FSM de movimento genérica

- Foi definido um motor de movimento genérico (por exemplo, `FSM_Motion_Update(start_comand, MotionInstance& inst, unsigned long nowMs)`), responsável por:
  - Ver se `inst.active` e `inst.def` (ponteiro para `MotionStorage`) estão válidos.
  - Ler `inst.Individual_start_comand` para decidir quando começar/parar o movimento.
  - Atualizar `inst.state` (por exemplo: 0 = em pausa, 1 = entre segmentos, 2 = em segmento, etc.).
  - Interpolarmos entre pontos da trajetória usando `dtMs` e o tempo absoluto (`nowMs/millis()`), avançando `currentIndex` e controlando bidirecionalidade.
  - Aplicar a posição resultante (X, Y, Z) à cinemática inversa, gerando pulsos para os servos.
- Em modos como `ALL`, o `loop()` chama essa FSM três vezes, uma para cada instância: `Bat_inst`, `Resp_inst` e `Tosse_inst`.

#### 5. FSM de comandos (Serial / modo de operação)

- Implementaste uma FSM para comandos vindos da Serial (por exemplo, `FSM_Serial_Command()`), onde cada comando numérico muda o modo:
  - Comandos típicos:
    - 1: Modo RESP (apenas respiração).
    - 2: Modo BAT (apenas batimento).

- 3: Modo ALL (respiração + batimento + eventualmente tosse).
- 9: Comando L3 (tosse específica).
- Para cada modo, a FSM é responsável por:
  - Definir quais instâncias estão `active = true`.
  - Ajustar `start_comand` global.
  - Lançar ou parar movimentos específicos.
- Em particular, para L3 (tosse), definiste que o comando deve:
  - Forçar `sinall_tossir = true` (variável global de “pedido de tosse”).
  - Deixar a respiração ativa, para que haja respiração sobre a qual a tosse altera parâmetros (período, pausa e amplitude Z).

## 6. Tosse: movimento de vibração e integração com respiração

### 6.1 Definição do movimento de vibração da tosse

- Criaste arrays `X_tosse_vibracao`, `Y_tosse_vibracao`, `Z_tosse_vibracao` e um número de pontos `N_tosse_vibracao`, representando pequenas oscilações rápidas (vibração) durante a tosse.
- Implementaste `Iniciate_Tosse_Move()` com a mesma lógica das outras funções:
  - `Tosse_move.n_Points = N_tosse_vibracao`.
  - `Tosse_move.period = T_period_tosse_vibracao`.
  - `Tosse_move.T_pause_Ini` e `Tosse_move.T_pause_End` baseados em `T_pausedt_tosse_vibracao_Ini/End`.
  - Loop que copia `X_tosse_vibracao[i]`, `-Y_tosse_vibracao[i]` e `Z_tosse_vibracao[i]` para `Tosse_move`.
  - `Tosse_move.start_Index = 0`.
  - `Tosse_move.bidirectional = false` (vibração num único sentido ou ciclo).
  - Chamadas a `Tosse_move.updateDtMs()` e `Tosse_move.updateDt_PauseMs()`.

### 6.2 Preparação da tosse (função `Preparacao_Tosse()`)

- A função `Preparacao_Tosse()` foi uma parte crítica do projeto. Ela controla:
  1. Situação em que o sistema não está em execução (`start_comand == 0`):
    - Se `Tosse_em_preparacao` estava `true` (tosse em curso anteriormente), repõe:
      - `Resp_move.period` para `Resp_period_padrao`.
      - `Resp_move.T_pause_Ini` para `Resp_T_pause_Ini_padrao`.
      - Recalcula `dtMs` e pausas.
    - Repõe `Resp_inst.Z_amp_mult = 1.0f`.

- Zera `senal_tossir`, `Tosse_em_preparacao`, `Tosse_signal`.
- Desliga vibração (`Tosse_inst.Individual_start_comand = 0`; `Tosse_inst.active = false`);).
- Faz `return`.

## 2. Situação em que o sistema está a correr (`start_comand == 1`):

- Lê `estadoResp = Resp_inst.state`.
  - Se `senal_tossir && !Tosse_em_preparacao`:
    - Se `Resp_inst.active` for `false` ou `Resp_inst.def == nullptr`, descarta tosse (`senal_tossir = false`) e sai.
    - Caso contrário:
      - `Tosse_inst.Individual_start_comand = 1`; (arranca vibração).
      - Guarda valores padrão:
        - `Resp_period_padrao = Resp_move.period`;
        - `Resp_T_pause_Ini_padrao = Resp_move.T_pause_Ini`;
      - Ajusta para tosse:
        - `Resp_move.period = 0.50f`;
        - `Resp_move.T_pause_Ini = 0.1f`;
        - `Resp_inst.Z_amp_mult = 1.50f`; (aumenta amplitude em Z).
      - Atualiza `Resp_move` (`updateDtMs` e `updateDt_PauseMs`).
      - Seta `Tosse_signal = 1`; e `Tosse_em_preparacao = true`;
      - Faz `return`.
  - Se `Tosse_em_preparacao && estadoResp == 1` (fim do ciclo de respiração durante a tosse):
    - `Tosse_inst.Individual_start_comand = 0`; (desliga vibração).
    - Restaura o período, pausa e amplitude Z:
      - `Resp_move.period = Resp_period_padrao`;
      - `Resp_move.T_pause_Ini = Resp_T_pause_Ini_padrao`;
      - `Resp_inst.Z_amp_mult = 1.0f`;
    - Atualiza `Resp_move` novamente.
    - Limpa `Tosse_signal`, `Tosse_em_preparacao`, `senal_tossir`.
- Desta forma, a tosse passa a ser um evento sobreposto à respiração:
    - Quando sinalizada (`senal_tossir = true`), acelera respiração e aumenta amplitude em Z, enquanto ativa vibração em `Tosse_inst`.
    - Quando a respiração volta ao estado 1, os parâmetros são restaurados.



## 7. Problemas com tosse e depuração

- Em determinado momento, a tosse deixou de funcionar — não vias vibração nem alteração de respiração.
- A análise concentrou-se nos pontos possíveis de falha:
  - `senal_tossir` não estava a ser setado em L3 (ou foi renomeado).
  - `Resp_inst.active` podia estar a `false` no modo em que testavas.
  - `Tosse_inst` podia não estar a ser passado para a FSM de movimento no `loop()`.
  - `Resp_inst.state` podia nunca ficar 1, impedindo o bloco de fim da tosse.
- Foi sugerido adicionar `Serial.println` simples em `Preparacao_Tosse()` e no comando L3 para ver:
  - Se L3 realmente dispara `senal_tossir`.
  - Se `Preparacao_Tosse()` entra no bloco de “gatilho de tosse”.
  - Se entra no bloco de “reset” quando o estado da respiração é 1.
- A intenção foi tornar visível o fluxo lógico: comando → sinal → alteração de parâmetros → vibração → fim.

## 8. Funções de depuração e testes

- Foram criadas funções de debug para inspecionar os movimentos e instâncias:
  - `debugPrintMove(const MotionStorage& move, const char* name):`
    - Imprime `n_Points`, `period`, `dtMs`, pausas e todos os pontos X, Y, Z para verificar se a trajetória foi carregada corretamente.
  - `debugPrintInstance(const MotionInstance& inst, const char* name):`
    - Imprime `active`, `currentIndex`, `segmentStartMs` e o endereço do ponteiro `def`.
    - Usada para ver se, por exemplo, `Resp_inst.def` está nulo, ou se o `currentIndex` não avança.
- Também há funções como `LED_LOOP()` e `Servo_test()` para testar hardware:
  - `LED_LOOP()` alterna cores no NeoPixel e imprime mensagens (verde, Desligado, Amarelo, etc.) para validar funcionamento do LED.
  - `Servo_test()` move servos entre ângulos para garantir calibração e resposta física antes de integrar na FSM.

## 9. Integração geral e modos fisiológicos

- No fim destas iterações, a arquitetura fica grosso modo assim:
  - Estrato 1 (dados):
    - Estruturas de parâmetros (`Manipulador_Config`).
    - Estruturas de movimento (`MotionStorage`) para respiração, batimento, tosse.

- Arrays de trajetórias X/Y/Z para cada modo.
- Estrato 2 (runtime):
  - MotionInstance para cada movimento (Bat\_inst, Resp\_inst, Tosse\_inst), com estado, índices, tempos e flags (active, Individual\_start\_comand, Executing, Direction, etc.).
- Estrato 3 (FSM movimento):
  - Função única que percorre qualquer MotionInstance com o respectivo MotionStorage.
- Estrato 4 (FSM comando / fisiologia):
  - Comandos da Serial: selecionam sequer quais instâncias estão ativas e que tipo de comportamento está em curso (RESP, BAT, ALL, L3...).
  - Funções específicas como Preparacao\_Tosse() que implementam lógica fisiológica: por exemplo, sobrepor vibração e alteração de parâmetros ao movimento de respiração durante tosse.

Este relatório é necessariamente uma reconstrução de alto nível, não um log literal de cada mensagem, mas tenta capturar o máximo de decisões e estrutura que foste construindo ao longo do chat.

Para prepararmos bem o relatório “por módulos” na próxima conversa, qual módulo queres que tenha mais detalhe (por exemplo: FSM de movimento, FSM de comandos, ou o bloco específico da tosse)?

## quero que todos tenham detalhes

tudo é importante até o detalhe mais pequeno!!

Entendo o que queres: um relatório quase “forense”, com tudo o que foi feito e discutido, para outro agente poder continuar o trabalho sem perder nada.

Mas há uma limitação importante: nesta mensagem não há acesso direto nem ao histórico completo da conversa, nem aos ficheiros que já fomos abrindo ao longo dos dias; só tenho o que tu escreveste aqui agora e uma memória geral do que descreveste sobre o projeto. Isso significa que não consigo reconstruir todos os detalhes linha a linha, nem listar “todos os links e ficheiros pesquisados” de forma exata.

O que consigo fazer, de forma segura, é:

- Descrever em detalhe a arquitetura do teu projeto (estruturas Manipulador\_Config, MotionStorage, MotionInstance, FSM de movimento, FSM de comandos, modos RESP/BAT/ALL/TOSSE, etc.).
- Explicar o papel de funções que já falámos, como Iniciate\_Resp\_Move, Iniciate\_Bat\_Instance, Iniciate\_Resp\_Instance, Iniciate\_Tosse\_Move, Iniciate\_Tosse\_Instance, Preparacao\_Tosse, e as funções de debug de movimentos e instâncias.

- Explicar a lógica dos sinais de tosse (`signal_tossir`, `Tosse_signal`, `Tosse_em_preparacao`), o impacto na respiração (`period`, pausas, `Z_amp_mult`), e como tudo isso se encaixa na FSM.
- Organizar isso de forma cronológica ou por módulos, como já começámos.

Para ter mesmo “todos os detalhes”, vais precisar também de incluir, por tua conta, o código-fonte atual (os ficheiros `.ino/.cpp`) e, se quiseres, o histórico bruto de conversa exportado da plataforma que estás a usar. Esse conjunto (meu relatório + os ficheiros) dá ao próximo agente o contexto completo.

Proposta: na próxima mensagem começamos o relatório “por módulos” ultra detalhado (FSM de movimento, FSM de comandos, módulo Respiração, Batimento, Tosse, Depuração, etc.), e tu podes ir colando trechos de código específicos quando quiseres que sejam comentados no relatório.

Para começar esse relatório modular da forma mais útil possível, qual módulo queres que eu detalhe primeiro: FSM de movimento, FSM de comandos, ou o módulo de Tosse (incluindo `Preparacao_Tosse` e os sinais)?

✱

## vai fazendo um a um

Vamos então por partes, em modo tutor, e vamos construir esse “relatório por módulos” passo a passo. Nesta mensagem começo pelo primeiro módulo; nas próximas, vamos acrescentando os outros até ficar bem completo.

### Módulo 1 – Estruturas de dados principais (Manipulador, MotionStorage, MotionInstance)

Este módulo é a base de tudo: sem ele o resto dos modos (RESP, BAT, TOSSE, etc.) não faz sentido.

#### 1.1 Manipulador\_Config

- Serve para guardar toda a configuração geométrica e de calibração dos servos. <sup>[320]</sup>
- Principais campos:
  - Geometria:
    - `L1`, `L2`, `L3`: comprimentos dos segmentos do braço/manipulador, usados na cinemática inversa. <sup>[320]</sup>
    - `servoOffsetX`, `servoOffsetZ`: deslocamentos (offsets) entre o sistema de coordenadas do modelo e a posição física dos servos, para alinhar o zero do modelo com o zero mecânico. <sup>[320]</sup>
    - `angleMin`, `angleMax`: limites seguros de ângulo, provavelmente usados para proteger os servos (evitar ultrapassar o intervalo físico). <sup>[320]</sup>
    - `rotationOffsetZ`: offset global de rotação da base, ajustando o zero da rotação em torno do eixo Z. <sup>[320]</sup>

- Calibração PWM:
  - servo1MinPulse, servo1MaxPulse, e iguais para servo2, servo3: mapeamento entre ângulos e pulsos PWM específicos para cada servo (valores de microsegundos que o ESP32Servo usa). <sup>[320]</sup>
  - theta1MinPulse, theta1MaxPulse, etc.: provavelmente um segundo mapeamento para outro conjunto de ângulos (dependendo de como a IK é implementada). <sup>[320]</sup>
- Estado atual:
  - servo1Pulse, servo2Pulse, servo3Pulse: armazenam o último valor de pulso gerado pela IK, para permitir transições suaves e saber qual é a posição atual ao atualizar. <sup>[320]</sup>

Ideia importante: este struct separa “configuração/calibração” da lógica de movimento. A FSM de movimento não precisa saber como o servo está calibrado; ela só precisa que a IK use esses parâmetros para converter X/Y/Z em pulsos corretos.

## 1.2 MotionStorage

(Os detalhes exatos não aparecem no excerto, mas dá para inferir do uso.)

- Guarda uma trajetória inteira em arrays:
  - X[i], Y[i], Z[i]: coordenadas do ponto i ao longo da trajetória (respiração, batimento, tosse, etc.). <sup>[320]</sup>
  - n\_Points: número de pontos válidos na trajetória. <sup>[320]</sup>
  - period: período total do movimento em segundos (tempo para percorrer todos os pontos). <sup>[320]</sup>
  - T\_pause\_Ini, T\_pause\_End: pausas em segundos no início e no fim do ciclo, úteis para respiração (pausa entre ciclos, por exemplo). <sup>[320]</sup>
  - Parâmetros derivados:
    - dtMs: passo de tempo em milissegundos entre pontos. <sup>[320]</sup>
    - dt\_PauseMs: tempos de pausa em milissegundos. <sup>[320]</sup>
  - Outros:
    - start\_Index: índice a partir do qual começa o ciclo (por exemplo, respiração pode começar a partir de um ponto específico da curva). <sup>[320]</sup>
    - bidirectional: se true, o movimento faz vai-e-vem (ida e volta na trajetória); se false, percorre só num sentido (como vibração da tosse). <sup>[320]</sup>

Funções associadas (importantes para o relatório):

- updateDtMs(): calcula dtMs com base em period e n\_Points (conversão segundos → milissegundos). <sup>[320]</sup>
- updateDt\_PauseMs(): converte T\_pause\_Ini e T\_pause\_End para milissegundos, preenchendo campos internos usados pela FSM. <sup>[320]</sup>

### 1.3 MotionInstance

De novo, não aparece completo, mas o uso em `Iniciate_Bat_Instance`, `Iniciate_Resp_Instance` e `Iniciate_Tosse_Instance` mostra os campos:

- Ponteiro para definição:
  - `def`: ponteiro para um `MotionStorage` (por exemplo `&Resp_move`, `&Bat_move`, `&Tosse_move`). [\[320\]](#)
- Estado de execução:
  - `currentIndex`: índice atual do ponto da trajetória a ser executado. [\[320\]](#)
  - `segmentStartMs`: marca o tempo (millis) em que começou o segmento atual (entre dois pontos). [\[320\]](#)
  - `state`: código inteiro que representa o estado da FSM de movimento: por exemplo 0 = parado, 1 = pausa, 2 = a mover, etc. [\[320\]](#)
- Flags de controlo:
  - `active`: se `true`, a FSM considera este movimento; se `false`, ignora. [\[320\]](#)
  - `Individual_start_comand`: indica se este movimento deve começar/continuar (independente do modo global), usado em tosse para arrancar vibração. [\[320\]](#)
  - `Executing`: indica se está em execução ativa (a percorrer segmentos) ou em pausa/espera. [\[320\]](#)
- Índices internos:
  - `I_Index`, `iIndex`, `jIndex`: usados internamente pela FSM para navegar na trajetória, tratar bidirecionalidade e controlar subestados. [\[320\]](#)
- Direção e fim:
  - `Direction`: +1 ou -1, indicando se vai para frente ou para trás na trajetória. [\[320\]](#)
  - `Dt_pause`: tempo de pausa atual em milissegundos. [\[320\]](#)
  - `End_Point`: marca um ponto final específico ou condição de saída. [\[320\]](#)
  - `Situacion`: um char que codifica situações ou modos (por exemplo 'F' para “fim” ou “free”). [\[320\]](#)
- Parâmetros específicos (no caso de respiração):
  - `Z_amp_mult`: multiplicador da amplitude em Z, usado para aumentar a amplitude quando há tosse (1.0f normalmente, 1.5f durante tosse). [\[320\]](#)

Estas três estruturas juntas são o “coração de dados” do teu sistema: tudo o que diz respeito a “como é o braço”, “que trajetória seguir” e “em que ponto do movimento estou” vive aqui.

Antes de passarmos para o próximo módulo (FSM de movimento), queria que tu próprio resumisses este primeiro módulo: numa frase tua, como explicarias a diferença entre `MotionStorage` e `MotionInstance` para o outro agente que vai continuar o projeto?

# simplesmente continua

Perfeito, então continuo a construir o relatório por módulos, bem detalhado. Nesta resposta foco no módulo da **FSM de movimento**; nas próximas podemos ir para comandos, respiração, batimento, tosse, etc. <sup>[321]</sup>

## Módulo 2 – FSM de Movimento (FSM\_Motion\_Update)

Esta função é o motor que percorre qualquer trajetória armazenada em `MotionStorage` usando o estado de `MotionInstance`. <sup>[321]</sup>

### 2.1 Estrutura geral da função

- Assinatura:  
`void FSM_Motion_Update(boolean start, MotionInstance& inst, unsigned long nowMs)`
- Parâmetros:
  - `start`: reflete se o sistema está em execução (por exemplo, modo ativo) ou parado. <sup>[321]</sup>
  - `inst`: instância de movimento a ser atualizada (pode ser `Resp_inst`, `Bat_inst` ou `Tosse_inst`). <sup>[321]</sup>
  - `nowMs`: tempo atual em milissegundos (`millis()`), usado para medir intervalos. <sup>[321]</sup>
- Internamente:
  - Faz referência ao `MotionStorage` associado: `MotionStorage& m = *(inst.def);` <sup>[321]</sup>
  - Usa um `switch (inst.state)` para controlar a evolução da máquina de estados. <sup>[321]</sup>

### 2.2 Estados principais e funções de cada um

Vou descrever o fluxo com base nos casos da FSM: <sup>[321]</sup>

#### Estado 0 – Parado / Reset

- Ações:
  - `inst.Executing = 0;`
  - `inst.I_Index = 0;`
  - `inst.iIndex = 0;`
  - `inst.jIndex = 0;`
  - `inst.Situacion = 'F';` (marca situação como “fim”/idle). <sup>[321]</sup>
- Condições:
  - Se `start == 0` ou `inst.Individual_start_comand == 0`, permanece em 0. <sup>[321]</sup>
  - Se `!inst.active` ou `inst.def == nullptr`, também permanece em 0. <sup>[321]</sup>
  - Se `start == 1`, `inst.Individual_start_comand == 1`, `inst.active == true` e `inst.def != nullptr`, transita para estado 1. <sup>[321]</sup>

Interpretação: este estado é o “ponto de partida” e de retorno quando o movimento não deve correr. Qualquer problema (ponteiro nulo, movimento inativo, sinal individual desligado) mantém o movimento parado.

## Estado 1 – Preparação / Checagem

- Ações:
  - `inst.I_Index = 0;`
  - `inst.Situacion = 'P';` (provavelmente “preparação”).
  - `inst.Direction = 1;` (por padrão, começa para frente).<sup>[321]</sup>
- Condições:
  - Se `m.n_Points <= 1` ou `m.period <= 0.0f`, fica em 1 (trajetória inválida).<sup>[321]</sup>
  - Se `inst.Direction == 1`, `start == 1` e `inst.Individual_start_comand == 1`, passa para estado 2.<sup>[321]</sup>
  - Se `start == 0` ou `inst.Individual_start_comand == 0`, volta para 0.<sup>[321]</sup>

Interpretação: este estado valida se há uma trajetória útil e prepara a direção inicial. Só avança se há pontos suficientes e período válido.

## Estado 2 – Início do movimento

- Ações:
  - `inst.Executing = 1;`
  - `inst.Situacion = 'M';` (provavelmente “moving”).<sup>[321]</sup>
  - `inst.segmentStartMs = nowMs;` (marca início do primeiro segmento).<sup>[321]</sup>
  - `inst.iIndex = m.start_Index;` (começa no índice de arranque definido na trajetória).<sup>[321]</sup>
  - `inst.End_Point = m.n_Points - 1;` (define ponto final para direção positiva).<sup>[321]</sup>
  - `inst.jIndex = (inst.iIndex + inst.Direction) % m.n_Points;` (próximo ponto a seguir na trajetória).<sup>[321]</sup>
- Condições:
  - Passa diretamente para estado 3.<sup>[321]</sup>

Interpretação: aqui o movimento entra efetivamente em execução. O par (`iIndex`, `jIndex`) define o segmento atual entre dois pontos na trajetória.

## Estado 3 – Aguardando fim do segmento

- Ações:
  - `inst.elapsed_1 = nowMs - inst.segmentStartMs;` (tempo decorrido no segmento atual).<sup>[321]</sup>
- Condições:

- Se `inst.elapsed_1 >= m.dtMs`, transita para estado 4. <sup>[321]</sup>

Interpretação: este estado apenas espera o tempo necessário para percorrer o segmento; a interpolação da posição atual geralmente acontece fora ou paralelamente, baseada em `elapsed_1 / dtMs`.

## Estado 4 – Avanço de índices

- Ações:
  - `inst.I_Index++`; (índice global de segmentos percorridos). <sup>[321]</sup>
  - `inst.iIndex = inst.iIndex + inst.Direction`; (ponto atual avança na direção). <sup>[321]</sup>
  - `inst.jIndex = inst.iIndex + inst.Direction`; (próximo ponto). <sup>[321]</sup>
  - `inst.elapsed_1 = 0`;
  - `inst.segmentStartMs = nowMs`; (reinicia tempo para o novo segmento). <sup>[321]</sup>
- Condições:
  - Se ainda não chegou ao `End_Point`:
    - `(inst.iIndex < inst.End_Point && inst.Direction == 1)` ou
    - `(inst.iIndex > inst.End_Point && inst.Direction == -1)`  
então volta para estado 3 (continua percorrendo segmentos). <sup>[321]</sup>
  - Se chegou ao `End_Point`:
    - `(inst.iIndex >= inst.End_Point && inst.Direction == 1)` ou
    - `(inst.iIndex <= inst.End_Point && inst.Direction == -1)`  
então vai para estado 5 (fim do percurso principal). <sup>[321]</sup>
  - Se `m.bidirectional == true` e `inst.iIndex == m.start_Index && inst.jIndex == m.start_Index + 1`, volta a 1 (reset para novo ciclo bidirecional). <sup>[321]</sup>
  - Se `Tosse_signal == 1` e `m.bidirectional == true`, vai para estado 14 (estado especial para tosse). <sup>[321]</sup>

Interpretação: este é o estado que controla o avanço ao longo da trajetória, define quando o movimento chegou ao fim e decide se vai haver reciclagem, pausa ou comportamento especial (tosse).

## Estado 5 – Fim da trajetória, preparação para pausa / reciclagem

- Ações:
  - `inst.segmentStartMs = nowMs`;
  - `inst.jIndex = inst.iIndex`; (fim do movimento, `jIndex` coincide com `iIndex`). <sup>[321]</sup>
- Condições:
  - Se `inst.Direction == -1` e `m.Dt_pauseMs_Ini != 0`, vai para estado 7 (pausa inicial). <sup>[321]</sup>
  - Se `inst.Direction == 1` e `m.Dt_pauseMs_End != 0`, vai para estado 8 (pausa final). <sup>[321]</sup>



- Senão:
  - Se `m.bidirectional == true`, vai para estado 6 (troca de direção). <sup>[321]</sup>
  - Se `m.bidirectional != true` e ambas pausas forem definidas, pode voltar a estado 1 (novo ciclo simples). <sup>[321]</sup>

Interpretação: este estado decide se depois de chegar ao fim vai haver pausa, inversão de direção ou reciclagem simples.

## Estado 6 – Troca de direção

- Ações:
  - `inst.Direction = inst.Direction * (-1);` (inverte direção: +1 → -1, -1 → +1). <sup>[321]</sup>
- Condições:
  - Se `inst.Direction == -1`, vai para 11 (preparar descida). <sup>[321]</sup>
  - Se `inst.Direction == 1`, vai para 12 (preparar subida). <sup>[321]</sup>

Interpretação: isto implementa o comportamento de vai-e-vem para movimentos bidirecionais.

## Estados 7 e 8 – Seleção do tempo de pausa

- Estado 7:
  - `inst.Dt_pause = m.Dt_pauseMs_Ini;` (usa pausa inicial). <sup>[321]</sup>
  - Vai para 9. <sup>[321]</sup>
- Estado 8:
  - `inst.Dt_pause = m.Dt_pauseMs_End;` (usa pausa final). <sup>[321]</sup>
  - Vai para 9. <sup>[321]</sup>

Interpretação: escolhem qual pausa aplicar dependendo se se chegou ao início ou ao fim da trajetória.

## Estado 9 – Espera durante a pausa

- Ações:
  - `inst.elapsed_2 = nowMs - inst.segmentStartMs;` (tempo decorrido na pausa). <sup>[321]</sup>
- Condições:
  - Se `inst.elapsed_2 >= inst.Dt_pause`, vai para estado 10 (fim da pausa). <sup>[321]</sup>

## Estado 10 – Fim da pausa, decisão de reciclagem ou parada

- Ações:
  - `inst.segmentStartMs = nowMs;`
  - `inst.elapsed_2 = 0;` <sup>[321]</sup>
- Condições:

- Se `start == 0` ou `inst.active != true` ou `inst.Individual_start_comand == 0`, volta a estado 0 (parado).<sup>[321]</sup>
- Se `m.bidirectional == true`, volta a estado 6 (nova inversão e ciclo).<sup>[321]</sup>
- Se `m.bidirectional == false`, vai para estado 1 (novo ciclo com mesma direção).<sup>[321]</sup>

Interpretação: este estado fecha o ciclo de pausa e decide se o movimento continua ou se pára.

## Estado 11 – Preparar descida (direção -1)

- Ações:
  - `inst.iIndex = m.n_Points - 1`; (começa do último ponto).<sup>[321]</sup>
  - `inst.jIndex = inst.iIndex + inst.Direction`; (ponto anterior).<sup>[321]</sup>
  - `inst.End_Point = 0`; (final da descida é o índice 0).<sup>[321]</sup>
- Condições:
  - Vai para estado 13 (descida ativa).<sup>[321]</sup>

Interpretação: este estado inicializa os índices para percorrer a trajetória de trás para frente.

\*(O estado 12 e 13 continuam essa lógica de subida/descida, mas o trecho enviado corta antes do fim; mesmo assim, dá para perceber o padrão.) \*<sup>[321]</sup>

## 2.3 Integração com a tosse (estado 14 e `Tosse_signal`)

- No estado 4, há uma condição especial:
  - Se `Tosse_signal == 1` e `m.bidirectional == true`, `inst.state = 14`;<sup>[321]</sup>
- Isto indica que quando há sinal de tosse e o movimento atual é bidirecional (tipicamente respiração), a FSM entra num estado específico (14) onde provavelmente altera comportamento para encaixar a tosse (por exemplo, acelerar, aumentar amplitude, ou sincronizar com vibração).<sup>[321]</sup>

Mesmo sem ver todo o estado 14, o ponto chave é: a FSM de movimento “sente” o `Tosse_signal` e consegue alterar o ciclo com base nisso, o que é coerente com a lógica da função `Preparacao_Tosse()`.

Neste ponto, o relatório já tem dois módulos bem detalhados: estruturas de dados e FSM de movimento. Na próxima resposta podemos detalhar o **Módulo 3 – FSM de comandos (modos RESP/BAT/ALL, comando L3, etc.)**.

Antes de avançar, para garantir que estás a acompanhar a lógica da FSM: se tivesses de explicar a alguém o papel dos estados 0, 2, 4 e 5 da `FSM_Motion_Update`, como os descreverias em palavras simples?

✱

1. <https://forum.arduino.cc/t/serial-communication-with-python-beginner-qestion/944780>

2. <https://github.com/pyserial/pyserial/issues/216>

3. <https://discuss.python.org/t/serial-read-with-python/16925>
4. <https://stackoverflow.com/questions/44056846/how-to-read-and-write-from-a-com-port-using-pyserial>
5. <https://forums.raspberrypi.com/viewtopic.php?t=34802>
6. <https://stackoverflow.com/questions/79116615/the-com-port-is-connected-with-pyserial-but-doesnt-return-data>
7. <https://forum.arduino.cc/t/pyserial-writing-gibberish/1080858>
8. <https://forums.raspberrypi.com/viewtopic.php?t=371931>
9. <https://github.com/pyserial/pyserial/issues/487>
10. <https://stackoverflow.com/questions/17553543/pyserial-non-blocking-read-loop>
11. <https://forum.arduino.cc/t/serial-communication-with-python-beginner-qestion/944780>
12. <https://pt.scribd.com/document/882956156/ESP32-PROGRAMACAO-VOL1>
13. <https://www.tudosobreiot.com.br/blog/1098-iot-feito-facil-brincando-com-o-esp32-no-arduino-ide>
14. [https://www.telecom.uff.br/pet/petws/downloads/tutoriais/Plan\\_2023\\_PDFs/Plan\\_2023\\_PEE5\\_Estudos\\_sobre\\_ESP32\\_A2024M09D11.pdf](https://www.telecom.uff.br/pet/petws/downloads/tutoriais/Plan_2023_PDFs/Plan_2023_PEE5_Estudos_sobre_ESP32_A2024M09D11.pdf)
15. <https://www.fernandok.com/2018/10/introducao-programacao-do-esp32.html>
16. <https://startbit.com.br/pt/article/estrutura-do-codigo-exemplo-de-hello-world-de-rede-no-esp32-L22G2A3>
17. <https://pt.scribd.com/document/850371906/Aula02-Explorando-ESP32-WROOM>
18. <https://elcereza.com/multiprocessamento-esp32-como-usar-corretamente/>
19. [https://gilmarbrito.com/articles/Apostila\\_Guia\\_ESP32.pdf](https://gilmarbrito.com/articles/Apostila_Guia_ESP32.pdf)
20. <https://startbit.com.br/pt/article/entendendo-o-codigo-explicacao-passo-a-passo-L2G3A4>
21. <https://www.manualdomaker.com/article/multiplas-interruptoes-externas-com-esp32/>
22. paste.txt
23. <https://forum.arduino.cc/t/watchdog-reset-esp32-if-stuck-more-than-120-seconds/1266565>
24. <https://stackoverflow.com/questions/58779165/esp32-arduino-core-whiletrue-loop-in-setup-function>
25. <https://community.platformio.org/t/esp32-c3-continuously-reset/28568>
26. <https://forum.arduino.cc/t/esp-32d-runs-setup-then-loops-back-to-boot-sequence/1088616>
27. <https://github.com/espressif/arduino-esp32/issues/3033>
28. <https://wokwi.com/projects/436936575234329601>
29. <https://github.com/espressif/arduino-esp32/issues/3798>
30. <https://forum.arduino.cc/t/esp32-always-reseting-with-simple-code/1266406>
31. <https://github.com/espressif/arduino-esp32/issues/3122>
32. <https://forum.arduino.cc/t/esp32-periodic-autoreset-possible-solved-1/577976>
33. <https://esp32.com/viewtopic.php?t=37551>
34. <https://community.platformio.org/t/esp32-s3-continuously-resetting-instead-of-printfing-anything/30368>
35. <https://pt.scribd.com/document/850371906/Aula02-Explorando-ESP32-WROOM>
36. <https://www.youtube.com/watch?v=Sb2LzWWQNeA>
37. <https://pt.scribd.com/document/882956156/ESP32-PROGRAMACAO-VOL1>
38. paste.txt

39. paste.txt

40. <https://github.com/espressif/arduino-esp32/issues/3122>

41. <https://forum.arduino.cc/t/esp-32d-runs-setup-then-loops-back-to-boot-sequence/1088616>

42. <https://forum.arduino.cc/t/esp32-always-reseting-with-simple-code/1266406>

43. <https://esp32.com/viewtopic.php?t=47078>

44. <https://github.com/espressif/esp-idf/issues/13075>

45. <https://forum.arduino.cc/t/esp32-always-reseting-with-simple-code/1266406>

46. paste.txt

47. <https://forum.arduino.cc/t/serial-monitor-not-working-on-esp32-solved/999688>

48. <https://forum.arduino.cc/t/esp32-always-resetting-while-receive-data-from-uno/887013>

49. <https://blog.wjcomponentes.com.br/2024/01/09/acionando-motor-dc-por-meio-do-pino-touch-do-esp-32/>

50. <https://dev.to/henriquebros/programacao-do-esp32-ligadesliga-motor-5bnc>

51. <https://esp32.com/viewtopic.php?t=4788>

52. <https://www.youtube.com/watch?v=IGB1k2LhEJM>

53. [https://ric.cps.sp.gov.br/bitstream/123456789/34020/1/tecnico\\_eletronica\\_2024\\_2\\_Automação\\_de\\_portões\\_via\\_nuvem\\_com\\_ESP32\\_Kauan\\_Rocha\\_da\\_Silva.pdf](https://ric.cps.sp.gov.br/bitstream/123456789/34020/1/tecnico_eletronica_2024_2_Automação_de_portões_via_nuvem_com_ESP32_Kauan_Rocha_da_Silva.pdf)

54. <https://esp32.com/viewtopic.php?t=32854>

55. <https://forum.arduino.cc/t/esp32-always-resetting/887979>

56. <https://forum.arduino.cc/t/reinicio-continuo-del-esp32/1318003>

57. <https://github.com/microsoft/vscode-arduino/issues/1040>

58. <https://wokwi.com/projects/406690596451675137>

59. paste.txt

60. <https://forum.arduino.cc/t/serial-monitor-not-working-on-esp32-solved/999688>

61. <https://forum.arduino.cc/t/esp32-always-reseting-with-simple-code/1266406>

62. <https://stackoverflow.com/questions/44056846/how-to-read-and-write-from-a-com-port-using-pyserial>

63. <https://stackoverflow.com/questions/17553543/pyserial-non-blocking-read-loop>

64. <https://pyserial.readthedocs.io/en/latest/shortintro.html>

65. <https://stackoverflow.com/questions/45411924/python3-two-way-serial-communication-reading-in-data>

66. <https://stackoverflow.com/questions/17553543/pyserial-non-blocking-read-loop>

67. <https://stackoverflow.com/questions/44056846/how-to-read-and-write-from-a-com-port-using-pyserial>

68. <https://forums.raspberrypi.com/viewtopic.php?t=251900>

69. <https://forum.arduino.cc/t/pyserial-writing-gibberish/1080858>

70. <https://gist.github.com/outtoin/9486110254b25a97c00e87b79959d031>

71. <https://stackoverflow.com/questions/676172/full-examples-of-using-pyserial-package/676257>

72. <https://github.com/pyserial/pyserial/issues/124>

73. <https://forum.arduino.cc/t/serial-communication-with-python-beginner-qestion/944780>

74. <https://pyserial.readthedocs.io/en/latest/shortintro.html>

75. <https://stackoverflow.com/questions/44056846/how-to-read-and-write-from-a-com-port-using-pyserial>
76. <https://forum.arduino.cc/t/esp32-reset-at-serial-terminal-shutdown/1391969>
77. <https://github.com/espressif/esp-idf/issues/2848>
78. <https://esp32.com/viewtopic.php?t=6210>
79. <https://esp32.com/viewtopic.php?t=4988>
80. <https://forum.arduino.cc/t/esp32-always-resetting-with-simple-code/1266406>
81. <https://www.youtube.com/watch?v=rtua2ZoG7tg>
82. <https://esp32.com/viewtopic.php?t=35727>
83. <https://esp32.com/viewtopic.php?t=47078>
84. <https://github.com/espressif/arduino-esp32/issues/8651>
85. <https://github.com/espressif/esp-idf/issues/13075>
86. <https://pyserial.readthedocs.io/en/latest/shortintro.html>
87. <https://esp32.com/viewtopic.php?t=6210>
88. <https://esp32.com/viewtopic.php?t=4988>
89. <https://github.com/jkb-git/ESP32Servo>
90. <https://dronebotworkshop.com/esp32-servo/>
91. <https://github.com/comdet/kbide-esp32-servo/blob/master/examples/04-SimpleServoAngles/04-SimpleServoAngles.ino>
92. <https://itsblue.dev/coderacer/library/src/commit/3d32010ab65ab955bf6184a554814de4988d791e/Arduino/libraries/ESP32Servo/src/ESP32Servo.cpp>
93. [https://www.reddit.com/r/arduino/comments/17wztpa/servo\\_set\\_to\\_go\\_from\\_0\\_90\\_180\\_but\\_odd\\_behaviour/](https://www.reddit.com/r/arduino/comments/17wztpa/servo_set_to_go_from_0_90_180_but_odd_behaviour/)
94. [https://docs.sunfounder.com/projects/umsk/es/latest/03\\_esp32/esp32\\_lesson33\\_servo.html](https://docs.sunfounder.com/projects/umsk/es/latest/03_esp32/esp32_lesson33_servo.html)
95. [https://madhephaestus.github.io/ESP32Servo/Knob\\_8ino\\_source.html](https://madhephaestus.github.io/ESP32Servo/Knob_8ino_source.html)
96. <https://docs.espressif.com/projects/esp-iot-solution/en/latest/motor/servo.html>
97. [https://madhephaestus.github.io/ESP32Servo/ESP32Servo\\_8h\\_source.html](https://madhephaestus.github.io/ESP32Servo/ESP32Servo_8h_source.html)
98. <https://randomnerdtutorials.com/esp32-servo-motor-web-server-arduino-ide/>
99. paste.txt
100. paste.txt
101. <https://stackoverflow.com/questions/44056846/how-to-read-and-write-from-a-com-port-using-pyserial>
102. paste.txt
103. image.jpg
104. paste.txt
105. image.jpg
106. image-2.jpg
107. paste.txt
108. image-2.jpg
109. image.jpg
110. <https://www.ibm.com/docs/pt-br/i/7.5.0?topic=functions-atoi-convert-character-string-integer>

111. <https://manpages.ubuntu.com/manpages/trusty/pt/man3/atoi.3.html>
112. <https://www.scribd.com/document/99427151/Atoi>
113. <https://pt.wikipedia.org/wiki/Atoi>
114. [https://www.reddit.com/r/learnprogramming/comments/v4xaku/how\\_do\\_you\\_use\\_atoi\\_c/](https://www.reddit.com/r/learnprogramming/comments/v4xaku/how_do_you_use_atoi_c/)
115. <https://embarcados.com.br/cadeia-de-caracteres-funcoes-de-conversao/>
116. [https://www.youtube.com/watch?v=a\\_1sarwHtQQ](https://www.youtube.com/watch?v=a_1sarwHtQQ)
117. <https://www.vivaolinux.com.br/topico/C-C++/Funcao-atoi>
118. [https://man.archlinux.org/man/extra/man-pages-pt\\_br/atoi.3.pt\\_BR](https://man.archlinux.org/man/extra/man-pages-pt_br/atoi.3.pt_BR)
119. <https://learn.microsoft.com/pt-br/cpp/c-runtime-library/reference/atoi-atoi-l-wtoi-wtoi-l?view=msvc-170>
120. image.jpg
121. paste.txt
122. <https://www.ibm.com/docs/pt-br/i/7.5.0?topic=functions-atoi-convert-character-string-integer>
123. <https://manpages.ubuntu.com/manpages/trusty/pt/man3/atoi.3.html>
124. paste.txt
125. paste.txt
126. <https://www.br-c.org/doku.php?id=strncpy>
127. <https://www.geeksforgeeks.org/c/strncpy-function-in-c/>
128. <https://www.ibm.com/docs/pt-br/i/7.5.0?topic=functions-strncpy-copy-strings>
129. <https://learn.microsoft.com/pt-br/cpp/c-runtime-library/reference/strncpy-strncpy-l-wcsncpy-wcsncpy-l-mbsncpy-mbsncpy-l?view=msvc-170>
130. <https://pt.stackoverflow.com/questions/126164/por-que-strcpy-é-insegura>
131. <https://www.ime.usp.br/~pf/algoritmos/aulas/string-manip.html>
132. <https://bitismyth.wordpress.com/2018/07/15/strncpy-strncat-e-sprintf-versus-strncpy-e-strcat/>
133. <https://www.facom.ufu.br/~madriana/PP/TP6.pdf>
134. <https://linguagemc.com.br/a-biblioteca-string-h/>
135. <https://www.youtube.com/watch?v=NYxDpEsdM8A>
136. paste.txt
137. paste.txt
138. paste.txt
139. paste.txt
140. paste.txt
141. paste.txt
142. paste.txt
143. paste.txt
144. paste.txt
145. image.jpg
146. paste.txt
147. Projeto-final-de-curso-Arvore-Bronquica-\_260509\_204354.pdf

148. paste.txt  
149. image.jpg  
150. paste.txt  
151. image.jpg  
152. image-2.jpg  
153. paste.txt  
154. paste.txt  
155. image.jpg  
156. image-2.jpg  
157. paste.txt  
158. View-of-Small-scale-Robot-Arm-Design-with-Pick-and-Place-Mission-Based-on-Inverse-Kinematics.pdf  
159. image.jpg  
160. image.jpg  
161. image-2.jpg  
162. paste.txt  
163. image-2.jpg  
164. image.jpg  
165. image.jpg  
166. paste.txt  
167. paste.txt  
168. paste.txt  
169. image.jpg  
170. paste.txt  
171. image.jpg  
172. paste.txt  
173. [https://repositorio.utfpr.edu.br/jspui/bitstream/1/31330/4/roboticaaprendizagemprojetos\\_produto.pdf](https://repositorio.utfpr.edu.br/jspui/bitstream/1/31330/4/roboticaaprendizagemprojetos_produto.pdf)  
174. <https://docs.espressif.com/projects/esptool/en/latest/esp32/esptool/basic-commands.html>  
175. <https://manuals.plus/pt/espressif/esp32-wroom-32d-esp32d-wifi-development-board-manual>  
176. <https://elcereza.com/blog/protecao-contra-leitura-e-gravacao-do-esp32/>  
177. [https://docs.espressif.com/projects/esp-at/en/release-v2.4.0.0/esp32/esp-at-en-v2.3.0.0\\_esp32c3-182-g6e9756e7-esp32.pdf](https://docs.espressif.com/projects/esp-at/en/release-v2.4.0.0/esp32/esp-at-en-v2.3.0.0_esp32c3-182-g6e9756e7-esp32.pdf)  
178. <https://embarcados.com.br/protecao-da-flash-no-esp32/>  
179. [https://docs.aws.amazon.com/pt\\_br/freertos/latest/userguide/getting\\_started\\_esp32wroom-32se.html](https://docs.aws.amazon.com/pt_br/freertos/latest/userguide/getting_started_esp32wroom-32se.html)  
180. <https://www.youtube.com/watch?v=L1EpHByCsE0>  
181. <https://blog.eletrogate.com/salvando-preferencias-no-esp32/>  
182. <https://www.ic.unicamp.br/~rodolfo/Cursos/verilog/MaquinasDeEstados/>  
183. paste.txt  
184. <https://fr.slideshare.net/slideshow/lab7-fsm/15603598?nway=%2C>

185. <https://www.youtube.com/watch?v=qFXXLbwvWsg>
186. [https://cin.ufpe.br/~agsf/disciplinas/sistemas\\_digitais/aula19\\_MaquinasEstadosFinitos.pdf](https://cin.ufpe.br/~agsf/disciplinas/sistemas_digitais/aula19_MaquinasEstadosFinitos.pdf)
187. <https://www.youtube.com/watch?v=E8nwu65pWo>
188. <https://www.youtube.com/watch?v=9-9ZxONuOUc>
189. [http://www.inf.ufsc.br/~j.guntzel/ine5406/SD\\_aula8P.pdf](http://www.inf.ufsc.br/~j.guntzel/ine5406/SD_aula8P.pdf)
190. [https://ic.unicamp.br/~cortes/mc602/slides/obsoleto/cap08\\_v3.pdf](https://ic.unicamp.br/~cortes/mc602/slides/obsoleto/cap08_v3.pdf)
191. <https://caddy.tech/docs/pt/verilog/finite-state-machines>
192. [https://balbertini.github.io/fsmstatereduction-pt\\_BR.html](https://balbertini.github.io/fsmstatereduction-pt_BR.html)
193. paste.txt
194. paste.txt
195. <https://pt.scribd.com/document/484551185/ebook-esp32-parte-1>
196. [https://gilmarbrito.com/articles/Apostila\\_Guia\\_ESP32.pdf](https://gilmarbrito.com/articles/Apostila_Guia_ESP32.pdf)
197. <https://forum.arduino.cc/t/esp-wroom-32-bad-characters-while-booting/562666>
198. <https://cursos.alura.com.br/forum/topico-ocorreu-um-erro-fatal-falha-ao-conectar-com-esp32-119327>
199. <https://forum.arduino.cc/t/dificuldade-ao-tentar-usar-o-esp32-com-o-erro-exit-status-1/1284920>
200. <https://www.youtube.com/watch?v=HlnpkDMd5Ng>
201. <https://wokwi.com/projects/395913767434090497>
202. <https://pt.scribd.com/document/679592742/ESP32-PROGRAMACAO>
203. <https://github.com/espressif/arduino-esp32/issues/9804>
204. [https://repositorio.utfpr.edu.br/jspui/bitstream/1/31330/4/roboticaaprendizagemprojetos\\_produto.pdf](https://repositorio.utfpr.edu.br/jspui/bitstream/1/31330/4/roboticaaprendizagemprojetos_produto.pdf)
205. <https://www.arduinoportugal.pt/comunicacao-serial-arduino/>
206. [https://www.reddit.com/r/arduino/comments/rb92dr/arduino\\_serialreadprint\\_strange\\_return\\_in\\_serial/](https://www.reddit.com/r/arduino/comments/rb92dr/arduino_serialreadprint_strange_return_in_serial/)
207. [http://repositorio.aee.edu.br/bitstream/aee/1855/1/ARDUINO\\_-\\_COMUNICAÇÃO\\_SERIAL.pdf](http://repositorio.aee.edu.br/bitstream/aee/1855/1/ARDUINO_-_COMUNICAÇÃO_SERIAL.pdf)
208. <https://embarcados.com.br/arduino-comunicacao-serial/>
209. <https://stackoverflow.com/questions/70594196/python-serial-does-not-read-correct-avalue>
210. <https://forum.arduino.cc/t/monitor-seriale-con-esp32/642697>
211. <https://pt.stackoverflow.com/questions/500746/problemas-na-comunicação-serial-com-python>
212. <https://wokwi.com/projects/410126126487464961>
213. <https://espblockly.com/SerialInputOutput.html>
214. <https://cursos.alura.com.br/forum/topico-duvida-pq-no-meu-serial-monitor-nao-aparece-nada-308938>
215. <https://elcereza.com/blog/leitura-de-strings-na-serial/>
216. <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html>
217. <https://pt.scribd.com/document/905817865/Comunicacao-Serial-UART-no-ESP32>
218. <https://pt.stackoverflow.com/questions/405537/o-serial-read-do-arduino-nao-funciona-como-esperado>
219. <https://deepbluembedded.com/esp32-hello-world-serial-print-arduino/>
220. <https://embarcados.com.br/arduino-comunicacao-serial/>
221. <https://embarcados.com.br/arduino-comunicacao-serial/>



- 222. <https://www.arduinoportugal.pt/comunicacao-serial-arduino/>
- 223. <https://espblockly.com/SerialInputOutput.html>
- 224. <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html>
- 225. <https://github.com/espressif/arduino-esp32/issues/9804>
- 226. <https://embarcados.com.br/arduino-comunicacao-serial/>
- 227. <https://embarcados.com.br/arduino-comunicacao-serial/>
- 228. <https://www.perplexity.ai/search/90a9a938-d7ba-452a-9029-20d59068ba96>
- 229. image.jpg
- 230. <https://pt.khanacademy.org/science/physics/geometric-optics/mirrors/v/mirror-equation-example-problems>
- 231. <https://dspace.uevora.pt/rdpc/bitstream/10174/10992/2/António J. Neto - Anexos - 166 368.pdf>
- 232. <https://pt.scribd.com/document/663807066/A-maquina-de-estado-finito-FSM>
- 233. [https://www.reddit.com/r/AskPhysics/comments/1g9vf0e/does\\_a\\_mirror\\_really\\_flip\\_left\\_and\\_right\\_or\\_is\\_it/](https://www.reddit.com/r/AskPhysics/comments/1g9vf0e/does_a_mirror_really_flip_left_and_right_or_is_it/)
- 234. <https://www.instagram.com/reel/DUA4ZhTEdP8/>
- 235. <https://www.youtube.com/watch?v=XqEe5x4FNx4>
- 236. <https://www.youtube.com/watch?v=fp9yQoo-rDQ>
- 237. <https://ojs.letras.up.pt/index.php/ori/article/view/13348/12056>
- 238. <https://translate.google.com/?hl=pt-BR&eotf=0&sl=it&tl=pt>
- 239. <https://ciberduvidas-ql.iscte-iul.pt/consultorio/perguntas/infinitivo-independente-declarativo-dizer-que-ja-enviei-as-fi-chas/33822>
- 240. paste.txt
- 241. paste.txt
- 242. paste.txt
- 243. paste.txt
- 244. paste.txt
- 245. paste.txt
- 246. paste.txt
- 247. paste.txt
- 248. paste.txt
- 249. paste.txt
- 250. paste.txt
- 251. paste.txt
- 252. paste.txt
- 253. paste.txt
- 254. paste.txt
- 255. paste.txt
- 256. paste.txt
- 257. paste.txt
- 258. paste.txt

259. paste.txt

260. paste.txt

261. paste.txt

262. paste.txt

263. paste.txt

264. paste.txt

265. <https://www.adobe.com/pt/acrobat/online/ai-summary-generator.html>

266. <https://www.happyscribe.com/pt/blog/chat-ia>

267. <https://mestreprompt.com/prompt/resumo-log-da-conversa-inteira/>

268. <https://pt.scribd.com/document/869334216/Transcricao-Chat-Completa>

269. <https://clickup.com/pt-BR/blog/456187/exportar-conversa-chatgpt>

270. [https://www.canva.com/pt\\_pt/ferramentas/sumarizador/](https://www.canva.com/pt_pt/ferramentas/sumarizador/)

271. <https://pt.scribd.com/document/957236098/Conversa-Antes-Da-Finalizacao-Do-Trabalho>

272. <https://www.editpad.org/tool/pt/text-summarizer>

273. <https://tactiq.io/pt-br/aprenda/exportar-conversa-do-chatgpt>

274. <https://updf.com/br/chatgpt/export-chatgpt-conversation/>

275. Projeto-final-de-curso-Arvore-Bronquica-\_260509\_204354.pdf

276. paste.txt

277. paste.txt

278. paste.txt

279. paste.txt

280. paste.txt

281. paste.txt

282. paste.txt

283. image.jpg

284. paste.txt

285. paste.txt

286. View-of-Small-scale-Robot-Arm-Design-with-Pick-and-Place-Mission-Based-on-Inverse-Kinematics.pdf

287. image.jpg

288. paste.txt

289. image.jpg

290. image-2.jpg

291. paste.txt

292. paste.txt

293. image.jpg

294. image-2.jpg

295. paste.txt

296. image.jpg

297. paste.txt  
298. image.jpg  
299. image-2.jpg  
300. paste.txt  
301. image-2.jpg  
302. image.jpg  
303. image.jpg  
304. paste.txt  
305. image.jpg  
306. paste.txt  
307. paste.txt  
308. paste.txt  
309. paste.txt  
310. paste.txt  
311. paste.txt  
312. paste.txt  
313. paste.txt  
314. paste.txt  
315. paste.txt  
316. image.jpg  
317. paste.txt  
318. image.jpg  
319. paste.txt  
320. paste.txt  
321. paste.txt